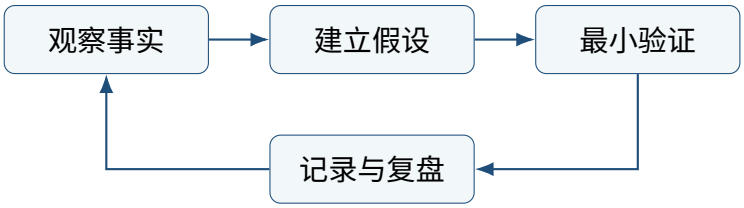
CTF 常见题型解题方法详解

从题型识别、证据保存到脚本化验证与赛后复盘



适用于本地靶场、赛事题目和获得明确授权的训练环境

2026 年 7 月 10 日

使用范围与安全边界

使用边界

本书只讨论 CTF 赛事、本地实验环境和获得明确书面授权的系统。所有命令、脚本、漏洞验证和流量分析都应限制在题目提供的目标、自己搭建的容器或专用虚拟机中。不要把书中的技巧用于公共网站、他人账户、生产网络或未知资产。遇到真实系统中的疑似漏洞，应停止进一步验证，并依照所属组织的漏洞披露流程报告。

本书把解题过程拆分为能够记录和验证的步骤。每章包含题型线索、标准流程、工具使用、证据保存、常见失误和小型演练。代码示例使用回环地址、示例域名、题目目录或本地容器。

阅读方法

初学者可以依次阅读第一篇，然后在 Web、密码学、二进制利用、逆向和取证中选择一个方向继续。已有经验的参赛者可以直接进入相应章节，再使用附录中的命令速查和检查清单完成复盘。完整结果应当说明题目入口，给出关键证据，能够重复执行得到相同结果，并解释方法成立的原因。

目录

使用范围与安全边界 **ii**

阅读方法 **iii**

第一部分　基础环境与通用方法 **1**

第一章　训练边界、隔离环境与证据目录 **2**

1.1 原理与题型边界 2
1.2 识别线索与优先动作 2
1.3 标准解题步骤 3
1.4 分析时持续追问的问题 3
1.5 命令、脚本与验证片段 3
1.6 证据保存与结果判定 4
1.7 常见失误与修正方法 4
1.8 小型演练 4
1.9 完成检查 5

第二章　通用解题循环：观察、假设、最小验证与复盘 **6**

2.1 原理与题型边界 6
2.2 识别线索与优先动作 6
2.3 标准解题步骤 7
2.4 分析时持续追问的问题 7
2.5 命令、脚本与验证片段 7
2.6 证据保存与结果判定 8
2.7 常见失误与修正方法 8
2.8 小型演练 8
2.9 完成检查 9

第三章　Linux 命令行、文件识别与文本处理 **10**

3.1 原理与题型边界 10

3.2 识别线索与优先动作 10
3.3 标准解题步骤 11
3.4 分析时持续追问的问题 11
3.5 命令、脚本与验证片段 11
3.6 证据保存与结果判定 12
3.7 常见失误与修正方法 12
3.8 小型演练 12
3.9 完成检查 13

第四章 字节、进制、编码、Unicode 与端序 14
4.1 原理与题型边界 14
4.2 识别线索与优先动作 14
4.3 标准解题步骤 15
4.4 分析时持续追问的问题 15
4.5 常用关系式 15
4.6 命令、脚本与验证片段 15
4.7 证据保存与结果判定 16
4.8 常见失误与修正方法 16
4.9 小型演练 17
4.10 完成检查 17

第五章 Python 解题脚本：输入输出、断言与确定性 18
5.1 原理与题型边界 18
5.2 识别线索与优先动作 18
5.3 标准解题步骤 18
5.4 分析时持续追问的问题 19
5.5 命令、脚本与验证片段 19
5.6 证据保存与结果判定 20
5.7 常见失误与修正方法 20
5.8 小型演练 20
5.9 完成检查 21

第六章 网络基础：TCP、UDP、DNS、TLS 与 HTTP 边界 22
6.1 原理与题型边界 22
6.2 识别线索与优先动作 22
6.3 标准解题步骤 23

6.4 分析时持续追问的问题 23
6.5 命令、脚本与验证片段 23
6.6 证据保存与结果判定 24
6.7 常见失误与修正方法 24
6.8 小型演练 24
6.9 完成检查 25

第二部分 Web 题型 26

第七章 HTTP 请求重构、基线与攻击面盘点 27
7.1 原理与题型边界 27
7.2 识别线索与优先动作 27
7.3 标准解题步骤 28
7.4 分析时持续追问的问题 28
7.5 命令、脚本与验证片段 28
7.6 证据保存与结果判定 29
7.7 常见失误与修正方法 29
7.8 小型演练 29
7.9 完成检查 30

第八章 源代码审查、静态资源与信息泄露 31
8.1 原理与题型边界 31
8.2 识别线索与优先动作 31
8.3 标准解题步骤 32
8.4 分析时持续追问的问题 32
8.5 命令、脚本与验证片段 32
8.6 证据保存与结果判定 33
8.7 常见失误与修正方法 33
8.8 小型演练 33
8.9 完成检查 34

第九章 认证、会话、访问控制与 IDOR 35
9.1 原理与题型边界 35
9.2 识别线索与优先动作 35
9.3 标准解题步骤 36

9.4 分析时持续追问的问题 36
9.5 命令、脚本与验证片段 36
9.6 证据保存与结果判定 37
9.7 常见失误与修正方法 37
9.8 小型演练 37
9.9 完成检查 38

第十章 SQL 注入：错误、布尔、联合与时间通道 39
10.1 原理与题型边界 39
10.2 识别线索与优先动作 39
10.3 标准解题步骤 40
10.4 分析时持续追问的问题 40
10.5 命令、脚本与验证片段 40
10.6 证据保存与结果判定 41
10.7 常见失误与修正方法 41
10.8 小型演练 41
10.9 完成检查 42

第十一章 NoSQL、LDAP 与 XPath 注入 43
11.1 原理与题型边界 43
11.2 识别线索与优先动作 43
11.3 标准解题步骤 44
11.4 分析时持续追问的问题 44
11.5 命令、脚本与验证片段 44
11.6 证据保存与结果判定 45
11.7 常见失误与修正方法 45
11.8 小型演练 45
11.9 完成检查 46

第十二章 XSS 与 DOM 数据流 47
12.1 原理与题型边界 47
12.2 识别线索与优先动作 47
12.3 标准解题步骤 48
12.4 分析时持续追问的问题 48
12.5 命令、脚本与验证片段 48
12.6 证据保存与结果判定 49

12.7 常见失误与修正方法 49
12.8 小型演练 49
12.9 完成检查 50

第十三章 CSRF、CORS 与点击劫持 51
13.1 原理与题型边界 51
13.2 识别线索与优先动作 51
13.3 标准解题步骤 52
13.4 分析时持续追问的问题 52
13.5 命令、脚本与验证片段 52
13.6 证据保存与结果判定 53
13.7 常见失误与修正方法 53
13.8 小型演练 53
13.9 完成检查 54

第十四章 文件上传：扩展名、MIME、内容检查与处理链 55
14.1 原理与题型边界 55
14.2 识别线索与优先动作 55
14.3 标准解题步骤 56
14.4 分析时持续追问的问题 56
14.5 命令、脚本与验证片段 56
14.6 证据保存与结果判定 57
14.7 常见失误与修正方法 57
14.8 小型演练 57
14.9 完成检查 58

第十五章 目录穿越、文件读取与本地包含 59
15.1 原理与题型边界 59
15.2 识别线索与优先动作 59
15.3 标准解题步骤 60
15.4 分析时持续追问的问题 60
15.5 命令、脚本与验证片段 60
15.6 证据保存与结果判定 61
15.7 常见失误与修正方法 61
15.8 小型演练 61
15.9 完成检查 62

第十六章　操作系统命令注入 **63**
16.1 原理与题型边界 63
16.2 识别线索与优先动作 63
16.3 标准解题步骤 63
16.4 分析时持续追问的问题 64
16.5 命令、脚本与验证片段 64
16.6 证据保存与结果判定 65
16.7 常见失误与修正方法 65
16.8 小型演练 65
16.9 完成检查 66

第十七章　SSRF：服务端请求、URL 解析与重定向 **67**
17.1 原理与题型边界 67
17.2 识别线索与优先动作 67
17.3 标准解题步骤 68
17.4 分析时持续追问的问题 68
17.5 命令、脚本与验证片段 68
17.6 证据保存与结果判定 69
17.7 常见失误与修正方法 69
17.8 小型演练 69
17.9 完成检查 70

第十八章　服务端模板注入 SSTI **71**
18.1 原理与题型边界 71
18.2 识别线索与优先动作 71
18.3 标准解题步骤 71
18.4 分析时持续追问的问题 72
18.5 命令、脚本与验证片段 72
18.6 证据保存与结果判定 73
18.7 常见失误与修正方法 73
18.8 小型演练 73
18.9 完成检查 74

第十九章　XXE 与 XML 解析器边界 **75**
19.1 原理与题型边界 75
19.2 识别线索与优先动作 75

19.3 标准解题步骤 76
19.4 分析时持续追问的问题 76
19.5 命令、脚本与验证片段 76
19.6 证据保存与结果判定 77
19.7 常见失误与修正方法 77
19.8 小型演练 77
19.9 完成检查 78

第二十章 不安全反序列化与对象重建 79
20.1 原理与题型边界 79
20.2 识别线索与优先动作 79
20.3 标准解题步骤 80
20.4 分析时持续追问的问题 80
20.5 命令、脚本与验证片段 80
20.6 证据保存与结果判定 81
20.7 常见失误与修正方法 81
20.8 小型演练 81
20.9 完成检查 82

第二十一章 JWT、签名 token 与 OAuth 常见错误 83
21.1 原理与题型边界 83
21.2 识别线索与优先动作 83
21.3 标准解题步骤 84
21.4 分析时持续追问的问题 84
21.5 命令、脚本与验证片段 84
21.6 证据保存与结果判定 85
21.7 常见失误与修正方法 85
21.8 小型演练 85
21.9 完成检查 86

第二十二章 API、GraphQL、批量赋值与参数污染 87
22.1 原理与题型边界 87
22.2 识别线索与优先动作 87
22.3 标准解题步骤 88
22.4 分析时持续追问的问题 88
22.5 命令、脚本与验证片段 88

22.6 证据保存与结果判定 89
22.7 常见失误与修正方法 89
22.8 小型演练 89
22.9 完成检查 90

第二十三章 竞态条件与业务逻辑漏洞 91
23.1 原理与题型边界 91
23.2 识别线索与优先动作 91
23.3 标准解题步骤 92
23.4 分析时持续追问的问题 92
23.5 命令、脚本与验证片段 92
23.6 证据保存与结果判定 93
23.7 常见失误与修正方法 93
23.8 小型演练 93
23.9 完成检查 94

第二十四章 Web 综合案例：从侦察到稳定复现 95
24.1 原理与题型边界 95
24.2 识别线索与优先动作 95
24.3 标准解题步骤 96
24.4 分析时持续追问的问题 96
24.5 命令、脚本与验证片段 96
24.6 证据保存与结果判定 97
24.7 常见失误与修正方法 97
24.8 小型演练 97
24.9 完成检查 98

第三部分 密码学题型 99

第二十五章 密码学解题流程、模运算与数论基础 100
25.1 原理与题型边界 100
25.2 识别线索与优先动作 100
25.3 标准解题步骤 101
25.4 分析时持续追问的问题 101
25.5 常用公式 101

25.6 命令、脚本与验证片段 102
25.7 证据保存与结果判定 102
25.8 常见失误与修正方法 102
25.9 小型演练 103
25.10 完成检查 103

第二十六章 古典密码、替换、移位与频率分析 104
26.1 原理与题型边界 104
26.2 识别线索与优先动作 104
26.3 标准解题步骤 105
26.4 分析时持续追问的问题 105
26.5 命令、脚本与验证片段 105
26.6 证据保存与结果判定 106
26.7 常见失误与修正方法 106
26.8 小型演练 106
26.9 完成检查 107

第二十七章 XOR、重复密钥与流密码误用 108
27.1 原理与题型边界 108
27.2 识别线索与优先动作 108
27.3 标准解题步骤 109
27.4 分析时持续追问的问题 109
27.5 命令、脚本与验证片段 109
27.6 证据保存与结果判定 110
27.7 常见失误与修正方法 110
27.8 小型演练 110
27.9 完成检查 111

第二十八章 哈希、MAC、密码存储与长度扩展 112
28.1 原理与题型边界 112
28.2 识别线索与优先动作 112
28.3 标准解题步骤 113
28.4 分析时持续追问的问题 113
28.5 命令、脚本与验证片段 113
28.6 证据保存与结果判定 114
28.7 常见失误与修正方法 114

28.8 小型演练 114
28.9 完成检查 115

第二十九章 RSA 基础：参数、填充与正确实现 116
29.1 原理与题型边界 116
29.2 识别线索与优先动作 116
29.3 标准解题步骤 117
29.4 分析时持续追问的问题 117
29.5 RSA 关系式 117
29.6 命令、脚本与验证片段 117
29.7 证据保存与结果判定 118
29.8 常见失误与修正方法 118
29.9 小型演练 118
29.10 完成检查 119

第三十章 RSA 常见弱点：共模、广播、低私钥与部分泄露 120
30.1 原理与题型边界 120
30.2 识别线索与优先动作 120
30.3 标准解题步骤 121
30.4 分析时持续追问的问题 121
30.5 命令、脚本与验证片段 121
30.6 证据保存与结果判定 122
30.7 常见失误与修正方法 122
30.8 小型演练 122
30.9 完成检查 123

第三十一章 椭圆曲线、ECDSA/DSA 与 nonce 重用 124
31.1 原理与题型边界 124
31.2 识别线索与优先动作 124
31.3 标准解题步骤 125
31.4 分析时持续追问的问题 125
31.5 签名方程 125
31.6 命令、脚本与验证片段 126
31.7 证据保存与结果判定 126
31.8 常见失误与修正方法 126
31.9 小型演练 127

31.10 完成检查 127

第三十二章 对称加密模式、填充与预言机 128

32.1 原理与题型边界 128
32.2 识别线索与优先动作 128
32.3 标准解题步骤 129
32.4 分析时持续追问的问题 129
32.5 命令、脚本与验证片段 129
32.6 证据保存与结果判定 130
32.7 常见失误与修正方法 130
32.8 小型演练 130
32.9 完成检查 131

第三十三章 伪随机数、LCG、MT19937 与状态恢复 132

33.1 原理与题型边界 132
33.2 识别线索与优先动作 132
33.3 标准解题步骤 133
33.4 分析时持续追问的问题 133
33.5 命令、脚本与验证片段 133
33.6 证据保存与结果判定 134
33.7 常见失误与修正方法 134
33.8 小型演练 134
33.9 完成检查 135

第三十四章 格、LLL、Coppersmith 与近似关系 136

34.1 原理与题型边界 136
34.2 识别线索与优先动作 136
34.3 标准解题步骤 137
34.4 分析时持续追问的问题 137
34.5 命令、脚本与验证片段 137
34.6 证据保存与结果判定 138
34.7 常见失误与修正方法 138
34.8 小型演练 138
34.9 完成检查 139

第三十五章　密码学综合案例：从参数审计到脚本验证 **140**
35.1 原理与题型边界 140
35.2 识别线索与优先动作 140
35.3 标准解题步骤 141
35.4 分析时持续追问的问题 141
35.5 命令、脚本与验证片段 141
35.6 证据保存与结果判定 142
35.7 常见失误与修正方法 142
35.8 小型演练 143
35.9 完成检查 143

第四部分　二进制利用 Pwn 题型 **144**

第三十六章　处理器架构、寄存器、端序与调用约定 **145**
36.1 原理与题型边界 145
36.2 识别线索与优先动作 145
36.3 标准解题步骤 146
36.4 分析时持续追问的问题 146
36.5 常见 ABI 摘要 146
36.6 命令、脚本与验证片段 147
36.7 证据保存与结果判定 147
36.8 常见失误与修正方法 147
36.9 小型演练 148
36.10 完成检查 148

第三十七章　ELF 结构、动态链接与保护机制 **149**
37.1 原理与题型边界 149
37.2 识别线索与优先动作 149
37.3 标准解题步骤 150
37.4 分析时持续追问的问题 150
37.5 命令、脚本与验证片段 150
37.6 证据保存与结果判定 151
37.7 常见失误与修正方法 151
37.8 小型演练 151
37.9 完成检查 152

第三十八章　GDB、断点、core 与崩溃定位　153
38.1 原理与题型边界 . . . 153
38.2 识别线索与优先动作 . . . 153
38.3 标准解题步骤 . . . 154
38.4 分析时持续追问的问题 . . . 154
38.5 命令、脚本与验证片段 . . . 154
38.6 证据保存与结果判定 . . . 155
38.7 常见失误与修正方法 . . . 155
38.8 小型演练 . . . 155
38.9 完成检查 . . . 156

第三十九章　栈溢出、偏移、Canary 与 ret2win　157
39.1 原理与题型边界 . . . 157
39.2 识别线索与优先动作 . . . 157
39.3 标准解题步骤 . . . 158
39.4 分析时持续追问的问题 . . . 158
39.5 命令、脚本与验证片段 . . . 158
39.6 证据保存与结果判定 . . . 159
39.7 常见失误与修正方法 . . . 159
39.8 小型演练 . . . 159
39.9 完成检查 . . . 160

第四十章　信息泄露、ret2libc 与动态基址计算　161
40.1 原理与题型边界 . . . 161
40.2 识别线索与优先动作 . . . 161
40.3 标准解题步骤 . . . 162
40.4 分析时持续追问的问题 . . . 162
40.5 命令、脚本与验证片段 . . . 162
40.6 证据保存与结果判定 . . . 163
40.7 常见失误与修正方法 . . . 163
40.8 小型演练 . . . 163
40.9 完成检查 . . . 164

第四十一章　ROP gadget、调用链与约束满足　165
41.1 原理与题型边界 . . . 165
41.2 识别线索与优先动作 . . . 165

41.3 标准解题步骤 166
41.4 分析时持续追问的问题 166
41.5 命令、脚本与验证片段 166
41.6 证据保存与结果判定 167
41.7 常见失误与修正方法 167
41.8 小型演练 167
41.9 完成检查 168

第四十二章 栈迁移、SROP 与 seccomp 约束 169
42.1 原理与题型边界 169
42.2 识别线索与优先动作 169
42.3 标准解题步骤 170
42.4 分析时持续追问的问题 170
42.5 命令、脚本与验证片段 170
42.6 证据保存与结果判定 171
42.7 常见失误与修正方法 171
42.8 小型演练 171
42.9 完成检查 172

第四十三章 格式化字符串：泄露、写入与参数定位 173
43.1 原理与题型边界 173
43.2 识别线索与优先动作 173
43.3 标准解题步骤 174
43.4 分析时持续追问的问题 174
43.5 命令、脚本与验证片段 174
43.6 证据保存与结果判定 175
43.7 常见失误与修正方法 175
43.8 小型演练 175
43.9 完成检查 176

第四十四章 Shellcode、系统调用与编码约束 177
44.1 原理与题型边界 177
44.2 识别线索与优先动作 177
44.3 标准解题步骤 178
44.4 分析时持续追问的问题 178
44.5 命令、脚本与验证片段 178

44.6 证据保存与结果判定 179
44.7 常见失误与修正方法 179
44.8 小型演练 179
44.9 完成检查 180

第四十五章 堆基础：chunk、bin、分配与释放轨迹 181
45.1 原理与题型边界 181
45.2 识别线索与优先动作 181
45.3 标准解题步骤 182
45.4 分析时持续追问的问题 182
45.5 命令、脚本与验证片段 182
45.6 证据保存与结果判定 183
45.7 常见失误与修正方法 183
45.8 小型演练 183
45.9 完成检查 184

第四十六章 UAF、Double Free、Tcache 与堆写原语 185
46.1 原理与题型边界 185
46.2 识别线索与优先动作 185
46.3 标准解题步骤 186
46.4 分析时持续追问的问题 186
46.5 命令、脚本与验证片段 186
46.6 证据保存与结果判定 187
46.7 常见失误与修正方法 187
46.8 小型演练 187
46.9 完成检查 188

第四十七章 Pwn 综合案例：泄露、基址、链条与远程稳定性 189
47.1 原理与题型边界 189
47.2 识别线索与优先动作 189
47.3 标准解题步骤 190
47.4 分析时持续追问的问题 190
47.5 命令、脚本与验证片段 190
47.6 证据保存与结果判定 191
47.7 常见失误与修正方法 191
47.8 小型演练 191

47.9 完成检查 192

第五部分　逆向工程 Reverse 题型 193

第四十八章　汇编基础、寄存器、标志位与反编译器阅读 194

48.1 原理与题型边界 194
48.2 识别线索与优先动作 194
48.3 标准解题步骤 195
48.4 分析时持续追问的问题 195
48.5 命令、脚本与验证片段 195
48.6 证据保存与结果判定 196
48.7 常见失误与修正方法 196
48.8 小型演练 197
48.9 完成检查 197

第四十九章　静态分析：字符串、导入、交叉引用与控制流 198

49.1 原理与题型边界 198
49.2 识别线索与优先动作 198
49.3 标准解题步骤 199
49.4 分析时持续追问的问题 199
49.5 命令、脚本与验证片段 199
49.6 证据保存与结果判定 200
49.7 常见失误与修正方法 200
49.8 小型演练 201
49.9 完成检查 201

第五十章　动态分析：断点、系统调用、库调用与输入差分 202

50.1 原理与题型边界 202
50.2 识别线索与优先动作 202
50.3 标准解题步骤 203
50.4 分析时持续追问的问题 203
50.5 命令、脚本与验证片段 203
50.6 证据保存与结果判定 204
50.7 常见失误与修正方法 204
50.8 小型演练 205

50.9 完成检查 205

第五十一章 常见数据变换、校验算法与轻量混淆 206

51.1 原理与题型边界 206
51.2 识别线索与优先动作 206
51.3 标准解题步骤 207
51.4 分析时持续追问的问题 207
51.5 命令、脚本与验证片段 207
51.6 证据保存与结果判定 208
51.7 常见失误与修正方法 208
51.8 小型演练 209
51.9 完成检查 209

第五十二章 壳、压缩、自修改代码与运行时解包 210

52.1 原理与题型边界 210
52.2 识别线索与优先动作 210
52.3 标准解题步骤 211
52.4 分析时持续追问的问题 211
52.5 命令、脚本与验证片段 211
52.6 证据保存与结果判定 212
52.7 常见失误与修正方法 212
52.8 小型演练 213
52.9 完成检查 213

第五十三章 反调试、反虚拟机、时间检测与环境检查 214

53.1 原理与题型边界 214
53.2 识别线索与优先动作 214
53.3 标准解题步骤 215
53.4 分析时持续追问的问题 215
53.5 命令、脚本与验证片段 215
53.6 证据保存与结果判定 216
53.7 常见失误与修正方法 216
53.8 小型演练 217
53.9 完成检查 217

第五十四章　自定义虚拟机、字节码与状态机逆向　218
54.1 原理与题型边界 . . . 218
54.2 识别线索与优先动作 . . . 218
54.3 标准解题步骤 . . . 219
54.4 分析时持续追问的问题 . . . 219
54.5 命令、脚本与验证片段 . . . 219
54.6 证据保存与结果判定 . . . 220
54.7 常见失误与修正方法 . . . 220
54.8 小型演练 . . . 221
54.9 完成检查 . . . 221

第五十五章　.NET、Java、Python 字节码与托管程序　222
55.1 原理与题型边界 . . . 222
55.2 识别线索与优先动作 . . . 222
55.3 标准解题步骤 . . . 223
55.4 分析时持续追问的问题 . . . 223
55.5 命令、脚本与验证片段 . . . 223
55.6 证据保存与结果判定 . . . 224
55.7 常见失误与修正方法 . . . 224
55.8 小型演练 . . . 225
55.9 完成检查 . . . 225

第五十六章　逆向综合案例：从样本盘点到独立求解器　226
56.1 原理与题型边界 . . . 226
56.2 识别线索与优先动作 . . . 226
56.3 标准解题步骤 . . . 227
56.4 分析时持续追问的问题 . . . 227
56.5 命令、脚本与验证片段 . . . 227
56.6 证据保存与结果判定 . . . 228
56.7 常见失误与修正方法 . . . 228
56.8 小型演练 . . . 228
56.9 完成检查 . . . 229

第六部分　数字取证、隐写与 Misc 题型 **230**

第五十七章　文件签名、格式结构、数据拼接与文件雕刻 **231**

57.1 原理与题型边界 231
57.2 识别线索与优先动作 231
57.3 标准解题步骤 232
57.4 分析时持续追问的问题 232
57.5 命令、脚本与验证片段 232
57.6 证据保存与结果判定 233
57.7 常见失误与修正方法 233
57.8 小型演练 233
57.9 完成检查 234

第五十八章　磁盘镜像、分区表与文件系统恢复 **235**

58.1 原理与题型边界 235
58.2 识别线索与优先动作 235
58.3 标准解题步骤 236
58.4 分析时持续追问的问题 236
58.5 命令、脚本与验证片段 236
58.6 证据保存与结果判定 237
58.7 常见失误与修正方法 237
58.8 小型演练 238
58.9 完成检查 238

第五十九章　网络流量 PCAP：会话重组、协议分析与对象提取 **239**

59.1 原理与题型边界 239
59.2 识别线索与优先动作 239
59.3 标准解题步骤 240
59.4 分析时持续追问的问题 240
59.5 命令、脚本与验证片段 240
59.6 证据保存与结果判定 241
59.7 常见失误与修正方法 241
59.8 小型演练 242
59.9 完成检查 242

第六十章　内存取证：进程、网络、命令行、注入区域与文件对象 **243**
60.1 原理与题型边界 243
60.2 识别线索与优先动作 243
60.3 标准解题步骤 244
60.4 分析时持续追问的问题 244
60.5 命令、脚本与验证片段 244
60.6 证据保存与结果判定 245
60.7 常见失误与修正方法 245
60.8 小型演练 246
60.9 完成检查 246

第六十一章　日志分析、时区统一与事件时间线 **247**
61.1 原理与题型边界 247
61.2 识别线索与优先动作 247
61.3 标准解题步骤 248
61.4 分析时持续追问的问题 248
61.5 命令、脚本与验证片段 248
61.6 证据保存与结果判定 249
61.7 常见失误与修正方法 249
61.8 小型演练 250
61.9 完成检查 250

第六十二章　图像隐写、像素通道、调色板与元数据 **251**
62.1 原理与题型边界 251
62.2 识别线索与优先动作 251
62.3 标准解题步骤 252
62.4 分析时持续追问的问题 252
62.5 命令、脚本与验证片段 252
62.6 证据保存与结果判定 253
62.7 常见失误与修正方法 253
62.8 小型演练 254
62.9 完成检查 254

第六十三章　音频、视频、频谱与帧级分析 **255**
63.1 原理与题型边界 255
63.2 识别线索与优先动作 255

63.3 标准解题步骤 256
63.4 分析时持续追问的问题 256
63.5 命令、脚本与验证片段 256
63.6 证据保存与结果判定 257
63.7 常见失误与修正方法 257
63.8 小型演练 258
63.9 完成检查 258

第六十四章 文档、压缩包、PDF、Office 与元数据取证 259
64.1 原理与题型边界 259
64.2 识别线索与优先动作 259
64.3 标准解题步骤 260
64.4 分析时持续追问的问题 260
64.5 命令、脚本与验证片段 260
64.6 证据保存与结果判定 261
64.7 常见失误与修正方法 261
64.8 小型演练 262
64.9 完成检查 262

第六十五章 硬件、串口、逻辑分析与简单总线数据 263
65.1 原理与题型边界 263
65.2 识别线索与优先动作 263
65.3 标准解题步骤 264
65.4 分析时持续追问的问题 264
65.5 命令、脚本与验证片段 264
65.6 证据保存与结果判定 265
65.7 常见失误与修正方法 265
65.8 小型演练 266
65.9 完成检查 266

第六十六章 OSINT 题：公开资料检索、时间验证与隐私边界 267
66.1 原理与题型边界 267
66.2 识别线索与优先动作 267
66.3 标准解题步骤 268
66.4 分析时持续追问的问题 268
66.5 命令、脚本与验证片段 268

66.6 证据保存与结果判定 269
66.7 常见失误与修正方法 269
66.8 小型演练 270
66.9 完成检查 270

第六十七章 Misc：二维码、条码、键盘轨迹、算法题与冷门语言 271
67.1 原理与题型边界 271
67.2 识别线索与优先动作 271
67.3 标准解题步骤 272
67.4 分析时持续追问的问题 272
67.5 命令、脚本与验证片段 272
67.6 证据保存与结果判定 273
67.7 常见失误与修正方法 274
67.8 小型演练 274
67.9 完成检查 274

第六十八章 取证综合案例：多源证据关联与可复现报告 275
68.1 原理与题型边界 275
68.2 识别线索与优先动作 275
68.3 标准解题步骤 276
68.4 分析时持续追问的问题 276
68.5 命令、脚本与验证片段 276
68.6 证据保存与结果判定 277
68.7 常见失误与修正方法 277
68.8 小型演练 278
68.9 完成检查 278

第七部分 移动端、智能合约、容器与云环境 279

第六十九章 Android APK：清单、组件、资源、字节码与本地库 280
69.1 原理与题型边界 280
69.2 识别线索与优先动作 280
69.3 标准解题步骤 281
69.4 分析时持续追问的问题 281
69.5 命令、脚本与验证片段 282

69.6 证据保存与结果判定 282
69.7 常见失误与修正方法 282
69.8 小型演练 283
69.9 完成检查 283

第七十章 iOS IPA、Mach-O 与应用数据分析 284
70.1 原理与题型边界 284
70.2 识别线索与优先动作 284
70.3 标准解题步骤 285
70.4 分析时持续追问的问题 285
70.5 命令、脚本与验证片段 285
70.6 证据保存与结果判定 286
70.7 常见失误与修正方法 286
70.8 小型演练 287
70.9 完成检查 287

第七十一章 智能合约：存储布局、调用上下文与常见漏洞原语 288
71.1 原理与题型边界 288
71.2 识别线索与优先动作 288
71.3 标准解题步骤 289
71.4 分析时持续追问的问题 289
71.5 存储槽的基本关系 289
71.6 命令、脚本与验证片段 290
71.7 证据保存与结果判定 290
71.8 常见失误与修正方法 290
71.9 小型演练 291
71.10 完成检查 291

第七十二章 Docker 镜像、层、Entrypoint、挂载与容器题 292
72.1 原理与题型边界 292
72.2 识别线索与优先动作 292
72.3 标准解题步骤 293
72.4 分析时持续追问的问题 293
72.5 命令、脚本与验证片段 293
72.6 证据保存与结果判定 294
72.7 常见失误与修正方法 294

72.8 小型演练 295
72.9 完成检查 295

第七十三章 云策略、IAM、K8s 清单与基础设施配置题 296
73.1 原理与题型边界 296
73.2 识别线索与优先动作 296
73.3 标准解题步骤 297
73.4 分析时持续追问的问题 297
73.5 命令、脚本与验证片段 298
73.6 证据保存与结果判定 298
73.7 常见失误与修正方法 298
73.8 小型演练 299
73.9 完成检查 299

第七十四章 现代平台综合案例：移动端、API、容器与链上状态联动 300
74.1 原理与题型边界 300
74.2 识别线索与优先动作 300
74.3 标准解题步骤 301
74.4 分析时持续追问的问题 301
74.5 命令、脚本与验证片段 301
74.6 证据保存与结果判定 302
74.7 常见失误与修正方法 302
74.8 小型演练 303
74.9 完成检查 303

第八部分 竞赛工程、协作与训练方法 304

第七十五章 开赛盘点、题目分流、动态分值与时间预算 305
75.1 原理与题型边界 305
75.2 识别线索与优先动作 305
75.3 标准解题步骤 306
75.4 分析时持续追问的问题 306
75.5 命令、脚本与验证片段 306
75.6 证据保存与结果判定 307
75.7 常见失误与修正方法 307

75.8 小型演练 308
75.9 完成检查 308

第七十六章 团队协作、共享笔记、证据交接与 Git 使用 309
76.1 原理与题型边界 309
76.2 识别线索与优先动作 309
76.3 标准解题步骤 310
76.4 分析时持续追问的问题 310
76.5 命令、脚本与验证片段 310
76.6 证据保存与结果判定 311
76.7 常见失误与修正方法 311
76.8 小型演练 312
76.9 完成检查 312

第七十七章 Writeup、复现实验、证据图与责任边界 313
77.1 原理与题型边界 313
77.2 识别线索与优先动作 313
77.3 标准解题步骤 314
77.4 分析时持续追问的问题 314
77.5 命令、脚本与验证片段 314
77.6 证据保存与结果判定 315
77.7 常见失误与修正方法 315
77.8 小型演练 316
77.9 完成检查 316

第七十八章 环境故障、工具差异、编码问题与系统化排错 317
78.1 原理与题型边界 317
78.2 识别线索与优先动作 317
78.3 标准解题步骤 318
78.4 分析时持续追问的问题 318
78.5 命令、脚本与验证片段 318
78.6 证据保存与结果判定 319
78.7 常见失误与修正方法 319
78.8 小型演练 320
78.9 完成检查 320

第七十九章　训练计划、题库选择、错题档案与能力增长 **321**
79.1 原理与题型边界 321
79.2 识别线索与优先动作 321
79.3 标准解题步骤 322
79.4 分析时持续追问的问题 322
79.5 十二周训练节奏建议 322
79.6 命令、脚本与验证片段 323
79.7 证据保存与结果判定 323
79.8 常见失误与修正方法 323
79.9 小型演练 324
79.10 完成检查 324

附录 A　实验目录、环境记录与题目工作表 **325**
A.1 环境快照字段 326
A.2 最小实验记录表 326

附录 B　Linux、网络与文本处理命令速查 **328**

附录 C　常见文件签名、容器结构与编码速查 **330**
C.1 编码识别线索 331

附录 D　HTTP 状态、头部、请求模板与 Web 检查表 **333**

附录 E　Python 解题模板库 **336**
E.1 健壮的命令行求解器 336
E.2 TCP 精确读取与带长度消息 337
E.3 二进制结构解析 337
E.4 HTTP 会话与差分 338

附录 F　密码学公式、判断表与脚本模板 **339**
F.1 基础关系 339

附录 G　Pwn、GDB 与 pwntools 速查 **341**

附录 H　逆向工程指令、跳转与工具速查 **344**

附录 I　取证命令、Wireshark 过滤器与 Volatility 插件速查 **346**

附录 J　移动端、合约、容器与云配置速查 **349**

附录 K　按题型执行的最终检查清单 **351**

K.1 通用 351
K.2 Web 351
K.3 Crypto 352
K.4 Pwn 352
K.5 Reverse 353
K.6 Forensics 353
K.7 Mobile/Contract/Cloud 354

附录 L　术语表 **355**

附录 M　可打印复盘表与训练记录 **358**

M.1 题目入口表 358
M.2 实验记录表 359
M.3 最终复现表 359
M.4 赛后复盘表 360

参考资料 **362**

Listings

1.1 本章常用命令与脚本 3
2.1 本章常用命令与脚本 7
3.1 本章常用命令与脚本 11
4.1 本章常用命令与脚本 15
5.1 本章常用命令与脚本 19
6.1 本章常用命令与脚本 23
7.1 本章常用命令与脚本 28
8.1 本章常用命令与脚本 33
9.1 本章常用命令与脚本 36
10.1 本章常用命令与脚本 40
11.1 本章常用命令与脚本 44
12.1 本章常用命令与脚本 49
13.1 本章常用命令与脚本 52
14.1 本章常用命令与脚本 56
15.1 本章常用命令与脚本 60
16.1 本章常用命令与脚本 64
17.1 本章常用命令与脚本 68
18.1 本章常用命令与脚本 72
19.1 本章常用命令与脚本 76
20.1 本章常用命令与脚本 80
21.1 本章常用命令与脚本 84
22.1 本章常用命令与脚本 88
23.1 本章常用命令与脚本 92
24.1 本章常用命令与脚本 96
25.1 本章常用命令与脚本 102
26.1 本章常用命令与脚本 105
27.1 本章常用命令与脚本 109
28.1 本章常用命令与脚本 113
29.1 本章常用命令与脚本 118
30.1 本章常用命令与脚本 121

31.1 本章常用命令与脚本 126
32.1 本章常用命令与脚本 129
33.1 本章常用命令与脚本 133
34.1 本章常用命令与脚本 137
35.1 本章常用命令与脚本 141
36.1 本章常用命令与脚本 147
37.1 本章常用命令与脚本 150
38.1 本章常用命令与脚本 154
39.1 本章常用命令与脚本 158
40.1 本章常用命令与脚本 162
41.1 本章常用命令与脚本 166
42.1 本章常用命令与脚本 170
43.1 本章常用命令与脚本 174
44.1 本章常用命令与脚本 178
45.1 本章常用命令与脚本 182
46.1 本章常用命令与脚本 186
47.1 本章常用命令与脚本 190
48.1 本章常用命令与脚本 196
49.1 本章常用命令与脚本 200
50.1 本章常用命令与脚本 204
51.1 本章常用命令与脚本 207
52.1 本章常用命令与脚本 211
53.1 本章常用命令与脚本 215
54.1 本章常用命令与脚本 220
55.1 本章常用命令与脚本 224
56.1 本章常用命令与脚本 227
57.1 本章常用命令与脚本 233
58.1 本章常用命令与脚本 237
59.1 本章常用命令与脚本 241
60.1 本章常用命令与脚本 245
61.1 本章常用命令与脚本 249
62.1 本章常用命令与脚本 253
63.1 本章常用命令与脚本 257
64.1 本章常用命令与脚本 261
65.1 本章常用命令与脚本 264

66.1 本章常用命令与脚本 269
67.1 本章常用命令与脚本 273
68.1 本章常用命令与脚本 276
69.1 本章常用命令与脚本 282
70.1 本章常用命令与脚本 286
71.1 本章常用命令与脚本 290
72.1 本章常用命令与脚本 294
73.1 本章常用命令与脚本 298
74.1 本章常用命令与脚本 302
75.1 本章常用命令与脚本 307
76.1 本章常用命令与脚本 311
77.1 本章常用命令与脚本 314
78.1 本章常用命令与脚本 318
79.1 本章常用命令与脚本 323
A.1 题目目录与 README 模板 325
B.1 安全批处理与中间结果保存 329
C.1 分层解码候选与结果记录 332
D.1 可重放 HTTP 基线模板 334
E.1 带输入哈希、验证和退出码的求解器 336
E.2 TCP 长度前缀消息模板 337
E.3 带边界检查的二进制解析器 337
E.4 保存响应证据的 requests 模板 338
F.1 模逆、整数根与共享因子模板 340
G.1 本地与远程双模式 pwntools 模板 342
H.1 静态、动态与固定位宽验证模板 345
I.1 网络与内存取证基线 347

第一部分

基础环境与通用方法

第一章　训练边界、隔离环境与证据目录

本章目标

建立适合 CTF 的工作目录、隔离层和证据记录习惯。目标是让题目原件不被修改，让未知程序只在受控环境运行，并让另一名队员能够从干净环境复现最终结果。

1.1　原理与题型边界

宿主机、分析虚拟机、容器和远程实例承担不同职责。原件只读保存，分析在副本上完成；容器适合固定用户态依赖，但涉及特权、设备或内核时应使用快照虚拟机。所有网络操作只指向赛事给出的实例或本地回环地址。

1.2　识别线索与优先动作

观察线索	可能含义	优先动作
提供 Dockerfile、依赖清单或单独 libc	运行环境可能是题目条件的一部分	保存原件并记录构建日志和版本
附件为 ELF、APK、宏文档或未知二进制	可能包含不可信可执行内容	在隔离虚拟机中先做静态识别
本地成功而远程失败	架构、库、编码、时区或随机化可能不同	逐项比较环境与启动参数
实例会重置或偶尔断开	存在生命周期、并发或连接限制	记录实例标识、时间和完整连接日志

1.3 标准解题步骤

1. **建立只读原件区**。创建 original、work、scripts、captures、notes 和 output 目录，原始附件只放入 original。
2. **计算哈希**。对下载文件计算 SHA-256，并在每次解压、修补和转换后记录新文件哈希。
3. **记录题面与范围**。保存题名、分值、flag 格式、主机、端口、实例限制和赛事规则。
4. **完成最小静态识别**。使用 file、xxd、strings、readelf 或格式专用工具确定类型、架构与元数据。
5. **选择隔离级别**。未知程序、宏文档、内核模块和特权容器使用快照虚拟机。
6. **固定依赖**。记录 Python 包、镜像标签、libc、JDK、Node 和关键工具版本。
7. **开启过程日志**。使用 script、tee、抓包和调试日志保存所有关键输入输出。

1.4 分析时持续追问的问题

- 当前观察是原始事实，还是工具给出的推断?
- 哪个最小输入变化能够区分主要假设?
- 结果能否在相同输入和干净环境中重复?
- 失败时是否保存了完整原始输出?
- 题目条件变化后，当前方法的前提是否仍成立?

1.5 命令、脚本与验证片段

以下内容只用于本地题目目录、回环地址、赛事提供的实例或明确授权的实验环境。每次运行都应保留原始输入和完整输出。

```
mkdir -p chall/{original,work,scripts,captures,notes,output}
cp challenge.zip chall/original/
cd chall
sha256sum original/* | tee notes/SHA256SUMS
```

```
file original/* | tee notes/file-types.txt
xxd -l 128 original/challenge.zip | tee notes/header.hex
script -q -f captures/session-$(date +%Y%m%d-%H%M%S).log
python3 --version | tee notes/versions.txt
openssl version | tee -a notes/versions.txt
gdb --version | head -n 1 | tee -a notes/versions.txt
```

Listing 1.1: 本章常用命令与脚本

1.6　证据保存与结果判定

保存原始输入、样本哈希、工具版本、完整命令行、标准输出、标准错误和关键中间结果。网络题保存原始请求与响应，二进制题保存地址、寄存器与内存，数据题保存每层转换前后的长度和哈希。结论必须能够由记录独立验证，不能只依赖终端历史。

1.7　常见失误与修正方法

- 直接在原件上解压、修复或保存，导致无法判断变化来源。
- 把附件上传到公共扫描或解码服务，可能泄露赛题。
- 使用 latest 镜像或未固定依赖，之后无法复现。
- 容器使用特权模式并挂载宿主机敏感目录。
- 只保留最终 exploit，没有保存题面和运行参数。

1.8　小型演练

题目场景

收到一个压缩包、ELF、单独的 libc 和远程端口。要求在不执行 ELF 的情况下确认架构并准备后续环境。

先把所有附件复制到 original 并计算哈希，用列表模式查看压缩包，再解压到 work。使用 file、readelf -h 和 readelf -l 确认架构、端序、解释器和入口点，对 libc 单独计算哈希。保存 Docker 构建日志和工具版本，README 记录远程参数与 flag 格式。此时尚未运行未知程序，但来源、环境和回滚路径已经明确。

1.9 完成检查

交付与复盘检查表

- ☐ 入口、输入、目标输出与限制条件已经写入笔记。
- ☐ 建立了可重复的基线输入与输出。
- ☐ 每个关键结论都有原始证据与验证步骤。
- ☐ 脚本能够识别失败，不会把异常输出当成成功。
- ☐ 从干净目录或新容器运行时能够得到同样结果。

第二章　通用解题循环：观察、假设、最小验证与复盘

本章目标

建立跨题型可使用的推理循环。先区分事实与解释，再提出少量可检验假设，通过一次只改变一个变量的最小实验减少不确定性，最后把结果和置信度写回笔记。

2.1　原理与题型边界

工具输出本身不是结论。事实应能由题面、文件、响应或调试器直接证明；解释是对事实的当前理解；行动是下一次实验。复杂题目需要维护事实表、假设表和实验记录，避免重复尝试与团队冲突。

2.2　识别线索与优先动作

观察线索	可能含义	优先动作
输出很多却说不出关键证据	采集信息没有围绕问题组织	写出当前需要区分的两个主要原因
同一工具不断更换参数	缺少基线与停止条件	保存确定输入，一次只改一个变量
队员结论冲突	观察条件或环境不同	交换原始请求、样本哈希与完整日志
偶尔成功但无法复现	可能有随机性、竞态、超时或隐藏状态	记录时间、重复次数和状态快照

2.3 标准解题步骤

1. **列出已知事实**。只写能够直接验证的观察，不把猜测混入事实。
2. **提出可检验假设**。每个假设写明支持条件和否定条件。
3. **选择高信息增益实验**。优先执行成本低、能排除多个原因的测试。
4. **建立基线**。保存一个确定输入和输出，后续只改变一个主要变量。
5. **记录结果与置信度**。保存命令、输入、输出、时间与环境，并区分已证明、强支持和待验证。
6. **设置停止条件**。到达时间检查点却没有新事实时，回读题面或切换方向。
7. **脚本化成功路径**。从干净环境重跑，删除无关步骤并检测失败。

2.4 分析时持续追问的问题

- 当前观察是原始事实，还是工具给出的推断?
- 哪个最小输入变化能够区分主要假设?
- 结果能否在相同输入和干净环境中重复?
- 失败时是否保存了完整原始输出?
- 题目条件变化后，当前方法的前提是否仍成立?

2.5 命令、脚本与验证片段

以下内容只用于本地题目目录、回环地址、赛事提供的实例或明确授权的实验环境。每次运行都应保留原始输入和完整输出。

```
cp request.txt notes/baseline-request.txt
cp response.txt notes/baseline-response.txt
diff -u notes/baseline-request.txt request-modified.txt | tee notes/request.
    diff
/usr/bin/time -v timeout 10s python3 scripts/solver.py \
  >captures/solver.stdout 2>captures/solver.stderr
printf 'exit=%s\n' "$?" | tee -a captures/solver.meta
printf '%s\n' '{"hypothesis":"length controls branch","result":"HTTP 403"}' \
  >> notes/observations.jsonl
```

Listing 2.1：本章常用命令与脚本

2.6　证据保存与结果判定

保存原始输入、样本哈希、工具版本、完整命令行、标准输出、标准错误和关键中间结果。网络题保存原始请求与响应，二进制题保存地址、寄存器与内存，数据题保存每层转换前后的长度和哈希。结论必须能够由记录独立验证，不能只依赖终端历史。

2.7　常见失误与修正方法

- 把工具没有报错当成假设成立。
- 一次修改多个字段，结果变化后无法判断因果。
- 只记录成功尝试，队友重复无效方向。
- 根据一次随机成功推断稳定漏洞。
- 拿到 flag 后没有从干净环境复现。

2.8　小型演练

题目场景

相同输入有时返回 OK，有时超时。设计三组实验区分竞态与网络抖动。

第一组固定输入与间隔连续运行，记录连接、首字节和总响应时间。第二组只改变并发数，保持总请求数不变。第三组在本地题目容器重复。若成功率随并发系统性变化且本地也出现同样模式，竞态假设得到支持；若结果只随网络延迟或实例重置变化，应转向传输与服务稳定性。

2.9 完成检查

交付与复盘检查表

- ☐ 入口、输入、目标输出与限制条件已经写入笔记。
- ☐ 建立了可重复的基线输入与输出。
- ☐ 每个关键结论都有原始证据与验证步骤。
- ☐ 脚本能够识别失败，不会把异常输出当成成功。
- ☐ 从干净目录或新容器运行时能够得到同样结果。

第三章　Linux 命令行、文件识别与文本处理

本章目标

掌握 file、xxd、strings、find、grep、awk、sed、sort、uniq、dd 等基础工具，在不破坏原件的前提下完成格式识别、批量搜索、字段提取、边界定位和差异比较。

3.1　原理与题型边界

扩展名只是提示，真实格式由魔数、结构和解析结果确定。长文本管道应分阶段保存中间结果，处理异常文件名时使用空字符分隔。时间线数据在排序前必须保留原始顺序和时区信息。

3.2　识别线索与优先动作

观察线索	可能含义	优先动作
扩展名与 file 结果不一致	文件可能被改名、拼接或多层封装	检查魔数、结束标记和容器目录
strings 出现路径、URL 或 flag 前缀	可能是入口、常量或诱饵	记录偏移并查看上下文字节
大量相似行只有少数字段变化	适合结构化提取和统计	确定分隔符后使用 awk 或 Python
解析器提示尾部额外数据	可能存在拼接或嵌入负载	比较真实结束位置与总长度

3.3 标准解题步骤

1. **盘点目录**。使用 find、stat 和 du 获取类型、大小、权限、时间与层级。
2. **识别真实格式**。使用 file、xxd、hexdump 和格式专用列表命令。
3. **提取可读信息**。运行 ASCII 与宽字符 strings，并保存偏移。
4. **定位边界**。用 xxd、od、dd 或 Python 按偏移读取。
5. **结构化文本**。先统计行数和唯一值，再按明确字段提取。
6. **安全批量处理**。使用 find -print0 与 xargs -0，并记录失败项。
7. **验证输出**。对切分和提取文件重新计算哈希并做格式校验。

3.4 分析时持续追问的问题

- 当前观察是原始事实，还是工具给出的推断?
- 哪个最小输入变化能够区分主要假设?
- 结果能否在相同输入和干净环境中重复?
- 失败时是否保存了完整原始输出?
- 题目条件变化后，当前方法的前提是否仍成立?

3.5 命令、脚本与验证片段

以下内容只用于本地题目目录、回环地址、赛事提供的实例或明确授权的实验环境。每次运行都应保留原始输入和完整输出。

```
find . -xdev -printf '%y\t%s\t%M\t%TY-%Tm-%TdT%TH:%TM:%TS\t%p\n' \
  | tee notes/inventory.tsv
stat --printf='%n\t%s\t%a\t%U:%G\t%y\n' work/* | tee notes/stat.tsv
file work/sample.bin
xxd -g 1 -l 256 work/sample.bin | tee notes/sample-head.hex
tail -c 256 work/sample.bin | xxd -g 1 | tee notes/sample-tail.hex
strings -a -t x -n 5 work/sample.bin | tee notes/sample-strings.txt
find work -type f -print0 | xargs -0 sha256sum | tee notes/all-hashes.txt
```

Listing 3.1: 本章常用命令与脚本

3.6　证据保存与结果判定

保存原始输入、样本哈希、工具版本、完整命令行、标准输出、标准错误和关键中间结果。网络题保存原始请求与响应，二进制题保存地址、寄存器与内存，数据题保存每层转换前后的长度和哈希。结论必须能够由记录独立验证，不能只依赖终端历史。

3.7　常见失误与修正方法

- 相信扩展名，没有检查魔数和结构。
- 找到可疑字符串后直接提交，没有验证其逻辑作用。
- 排序日志后覆盖原文件，丢失原始顺序。
- shell 循环不能正确处理特殊文件名。
- dd 切分时混淆十进制和十六进制偏移。

3.8　小型演练

题目场景

photo.jpg 能正常显示，但工具提示尾部存在 ZIP。提取尾部并证明边界合理。

计算原件哈希并检查头尾。搜索图片结束标记与 ZIP 本地文件头，但不能只依赖第一个匹配。切分后验证前段仍通过图片解析，后段能被 unzip 列表模式读取。记录偏移、命令、两个输出哈希和格式识别结果，以结构一致性而非文件名作为判定依据。

3.9 完成检查

交付与复盘检查表

- ☐ 入口、输入、目标输出与限制条件已经写入笔记。
- ☐ 建立了可重复的基线输入与输出。
- ☐ 每个关键结论都有原始证据与验证步骤。
- ☐ 脚本能够识别失败，不会把异常输出当成成功。
- ☐ 从干净目录或新容器运行时能够得到同样结果。

第四章　字节、进制、编码、Unicode 与端序

本章目标

建立字节优先的分析习惯，区分十六进制文本、Base64、URL 编码、Unicode 编码、大小端整数、补码、压缩与加密。每次转换只处理一层，并验证输出结构。

4.1　原理与题型边界

同一组字节可能被解释为文本、整数、地址、像素或协议字段。可打印并不等于正确。应记录每层输入输出长度、哈希和可逆性，结合魔数、语法、字段范围和 flag 格式判断。

4.2　识别线索与优先动作

观察线索	可能含义	优先动作
仅含十六进制字符且长度为偶数	可能是十六进制文本	解码后检查魔数和可打印率
字符集为字母数字、加号、斜杠和等号	可能是标准 Base64	严格校验填充与输出结构
大量百分号序列或加号	可能是 URL 或表单编码	区分路径编码与表单空格规则
可见字符之间夹零字节	可能是 UTF-16LE 或宽字符	按候选编码解码并保留原始字节
同一字节序列产生两个合理整数	端序需要结合上下文确定	用长度、时间戳或地址范围验证

4.3 标准解题步骤

1. **保存原始字节**。以二进制模式读取，记录长度、哈希与头尾。
2. **统计字符集与长度约束**。检查字符种类、模数、填充和分隔符。
3. **一次只解一层**。记录操作、参数和输出长度。
4. **验证语义**。检查魔数、语法、校验和和字段范围。
5. **明确 Unicode 层次**。区分码点、编码字节、规范化与显示字形。
6. **明确整数表示**。写清位宽、端序、符号和溢出。
7. **构建可逆脚本**。重新编码或加入断言，验证中间层。

4.4 分析时持续追问的问题

- 当前观察是原始事实，还是工具给出的推断?
- 哪个最小输入变化能够区分主要假设?
- 结果能否在相同输入和干净环境中重复?
- 失败时是否保存了完整原始输出?
- 题目条件变化后，当前方法的前提是否仍成立?

4.5 常用关系式

若字节为 $b_0, \ldots, b_{n-1}$，小端整数为 $N_{LE} = \sum_{i=0}^{n-1} b_i 256^i$，大端整数为 $N_{BE} = \sum_{i=0}^{n-1} b_i 256^{n-1-i}$。对于 w 位补码，若无符号值 $u \geq 2^{w-1}$，对应有符号值为 $u - 2^w$。

4.6 命令、脚本与验证片段

以下内容只用于本地题目目录、回环地址、赛事提供的实例或明确授权的实验环境。每次运行都应保留原始输入和完整输出。

```
from base64 import b64decode
from pathlib import Path
from urllib.parse import unquote_to_bytes
```

```
import hashlib, struct
raw = Path('work/input.bin').read_bytes()
print(len(raw), hashlib.sha256(raw).hexdigest(), raw[:32].hex())
hex_text=b'504b0304'
decoded=bytes.fromhex(hex_text.decode('ascii'))
assert decoded.hex().encode('ascii') == hex_text.lower()
sample=bytes.fromhex('78563412')
print(int.from_bytes(sample,'little'), int.from_bytes(sample,'big'))
print(struct.unpack('<I',sample)[0])
print(unquote_to_bytes('flag%7Bbyte%5Ffirst%7D'))
print(b64decode(b'ZmxhZ3tiYXNlNjRfaXNfZW5jb2Rpbmd9',validate=True))
```

Listing 4.1: 本章常用命令与脚本

4.7　证据保存与结果判定

保存原始输入、样本哈希、工具版本、完整命令行、标准输出、标准错误和关键中间结果。网络题保存原始请求与响应，二进制题保存地址、寄存器与内存，数据题保存每层转换前后的长度和哈希。结论必须能够由记录独立验证，不能只依赖终端历史。

4.8　常见失误与修正方法

- 把可打印输出当成正确结果。
- 混淆十六进制文本与字节数组。
- 随意为 Base64 补填充。
- 使用默认文本编码读取二进制。
- 忽略位宽和溢出规则。

4.9 小型演练

题目场景

文本 343836353663366336663030 解码一次仍不可读。给出逐层处理和验证条件。

第一层满足十六进制字符与偶数长度约束，先转为字节，得到另一串 ASCII 十六进制。验证重新编码能恢复第一层，再进行第二次十六进制解码。最后检查 flag 格式。任何一层若输出长度或字符集不合理，都应回到上一层，不继续盲目解码。

4.10 完成检查

交付与复盘检查表

- □ 入口、输入、目标输出与限制条件已经写入笔记。
- □ 建立了可重复的基线输入与输出。
- □ 每个关键结论都有原始证据与验证步骤。
- □ 脚本能够识别失败，不会把异常输出当成成功。
- □ 从干净目录或新容器运行时能够得到同样结果。

第五章　Python 解题脚本：输入输出、断言与确定性

本章目标

把重复操作、字节转换、候选枚举和网络交互写成能够检测失败、保存证据并在干净环境重复运行的 Python 脚本。

5.1　原理与题型边界

实用脚本应把输入输出、解析、求解和验证分离。日志写到标准错误，最终 flag 或机器可读结果写到标准输出。网络读取按协议边界而非固定 sleep，枚举前先估算搜索空间并设计剪枝。

5.2　识别线索与优先动作

观察线索	可能含义	优先动作
相同操作需要处理多个候选	适合循环与结构化输出	先写处理单个输入的纯函数
手工转换十六进制、异或或端序	容易发生一字节错误	使用 bytes、int.from_bytes 与断言
远程服务有菜单或多轮交互	手工操作不稳定	封装收发函数并保存原始字节
脚本偶尔卡住	缺少超时或协议边界	按分隔符或长度读取并设置总超时

5.3　标准解题步骤

1. **定义输入输出契约**。明确参数、成功输出、失败退出码与依赖。

2. **分离解析与求解**。解析函数只生成结构，求解函数只处理明确类型。
3. **加入边界检查**。验证长度、魔数、字段范围、候选数量和 flag 格式。
4. **保存原始数据**。将响应和中间字节写入 captures。
5. **保证确定性**。固定随机种子和排序，不依赖当前目录。
6. **处理异常与超时**。区分连接、协议、解析和候选为空。
7. **从干净环境重跑**。一条命令从原件得到结果。

5.4　分析时持续追问的问题

- 当前观察是原始事实，还是工具给出的推断?
- 哪个最小输入变化能够区分主要假设?
- 结果能否在相同输入和干净环境中重复?
- 失败时是否保存了完整原始输出?
- 题目条件变化后，当前方法的前提是否仍成立?

5.5　命令、脚本与验证片段

以下内容只用于本地题目目录、回环地址、赛事提供的实例或明确授权的实验环境。每次运行都应保留原始输入和完整输出。

```
from __future__ import annotations
import argparse, hashlib, logging
from pathlib import Path
LOG=logging.getLogger('solver')
def load_input(path:Path)->bytes:
    data=path.read_bytes()
    if not data: raise ValueError(f'empty input: {path}')
    LOG.info('bytes=%d sha256=%s',len(data),hashlib.sha256(data).hexdigest())
    return data
def solve(data:bytes)->bytes:
    text=data.strip()
    if len(text)%2: raise ValueError(f'odd hex length: {len(text)}')
    return bytes(b^0x23 for b in bytes.fromhex(text.decode('ascii')))
def validate(x:bytes):
    if not (x.startswith(b'flag{') and x.endswith(b'}')):
```

```
        raise ValueError(f'unexpected result: {x[:64]!r}')
def main()->int:
    p=argparse.ArgumentParser(); p.add_argument('input',type=Path); a=p.parse_args()
    logging.basicConfig(level=logging.INFO)
    out=solve(load_input(a.input)); validate(out); print(out.decode()); return 0
if __name__=='__main__': raise SystemExit(main())
```

Listing 5.1：本章常用命令与脚本

5.6　证据保存与结果判定

保存原始输入、样本哈希、工具版本、完整命令行、标准输出、标准错误和关键中间结果。网络题保存原始请求与响应，二进制题保存地址、寄存器与内存，数据题保存每层转换前后的长度和哈希。结论必须能够由记录独立验证，不能只依赖终端历史。

5.7　常见失误与修正方法

- 使用字符串处理任意二进制数据。
- 依赖 sleep 猜响应时间。
- 把主机、端口和路径硬编码。
- 捕获所有异常后忽略。
- 把调试信息与最终结果混在标准输出。

5.8　小型演练

题目场景

把“读十六进制、异或 0x23、检查 flag”的手工流程改成一条命令可运行的脚本。

使用 pathlib 以字节读取，检查空文件、合法十六进制和偶数长度。解码后逐字节异或，验证 flag 前后缀。argparse 接受输入路径，日志写入标准错误，成功

结果单独写入标准输出。用合法输入、空文件、奇数长度和非法字符分别测试退出行为。

5.9 完成检查

交付与复盘检查表

- ☐ 入口、输入、目标输出与限制条件已经写入笔记。
- ☐ 建立了可重复的基线输入与输出。
- ☐ 每个关键结论都有原始证据与验证步骤。
- ☐ 脚本能够识别失败，不会把异常输出当成成功。
- ☐ 从干净目录或新容器运行时能够得到同样结果。

第六章　网络基础：TCP、UDP、DNS、TLS 与 HTTP 边界

本章目标

理解网络题中的分层与消息边界。TCP 是有序字节流，UDP 保留数据报，DNS 有多种记录类型，TLS 位于应用协议之前，HTTP 又有请求行、头部、消息体和连接复用。

6.1　原理与题型边界

recv 返回当前可用字节，不保证是一条完整消息。协议必须通过换行、长度字段、固定头、分隔符或连接关闭定义边界。TLS 握手成功只说明通道建立，不说明应用请求正确；DNS 结果受记录类型、解析器、缓存与 TTL 影响。

6.2　识别线索与优先动作

观察线索	可能含义	优先动作
一次 recv 得到半条或多条消息	TCP 不保留应用消息边界	按长度、换行或固定标记循环读取
UDP 端口存在但无响应	数据报格式、大小或校验不满足	抓包并逐字段比较
域名有 TXT 或多级 CNAME	信息可能编码在记录与查询名中	使用 dig 指定类型保存完整回答
TLS 握手失败	可能是 SNI、版本、证书或 ALPN	使用 openssl s_client 查看细节

6.3 标准解题步骤

1. **确定传输与应用层**。记录 TCP 或 UDP、端口、是否 TLS 和首批字节。
2. **保存原始流量**。用 tcpdump 或 Wireshark 抓取本地测试。
3. **识别消息边界**。检查换行、长度字段、固定头和关闭语义。
4. **建立手工基线**。用 nc、socat、curl、openssl 或 dig 完成最小交互。
5. **编写状态机客户端**。按协议状态读写，不用固定 sleep。
6. **比较本地与远程**。检查 DNS、IPv4/IPv6、SNI、换行和超时。
7. **限制重试与速率**。遵守赛事规则并使用有上限退避。

6.4 分析时持续追问的问题

- 当前观察是原始事实，还是工具给出的推断?
- 哪个最小输入变化能够区分主要假设?
- 结果能否在相同输入和干净环境中重复?
- 失败时是否保存了完整原始输出?
- 题目条件变化后，当前方法的前提是否仍成立?

6.5 命令、脚本与验证片段

以下内容只用于本地题目目录、回环地址、赛事提供的实例或明确授权的实验环境。每次运行都应保留原始输入和完整输出。

```
nc -nv 127.0.0.1 31337 | tee captures/nc-session.txt
socat -v - TCP:127.0.0.1:31337,connect-timeout=3 2>captures/socat.trace
openssl s_client -connect 127.0.0.1:8443 -servername challenge.local \
  -brief < /dev/null 2>&1 | tee captures/tls.txt
dig challenge.local TXT +noall +answer | tee captures/dns-txt.txt
sudo tcpdump -i lo -nn -s0 -w captures/challenge.pcap 'port 31337'
curl --http1.1 --include --silent http://127.0.0.1:8000/ \
  -o captures/http-response.txt
```

Listing 6.1: 本章常用命令与脚本

6.6　证据保存与结果判定

保存原始输入、样本哈希、工具版本、完整命令行、标准输出、标准错误和关键中间结果。网络题保存原始请求与响应，二进制题保存地址、寄存器与内存，数据题保存每层转换前后的长度和哈希。结论必须能够由记录独立验证，不能只依赖终端历史。

6.7　常见失误与修正方法

- 把一次 recv 当成完整消息。
- 只根据端口号猜协议。
- 调试 HTTPS 时忽略 SNI。
- 抓包过滤器过窄导致丢失握手或重传。
- 无限重试远程实例。

6.8　小型演练

题目场景

TCP 服务先发四字节小端长度，再发 JSON。简单脚本 recv(4) 后 recv(length)，有时失败。

编写 read_exactly 循环读取，直到累计字节满足要求或遇到 EOF。先精确读取四字节并按小端解析，检查长度上限，再精确读取消息体并解析 JSON。日志保存每次分块长度与原始十六进制，能够区分协议错误和连接中断。

6.9 完成检查

交付与复盘检查表

- ☐ 入口、输入、目标输出与限制条件已经写入笔记。
- ☐ 建立了可重复的基线输入与输出。
- ☐ 每个关键结论都有原始证据与验证步骤。
- ☐ 脚本能够识别失败，不会把异常输出当成成功。
- ☐ 从干净目录或新容器运行时能够得到同样结果。

第二部分

Web 题型

第七章　HTTP 请求重构、基线与攻击面盘点

本章目标

Web 题的第一步是把浏览器或题目客户端发出的请求完整重构出来。明确方法、路径、查询参数、头部、Cookie、消息体、重定向和响应差异后，后续测试才有可靠基线。

7.1　原理与题型边界

HTTP 请求会经过浏览器、代理、Web 服务器、框架和应用。每一层可能做 URL 解码、路径归一化、头部合并、参数重复处理和内容类型解析。攻击面不仅是页面表单，还包括 API、静态资源、JavaScript、源映射、隐藏路由和异常方法。

7.2　识别线索与优先动作

观察线索	可能含义	优先动作
浏览器成功而 curl 失败	缺少 Cookie、Origin、Referer、CSRF token 或协议细节	从开发者工具复制请求并逐项删减
页面加载后继续请求多个接口	主要逻辑位于 API 或前端脚本	导出 HAR 并保存所有 XHR/fetch
状态码相同但内容不同	差异可能在字段、长度、头部或模板分支	规范化动态字段后做 diff
存在多次 30x 跳转	认证或路径规范化改变入口	禁止自动跳转并逐跳记录 Location

7.3　标准解题步骤

1. **保存原始请求基线**。记录方法、路径、查询、头部、Cookie、消息体、重定向、状态码、长度和稳定响应片段。
2. **定位输入到危险点的数据流**。列出路径段、查询参数、表单字段、JSON、Cookie、头部和文件名等输入位置。
3. **一次只改变一个条件**。从最小请求开始，每次只增加或修改一个字段。
4. **建立正反对照**。使用相同会话发送两次完全相同请求，确认基线稳定。
5. **确认影响与边界**。把每个入口标注认证状态、角色、数据类型、服务端动作和返回特征。
6. **检查编码与多层解析**。分别观察浏览器、代理、Web 服务器、框架和应用对输入的解码、归一化与类型转换。
7. **脚本化并从新会话复现**。把登录、token 获取、请求发送、结果判定和失败处理写入脚本，使用新会话重复。

7.4　分析时持续追问的问题

- 当前观察是原始事实，还是工具给出的推断?
- 哪个最小输入变化能够区分主要假设?
- 结果能否在相同输入和干净环境中重复?
- 失败时是否保存了完整原始输出?
- 题目条件变化后，当前方法的前提是否仍成立?

7.5　命令、脚本与验证片段

以下内容只用于本地题目目录、回环地址、赛事提供的实例或明确授权的实验环境。每次运行都应保留原始输入和完整输出。

```
curl --http1.1 --silent --show-error --include --max-redirs 0 \
  'http://127.0.0.1:8000/profile?id=1' \
  -H 'Host: challenge.local' \
```

```
  -H 'Cookie: session=LOCAL_TEST_TOKEN' \
  -o captures/profile-id-1.txt
curl --silent --cookie-jar captures/cookies.txt --cookie captures/cookies.txt \
  -D captures/headers.txt \
  -w 'code=%{http_code} bytes=%{size_download} time=%{time_total}\n' \
  'http://127.0.0.1:8000/api/items?limit=10' -o captures/body.json
```

Listing 7.1: 本章常用命令与脚本

7.6 证据保存与结果判定

保存未经编辑的原始请求与响应、会话状态、角色、时间、重定向链、响应长度和关键差异。若使用浏览器，还要保存开发者工具网络记录与相关客户端代码。最终证明应包含一组可重复的正反对照，说明变化来自被测试字段，而不是缓存、随机数或登录状态。

7.7 常见失误与修正方法

- 只看页面显示，没有保存底层接口。
- 让代理自动修正请求，丢失原始字节。
- 一次修改多个字段。
- 把动态时间戳造成的差异误认为漏洞。
- 自动跟随重定向后不知道在哪一步改变状态。

7.8 小型演练

题目场景

浏览器能访问 /admin-preview，直接 curl 返回 403。重构一个最小可重复请求。

从网络面板复制请求，保存 Cookie、Host、Origin、Referer 和重定向链。逐项删除非必要浏览器头，确认哪个字段决定行为。若 CSRF token 来自前一页，脚本先获取页面并解析 token，再发送目标请求。最后用全新 Cookie 运行，证明流程不依赖旧会话。

7.9　完成检查

交付与复盘检查表

- ☐ 入口、输入、目标输出与限制条件已经写入笔记。
- ☐ 建立了可重复的基线输入与输出。
- ☐ 每个关键结论都有原始证据与验证步骤。
- ☐ 脚本能够识别失败，不会把异常输出当成成功。
- ☐ 从干净目录或新容器运行时能够得到同样结果。

第八章　源代码审查、静态资源与信息泄露

本章目标

很多 Web 题并不要求先发现复杂漏洞，入口可能直接存在于客户端代码、备份文件、调试响应、配置、错误栈、源映射或版本控制残留中。

8.1　原理与题型边界

客户端代码对用户可见，不能承担保密职责。服务端若暴露源码、配置、日志或错误栈，可能泄露路由、参数名、密钥位置和内部路径。信息本身未必是 flag，但常用于构建后续请求。

8.2　识别线索与优先动作

观察线索	可能含义	优先动作
JavaScript 中出现未在界面显示的路径	可能存在隐藏 API 或管理功能	搜索 fetch、axios、route 和 URL 字符串
存在.map 文件	源映射可能还原原始源码与文件名	下载并检查 sourcesContent
错误页包含堆栈和绝对路径	调试模式或异常处理泄露内部结构	触发最小错误并保存完整响应
目录出现.bak、.old、~ 或压缩包	部署残留可能包含源码和配置	在题目范围内验证明确候选，不做无边界扫描

8.3　标准解题步骤

1. **保存原始请求基线**。记录方法、路径、查询、头部、Cookie、消息体、重定向、状态码、长度和稳定响应片段。
2. **定位输入到危险点的数据流**。建立静态资源清单，并按 MIME、大小和引用关系排序。
3. **一次只改变一个条件**。搜索路由、参数、环境变量名、默认凭据占位符和调试开关。
4. **建立正反对照**。对备份候选建立明确来源，不使用大规模目录字典攻击远程实例。
5. **确认影响与边界**。把泄露信息与实际服务行为交叉验证，避免相信过期源码。
6. **检查编码与多层解析**。分别观察浏览器、代理、Web 服务器、框架和应用对输入的解码、归一化与类型转换。
7. **脚本化并从新会话复现**。把登录、token 获取、请求发送、结果判定和失败处理写入脚本，使用新会话重复。

8.4　分析时持续追问的问题

- 当前观察是原始事实，还是工具给出的推断?
- 哪个最小输入变化能够区分主要假设?
- 结果能否在相同输入和干净环境中重复?
- 失败时是否保存了完整原始输出?
- 题目条件变化后，当前方法的前提是否仍成立?

8.5　命令、脚本与验证片段

以下内容只用于本地题目目录、回环地址、赛事提供的实例或明确授权的实验环境。每次运行都应保留原始输入和完整输出。

```
find work/site -type f -maxdepth 4 -printf '%s\t%p\n' | sort -nr \
  | tee notes/static-files.tsv
rg -n --hidden -S 'fetch\(|axios|/api/|admin|debug|secret|token|
    sourceMappingURL' \
  work/site | tee notes/source-hits.txt
curl -sS 'http://127.0.0.1:8000/static/app.js.map' \
  -o work/app.js.map
python3 -m json.tool work/app.js.map > notes/app-map.pretty.json
```

Listing 8.1: 本章常用命令与脚本

8.6 证据保存与结果判定

保存未经编辑的原始请求与响应、会话状态、角色、时间、重定向链、响应长度和关键差异。若使用浏览器，还要保存开发者工具网络记录与相关客户端代码。最终证明应包含一组可重复的正反对照，说明变化来自被测试字段，而不是缓存、随机数或登录状态。

8.7 常见失误与修正方法

- 把客户端硬编码值直接当作有效服务器密钥。
- 只搜索 flag 字样，忽略路由和参数。
- 下载源码后不比较部署版本。
- 触发破坏性错误以获取堆栈。
- 对远程实例进行无边界目录爆破。

8.8 小型演练

题目场景

页面只显示一个按钮，app.js 引用源映射。请从静态资源确定隐藏接口和所需参数。

保存 app.js 与 map 文件，检查 sources、names 和 sourcesContent。搜索 fetch 调用、路由常量和请求体结构，再用 curl 向本地实例发送最小请求。若接

口行为与源码不一致，记录响应并检查版本标识或后端验证，不把源映射中的旧逻辑当作事实。

8.9　完成检查

交付与复盘检查表

- ☐ 入口、输入、目标输出与限制条件已经写入笔记。
- ☐ 建立了可重复的基线输入与输出。
- ☐ 每个关键结论都有原始证据与验证步骤。
- ☐ 脚本能够识别失败，不会把异常输出当成成功。
- ☐ 从干净目录或新容器运行时能够得到同样结果。

第九章　认证、会话、访问控制与 IDOR

本章目标

区分身份认证、会话管理和授权。认证回答“你是谁”，会话维持状态，授权决定某个身份能否访问对象或执行动作。CTF 常把前端隐藏、可预测对象标识和角色检查缺失组合在一起。

9.1　原理与题型边界

安全判断必须在服务端对每个对象和动作执行。仅隐藏按钮、依赖客户端角色字段、检查是否登录但不检查对象归属，都可能导致水平或垂直越权。会话 Cookie 的签名、过期、轮换和固定同样需要观察。

9.2　识别线索与优先动作

观察线索	可能含义	优先动作
修改对象 id 后返回另一名用户数据	可能存在水平 IDOR	用两个自有测试账户建立所有权对照
前端隐藏管理员按钮但接口可直接调用	服务端可能缺少角色检查	重放相同接口并比较角色
登录前后 Cookie 不变化	可能有会话固定或前置状态被复用	记录登录前后 Set-Cookie
登出后旧请求仍成功	会话未失效或缓存掩盖状态	使用新连接与禁缓存头复测

9.3　标准解题步骤

1. **保存原始请求基线**。记录方法、路径、查询、头部、Cookie、消息体、重定向、状态码、长度和稳定响应片段。
2. **定位输入到危险点的数据流**。建立匿名、普通用户、另一普通用户和管理员题目角色的权限矩阵。
3. **一次只改变一个条件**。只修改对象标识，保持会话和其他字段不变。
4. **建立正反对照**。分别测试读取、修改、删除和敏感动作，因为授权可能只覆盖部分方法。
5. **确认影响与边界**。确认成功响应真正改变服务端状态，而不只是前端显示。
6. **检查编码与多层解析**。分别观察浏览器、代理、Web 服务器、框架和应用对输入的解码、归一化与类型转换。
7. **脚本化并从新会话复现**。把登录、token 获取、请求发送、结果判定和失败处理写入脚本，使用新会话重复。

9.4　分析时持续追问的问题

- 当前观察是原始事实，还是工具给出的推断?
- 哪个最小输入变化能够区分主要假设?
- 结果能否在相同输入和干净环境中重复?
- 失败时是否保存了完整原始输出?
- 题目条件变化后，当前方法的前提是否仍成立?

9.5　命令、脚本与验证片段

以下内容只用于本地题目目录、回环地址、赛事提供的实例或明确授权的实验环境。每次运行都应保留原始输入和完整输出。

```
# Compare the same object request under two local challenge accounts.
curl -sS -b captures/alice.cookies \
  'http://127.0.0.1:8000/api/profile/1001' -o captures/alice-own.json
```

```
curl -sS -b captures/alice.cookies \
  'http://127.0.0.1:8000/api/profile/1002' -o captures/alice-other.json
diff -u captures/alice-own.json captures/alice-other.json
# Record login cookie rotation.
curl -sS -c captures/pre.cookies http://127.0.0.1:8000/login -o /dev/null
curl -sS -b captures/pre.cookies -c captures/post.cookies \
  -d 'username=alice&password=challenge' http://127.0.0.1:8000/login -o /dev/
    null
diff -u captures/pre.cookies captures/post.cookies
```

Listing 9.1: 本章常用命令与脚本

9.6 证据保存与结果判定

保存未经编辑的原始请求与响应、会话状态、角色、时间、重定向链、响应长度和关键差异。若使用浏览器，还要保存开发者工具网络记录与相关客户端代码。最终证明应包含一组可重复的正反对照，说明变化来自被测试字段，而不是缓存、随机数或登录状态。

9.7 常见失误与修正方法

- 只修改前端 HTML 或 JavaScript，没有验证后端接口。
- 使用真实他人账户而不是题目提供的测试身份。
- 看到 200 就认为越权成功，没有检查返回对象。
- 混淆认证失败与对象不存在。
- 复用旧 Cookie，未验证会话轮换和失效。

9.8 小型演练

题目场景

题目提供两个普通账户。账户 A 请求 /api/note/41，修改为 42 后返回内容。证明是否为 IDOR。

分别用 A 和 B 创建自有笔记并记录 ID。以 A 会话访问 A 的笔记、B 的笔记和不存在的 id，比较状态码、字段和内容。再以 B 会话做反向测试。若对象归属变

化是唯一变量且服务端返回对方私有内容，才能证明水平越权。保存权限矩阵与完整响应。

9.9　完成检查

交付与复盘检查表

- □ 入口、输入、目标输出与限制条件已经写入笔记。
- □ 建立了可重复的基线输入与输出。
- □ 每个关键结论都有原始证据与验证步骤。
- □ 脚本能够识别失败，不会把异常输出当成成功。
- □ 从干净目录或新容器运行时能够得到同样结果。

第十章　SQL 注入：错误、布尔、联合与时间通道

本章目标

SQL 注入题要求判断用户输入是否改变了查询结构。常见通道包括错误信息、布尔差异、联合查询、排序与时间差异。目标是先证明结构可控，再根据数据库语法和回显能力选择最小提取方式。

10.1　原理与题型边界

注入产生于字符串拼接、动态标识符、排序字段或不完整参数化。参数化查询通常保护值，但表名、列名和排序方向仍需白名单。时间通道必须使用重复采样与对照，不能把一次慢响应视为证据。

10.2　识别线索与优先动作

观察线索	可能含义	优先动作
单引号触发数据库语法错误	输入进入字符串上下文或错误处理泄露	用成对引号与转义建立对照
真条件与假条件响应稳定不同	存在布尔通道	固定会话并比较长度和关键字段
ORDER BY 序号变化触发错误	可能控制排序或列数	逐步确定边界并停止无意义枚举
特定表达式使响应延迟	可能存在时间通道	重复多次比较中位数和分布

10.3　标准解题步骤

1. **保存原始请求基线**。记录方法、路径、查询、头部、Cookie、消息体、重定向、状态码、长度和稳定响应片段。
2. **定位输入到危险点的数据流**。确定输入位于字符串、数字、LIKE、ORDER BY 还是标识符上下文。
3. **一次只改变一个条件**。构造语法闭合的真条件与假条件，确保响应差异稳定。
4. **建立正反对照**。根据错误、回显和数据库特征选择联合、错误或盲注，不同时混用。
5. **确认影响与边界**。只提取完成题目所需的最小数据，并验证字符集与长度。
6. **检查编码与多层解析**。分别观察浏览器、代理、Web 服务器、框架和应用对输入的解码、归一化与类型转换。
7. **脚本化并从新会话复现**。把登录、token 获取、请求发送、结果判定和失败处理写入脚本，使用新会话重复。

10.4　分析时持续追问的问题

- 当前观察是原始事实，还是工具给出的推断?
- 哪个最小输入变化能够区分主要假设?
- 结果能否在相同输入和干净环境中重复?
- 失败时是否保存了完整原始输出?
- 题目条件变化后，当前方法的前提是否仍成立?

10.5　命令、脚本与验证片段

以下内容只用于本地题目目录、回环地址、赛事提供的实例或明确授权的实验环境。每次运行都应保留原始输入和完整输出。

```
# Local challenge baselines. Quoting prevents the shell from altering payloads.
curl -sS 'http://127.0.0.1:8000/item?id=1' -o captures/sql-base.html
curl -sS --get --data-urlencode "id=1'" \
```

```
  'http://127.0.0.1:8000/item' -o captures/sql-quote.html
curl -sS --get --data-urlencode "id=1 AND 1=1" \
  'http://127.0.0.1:8000/item' -o captures/sql-true.html
curl -sS --get --data-urlencode "id=1 AND 1=2" \
  'http://127.0.0.1:8000/item' -o captures/sql-false.html
wc -c captures/sql-*.html
```

Listing 10.1: 本章常用命令与脚本

10.6 证据保存与结果判定

保存未经编辑的原始请求与响应、会话状态、角色、时间、重定向链、响应长度和关键差异。若使用浏览器，还要保存开发者工具网络记录与相关客户端代码。最终证明应包含一组可重复的正反对照，说明变化来自被测试字段，而不是缓存、随机数或登录状态。

10.7 常见失误与修正方法

- 一上来使用长 payload，无法判断哪部分生效。
- 忽略字符串与数字上下文。
- 把 WAF 拒绝页面当成数据库响应。
- 时间型测试只运行一次。
- 提取超过题目需要的数据。

10.8 小型演练

题目场景

本地搜索接口对 id=1 返回商品，id=1 AND 1=1 与 id=1 AND 1=2 长度稳定不同。下一步如何确认并提取一位信息。

先重复两组对照并规范化动态字段，确认差异不是缓存。确定数字上下文后，用一个已知可验证条件测试数据库表达式。随后把目标问题缩小为单个布尔判断，例如某字段长度是否大于给定值，使用二分减少请求。每一步保存条件、响应特征和判断，不依赖单次肉眼观察。

10.9　完成检查

交付与复盘检查表

- ☐ 入口、输入、目标输出与限制条件已经写入笔记。
- ☐ 建立了可重复的基线输入与输出。
- ☐ 每个关键结论都有原始证据与验证步骤。
- ☐ 脚本能够识别失败，不会把异常输出当成成功。
- ☐ 从干净目录或新容器运行时能够得到同样结果。

第十一章　NoSQL、LDAP 与 XPath 注入

本章目标

非关系型查询同样可能因类型、操作符或表达式拼接而被改变。MongoDB 风格 JSON、LDAP 过滤器和 XPath 条件的语法不同，但共同方法是识别输入进入何种查询语言，并建立合法、真、假与错误对照。

11.1　原理与题型边界

JSON 参数可能从字符串变成对象或数组，使操作符进入查询。LDAP 过滤器和 XPath 依赖括号、通配符、引号和逻辑运算。框架对重复参数和嵌套键的解析方式经常决定是否能够改变类型。

11.2　识别线索与优先动作

观察线索	可能含义	优先动作
JSON 字符串改为对象后响应变化	后端可能直接使用用户对象构建查询	比较类型错误与操作符行为
参数名带方括号后被解析为嵌套对象	框架支持深层参数解析	保存实际服务端类型或错误
星号改变 LDAP 搜索结果	通配符进入过滤表达式	使用精确值与不存在值对照
单引号或方括号影响 XML 查询	可能进入 XPath 表达式	构造语法正确的真与假条件

11.3　标准解题步骤

1. **保存原始请求基线**。记录方法、路径、查询、头部、Cookie、消息体、重定向、状态码、长度和稳定响应片段。
2. **定位输入到危险点的数据流**。确认 Content-Type 和框架如何把请求转换为数据类型。
3. **一次只改变一个条件**。从单个字段的类型变化开始，不混入多个操作符。
4. **建立正反对照**。分别建立匹配一个、匹配零个和语法错误三个响应。
5. **确认影响与边界**。用服务端错误或本地源码确认实际查询语言。
6. **检查编码与多层解析**。分别观察浏览器、代理、Web 服务器、框架和应用对输入的解码、归一化与类型转换。
7. **脚本化并从新会话复现**。把登录、token 获取、请求发送、结果判定和失败处理写入脚本，使用新会话重复。

11.4　分析时持续追问的问题

- 当前观察是原始事实，还是工具给出的推断?
- 哪个最小输入变化能够区分主要假设?
- 结果能否在相同输入和干净环境中重复?
- 失败时是否保存了完整原始输出?
- 题目条件变化后，当前方法的前提是否仍成立?

11.5　命令、脚本与验证片段

以下内容只用于本地题目目录、回环地址、赛事提供的实例或明确授权的实验环境。每次运行都应保留原始输入和完整输出。

```
# JSON type comparison against a local toy API.
curl -sS -H 'Content-Type: application/json' \
  -d '{"username":"alice","password":"wrong"}' \
  http://127.0.0.1:8000/api/login -o captures/nosql-string.json
```

```
curl -sS -H 'Content-Type: application/json' \
  -d '{"username":{"$eq":"alice"},"password":"wrong"}' \
  http://127.0.0.1:8000/api/login -o captures/nosql-object.json
# URL-encoded nested parameter parsing.
curl -sS --data-urlencode 'username[$eq]=alice' \
  --data-urlencode 'password=wrong' http://127.0.0.1:8000/login
```

Listing 11.1: 本章常用命令与脚本

11.6 证据保存与结果判定

保存未经编辑的原始请求与响应、会话状态、角色、时间、重定向链、响应长度和关键差异。若使用浏览器，还要保存开发者工具网络记录与相关客户端代码。最终证明应包含一组可重复的正反对照，说明变化来自被测试字段，而不是缓存、随机数或登录状态。

11.7 常见失误与修正方法

- 把 SQL payload 原样套用到不同查询语言。
- 忽略 JSON 类型变化，只关注字符串内容。
- 不能区分语法错误与零结果。
- 未确认框架是否解析嵌套参数。
- 从真实目录服务提取不必要信息。

11.8 小型演练

题目场景

登录 API 接受 JSON。username 为字符串时失败，改成对象后返回不同错误。判断是否存在操作符注入。

先用空对象、数组、数字和未知键测试类型验证边界，记录不同错误。再在本地源码或错误信息中确认后端查询构造。构造一个不会绕过认证但能明确证明操作符被解释的正反对照，例如等于存在用户名与不存在用户名。只有响应差异稳定且能够由查询语义解释时，才继续完成题目所需验证。

11.9　完成检查

交付与复盘检查表

- ☐ 入口、输入、目标输出与限制条件已经写入笔记。
- ☐ 建立了可重复的基线输入与输出。
- ☐ 每个关键结论都有原始证据与验证步骤。
- ☐ 脚本能够识别失败，不会把异常输出当成成功。
- ☐ 从干净目录或新容器运行时能够得到同样结果。

第十二章　XSS 与 DOM 数据流

本章目标

跨站脚本题需要区分反射型、存储型和 DOM 型，并分析输入进入 HTML、属性、JavaScript、URL 还是 CSS 上下文。目标是证明浏览器执行了题目控制的数据，而不是仅在响应文本中出现。

12.1　原理与题型边界

浏览器解析器会进行实体解码、DOM 修复和上下文切换。服务端转义只对对应上下文有效。DOM 型问题由 source 到 sink 的客户端数据流产生，例如 location、postMessage 或存储数据进入 innerHTML、document.write、eval 或事件属性。

12.2　识别线索与优先动作

观察线索	可能含义	优先动作
输入原样出现在 HTML 文本节点	可能存在反射但未必可执行	查看实际 DOM 与响应源码
输入进入引号包围的属性	需要判断能否闭合属性或触发 URL 语义	使用无害事件记录作为本地验证
响应不含输入但页面执行后出现	可能是 DOM 型	跟踪 location、storage、postMessage 到 sink
内容保存后由另一页面展示	可能是存储型	用两个本地测试会话验证持久化边界

12.3　标准解题步骤

1. **保存原始请求基线**。记录方法、路径、查询、头部、Cookie、消息体、重定向、状态码、长度和稳定响应片段。
2. **定位输入到危险点的数据流**。确定 source、转换步骤、sink 和最终浏览器上下文。
3. **一次只改变一个条件**。先使用纯文本标记证明反射位置，再测试最小无害执行标记。
4. **建立正反对照**。同时检查响应源码和浏览器构建后的 DOM。
5. **确认影响与边界**。记录 CSP、sandbox、Trusted Types 和浏览器控制台错误对执行链的影响。
6. **检查编码与多层解析**。分别观察浏览器、代理、Web 服务器、框架和应用对输入的解码、归一化与类型转换。
7. **脚本化并从新会话复现**。把登录、token 获取、请求发送、结果判定和失败处理写入脚本，使用新会话重复。

12.4　分析时持续追问的问题

- 当前观察是原始事实，还是工具给出的推断?
- 哪个最小输入变化能够区分主要假设?
- 结果能否在相同输入和干净环境中重复?
- 失败时是否保存了完整原始输出?
- 题目条件变化后，当前方法的前提是否仍成立?

12.5　命令、脚本与验证片段

以下内容只用于本地题目目录、回环地址、赛事提供的实例或明确授权的实验环境。每次运行都应保留原始输入和完整输出。

```
# Harmless local payloads only demonstrate execution in the challenge page.
curl -sS --get --data-urlencode 'q=CTF_MARKER_123' \
  http://127.0.0.1:8000/search -o captures/xss-marker.html
rg -n 'CTF_MARKER_123|innerHTML|document.write|location|postMessage|eval\(' \
  work/site captures/xss-marker.html
# In a local browser console, inspect the final DOM rather than only response
    source.
# document.querySelector('main').innerHTML
```

Listing 12.1: 本章常用命令与脚本

12.6 证据保存与结果判定

保存未经编辑的原始请求与响应、会话状态、角色、时间、重定向链、响应长度和关键差异。若使用浏览器，还要保存开发者工具网络记录与相关客户端代码。最终证明应包含一组可重复的正反对照，说明变化来自被测试字段，而不是缓存、随机数或登录状态。

12.7 常见失误与修正方法

- 看到输入被反射就断言 XSS。
- 忽略 HTML、属性和脚本上下文差异。
- 只看响应源码，不看最终 DOM。
- 使用外部回连或数据窃取代码，越出题目范围。
- 忽略 CSP 控制台错误与 sandbox。

12.8 小型演练

题目场景

搜索词出现在页面源码的 data-name 属性，页面脚本又把该属性写入 innerHTML。如何证明数据流。

先用唯一纯文本标记定位服务端输出和属性边界。查看客户端脚本如何读取 data-name，并在断点处记录字符串变化。使用只在本地控制台输出固定标记的

最小事件验证执行，不发送网络请求。保存响应源码、最终 DOM、调用栈与 CSP 信息，说明漏洞发生在第二次写入 innerHTML，而不仅是初始属性反射。

12.9　完成检查

交付与复盘检查表

- ☐ 入口、输入、目标输出与限制条件已经写入笔记。
- ☐ 建立了可重复的基线输入与输出。
- ☐ 每个关键结论都有原始证据与验证步骤。
- ☐ 脚本能够识别失败，不会把异常输出当成成功。
- ☐ 从干净目录或新容器运行时能够得到同样结果。

第十三章　CSRF、CORS 与点击劫持

本章目标

CSRF 利用浏览器自动携带身份凭据发起状态改变请求；CORS 决定脚本能否读取跨源响应；点击劫持依赖页面可被嵌入并诱导交互。三者经常混淆，需要分别验证请求发送、响应读取与界面嵌入。

13.1　原理与题型边界

SameSite Cookie、CSRF token、Origin/Referer 校验和自定义头会影响 CSRF。CORS 的 Access-Control-Allow-Origin 与 Allow-Credentials 组合决定读取权限，但 CORS 不阻止所有请求发送。X-Frame-Options 和 CSP frame-ancestors 控制嵌入。

13.2　识别线索与优先动作

观察线索	可能含义	优先动作
状态改变只需 GET 或简单表单 POST	可能可由跨站页面触发	用本地第二源构造无害请求
服务器反射任意 Origin 并允许凭据	可能存在凭据化 CORS	比较预检与实际响应头
token 固定、未绑定会话或可预测	CSRF 防护可能无效	用两个会话比较 token
页面可被 iframe 加载	可能缺少防嵌入策略	检查响应头并在本地页面验证

13.3　标准解题步骤

1. **保存原始请求基线**。记录方法、路径、查询、头部、Cookie、消息体、重定向、状态码、长度和稳定响应片段。
2. **定位输入到危险点的数据流**。明确动作是否改变服务端状态，并建立动作前后查询。
3. **一次只改变一个条件**。区分简单请求与需要预检的请求。
4. **建立正反对照**。分别测试请求能否发送、响应能否被脚本读取。
5. **确认影响与边界**。使用两个本地来源与题目测试账户验证，不访问第三方站点。
6. **检查编码与多层解析**。分别观察浏览器、代理、Web 服务器、框架和应用对输入的解码、归一化与类型转换。
7. **脚本化并从新会话复现**。把登录、token 获取、请求发送、结果判定和失败处理写入脚本，使用新会话重复。

13.4　分析时持续追问的问题

- 当前观察是原始事实，还是工具给出的推断?
- 哪个最小输入变化能够区分主要假设?
- 结果能否在相同输入和干净环境中重复?
- 失败时是否保存了完整原始输出?
- 题目条件变化后，当前方法的前提是否仍成立?

13.5　命令、脚本与验证片段

以下内容只用于本地题目目录、回环地址、赛事提供的实例或明确授权的实验环境。每次运行都应保留原始输入和完整输出。

```
# Inspect anti-CSRF and framing headers.
curl -sS -D captures/action-headers.txt -o /dev/null \
  http://127.0.0.1:8000/account
rg -i 'set-cookie|access-control|x-frame-options|content-security-policy' \
```

```
  captures/action-headers.txt
# Local origin comparison.
curl -sS -i -X OPTIONS http://127.0.0.1:8000/api/profile \
  -H 'Origin: http://127.0.0.1:9000' \
  -H 'Access-Control-Request-Method: POST' \
  -H 'Access-Control-Request-Headers: content-type'
```

Listing 13.1: 本章常用命令与脚本

13.6 证据保存与结果判定

保存未经编辑的原始请求与响应、会话状态、角色、时间、重定向链、响应长度和关键差异。若使用浏览器，还要保存开发者工具网络记录与相关客户端代码。最终证明应包含一组可重复的正反对照，说明变化来自被测试字段，而不是缓存、随机数或登录状态。

13.7 常见失误与修正方法

- 把 CORS 错误等同于 CSRF。
- 只看到请求 200，没有检查状态是否改变。
- 忽略 SameSite、token 与 Origin 校验。
- 使用真实用户会话进行测试。
- 未区分 frame-ancestors 与脚本同源策略。

13.8 小型演练

题目场景

本地账户修改邮箱接口使用 POST 表单，没有 token。证明是否可被第二个本地源触发。

先查询当前邮箱作为基线。建立另一个回环端口上的静态页面，提交普通 HTML form 到题目接口，使用专用测试会话访问。提交后重新查询账户状态。记录 Cookie SameSite、Origin/Referer 和响应。若浏览器自动携带会话并确实改变状态，才能证明 CSRF。不要依赖是否能读取响应。

13.9　完成检查

交付与复盘检查表

- ☐ 入口、输入、目标输出与限制条件已经写入笔记。
- ☐ 建立了可重复的基线输入与输出。
- ☐ 每个关键结论都有原始证据与验证步骤。
- ☐ 脚本能够识别失败，不会把异常输出当成成功。
- ☐ 从干净目录或新容器运行时能够得到同样结果。

第十四章　文件上传：扩展名、MIME、内容检查与处理链

本章目标

文件上传题的入口不只在“能否上传”，还包括文件名规范化、扩展名、MIME、内容嗅探、图片重编码、存储路径、下载响应和后续解析。目标是找出校验发生在哪一层以及上传后文件如何被使用。

14.1　原理与题型边界

客户端 accept 属性不是安全控制。服务端可能按扩展名、Content-Type、魔数或解析结果校验，并可能重命名、转码或放入独立存储。上传成功不代表文件可执行；执行、解析、下载和引用是不同影响。

14.2　识别线索与优先动作

观察线索	可能含义	优先动作
前端拒绝但直接请求能上传	校验只在客户端	重放请求并观察服务端验证
修改 Content-Type 后结果变化	服务端依赖请求头或解析器	保持内容不变建立对照
上传后文件被重命名或转码	存在处理流水线	下载处理后文件并比较哈希和结构
文件名中的路径符号被改变	服务器做规范化或安全重命名	记录原始名、存储名和访问 URL

14.3　标准解题步骤

1. **保存原始请求基线**。记录方法、路径、查询、头部、Cookie、消息体、重定向、状态码、长度和稳定响应片段。
2. **定位输入到危险点的数据流**。使用无害小文件建立上传、存储、下载和删除基线。
3. **一次只改变一个条件**。一次只改变扩展名、MIME、魔数、文件名或内容。
4. **建立正反对照**。下载服务端保存的文件并比较哈希、元数据和格式。
5. **确认影响与边界**。确定文件是被静态提供、解析、转码还是交给其他服务。
6. **检查编码与多层解析**。分别观察浏览器、代理、Web 服务器、框架和应用对输入的解码、归一化与类型转换。
7. **脚本化并从新会话复现**。把登录、token 获取、请求发送、结果判定和失败处理写入脚本，使用新会话重复。

14.4　分析时持续追问的问题

- 当前观察是原始事实，还是工具给出的推断?
- 哪个最小输入变化能够区分主要假设?
- 结果能否在相同输入和干净环境中重复?
- 失败时是否保存了完整原始输出?
- 题目条件变化后，当前方法的前提是否仍成立?

14.5　命令、脚本与验证片段

以下内容只用于本地题目目录、回环地址、赛事提供的实例或明确授权的实验环境。每次运行都应保留原始输入和完整输出。

```
printf 'CTF_UPLOAD_MARKER\n' > work/marker.txt
curl -sS -F 'file=@work/marker.txt;type=text/plain;filename=marker.txt' \
  http://127.0.0.1:8000/upload -o captures/upload.json
# Image processing comparison.
```

```
file work/input.png
sha256sum work/input.png
curl -sS -F 'file=@work/input.png;type=image/png;filename=input.png' \
  http://127.0.0.1:8000/upload -o captures/upload-image.json
# Download the local challenge result and compare it separately.
```

Listing 14.1: 本章常用命令与脚本

14.6 证据保存与结果判定

保存未经编辑的原始请求与响应、会话状态、角色、时间、重定向链、响应长度和关键差异。若使用浏览器，还要保存开发者工具网络记录与相关客户端代码。最终证明应包含一组可重复的正反对照，说明变化来自被测试字段，而不是缓存、随机数或登录状态。

14.7 常见失误与修正方法

- 把前端 accept 属性当作服务端防护。
- 同时修改扩展名、MIME 和内容，无法判断哪项生效。
- 上传成功就断言代码执行。
- 没有下载服务端处理后的文件。
- 使用具有破坏性或持久化功能的上传内容。

14.8 小型演练

题目场景

头像接口只允许 PNG，上传后图片尺寸和哈希改变。分析校验与处理链。

用最小合法 PNG 建立基线，记录请求 Content-Type、文件名、响应 URL、下载文件哈希与像素。分别改变扩展名、MIME 和尾随数据，每次只改一项。若服务端重编码，原始尾随数据会消失；若只检查魔数，结构异常可能被保留。结论应说明哪一层接受、哪一层重写，以及题目后续如何使用结果。

14.9　完成检查

交付与复盘检查表

- ☐ 入口、输入、目标输出与限制条件已经写入笔记。
- ☐ 建立了可重复的基线输入与输出。
- ☐ 每个关键结论都有原始证据与验证步骤。
- ☐ 脚本能够识别失败，不会把异常输出当成成功。
- ☐ 从干净目录或新容器运行时能够得到同样结果。

第十五章　目录穿越、文件读取与本地包含

本章目标

目录穿越发生在用户输入参与路径拼接而规范化或根目录约束不足。本地文件包含还会把读取结果交给模板或解释器。解题时先证明路径可控和读取边界，再判断是否存在进一步解析。

15.1　原理与题型边界

路径可能经过 URL 解码、反斜杠转换、重复解码、软链接解析和操作系统规范化。安全实现应在规范化后验证结果仍位于允许根目录。文件读取与代码执行是不同影响，不能从“能读文件”直接推导“可执行”。

15.2　识别线索与优先动作

观察线索	可能含义	优先动作
filename 参数能读取不同模板	输入进入文件选择或路径拼接	使用两个合法文件建立基线
../ 被过滤但编码形式行为不同	多层解码或黑名单可能不一致	记录每层原始请求和响应
错误页泄露绝对路径	有助于理解根目录和工作目录	用不存在文件与合法文件对照
同一文件被下载与 include 时行为不同	后端后续处理不同	分别测试读取和渲染端点

15.3　标准解题步骤

1. **保存原始请求基线**。记录方法、路径、查询、头部、Cookie、消息体、重定向、状态码、长度和稳定响应片段。
2. **定位输入到危险点的数据流**。确定输入是完整路径、文件名、模板名还是目录名。
3. **一次只改变一个条件**。用题目目录内两个已知文件证明路径选择能力。
4. **建立正反对照**。逐步尝试单层规范化和编码差异，不堆叠复杂变体。
5. **确认影响与边界**。确认读取内容、错误类型和允许根目录，完成题目所需最小影响。
6. **检查编码与多层解析**。分别观察浏览器、代理、Web 服务器、框架和应用对输入的解码、归一化与类型转换。
7. **脚本化并从新会话复现**。把登录、token 获取、请求发送、结果判定和失败处理写入脚本，使用新会话重复。

15.4　分析时持续追问的问题

- 当前观察是原始事实，还是工具给出的推断?
- 哪个最小输入变化能够区分主要假设?
- 结果能否在相同输入和干净环境中重复?
- 失败时是否保存了完整原始输出?
- 题目条件变化后，当前方法的前提是否仍成立?

15.5　命令、脚本与验证片段

以下内容只用于本地题目目录、回环地址、赛事提供的实例或明确授权的实验环境。每次运行都应保留原始输入和完整输出。

```
# Local challenge path selection baselines.
curl -sS --get --data-urlencode 'file=public/readme.txt' \
  http://127.0.0.1:8000/download -o captures/readme.txt
```

```
curl -sS --get --data-urlencode 'file=missing.txt' \
  http://127.0.0.1:8000/download -o captures/missing.txt
curl -sS --get --data-urlencode 'file=../config/example.txt' \
  http://127.0.0.1:8000/download -o captures/traversal.txt
wc -c captures/*.txt
```

Listing 15.1: 本章常用命令与脚本

15.6 证据保存与结果判定

保存未经编辑的原始请求与响应、会话状态、角色、时间、重定向链、响应长度和关键差异。若使用浏览器，还要保存开发者工具网络记录与相关客户端代码。最终证明应包含一组可重复的正反对照，说明变化来自被测试字段，而不是缓存、随机数或登录状态。

15.7 常见失误与修正方法

- 一开始使用多层编码，无法知道哪层解码。
- 把错误页路径当作已读取文件。
- 把任意文件读取与代码执行混为一谈。
- 忽略 Windows 与 POSIX 路径差异。
- 读取超出题目需要的敏感数据。

15.8 小型演练

题目场景

下载接口 file=public/a.txt 可用，file=../config/example.txt 返回另一内容。如何证明目录穿越并确定根目录。

先分别请求两个合法文件和不存在文件，记录错误模式。保持会话与其他参数不变，只改变 file。若上级路径返回可预测的题目配置内容，说明规范化后越出 public。再通过相邻已知文件与错误中的绝对路径推断根目录，但不要进行无边界文件枚举。保存原始请求、响应和本地题目目录结构对照。

15.9　完成检查

交付与复盘检查表

- ☐ 入口、输入、目标输出与限制条件已经写入笔记。
- ☐ 建立了可重复的基线输入与输出。
- ☐ 每个关键结论都有原始证据与验证步骤。
- ☐ 脚本能够识别失败，不会把异常输出当成成功。
- ☐ 从干净目录或新容器运行时能够得到同样结果。

第十六章　操作系统命令注入

本章目标

命令注入产生于用户输入进入 shell 或命令字符串。解题时先区分输入被作为单个参数传给程序，还是由 shell 重新解释。使用无害、可观察的本地标记证明控制字符是否改变命令结构。

16.1　原理与题型边界

安全调用通常传递参数数组并避免 shell。危险实现会拼接字符串后交给 sh、cmd 或类似解释器。引号、空白、换行、管道和命令替换在不同 shell 中行为不同。时间差异与错误信息都需要对照。

16.2　识别线索与优先动作

观察线索	可能含义	优先动作
输入中的分隔符导致额外输出	可能进入 shell 命令字符串	使用固定标记而非系统枚举
空格被过滤但换行或编码行为不同	过滤器与 shell 解析边界不一致	一次测试一种分隔方式
错误中出现完整命令或 shell 名称	可确认调用方式与引号上下文	保存错误并构造闭合对照
延迟随输入表达式稳定变化	可能执行了额外命令	重复采样并与等长输入比较

16.3　标准解题步骤

1. **保存原始请求基线**。记录方法、路径、查询、头部、Cookie、消息体、重定向、状态码、长度和稳定响应片段。

2. **定位输入到危险点的数据流**。建立普通参数基线并观察预期程序输出。
3. **一次只改变一个条件**。使用不会修改系统的固定文本标记测试命令边界。
4. **建立正反对照**。根据错误确定单引号、双引号、无引号或参数数组上下文。
5. **确认影响与边界**。证明影响后停止扩展，只读取题目明确提供的数据。
6. **检查编码与多层解析**。分别观察浏览器、代理、Web 服务器、框架和应用对输入的解码、归一化与类型转换。
7. **脚本化并从新会话复现**。把登录、token 获取、请求发送、结果判定和失败处理写入脚本，使用新会话重复。

16.4　分析时持续追问的问题

- 当前观察是原始事实，还是工具给出的推断?
- 哪个最小输入变化能够区分主要假设?
- 结果能否在相同输入和干净环境中重复?
- 失败时是否保存了完整原始输出?
- 题目条件变化后，当前方法的前提是否仍成立?

16.5　命令、脚本与验证片段

以下内容只用于本地题目目录、回环地址、赛事提供的实例或明确授权的实验环境。每次运行都应保留原始输入和完整输出。

```
# Harmless marker against a local toy endpoint.
curl -sS --get --data-urlencode 'host=127.0.0.1' \
  http://127.0.0.1:8000/ping -o captures/cmd-base.txt
curl -sS --get --data-urlencode 'host=127.0.0.1;printf CTF_CMD_MARKER' \
  http://127.0.0.1:8000/ping -o captures/cmd-marker.txt
diff -u captures/cmd-base.txt captures/cmd-marker.txt
```

Listing 16.1: 本章常用命令与脚本

16.6　证据保存与结果判定

保存未经编辑的原始请求与响应、会话状态、角色、时间、重定向链、响应长度和关键差异。若使用浏览器，还要保存开发者工具网络记录与相关客户端代码。最终证明应包含一组可重复的正反对照，说明变化来自被测试字段，而不是缓存、随机数或登录状态。

16.7　常见失误与修正方法

- 直接使用破坏性命令。
- 把程序参数注入误判为 shell 注入。
- 同时尝试多个分隔符和编码。
- 时间测试只运行一次。
- 发现执行后继续枚举主机或持久化。

16.8　小型演练

题目场景

本地 ping 功能接受 host。加入分号和固定 printf 后响应多出标记。给出证据链。

保存普通请求与标记请求，确保唯一变化是 host。重复多次确认标记稳定出现，检查错误输出和本地源码以确定是否使用 shell。使用只输出固定文本的命令，不访问网络、不写文件。最终说明输入如何闭合原命令、shell 如何解释分隔符、响应如何包含额外输出。

16.9　完成检查

交付与复盘检查表

- ☐ 入口、输入、目标输出与限制条件已经写入笔记。
- ☐ 建立了可重复的基线输入与输出。
- ☐ 每个关键结论都有原始证据与验证步骤。
- ☐ 脚本能够识别失败，不会把异常输出当成成功。
- ☐ 从干净目录或新容器运行时能够得到同样结果。

第十七章　SSRF：服务端请求、URL 解析与重定向

本章目标

SSRF 题让应用服务器根据用户提供的 URL 发起请求。关键是确认请求由服务端产生、允许的协议与目标范围、DNS 和重定向处理，以及响应是否回显。所有示例只访问本地题目容器内的测试服务。

17.1　原理与题型边界

URL 会经过应用校验、URL 解析器、DNS、代理和 HTTP 客户端。字符串前缀检查可能与最终解析结果不一致；重定向可能在校验后改变目标；响应不回显时可通过本地日志、状态差异或时间证明请求发生。

17.2　识别线索与优先动作

观察线索	可能含义	优先动作
输入 URL 后页面返回远端标题或状态	服务端可能主动抓取	用两个本地端口建立对照
只允许某个域名但会跟随跳转	校验与最终目标可能分离	使用本地重定向服务记录第二跳
不同协议产生不同错误	客户端支持协议集合可被识别	只测试题目必要的安全协议
响应不回显但本地服务收到请求	存在盲 SSRF 通道	保存接收日志与关联标识

17.3　标准解题步骤

1. **保存原始请求基线**。记录方法、路径、查询、头部、Cookie、消息体、重定向、状态码、长度和稳定响应片段。
2. **定位输入到危险点的数据流**。确认请求来自服务器而不是浏览器 JavaScript。
3. **一次只改变一个条件**。使用回环上的两个自有测试服务记录源地址、头部和路径。
4. **建立正反对照**。分别测试解析、DNS、端口、重定向和响应处理，不混在一次请求中。
5. **确认影响与边界**。确认题目允许的内部资源并只访问提供的本地目标。
6. **检查编码与多层解析**。分别观察浏览器、代理、Web 服务器、框架和应用对输入的解码、归一化与类型转换。
7. **脚本化并从新会话复现**。把登录、token 获取、请求发送、结果判定和失败处理写入脚本，使用新会话重复。

17.4　分析时持续追问的问题

- 当前观察是原始事实，还是工具给出的推断?
- 哪个最小输入变化能够区分主要假设?
- 结果能否在相同输入和干净环境中重复?
- 失败时是否保存了完整原始输出?
- 题目条件变化后，当前方法的前提是否仍成立?

17.5　命令、脚本与验证片段

以下内容只用于本地题目目录、回环地址、赛事提供的实例或明确授权的实验环境。每次运行都应保留原始输入和完整输出。

```
# Start a local observation server in another terminal:
# python3 -m http.server 9001 --bind 127.0.0.1
curl -sS --get --data-urlencode 'url=http://127.0.0.1:9001/ssrf-marker' \
```

```
  http://127.0.0.1:8000/fetch -o captures/ssrf-response.txt
# Compare URL parser behavior locally.
python3 - <<'PYCODE'
from urllib.parse import urlsplit
for u in ['http://127.0.0.1:9001/a','http://challenge.local@127.0.0.1:9001/b']:
    print(u, urlsplit(u))
PYCODE
```

Listing 17.1: 本章常用命令与脚本

17.6 证据保存与结果判定

保存未经编辑的原始请求与响应、会话状态、角色、时间、重定向链、响应长度和关键差异。若使用浏览器，还要保存开发者工具网络记录与相关客户端代码。最终证明应包含一组可重复的正反对照，说明变化来自被测试字段，而不是缓存、随机数或登录状态。

17.7 常见失误与修正方法

- 使用公共回连服务泄露题目信息。
- 只看页面错误，没有证明请求来自服务端。
- 同时改变主机、协议、端口和重定向。
- 忽略 DNS 缓存与重绑定时序。
- 访问题目未明确提供的内部资产。

17.8 小型演练

题目场景

fetch 接口只显示“成功”，不回显内容。证明它请求了本地 9001 端口。

在同一题目环境启动记录路径和头部的本地 HTTP 服务，为每次测试使用唯一标识。向 fetch 提交回环 URL，保存接口响应和本地服务日志，并对不存在端口建立失败对照。若日志中的唯一路径在对应时间出现，且源地址来自题目应用环境，即可证明服务端请求。

17.9　完成检查

交付与复盘检查表

- ☐ 入口、输入、目标输出与限制条件已经写入笔记。
- ☐ 建立了可重复的基线输入与输出。
- ☐ 每个关键结论都有原始证据与验证步骤。
- ☐ 脚本能够识别失败，不会把异常输出当成成功。
- ☐ 从干净目录或新容器运行时能够得到同样结果。

第十八章　服务端模板注入 SSTI

本章目标

SSTI 发生在用户输入被当作模板源码而非普通数据渲染。不同模板引擎语法、对象模型和沙箱不同。先用无害算术或字符串表达式识别求值，再根据题目源码确认引擎与可访问对象。

18.1　原理与题型边界

安全模板把用户输入作为变量值。危险实现会拼接模板字符串后重新编译。相似的大括号语法可能属于不同引擎，也可能只是前端模板。错误信息、转义行为和算术结果是识别线索。

18.2　识别线索与优先动作

观察线索	可能含义	优先动作
输入 {{7*7}} 显示 49	可能被模板引擎求值	用纯文本与另一表达式建立对照
错误页出现模板引擎类名或文件	可确定引擎和版本线索	保存堆栈并查本地依赖
同一值在 HTML 与文本端点转义不同	存在不同模板上下文	分别记录自动转义行为
前端源码含双大括号但服务器响应不变	可能只是客户端模板	禁用脚本或查看原始响应区分

18.3　标准解题步骤

1. **保存原始请求基线**。记录方法、路径、查询、头部、Cookie、消息体、重定向、状态码、长度和稳定响应片段。

2. **定位输入到危险点的数据流**。先证明服务端渲染而非客户端替换。
3. **一次只改变一个条件**。使用无害算术、字符串拼接和不存在变量识别语法。
4. **建立正反对照**。结合依赖文件或错误确定引擎，不从长 payload 猜测。
5. **确认影响与边界**。只访问题目明确暴露的模板变量与本地 flag 数据。
6. **检查编码与多层解析**。分别观察浏览器、代理、Web 服务器、框架和应用对输入的解码、归一化与类型转换。
7. **脚本化并从新会话复现**。把登录、token 获取、请求发送、结果判定和失败处理写入脚本，使用新会话重复。

18.4　分析时持续追问的问题

- 当前观察是原始事实，还是工具给出的推断?
- 哪个最小输入变化能够区分主要假设?
- 结果能否在相同输入和干净环境中重复?
- 失败时是否保存了完整原始输出?
- 题目条件变化后，当前方法的前提是否仍成立?

18.5　命令、脚本与验证片段

以下内容只用于本地题目目录、回环地址、赛事提供的实例或明确授权的实验环境。每次运行都应保留原始输入和完整输出。

```
for p in 'CTF_MARKER' '{{7*7}}' '{{7*8}}' '{{unknown_name}}'; do
  curl -sS --get --data-urlencode "name=$p" \
    http://127.0.0.1:8000/greet \
    -o "captures/ssti-$(printf %s "$p" | sha256sum | cut -c1-8).txt"
done
rg -n '49|56|unknown|Template|Jinja|Twig|Freemarker' captures/ssti-*
```

Listing 18.1: 本章常用命令与脚本

18.6 证据保存与结果判定

保存未经编辑的原始请求与响应、会话状态、角色、时间、重定向链、响应长度和关键差异。若使用浏览器，还要保存开发者工具网络记录与相关客户端代码。最终证明应包含一组可重复的正反对照，说明变化来自被测试字段，而不是缓存、随机数或登录状态。

18.7 常见失误与修正方法

- 直接套用某个引擎的长利用链。
- 把客户端模板语法误判为服务端求值。
- 只使用一个表达式，没有正反对照。
- 忽略自动转义与错误处理。
- 在不明确沙箱边界时执行系统命令。

18.8 小型演练

题目场景

greet 页面把 {{7*7}} 显示为 49，{{7*8}} 显示为 56。确定是否为 Jinja 类引擎。

先确认结果出现在原始 HTTP 响应而不是前端脚本生成。再测试字符串操作、不存在变量和语法错误，保存错误文本。检查 requirements 或服务端源码确定引擎。证明输入被当作模板源码后，根据题目提供的对象寻找 flag，避免跳到系统命令执行。结论应说明渲染调用发生在哪里。

18.9　完成检查

交付与复盘检查表

- ☐ 入口、输入、目标输出与限制条件已经写入笔记。
- ☐ 建立了可重复的基线输入与输出。
- ☐ 每个关键结论都有原始证据与验证步骤。
- ☐ 脚本能够识别失败，不会把异常输出当成成功。
- ☐ 从干净目录或新容器运行时能够得到同样结果。

第十九章　XXE 与 XML 解析器边界

本章目标

XXE 题利用 XML 解析器对外部实体、DTD、XInclude 或网络资源的处理。现代库默认设置差异很大，必须通过本地题目解析器行为验证。目标通常是读取容器内题目文件或发起本地请求。

19.1　原理与题型边界

XML 有文档类型声明、实体展开和命名空间等机制。风险取决于是否允许 DTD、外部通用实体、参数实体、网络访问以及解析结果是否回显。JSON 接口也可能在特定 Content-Type 下切换到 XML 解析。

19.2　识别线索与优先动作

观察线索	可能含义	优先动作
Content-Type 改为 XML 后进入不同错误	端点可能支持 XML 解析器	先发送最小合法 XML
DOCTYPE 触发明确禁用或解析错误	可以判断 DTD 配置	比较无 DTD、内部实体和外部实体
实体值出现在响应字段	存在回显实体展开	使用本地固定文本文件验证
没有回显但本地 HTTP 服务收到请求	可能是盲 XXE	使用唯一路径和时间关联

19.3　标准解题步骤

1. **保存原始请求基线**。记录方法、路径、查询、头部、Cookie、消息体、重定向、状态码、长度和稳定响应片段。
2. **定位输入到危险点的数据流**。发送最小合法 XML 建立字段映射。
3. **一次只改变一个条件**。依次测试内部实体、外部本地文件和本地 HTTP，不同时混用。
4. **建立正反对照**。记录解析错误、状态码、响应体和本地接收日志。
5. **确认影响与边界**。只读取题目容器明确提供的测试文件。
6. **检查编码与多层解析**。分别观察浏览器、代理、Web 服务器、框架和应用对输入的解码、归一化与类型转换。
7. **脚本化并从新会话复现**。把登录、token 获取、请求发送、结果判定和失败处理写入脚本，使用新会话重复。

19.4　分析时持续追问的问题

- 当前观察是原始事实，还是工具给出的推断?
- 哪个最小输入变化能够区分主要假设?
- 结果能否在相同输入和干净环境中重复?
- 失败时是否保存了完整原始输出?
- 题目条件变化后，当前方法的前提是否仍成立?

19.5　命令、脚本与验证片段

以下内容只用于本地题目目录、回环地址、赛事提供的实例或明确授权的实验环境。每次运行都应保留原始输入和完整输出。

```
cat > work/xxe.xml <<'XML'
<?xml version="1.0"?>
<!DOCTYPE root [<!ENTITY demo SYSTEM "file:///etc/hostname">]>
<root><name>&demo;</name></root>
```

```
XML
curl -sS -H 'Content-Type: application/xml' --data-binary @work/xxe.xml \
  http://127.0.0.1:8000/api/xml -o captures/xxe-response.txt
cat > work/xml-baseline.xml <<'XML'
<?xml version="1.0"?><root><name>CTF_MARKER</name></root>
XML
```

Listing 19.1: 本章常用命令与脚本

19.6 证据保存与结果判定

保存未经编辑的原始请求与响应、会话状态、角色、时间、重定向链、响应长度和关键差异。若使用浏览器，还要保存开发者工具网络记录与相关客户端代码。最终证明应包含一组可重复的正反对照，说明变化来自被测试字段，而不是缓存、随机数或登录状态。

19.7 常见失误与修正方法

- 没有先验证端点是否真正解析 XML。
- 把内部实体展开与外部实体混淆。
- 使用外部公共服务。
- 读取题目范围外的文件。
- 忽略实体禁用错误已经给出的防护信息。

19.8 小型演练

题目场景

XML API 将 name 字段回显。内部实体可展开，外部实体是否允许未知。设计逐步验证。

先发送无 DTD 的合法 XML，确认字段映射。第二个请求定义内部实体为固定文本，证明实体展开。第三个请求把外部实体指向容器中允许的 /etc/hostname 或题目测试文件，比较响应。若不回显，使用同容器本地 HTTP 记录服务。每一步只改变实体类型，保存原始 XML 和解析错误。

19.9　完成检查

交付与复盘检查表

- ☐ 入口、输入、目标输出与限制条件已经写入笔记。
- ☐ 建立了可重复的基线输入与输出。
- ☐ 每个关键结论都有原始证据与验证步骤。
- ☐ 脚本能够识别失败，不会把异常输出当成成功。
- ☐ 从干净目录或新容器运行时能够得到同样结果。

第二十章　不安全反序列化与对象重建

本章目标

反序列化题要求识别客户端数据是否被服务器还原为对象，以及还原过程是否触发类型选择、魔术方法或危险构造。首先确定格式和完整性保护，再使用静态检查或本地副本分析，不对未知数据直接执行。

20.1　原理与题型边界

JSON 通常只重建基本类型，而 Python pickle、Java 序列化、PHP serialize 和某些框架 token 可以携带类型与对象图。风险来自反序列化期间调用构造、解析器钩子或已有类的方法。签名或 MAC 若正确验证，可以阻止篡改。

20.2　识别线索与优先动作

观察线索	可能含义	优先动作
Cookie 或参数是长 Base64 二进制	可能是序列化对象或压缩 token	解码后检查魔数和字符串，不执行
修改一个字段导致类型或类名错误	服务器可能重建对象图	保存错误并确认格式
源码调用 pickle.loads、ObjectInputStream 或 unserialize	明确存在对象反序列化	检查数据来源与完整性验证
token 末尾固定长度且修改后报签名错误	可能有 MAC 或签名保护	先分析验证顺序，不盲目构造对象

20.3　标准解题步骤

1. **保存原始请求基线**。记录方法、路径、查询、头部、Cookie、消息体、重定向、状态码、长度和稳定响应片段。
2. **定位输入到危险点的数据流**。解码并静态识别格式，计算原始哈希。
3. **一次只改变一个条件**。检查是否有签名、MAC、加密或长度前缀。
4. **建立正反对照**。使用格式专用反汇编或解析工具查看对象图，不直接加载未知对象。
5. **确认影响与边界**。在本地题目源码中寻找可控字段、类型选择和魔术方法。
6. **检查编码与多层解析**。分别观察浏览器、代理、Web 服务器、框架和应用对输入的解码、归一化与类型转换。
7. **脚本化并从新会话复现**。把登录、token 获取、请求发送、结果判定和失败处理写入脚本，使用新会话重复。

20.4　分析时持续追问的问题

- 当前观察是原始事实，还是工具给出的推断?
- 哪个最小输入变化能够区分主要假设?
- 结果能否在相同输入和干净环境中重复?
- 失败时是否保存了完整原始输出?
- 题目条件变化后，当前方法的前提是否仍成立?

20.5　命令、脚本与验证片段

以下内容只用于本地题目目录、回环地址、赛事提供的实例或明确授权的实验环境。每次运行都应保留原始输入和完整输出。

```
# Static inspection of a local Python pickle without executing it.
python3 -m pickletools work/session.bin > notes/pickle-disassembly.txt
strings -a -t x work/session.bin | tee notes/session-strings.txt
xxd -g 1 -l 128 work/session.bin | tee notes/session-head.hex
```

```
sha256sum work/session.bin | tee notes/session.sha256
# Java stream header is commonly AC ED 00 05; inspect only.
file work/token.bin
```

Listing 20.1: 本章常用命令与脚本

20.6 证据保存与结果判定

保存未经编辑的原始请求与响应、会话状态、角色、时间、重定向链、响应长度和关键差异。若使用浏览器，还要保存开发者工具网络记录与相关客户端代码。最终证明应包含一组可重复的正反对照，说明变化来自被测试字段，而不是缓存、随机数或登录状态。

20.7 常见失误与修正方法

- 直接 pickle.loads 未知附件。
- 只看到 Base64 就断言是序列化。
- 忽略签名验证和压缩层。
- 寻找不存在于目标类路径中的通用 gadget。
- 构造会修改系统或联网的对象链。

20.8 小型演练

题目场景

Cookie 解码后以 pickle 协议头开头。服务端源码先验证 HMAC，再 loads。能否直接修改 role。

先用 pickletools 静态反汇编确认字段与类型，不执行。检查 HMAC 覆盖的是原始序列化字节还是解码后内容，并确认密钥是否真正未知。任意修改都会导致签名失败时，不能仅靠改变 role。若题目另有密钥泄露或验证顺序错误，再在本地构造最小对象并重新签名；否则应转向密钥来源或逻辑漏洞。

20.9　完成检查

交付与复盘检查表

- ☐ 入口、输入、目标输出与限制条件已经写入笔记。
- ☐ 建立了可重复的基线输入与输出。
- ☐ 每个关键结论都有原始证据与验证步骤。
- ☐ 脚本能够识别失败，不会把异常输出当成成功。
- ☐ 从干净目录或新容器运行时能够得到同样结果。

第二十一章　JWT、签名 token 与 OAuth 常见错误

本章目标

JWT 是编码后的头部、载荷和签名，不等同于加密。解题重点是确认服务器实际允许的算法、密钥选择、声明验证和会话绑定。OAuth 题还要区分授权请求、回调、state、redirect_uri、code 与 token。

21.1　原理与题型边界

JWT 安全依赖服务器固定算法、正确验证签名和检查 exp、nbf、iss、aud 等声明。客户端提供的 alg 或 kid 不能无条件决定验证逻辑。OAuth 的 redirect_uri、state 和 code 绑定用于阻止授权结果被错误发送或混用。

21.2　识别线索与优先动作

观察线索	可能含义	优先动作
JWT 载荷可直接 Base64 解码	这是格式特性，不表示可篡改	先记录 header、claims 与签名算法
修改 payload 后仍被接受	可能未验签或使用错误密钥	用一位变化与无效签名对照
kid 影响错误或读取路径	密钥选择可能受控	检查白名单与规范化
OAuth 回调缺少 state 或允许宽松 redirect_uri	授权响应可能未绑定原请求	用两个本地会话建立绑定对照

21.3　标准解题步骤

1. **保存原始请求基线**。记录方法、路径、查询、头部、Cookie、消息体、重定向、状态码、长度和稳定响应片段。
2. **定位输入到危险点的数据流**。解码但不信任 token 内容，记录算法和声明。
3. **一次只改变一个条件**。用原 token、仅改载荷、仅改签名和过期 token 建立矩阵。
4. **建立正反对照**。确认服务器固定算法、密钥来源和声明校验。
5. **确认影响与边界**。OAuth 流程逐跳记录 authorize、callback、code 交换和最终会话。
6. **检查编码与多层解析**。分别观察浏览器、代理、Web 服务器、框架和应用对输入的解码、归一化与类型转换。
7. **脚本化并从新会话复现**。把登录、token 获取、请求发送、结果判定和失败处理写入脚本，使用新会话重复。

21.4　分析时持续追问的问题

- 当前观察是原始事实，还是工具给出的推断?
- 哪个最小输入变化能够区分主要假设?
- 结果能否在相同输入和干净环境中重复?
- 失败时是否保存了完整原始输出?
- 题目条件变化后，当前方法的前提是否仍成立?

21.5　命令、脚本与验证片段

以下内容只用于本地题目目录、回环地址、赛事提供的实例或明确授权的实验环境。每次运行都应保留原始输入和完整输出。

```
python3 - <<'PYCODE'
import base64, json
jwt=open('work/token.txt').read().strip()
```

```
parts=jwt.split('.')
for name,p in zip(('header','payload'),parts[:2]):
    p += '='*(-len(p)%4)
    print(name, json.dumps(json.loads(base64.urlsafe_b64decode(p)),indent=2))
print('signature-bytes', len(base64.urlsafe_b64decode(parts[2]+'='*(-len(parts
    [2])%4))))
PYCODE
# Record OAuth redirects without following them automatically.
curl -sS -D captures/oauth-headers.txt -o /dev/null --max-redirs 0 \
  'http://127.0.0.1:8000/oauth/start'
```

Listing 21.1: 本章常用命令与脚本

21.6　证据保存与结果判定

保存未经编辑的原始请求与响应、会话状态、角色、时间、重定向链、响应长度和关键差异。若使用浏览器，还要保存开发者工具网络记录与相关客户端代码。最终证明应包含一组可重复的正反对照，说明变化来自被测试字段，而不是缓存、随机数或登录状态。

21.7　常见失误与修正方法

- 把可解码的 JWT 当成未加密漏洞。
- 只改 role 后看到 401 就停止分析所有验证条件。
- 让 token 中 alg 无条件决定本地验证。
- 忽略 exp、aud、iss 和会话绑定。
- OAuth 测试混用同一浏览器会话，无法判断 state。

21.8　小型演练

题目场景

JWT 修改 role 后仍被本地 API 接受。需要证明服务器是否完全不验签。

准备四个 token：原始、只改 payload 保留旧签名、破坏一个签名字节、合法签名但过期。用全新会话分别请求同一接口。若只改 payload 和破坏签名都被接受，

说明签名未验证；若只有某种算法或 kid 组合被接受，需要继续分析算法与密钥选择。保存 token 哈希和响应矩阵，不把单次成功泛化。

21.9　完成检查

交付与复盘检查表

- □ 入口、输入、目标输出与限制条件已经写入笔记。
- □ 建立了可重复的基线输入与输出。
- □ 每个关键结论都有原始证据与验证步骤。
- □ 脚本能够识别失败，不会把异常输出当成成功。
- □ 从干净目录或新容器运行时能够得到同样结果。

第二十二章　API、GraphQL、批量赋值与参数污染

本章目标

现代 Web 题常把入口放在 JSON API、GraphQL、对象绑定和批量更新中。需要枚举字段、类型、操作、认证规则与错误差异，同时注意重复参数、未知字段和嵌套对象如何被框架处理。

22.1　原理与题型边界

GraphQL 通过 schema、query、mutation、变量和解析器工作。即使关闭 introspection，错误、客户端代码和已知查询仍可暴露字段。批量赋值发生在请求对象被直接映射到模型，导致 role、owner 或状态字段被意外更新。HTTP 参数污染来自前后端对重复参数取首值、末值或数组的差异。

22.2　识别线索与优先动作

观察线索	可能含义	优先动作
JSON 中加入未显示字段后状态变化	可能存在批量赋值	一次添加一个候选字段并查询对象
GraphQL 错误提示相近字段名	schema 可通过错误反馈逐步确定	使用最小查询与变量类型对照
重复参数时代理与应用响应不同	可能有参数污染或解析分歧	保存原始请求字节并比较顺序
批量接口只检查整体权限	单个对象可能缺少授权	混合自有与不允许对象做事务对照

22.3　标准解题步骤

1. **保存原始请求基线**。记录方法、路径、查询、头部、Cookie、消息体、重定向、状态码、长度和稳定响应片段。
2. **定位输入到危险点的数据流**。建立 API 路由、方法、字段、类型和角色矩阵。
3. **一次只改变一个条件**。对 GraphQL 从 __typename、单字段和错误开始，不发送巨大查询。
4. **建立正反对照**。对批量赋值一次增加一个字段，并在独立 GET 中验证持久化。
5. **确认影响与边界**。重复参数分别测试顺序、编码和内容类型，记录每层解释。
6. **检查编码与多层解析**。分别观察浏览器、代理、Web 服务器、框架和应用对输入的解码、归一化与类型转换。
7. **脚本化并从新会话复现**。把登录、token 获取、请求发送、结果判定和失败处理写入脚本，使用新会话重复。

22.4　分析时持续追问的问题

- 当前观察是原始事实，还是工具给出的推断?
- 哪个最小输入变化能够区分主要假设?
- 结果能否在相同输入和干净环境中重复?
- 失败时是否保存了完整原始输出?
- 题目条件变化后，当前方法的前提是否仍成立?

22.5　命令、脚本与验证片段

以下内容只用于本地题目目录、回环地址、赛事提供的实例或明确授权的实验环境。每次运行都应保留原始输入和完整输出。

```
# Minimal local GraphQL request.
curl -sS -H 'Content-Type: application/json' \
  -d '{"query":"query { viewer { id username } }"}' \
  http://127.0.0.1:8000/graphql -o captures/graphql-viewer.json
```

```
# Mass-assignment field comparison.
curl -sS -X PATCH -H 'Content-Type: application/json' \
  -b captures/user.cookies \
  -d '{"display_name":"alice"}' \
  http://127.0.0.1:8000/api/me -o captures/update-base.json
curl -sS -X PATCH -H 'Content-Type: application/json' \
  -b captures/user.cookies \
  -d '{"display_name":"alice","role":"admin"}' \
  http://127.0.0.1:8000/api/me -o captures/update-role.json
```

Listing 22.1: 本章常用命令与脚本

22.6　证据保存与结果判定

保存未经编辑的原始请求与响应、会话状态、角色、时间、重定向链、响应长度和关键差异。若使用浏览器，还要保存开发者工具网络记录与相关客户端代码。最终证明应包含一组可重复的正反对照，说明变化来自被测试字段，而不是缓存、随机数或登录状态。

22.7　常见失误与修正方法

- 一开始发送完整 introspection，忽略题目限制和速率。
- 只看更新响应，没有重新读取对象。
- 一次添加多个隐藏字段。
- 把 GraphQL 字段存在等同于当前角色可访问。
- 代理自动合并重复参数，丢失原始顺序。

22.8　小型演练

题目场景

PATCH /api/me 接受 display_name。加入 role 字段响应仍为 200。判断是否批量赋值。

先读取当前对象并保存角色。发送只含 display_name 的基线请求，再发送增加 role 的请求，随后用独立 GET 和重新登录验证持久化。比较服务器是否忽略、

拒绝、暂存或真正修改字段。若 role 变化是唯一差异且影响授权，才可判定批量赋值。

22.9　完成检查

交付与复盘检查表

- □ 入口、输入、目标输出与限制条件已经写入笔记。
- □ 建立了可重复的基线输入与输出。
- □ 每个关键结论都有原始证据与验证步骤。
- □ 脚本能够识别失败，不会把异常输出当成成功。
- □ 从干净目录或新容器运行时能够得到同样结果。

第二十三章　竞态条件与业务逻辑漏洞

本章目标

业务逻辑题关注合法功能组合后产生的错误状态，例如重复兑换、余额检查与扣减分离、状态机跳步、优惠叠加和库存竞争。竞态条件要求两个或更多请求在关键时间窗口交错。

23.1　原理与题型边界

单请求看似安全并不保证多请求原子性。典型模式是先检查条件，再执行更新，两个并发请求都通过检查。业务逻辑漏洞还可能完全不依赖并发，而是因为操作顺序、边界值或角色流程不完整。

23.2　识别线索与优先动作

观察线索	可能含义	优先动作
同一优惠或兑换码偶尔能重复使用	检查与标记可能非原子	先用顺序请求建立基线，再有限并发
余额在请求后出现负数或重复扣减	更新缺少事务或边界检查	记录每次前后余额和事务标识
跳过页面直接调用后续接口成功	状态机只在前端执行	按步骤缺失建立请求矩阵
取消、退款与完成可异常组合	状态转换约束不完整	画出允许状态图并测试单条异常边

23.3　标准解题步骤

1. **保存原始请求基线**。记录方法、路径、查询、头部、Cookie、消息体、重定向、状态码、长度和稳定响应片段。
2. **定位输入到危险点的数据流**。画出对象状态机、角色、前置条件和不变量。
3. **一次只改变一个条件**。先用顺序请求确认正常行为与一次性限制。
4. **建立正反对照**。使用少量并发和同步起点测试关键窗口，限制请求数量。
5. **确认影响与边界**。每次测试前恢复干净状态，记录所有响应和最终对象状态。
6. **检查编码与多层解析**。分别观察浏览器、代理、Web 服务器、框架和应用对输入的解码、归一化与类型转换。
7. **脚本化并从新会话复现**。把登录、token 获取、请求发送、结果判定和失败处理写入脚本，使用新会话重复。

23.4　分析时持续追问的问题

- 当前观察是原始事实，还是工具给出的推断?
- 哪个最小输入变化能够区分主要假设?
- 结果能否在相同输入和干净环境中重复?
- 失败时是否保存了完整原始输出?
- 题目条件变化后，当前方法的前提是否仍成立?

23.5　命令、脚本与验证片段

以下内容只用于本地题目目录、回环地址、赛事提供的实例或明确授权的实验环境。每次运行都应保留原始输入和完整输出。

```
# Bounded local concurrency test with two requests.
python3 - <<'PYCODE'
import concurrent.futures, requests
URL='http://127.0.0.1:8000/redeem'
COOKIES={'session':'LOCAL_TEST_TOKEN'}
def one(i):
```

```
    r=requests.post(URL,json={'code':'CTF-DEMO'},cookies=COOKIES,timeout=3)
    return i,r.status_code,r.text[:120]
with concurrent.futures.ThreadPoolExecutor(max_workers=2) as ex:
    print(list(ex.map(one,range(2))))
PYCODE
```

Listing 23.1: 本章常用命令与脚本

23.6 证据保存与结果判定

保存未经编辑的原始请求与响应、会话状态、角色、时间、重定向链、响应长度和关键差异。若使用浏览器，还要保存开发者工具网络记录与相关客户端代码。最终证明应包含一组可重复的正反对照，说明变化来自被测试字段，而不是缓存、随机数或登录状态。

23.7 常见失误与修正方法

- 未建立顺序基线就直接高并发。
- 请求数量过大导致服务异常而非竞态。
- 只看两个响应，没有查询最终状态。
- 测试间未重置账户或对象。
- 把普通业务设计选择当成安全漏洞。

23.8 小型演练

题目场景

兑换码正常只能使用一次。两个并发请求偶尔都返回成功，但最终余额只增加一次。是否构成题目所需竞态。

同时保存两个响应、兑换记录和最终余额。若只有响应错误但持久状态保持一次，影响可能只是状态报告不一致；若积分、商品或记录实际重复，则不变量被破坏。重复少量测试并重置账户，统计成功条件。题目要求拿到 flag 时还要确认 flag 触发依据是响应还是最终状态。

23.9　完成检查

交付与复盘检查表

- ☐ 入口、输入、目标输出与限制条件已经写入笔记。
- ☐ 建立了可重复的基线输入与输出。
- ☐ 每个关键结论都有原始证据与验证步骤。
- ☐ 脚本能够识别失败，不会把异常输出当成成功。
- ☐ 从干净目录或新容器运行时能够得到同样结果。

第二十四章　Web 综合案例：从侦察到稳定复现

本章目标

综合题通常不是单一 payload，而是多阶段链条，例如从客户端源码发现隐藏接口，经访问控制缺失读取配置，再利用内部请求或模板渲染取得 flag。本章强调每阶段独立证明和最小化。

24.1　原理与题型边界

漏洞链的强度取决于最弱证据。每一步都要明确输入、前置状态、输出和下一步所需信息。若某一步依赖旧会话、缓存、随机实例或手工复制，整条链就不稳定。

24.2　识别线索与优先动作

观察线索	可能含义	优先动作
前端脚本暴露隐藏 API	可作为第一阶段入口	保存源码位置和最小请求
接口返回内部 URL 或对象 id	可为下一阶段提供可控参数	验证数据来源和权限
内部服务响应经主应用回显	可能形成 SSRF 或代理链	使用本地标记证明每一跳
最终页面模板渲染可控字段	可能完成题目目标	用无害表达式先证明求值

24.3　标准解题步骤

1. **保存原始请求基线**。记录方法、路径、查询、头部、Cookie、消息体、重定向、状态码、长度和稳定响应片段。
2. **定位输入到危险点的数据流**。把每个阶段写成独立函数，输入输出明确。
3. **一次只改变一个条件**。为每阶段建立正反对照和失败判定。
4. **建立正反对照**。减少不必要请求，避免把侦察噪声放入最终脚本。
5. **确认影响与边界**。从新实例、新账户和空目录执行整条链。
6. **检查编码与多层解析**。分别观察浏览器、代理、Web 服务器、框架和应用对输入的解码、归一化与类型转换。
7. **脚本化并从新会话复现**。把登录、token 获取、请求发送、结果判定和失败处理写入脚本，使用新会话重复。

24.4　分析时持续追问的问题

- 当前观察是原始事实，还是工具给出的推断?
- 哪个最小输入变化能够区分主要假设?
- 结果能否在相同输入和干净环境中重复?
- 失败时是否保存了完整原始输出?
- 题目条件变化后，当前方法的前提是否仍成立?

24.5　命令、脚本与验证片段

以下内容只用于本地题目目录、回环地址、赛事提供的实例或明确授权的实验环境。每次运行都应保留原始输入和完整输出。

```
# Skeleton for a local multi-stage challenge.
python3 - <<'PYCODE'
import requests
S=requests.Session(); BASE='http://127.0.0.1:8000'
r=S.get(BASE+'/static/app.js',timeout=3); r.raise_for_status()
assert '/api/preview' in r.text
```

```
r=S.get(BASE+'/api/preview',params={'id':'demo'},timeout=3); r.raise_for_status()
internal=r.json()['source_url']; print('stage1',internal)
r=S.post(BASE+'/fetch',json={'url':internal},timeout=3); r.raise_for_status()
print('stage2',r.text[:160])
PYCODE
```

Listing 24.1: 本章常用命令与脚本

24.6 证据保存与结果判定

保存未经编辑的原始请求与响应、会话状态、角色、时间、重定向链、响应长度和关键差异。若使用浏览器，还要保存开发者工具网络记录与相关客户端代码。最终证明应包含一组可重复的正反对照，说明变化来自被测试字段，而不是缓存、随机数或登录状态。

24.7 常见失误与修正方法

- 把所有阶段写在一个无法定位错误的长请求中。
- 只保存最终 flag，没有保存中间证据。
- 把偶然的缓存响应当成阶段成功。
- 链条中使用过期 token 或硬编码实例地址。
- 复现时没有检查每阶段返回类型和字段。

24.8 小型演练

题目场景

app.js 暴露 /api/preview，接口返回内部 report URL，主站 fetch 功能可读取该 URL，报告标题又进入模板。设计稳定链条。

第一阶段保存 app.js 哈希和接口路径。第二阶段用普通用户调用 preview，记录内部 URL 与对象 id。第三阶段以唯一标记验证 fetch 请求确由服务器发起并回显。第四阶段使用无害算术证明标题被服务器模板求值，再读取题目提供的 flag 变

量。脚本为每阶段检查状态码、Content-Type 和字段，失败立即停止。最后用新实例完整运行并保存日志。

24.9　完成检查

交付与复盘检查表

- ☐ 入口、输入、目标输出与限制条件已经写入笔记。
- ☐ 建立了可重复的基线输入与输出。
- ☐ 每个关键结论都有原始证据与验证步骤。
- ☐ 脚本能够识别失败，不会把异常输出当成成功。
- ☐ 从干净目录或新容器运行时能够得到同样结果。

第三部分

密码学题型

第二十五章　密码学解题流程、模运算与数论基础

本章目标

密码学题首先是数据建模题。需要分清编码、压缩、加密、哈希与签名，明确整数位长、模数、群、随机数和已知关系，再选择对应算法。很多失败来自数据类型与数学模型不一致。

25.1　原理与题型边界

模运算中的等式在模数下成立。逆元存在当且仅当两个数互素；中国剩余定理把多个互素模数下的同余合并；扩展欧几里得求逆；快速幂计算大指数。所有算法都应同时检查成立条件。

25.2　识别线索与优先动作

观察线索	可能含义	优先动作
给出大量大整数 n、e、c	可能是 RSA 或模幂模型	记录位长、gcd 和参数关系
多个样本共享模数或随机参数	重复结构可能破坏安全假设	两两计算 gcd 或差分
输出像十六进制但长度异常	可能有前导零、位宽或编码层	保留固定字节长度再解码
题面强调 prime、inverse、congruence	需要直接建立数论等式	先用小参数手算验证模型

25.3 标准解题步骤

1. **抄录参数与数据类型**。把整数、字节、十六进制、模数、曲线、随机数和编码层分别记录，避免在字符串与整数之间混淆。
2. **检查基本约束**。检查 gcd、逆元存在性、范围、素性和模数是否互素。
3. **寻找重复与关系**。比较多个样本的模数、指数、nonce、前缀、长度和差值。
4. **建立最小数学模型**。把题面算法逐行翻译成数学符号，标出未知量和方程数量。
5. **先求可验证的中间量**。先验证一个逆元、一个同余或一个小样本，再扩展到完整数据。
6. **实现并加入断言**。脚本对范围、同余、长度、编码和重新加密结果做断言，不只打印候选。
7. **验证并还原字节**。将数学结果按明确位宽和端序转成字节，检查填充、结构和 flag 格式。

25.4 分析时持续追问的问题

- 当前观察是原始事实，还是工具给出的推断?
- 哪个最小输入变化能够区分主要假设?
- 结果能否在相同输入和干净环境中重复?
- 失败时是否保存了完整原始输出?
- 题目条件变化后，当前方法的前提是否仍成立?

25.5 常用公式

扩展欧几里得给出 $ax + ny = \gcd(a, n)$。当 $\gcd(a, n) = 1$ 时，x 是 a 在模 n 下的逆元。中国剩余定理在模数两两互素时给出模 $N = \prod n_i$ 的唯一解。

25.6　命令、脚本与验证片段

以下内容只用于本地题目目录、回环地址、赛事提供的实例或明确授权的实验环境。每次运行都应保留原始输入和完整输出。

```
from math import gcd

def egcd(a,b):
    if b==0: return a,1,0
    g,x1,y1=egcd(b,a%b)
    return g,y1,x1-(a//b)*y1

def invmod(a,n):
    g,x,_=egcd(a,n)
    if g!=1: raise ValueError(f'no inverse: gcd={g}')
    return x%n

n=101*113; a=37
print('gcd',gcd(a,n),'inverse',invmod(a,n))
assert (a*invmod(a,n))%n==1
print('pow',pow(7,12345,n))
```

Listing 25.1: 本章常用命令与脚本

25.7　证据保存与结果判定

保存题目原始参数、每个整数的进制与位长、使用的数学关系、脚本版本、候选数量和所有验证等式。对随机性问题保存样本顺序与时间；对加密题保存重新加密或重新签名得到原始数据的检查。

25.8　常见失误与修正方法

- 在没有检查 gcd 的情况下直接求逆。
- 把十六进制字符串当作十进制整数。
- 忽略前导零和固定块长度。
- 公式能算出一个值就认为模型正确。
- 使用浮点数处理大整数根或同余。

25.9　小型演练

题目场景

已知两个大整数 a、n 和值 y=a*x mod n。求 x，并说明何时无唯一解。

先计算 gcd(a,n)。若为 1，使用扩展欧几里得得到 a 的模逆，x=y*a^{-1} mod n，并重新代入验证。若 gcd 大于 1，需要检查 y 是否可被该 gcd 整除；可能无解或有多个同余类，不能直接调用逆元函数。最终按题面位宽转回字节。

25.10　完成检查

交付与复盘检查表

- □ 入口、输入、目标输出与限制条件已经写入笔记。
- □ 建立了可重复的基线输入与输出。
- □ 每个关键结论都有原始证据与验证步骤。
- □ 脚本能够识别失败，不会把异常输出当成成功。
- □ 从干净目录或新容器运行时能够得到同样结果。

第二十六章　古典密码、替换、移位与频率分析

本章目标

古典密码题常包含 Caesar、仿射、Vigenère、单表替换、栅栏、列置换和转轮式变换。关键是识别字母表、保留字符、密钥周期与变换方向，而不是盲目调用所有解码器。

26.1　原理与题型边界

移位和仿射密码在有限字母表上做线性变换；Vigenère 使用周期密钥；置换只改变位置而不改变字符频率；替换改变符号但通常保留语言统计。题目可能使用自定义字母表、大小写或字节范围。

26.2　识别线索与优先动作

观察线索	可能含义	优先动作
字母频率类似自然语言但字符被替换	可能是单表替换或移位	统计频率、重复模式和单词长度
每隔固定距离出现相似关系	可能是周期密钥	做重合指数与周期分组
字符集合不变但顺序混乱	可能是置换或栅栏	检查长度因子与二维排列
已知明文与密文字符呈线性关系	可能是仿射变换	解两个同余求参数并验证全部样本

26.3 标准解题步骤

1. **抄录参数与数据类型**。把整数、字节、十六进制、模数、曲线、随机数和编码层分别记录，避免在字符串与整数之间混淆。
2. **检查基本约束**。确定字母表、模数、大小写、空格和标点是否参与变换。
3. **寻找重复与关系**。先测试保持频率的置换与改变频率的替换特征。
4. **建立最小数学模型**。使用已知前缀、语言得分和题面提示缩小密钥空间。
5. **先求可验证的中间量**。在完整文本上重新加密，确认所有字符都匹配。
6. **实现并加入断言**。脚本对范围、同余、长度、编码和重新加密结果做断言，不只打印候选。
7. **验证并还原字节**。将数学结果按明确位宽和端序转成字节，检查填充、结构和 flag 格式。

26.4 分析时持续追问的问题

- 当前观察是原始事实，还是工具给出的推断?
- 哪个最小输入变化能够区分主要假设?
- 结果能否在相同输入和干净环境中重复?
- 失败时是否保存了完整原始输出?
- 题目条件变化后，当前方法的前提是否仍成立?

26.5 命令、脚本与验证片段

以下内容只用于本地题目目录、回环地址、赛事提供的实例或明确授权的实验环境。每次运行都应保留原始输入和完整输出。

```
from collections import Counter
import string
ALPH=string.ascii_uppercase
ct='KHOORZRUOG'
print(Counter(ct).most_common())
for k in range(26):
```

```
    pt=''.join(ALPH[(ALPH.index(c)-k)%26] if c in ALPH else c for c in ct)
    print(k,pt)

def affine_dec(c,a_inv,b):
    return ALPH[(a_inv*(ALPH.index(c)-b))%26]
```

Listing 26.1: 本章常用命令与脚本

26.6　证据保存与结果判定

保存题目原始参数、每个整数的进制与位长、使用的数学关系、脚本版本、候选数量和所有验证等式。对随机性问题保存样本顺序与时间；对加密题保存重新加密或重新签名得到原始数据的检查。

26.7　常见失误与修正方法

- 默认字母表为 26 个大写字母。
- 只凭一句可读片段选择密钥。
- 置换题中忘记保留填充位置。
- 频率分析忽略文本过短导致的噪声。
- 解密成功后没有重新加密验证。

26.8　小型演练

题目场景

密文仅含大写字母，题面给出 flag 前缀对应的三个密文字母。判断是否为仿射密码。

把已知明密文对转换为 0 至 25 的整数，建立 c=a*m+b mod 26。用两组差分求 a，但先检查差值是否可逆。求得 a、b 后用第三组已知字符验证，再解全部文本。若差值不可逆，枚举满足方程的少量 a 并用所有已知对筛选。

26.9 完成检查

交付与复盘检查表

- ☐ 入口、输入、目标输出与限制条件已经写入笔记。
- ☐ 建立了可重复的基线输入与输出。
- ☐ 每个关键结论都有原始证据与验证步骤。
- ☐ 脚本能够识别失败，不会把异常输出当成成功。
- ☐ 从干净目录或新容器运行时能够得到同样结果。

第二十七章　XOR、重复密钥与流密码误用

本章目标

XOR 题覆盖单字节 XOR、重复密钥 XOR、已知明文、双重加密抵消和流密码密钥流复用。XOR 可逆且线性，核心关系是 c=p xor k。任何两个量已知即可求第三个。

27.1　原理与题型边界

重复密钥会在固定周期重复同一密钥字节。流密码或 CTR 模式若重复 nonce，两个密文异或会消去密钥流，得到两个明文异或。自然语言、格式头和 flag 前缀可提供拖拽猜测，但每个猜测必须回到所有样本验证。

27.2　识别线索与优先动作

观察线索	可能含义	优先动作
密文长度与明文相同且无明显块结构	可能是 XOR 或流密码	检查已知前缀与逐字节关系
多个密文长度不同但前缀关系异常	可能复用同一密钥流	对齐后两两异或
每隔固定长度统计特征重复	可能是重复密钥 XOR	用归一化汉明距离估计周期
同一数据 XOR 两次恢复原文	利用了 XOR 自反性	写出运算顺序并化简

27.3 标准解题步骤

1. **抄录参数与数据类型**。把整数、字节、十六进制、模数、曲线、随机数和编码层分别记录，避免在字符串与整数之间混淆。
2. **检查基本约束**。明确 XOR 的字节对齐、密钥周期和截断规则。
3. **寻找重复与关系**。使用已知文件头、JSON 语法或 flag 前缀恢复局部密钥流。
4. **建立最小数学模型**。对所有密文应用候选密钥流，检查是否同时产生合理结构。
5. **先求可验证的中间量**。避免只按可打印率评分，加入语法、字典和校验约束。
6. **实现并加入断言**。脚本对范围、同余、长度、编码和重新加密结果做断言，不只打印候选。
7. **验证并还原字节**。将数学结果按明确位宽和端序转成字节，检查填充、结构和 flag 格式。

27.4 分析时持续追问的问题

- 当前观察是原始事实，还是工具给出的推断?
- 哪个最小输入变化能够区分主要假设?
- 结果能否在相同输入和干净环境中重复?
- 失败时是否保存了完整原始输出?
- 题目条件变化后，当前方法的前提是否仍成立?

27.5 命令、脚本与验证片段

以下内容只用于本地题目目录、回环地址、赛事提供的实例或明确授权的实验环境。每次运行都应保留原始输入和完整输出。

```
from itertools import cycle

def xor(a,b): return bytes(x^y for x,y in zip(a,b))
def repeat_xor(data,key): return bytes(x^y for x,y in zip(data,cycle(key)))
ct=bytes.fromhex('2f0d0a0c4d')
known=b'flag{'
```

```
ks=xor(ct,known)
print('keystream',ks.hex())
print('roundtrip',repeat_xor(repeat_xor(b'example',b'ICE'),b'ICE'))

def hamming(a,b): return sum((x^y).bit_count() for x,y in zip(a,b))
```

Listing 27.1: 本章常用命令与脚本

27.6　证据保存与结果判定

保存题目原始参数、每个整数的进制与位长、使用的数学关系、脚本版本、候选数量和所有验证等式。对随机性问题保存样本顺序与时间；对加密题保存重新加密或重新签名得到原始数据的检查。

27.7　常见失误与修正方法

- 把十六进制字符串的字符逐个 XOR，而不是先转字节。
- 不同长度数据用 zip 后静默截断。
- 仅凭可打印率选择密钥。
- 已知明文位置错一字节导致整段密钥流错误。
- 把重复密钥 XOR 与安全随机一次一密混为一谈。

27.8　小型演练

题目场景

两个 CTR 密文使用同一 nonce，已知第一个明文前 16 字节。恢复第二个明文对应部分。

由 c1=p1 xor ks 得到前 16 字节密钥流 ks=c1 xor p1，再计算 p2=c2 xor ks。检查第二个明文的结构和编码。如果需要扩展，使用第二个明文中可靠格式继续恢复后续密钥流，但每一步标明位置和置信度。不能从局部恢复直接声称得到完整密钥。

27.9 完成检查

交付与复盘检查表

- ☐ 入口、输入、目标输出与限制条件已经写入笔记。
- ☐ 建立了可重复的基线输入与输出。
- ☐ 每个关键结论都有原始证据与验证步骤。
- ☐ 脚本能够识别失败，不会把异常输出当成成功。
- ☐ 从干净目录或新容器运行时能够得到同样结果。

第二十八章　哈希、MAC、密码存储与长度扩展

本章目标

哈希题可能涉及碰撞、前像、截断、弱校验、Merkle-Damgård 长度扩展、HMAC 误用和密码哈希。首先区分无密钥哈希、带密钥 MAC、数字签名与密码派生。

28.1　原理与题型边界

普通 hash(secret || message) 对某些迭代哈希可能受长度扩展影响, HMAC 的内外两层结构不受此问题。截断哈希降低穷举复杂度；无盐快速哈希适合离线字典，而专用密码哈希通过盐和成本参数增加代价。

28.2　识别线索与优先动作

观察线索	可能含义	优先动作
签名长度固定且等于 MD5/SHA 摘要	可能是简单哈希或 MAC	修改消息并观察服务器错误
服务端验证 hash(secret||msg)	可能存在长度扩展条件	确认算法、秘密长度范围与原始填充
只比较摘要前若干位	有效安全强度被截断	计算需要枚举的期望规模
相同密码产生相同摘要且无盐	可能可用题目字典离线验证	先确认哈希算法和编码

28.3 标准解题步骤

1. **抄录参数与数据类型**。把整数、字节、十六进制、模数、曲线、随机数和编码层分别记录，避免在字符串与整数之间混淆。
2. **检查基本约束**。确认算法、消息字节、连接顺序、编码、盐与密钥位置。
3. **寻找重复与关系**。判断目标是碰撞、前像、第二前像还是伪造 MAC。
4. **建立最小数学模型**。计算实际比较位数和搜索空间，不盲目暴力。
5. **先求可验证的中间量**。对候选重新计算服务器相同格式的摘要并验证。
6. **实现并加入断言**。脚本对范围、同余、长度、编码和重新加密结果做断言，不只打印候选。
7. **验证并还原字节**。将数学结果按明确位宽和端序转成字节，检查填充、结构和 flag 格式。

28.4 分析时持续追问的问题

- 当前观察是原始事实，还是工具给出的推断?
- 哪个最小输入变化能够区分主要假设?
- 结果能否在相同输入和干净环境中重复?
- 失败时是否保存了完整原始输出?
- 题目条件变化后，当前方法的前提是否仍成立?

28.5 命令、脚本与验证片段

以下内容只用于本地题目目录、回环地址、赛事提供的实例或明确授权的实验环境。每次运行都应保留原始输入和完整输出。

```
import hashlib, hmac
msg=b'action=view&file=note'
key=b'local-demo-key'
print('sha256',hashlib.sha256(msg).hexdigest())
print('prefix-mac',hashlib.sha256(key+msg).hexdigest())
print('hmac',hmac.new(key,msg,hashlib.sha256).hexdigest())
```

```
# Constant-time comparison for a local verifier.
tag=hmac.new(key,msg,hashlib.sha256).digest()
assert hmac.compare_digest(tag,hmac.new(key,msg,hashlib.sha256).digest())
```

Listing 28.1: 本章常用命令与脚本

28.6　证据保存与结果判定

保存题目原始参数、每个整数的进制与位长、使用的数学关系、脚本版本、候选数量和所有验证等式。对随机性问题保存样本顺序与时间；对加密题保存重新加密或重新签名得到原始数据的检查。

28.7　常见失误与修正方法

- 把哈希输出称为加密结果。
- 没有确认消息的精确字节和参数顺序。
- 对 HMAC 错误应用长度扩展。
- 忽略摘要截断后的实际难度。
- 在远程服务上进行无界字典攻击。

28.8　小型演练

题目场景

本地接口 token=SHA256(secret || query), query 与 token 可见, secret 长度在 8 到 16。判断是否适合长度扩展。

确认算法确为原始 SHA-256 前缀 MAC, 服务器直接对 secret 与原始 query 字节拼接。对每个候选秘密长度构造正确 Merkle-Damgård 填充和新增字段，计算延续状态。每个候选只向本地接口验证一次。若实现使用 HMAC、先解码参数后再哈希或重新排序字段，则模型不成立，必须重新检查字节序列。

28.9 完成检查

交付与复盘检查表

- ☐ 入口、输入、目标输出与限制条件已经写入笔记。
- ☐ 建立了可重复的基线输入与输出。
- ☐ 每个关键结论都有原始证据与验证步骤。
- ☐ 脚本能够识别失败，不会把异常输出当成成功。
- ☐ 从干净目录或新容器运行时能够得到同样结果。

第二十九章　RSA 基础：参数、填充与正确实现

本章目标

RSA 题首先检查 n、e、c、p、q、phi、d 的关系，再判断是否使用 OAEP、PKCS#1 v1.5 或无填充。很多 CTF 弱点来自参数生成与填充，而不是求解任意大整数分解。

29.1　原理与题型边界

RSA 在模 n 下计算 c=m^e，解密 m=c^d。私钥指数满足 ed=1 mod lambda(n) 或 phi(n)。安全加密必须使用随机填充；直接对短消息做模幂会暴露代数结构。消息整数必须小于 n。

29.2　识别线索与优先动作

观察线索	可能含义	优先动作
题目直接给出 p、q 或 phi	可直接计算私钥	验证 p*q=n 后求 d
e 很小且消息短、无填充	可能满足 m^e<n	尝试精确整数 e 次根并验证
多个模数共享素因子	随机数生成失败	两两 gcd 查找因子
解密整数无法直接显示	可能包含填充或固定长度前导零	按模数字节长度恢复并解析

29.3 标准解题步骤

1. **抄录参数与数据类型**。把整数、字节、十六进制、模数、曲线、随机数和编码层分别记录，避免在字符串与整数之间混淆。
2. **检查基本约束**。检查 n 位长、e、gcd(e,phi) 和消息范围。
3. **寻找重复与关系**。查找给定参数、共享因子、泄露位和无填充线索。
4. **建立最小数学模型**。先在小参数上验证公式与字节转换。
5. **先求可验证的中间量**。解密后按填充规范解析，不用 strip 随意删除零字节。
6. **实现并加入断言**。脚本对范围、同余、长度、编码和重新加密结果做断言，不只打印候选。
7. **验证并还原字节**。将数学结果按明确位宽和端序转成字节，检查填充、结构和 flag 格式。

29.4 分析时持续追问的问题

- 当前观察是原始事实，还是工具给出的推断?
- 哪个最小输入变化能够区分主要假设?
- 结果能否在相同输入和干净环境中重复?
- 失败时是否保存了完整原始输出?
- 题目条件变化后，当前方法的前提是否仍成立?

29.5 RSA 关系式

$n = pq$，$\varphi(n) = (p-1)(q-1)$，私钥满足 $ed \equiv 1 \pmod{\lambda(n)}$。无填充时 $c \equiv m^e \pmod{n}$。

29.6 命令、脚本与验证片段

以下内容只用于本地题目目录、回环地址、赛事提供的实例或明确授权的实验环境。每次运行都应保留原始输入和完整输出。

```python
from math import gcd
p=1000003; q=1000033; n=p*q; e=65537
phi=(p-1)*(q-1)
assert gcd(e,phi)==1
d=pow(e,-1,phi)
m=int.from_bytes(b'CTF','big')
c=pow(m,e,n)
assert pow(c,d,n)==m
k=(n.bit_length()+7)//8
print(pow(c,d,n).to_bytes(k,'big'))
```

Listing 29.1: 本章常用命令与脚本

29.7　证据保存与结果判定

保存题目原始参数、每个整数的进制与位长、使用的数学关系、脚本版本、候选数量和所有验证等式。对随机性问题保存样本顺序与时间；对加密题保存重新加密或重新签名得到原始数据的检查。

29.8　常见失误与修正方法

- 没有验证 p*q=n。
- 使用 phi 与 lambda 时混淆条件。
- 把浮点根当作精确整数根。
- 随意去除解密结果的零字节。
- 忽略 OAEP 或 PKCS#1 填充导致的结构。

29.9　小型演练

题目场景

题目给出 p、q、e、c。编写脚本恢复明文并处理前导零。

验证 p 与 q 的乘积等于 n，计算 phi 或 lambda，确认 e 与其互素，求 d。计算 m=c^d mod n，再以模数字节长度 k 固定转换为字节。若题面说明无填充，可定

位消息长度；若有 PKCS#1 结构，应按格式检查前缀和分隔符，而不是简单 lstrip。最后重新加密验证 c。

29.10　完成检查

交付与复盘检查表

- ☐ 入口、输入、目标输出与限制条件已经写入笔记。
- ☐ 建立了可重复的基线输入与输出。
- ☐ 每个关键结论都有原始证据与验证步骤。
- ☐ 脚本能够识别失败，不会把异常输出当成成功。
- ☐ 从干净目录或新容器运行时能够得到同样结果。

第三十章　RSA 常见弱点：共模、广播、低私钥与部分泄露

本章目标

这一章覆盖 CTF 中常见的 RSA 结构性弱点：共享素因子、共模攻击、低指数广播、Wiener 低私钥、p 与 q 接近、部分比特泄露和相关消息。选择方法前必须核对严格前提。

30.1　原理与题型边界

共模攻击要求同一 n、同一 m、互素指数；Håstad 广播要求同一未填充消息、相同小指数和足够多互素模数；Wiener 要求 d 异常小；Fermat 适合 p、q 接近。条件不满足时公式可能给出貌似合理但错误的整数。

30.2　识别线索与优先动作

观察线索	可能含义	优先动作
多个 n 两两 gcd 非 1	存在共享素因子	计算 gcd 并验证因子
同一 n 下有两个互素 e 与对应 c	可能共模攻击	用扩展欧几里得组合指数
e=3 且同一消息发给至少三个互素 n	可能广播攻击	CRT 合并后取精确立方根
d 可能很小或 e/n 连分数异常	可能 Wiener 攻击	检查收敛分数给出的 phi 是否有效

30.3 标准解题步骤

1. **抄录参数与数据类型**。把整数、字节、十六进制、模数、曲线、随机数和编码层分别记录，避免在字符串与整数之间混淆。
2. **检查基本约束**。对所有模数构建 gcd 矩阵和位长表。
3. **寻找重复与关系**。逐项检查攻击前提并写入断言。
4. **建立最小数学模型**。求出候选因子、消息或私钥后，必须重新加密验证所有样本。
5. **先求可验证的中间量**。对部分泄露问题先估计未知位数，再决定枚举、格或代数方法。
6. **实现并加入断言**。脚本对范围、同余、长度、编码和重新加密结果做断言，不只打印候选。
7. **验证并还原字节**。将数学结果按明确位宽和端序转成字节，检查填充、结构和 flag 格式。

30.4 分析时持续追问的问题

- 当前观察是原始事实，还是工具给出的推断?
- 哪个最小输入变化能够区分主要假设?
- 结果能否在相同输入和干净环境中重复?
- 失败时是否保存了完整原始输出?
- 题目条件变化后，当前方法的前提是否仍成立?

30.5 命令、脚本与验证片段

以下内容只用于本地题目目录、回环地址、赛事提供的实例或明确授权的实验环境。每次运行都应保留原始输入和完整输出。

```
from math import gcd, isqrt
mods=[int(x) for x in open('work/moduli.txt')]
for i in range(len(mods)):
```

```
    for j in range(i):
        g=gcd(mods[i],mods[j])
        if 1<g<mods[i]: print('shared factor',i,j,g)

def iroot3(n):
    lo,hi=0,1
    while hi**3<n: hi*=2
    while lo+1<hi:
        mid=(lo+hi)//2
        if mid**3<n: lo=mid
        else: hi=mid
    return hi,hi**3==n
```

Listing 30.1: 本章常用命令与脚本

30.6　证据保存与结果判定

保存题目原始参数、每个整数的进制与位长、使用的数学关系、脚本版本、候选数量和所有验证等式。对随机性问题保存样本顺序与时间；对加密题保存重新加密或重新签名得到原始数据的检查。

30.7　常见失误与修正方法

- 看到 e=3 就直接取 c 的立方根，忽略模约简。
- 共模攻击时指数不互素。
- CRT 模数不互素却套用普通公式。
- Wiener 候选没有验证 p、q 判别式。
- 部分泄露问题未知空间过大仍直接暴力。

30.8　小型演练

题目场景

三个互素模数使用 e=3 加密同一无填充短消息。恢复明文。

先验证三个模数两两互素和密文范围。使用 CRT 合并 c_i=m^3 mod n_i，得到模 N 的值。若消息足够短，m^3<N，合并结果就是普通整数 m^3。取精确整数立方根，要求余数为零，再把 m 转字节并对三个模数重新加密。若根不精确，说明消息、填充或前提不符。

30.9 完成检查

交付与复盘检查表

- ☐ 入口、输入、目标输出与限制条件已经写入笔记。
- ☐ 建立了可重复的基线输入与输出。
- ☐ 每个关键结论都有原始证据与验证步骤。
- ☐ 脚本能够识别失败，不会把异常输出当成成功。
- ☐ 从干净目录或新容器运行时能够得到同样结果。

第三十一章　椭圆曲线、ECDSA/DSA 与 nonce 重用

本章目标

椭圆曲线题需要明确曲线方程、有限域、基点、阶和点表示。签名题最常见弱点是 nonce 重用、nonce 偏差、私钥与 nonce 关系或错误验证。

31.1　原理与题型边界

在有限域上的点加法构成群。ECDSA 签名中 r 来自 kG，s=k^{-1}(z+rd) mod q。若同一私钥重复使用 k，两个签名的 r 相同，可由 s 差和消息哈希差恢复 k，再求私钥 d。所有运算在子群阶 q 下。

31.2　识别线索与优先动作

观察线索	可能含义	优先动作
两个签名 r 相同但消息不同	很可能复用了 nonce k	确认哈希 z 与曲线阶后解线性方程
nonce 来自短时间戳或小范围随机数	搜索空间可能可控	先估算范围并本地验证公钥
给出点坐标但不在曲线上	编码、域或曲线参数可能读错	代入曲线方程检查
验签只比较部分坐标或忽略范围	验证器实现可能错误	建立合法与边界签名矩阵

31.3 标准解题步骤

1. **抄录参数与数据类型**。把整数、字节、十六进制、模数、曲线、随机数和编码层分别记录，避免在字符串与整数之间混淆。
2. **检查基本约束**。记录域素数 p、曲线参数、基点 G、阶 q、公钥和签名编码。
3. **寻找重复与关系**。检查点在曲线上、阶关系和 r、s 范围。
4. **建立最小数学模型**。对重复 r、相关 k 或已知 k 建立模 q 方程。
5. **先求可验证的中间量**。恢复私钥后计算 dG 与给定公钥比较，并重新签名或验签。
6. **实现并加入断言**。脚本对范围、同余、长度、编码和重新加密结果做断言，不只打印候选。
7. **验证并还原字节**。将数学结果按明确位宽和端序转成字节，检查填充、结构和 flag 格式。

31.4 分析时持续追问的问题

- 当前观察是原始事实，还是工具给出的推断?
- 哪个最小输入变化能够区分主要假设?
- 结果能否在相同输入和干净环境中重复?
- 失败时是否保存了完整原始输出?
- 题目条件变化后，当前方法的前提是否仍成立?

31.5 签名方程

ECDSA 中 $s \equiv k^{-1}(z + rd) \pmod{q}$。当两个不同消息重复使用 k 时，$k \equiv (z_1 - z_2)(s_1 - s_2)^{-1} \pmod{q}$。

31.6　命令、脚本与验证片段

以下内容只用于本地题目目录、回环地址、赛事提供的实例或明确授权的实验环境。每次运行都应保留原始输入和完整输出。

```
# ECDSA nonce reuse algebra with toy integers.
q=10177
r=4242; s1=3131; s2=8181
z1=1234; z2=6789
k=((z1-z2)*pow((s1-s2)%q,-1,q))%q
d=((s1*k-z1)*pow(r,-1,q))%q
print('k',k,'d',d)
assert (s1*k-z1-r*d)%q==0
assert (s2*k-z2-r*d)%q==0
```

Listing 31.1: 本章常用命令与脚本

31.7　证据保存与结果判定

保存题目原始参数、每个整数的进制与位长、使用的数学关系、脚本版本、候选数量和所有验证等式。对随机性问题保存样本顺序与时间；对加密题保存重新加密或重新签名得到原始数据的检查。

31.8　常见失误与修正方法

- 在域素数 p 下而不是子群阶 q 下求签名逆元。
- 只凭 r 相同就忽略消息是否不同。
- 恢复 d 后没有与公钥比较。
- 点坐标编码与压缩格式解析错误。
- 使用浮点数处理有限域点。

31.9 小型演练

题目场景

两个 ECDSA 签名具有相同 r，已知消息和 q。恢复私钥并验证。

按签名所用哈希把消息转换为 z, 注意截断规则。计算 k=(z1-z2)/(s1-s2) mod q，再计算 d=(s1*k-z1)/r mod q。检查两个签名方程都成立，并使用曲线库计算 dG 与给定公钥相等。若 s1=s2 或消息哈希相同，需要重新判断是否只是重复签名。

31.10 完成检查

交付与复盘检查表

- □ 入口、输入、目标输出与限制条件已经写入笔记。
- □ 建立了可重复的基线输入与输出。
- □ 每个关键结论都有原始证据与验证步骤。
- □ 脚本能够识别失败，不会把异常输出当成成功。
- □ 从干净目录或新容器运行时能够得到同样结果。

第三十二章　对称加密模式、填充与预言机

本章目标

对称密码题需要区分分组密码与流模式，明确 IV、nonce、填充和认证。ECB 泄露相同块，CBC 具有可预测的按位修改关系，CTR nonce 复用等同密钥流复用，AEAD 同时提供机密性和完整性。

32.1　原理与题型边界

CBC 解密为 p_i=D(c_i) xor c_{i-1}，修改前一密文块会可控改变后一明文块。填充预言机通过不同错误泄露填充是否有效。任何预言机测试都应在本地题目中限速、缓存结果并使用多次确认。

32.2　识别线索与优先动作

观察线索	可能含义	优先动作
密文中出现重复 16 字节块	可能是 ECB 或重复输入块	按块分组并统计重复
修改前一块导致后一明文对应位翻转	符合 CBC 位翻转关系	用已知字段建立最小修改
相同 nonce 下 CTR 密文前缀关系异常	可能复用密钥流	两两异或并使用已知明文
无效密文返回填充与 MAC 不同错误	可能形成预言机	建立错误分类与重复验证

32.3 标准解题步骤

1. **抄录参数与数据类型**。把整数、字节、十六进制、模数、曲线、随机数和编码层分别记录，避免在字符串与整数之间混淆。
2. **检查基本约束**。确定算法、块长、模式、IV/nonce 位置、填充与认证顺序。
3. **寻找重复与关系**。按块显示十六进制，比较重复和变化传播。
4. **建立最小数学模型**。对位翻转写出目标明文、原明文和差分掩码。
5. **先求可验证的中间量**。对预言机使用本地接口、有限请求和稳定错误判定。
6. **实现并加入断言**。脚本对范围、同余、长度、编码和重新加密结果做断言，不只打印候选。
7. **验证并还原字节**。将数学结果按明确位宽和端序转成字节，检查填充、结构和 flag 格式。

32.4 分析时持续追问的问题

- 当前观察是原始事实，还是工具给出的推断?
- 哪个最小输入变化能够区分主要假设?
- 结果能否在相同输入和干净环境中重复?
- 失败时是否保存了完整原始输出?
- 题目条件变化后，当前方法的前提是否仍成立?

32.5 命令、脚本与验证片段

以下内容只用于本地题目目录、回环地址、赛事提供的实例或明确授权的实验环境。每次运行都应保留原始输入和完整输出。

```
from pathlib import Path
ct=Path('work/cipher.bin').read_bytes(); BS=16
blocks=[ct[i:i+BS] for i in range(0,len(ct),BS)]
for i,b in enumerate(blocks): print(i,b.hex())
print('duplicates',len(blocks)-len(set(blocks)))
# CBC bit-flip mask for a known local plaintext field.
```

```
old=b'role=user;......'; new=b'role=admin;.....'
mask=bytes(a^b for a,b in zip(old,new))
print(mask.hex())
```

Listing 32.1: 本章常用命令与脚本

32.6　证据保存与结果判定

保存题目原始参数、每个整数的进制与位长、使用的数学关系、脚本版本、候选数量和所有验证等式。对随机性问题保存样本顺序与时间；对加密题保存重新加密或重新签名得到原始数据的检查。

32.7　常见失误与修正方法

- 没有确认块长和 IV 存储位置。
- 把 CBC 位翻转当作能够任意解密。
- 预言机错误分类不稳定。
- 忽略 MAC 或 AEAD 会阻止篡改。
- CTR 中把 nonce 与计数器布局混淆。

32.8　小型演练

题目场景

CBC token 的已知明文块为 role=user，服务端不验证 MAC。修改密文使其变为 role=admin。

定位包含目标文本的明文块和前一密文块。计算 delta=old xor new，把 delta XOR 到前一密文块的对应位置。其他字节保持不变。提交前先在本地实现或已知密钥环境验证修改只影响目标块，并注意前一明文块会被不可预测地破坏。若目标位于第一块，需要修改 IV。

32.9 完成检查

交付与复盘检查表

- ☐ 入口、输入、目标输出与限制条件已经写入笔记。
- ☐ 建立了可重复的基线输入与输出。
- ☐ 每个关键结论都有原始证据与验证步骤。
- ☐ 脚本能够识别失败，不会把异常输出当成成功。
- ☐ 从干净目录或新容器运行时能够得到同样结果。

第三十三章　伪随机数、LCG、MT19937 与状态恢复

本章目标

随机数题常给出若干输出、时间范围或自定义生成器。解题目标是识别状态更新方程、输出变换、取模或截断，并判断观察数量是否足以恢复状态或预测下一值。

33.1　原理与题型边界

LCG 满足 x_{n+1}=a x_n+c mod m，连续输出可形成同余方程。MT19937 状态大且输出经过 tempering，但足够完整输出可以逆变换恢复状态。时间种子题需要估计真实时间窗口、单位和时区，不能无界枚举。

33.2　识别线索与优先动作

观察线索	可能含义	优先动作
连续整数满足近似线性关系	可能是 LCG 或线性递推	计算差分并建立模方程
输出为 32 位且数量很多	可能是 MT19937 或类似生成器	检查 tempering 和样本数量
种子来自当前时间	搜索空间由时间窗口决定	记录服务时间、单位和偏移
只看到输出低位或取模结果	状态信息被截断	估计约束数量再选择格或枚举

33.3 标准解题步骤

1. **抄录参数与数据类型**。把整数、字节、十六进制、模数、曲线、随机数和编码层分别记录，避免在字符串与整数之间混淆。
2. **检查基本约束**。按生成顺序保存原始输出，不做排序。
3. **寻找重复与关系**。写出状态更新、输出函数、位宽和取模。
4. **建立最小数学模型**。用少量样本求候选参数，再用未参与求解的样本验证。
5. **先求可验证的中间量**。预测下一输出前确认服务是否额外消耗随机数。
6. **实现并加入断言**。脚本对范围、同余、长度、编码和重新加密结果做断言，不只打印候选。
7. **验证并还原字节**。将数学结果按明确位宽和端序转成字节，检查填充、结构和 flag 格式。

33.4 分析时持续追问的问题

- 当前观察是原始事实，还是工具给出的推断?
- 哪个最小输入变化能够区分主要假设?
- 结果能否在相同输入和干净环境中重复?
- 失败时是否保存了完整原始输出?
- 题目条件变化后，当前方法的前提是否仍成立?

33.5 命令、脚本与验证片段

以下内容只用于本地题目目录、回环地址、赛事提供的实例或明确授权的实验环境。每次运行都应保留原始输入和完整输出。

```
from math import gcd
xs=[123,456,789,111]
# For a known modulus m, solve LCG a and c from three outputs.
m=1009
num=(xs[2]-xs[1])%m; den=(xs[1]-xs[0])%m
if gcd(den,m)==1:
```

```
    a=num*pow(den,-1,m)%m
    c=(xs[1]-a*xs[0])%m
    print(a,c,(a*xs[2]+c)%m)
```

Listing 33.1: 本章常用命令与脚本

33.6　证据保存与结果判定

保存题目原始参数、每个整数的进制与位长、使用的数学关系、脚本版本、候选数量和所有验证等式。对随机性问题保存样本顺序与时间；对加密题保存重新加密或重新签名得到原始数据的检查。

33.7　常见失误与修正方法

- 排序输出后破坏生成顺序。
- 只用参与拟合的样本验证模型。
- 忽略额外随机调用导致预测错位。
- 时间种子范围没有时区与单位依据。
- 把密码学安全随机数当作可由少量输出恢复。

33.8　小型演练

题目场景

已知 LCG 模数 m 和三个连续完整输出，求 a、c 并预测第四个。

由 x2-x1=a(x1-x0) mod m。若分母在模 m 下可逆，求 a，再求 c=x1-a*x0。若不可逆，需要解线性同余并得到多个候选。使用第四个已知样本筛选，最终预测第五个。所有下标和输出顺序都要与题面一致。

33.9 完成检查

交付与复盘检查表

- ☐ 入口、输入、目标输出与限制条件已经写入笔记。
- ☐ 建立了可重复的基线输入与输出。
- ☐ 每个关键结论都有原始证据与验证步骤。
- ☐ 脚本能够识别失败，不会把异常输出当成成功。
- ☐ 从干净目录或新容器运行时能够得到同样结果。

第三十四章　格、LLL、Coppersmith 与近似关系

本章目标

格方法用于把多个小误差、部分泄露或小根问题转化为短向量。CTF 常见场景包括 Hidden Number Problem、近似公因子、RSA 小根、背包和有界 nonce。格不是“把数字扔进 LLL”，矩阵缩放与变量边界决定结果。

34.1　原理与题型边界

格是整数向量的离散集合，LLL 找到较短且近似正交的基。构造目标是让正确未知量对应一个异常短向量或多项式小根。必须明确每一行表示什么、各坐标量级如何平衡，以及恢复结果如何验证。

34.2　识别线索与优先动作

观察线索	可能含义	优先动作
多个方程含同一秘密与小误差	可能形成 HNP 或近似关系	写出误差界与方程数量
RSA 已知 p 的高位或低位	未知部分较小	建立多项式 f(x)=0 mod n
nonce 只泄露若干位或有偏差	签名可形成格问题	把界和模 q 方程标准化
背包权重有特殊密度或结构	可尝试子集和格嵌入	先计算密度和目标范围

34.3 标准解题步骤

1. **抄录参数与数据类型**。把整数、字节、十六进制、模数、曲线、随机数和编码层分别记录，避免在字符串与整数之间混淆。
2. **检查基本约束**。把问题写成精确整数方程与未知量边界。
3. **寻找重复与关系**。估计变量量级并设计缩放，避免某坐标支配长度。
4. **建立最小数学模型**。在小型实例验证矩阵构造，再运行 LLL。
5. **先求可验证的中间量**。从候选向量恢复未知量，并代入所有原方程与公钥验证。
6. **实现并加入断言**。脚本对范围、同余、长度、编码和重新加密结果做断言，不只打印候选。
7. **验证并还原字节**。将数学结果按明确位宽和端序转成字节，检查填充、结构和 flag 格式。

34.4 分析时持续追问的问题

- 当前观察是原始事实，还是工具给出的推断?
- 哪个最小输入变化能够区分主要假设?
- 结果能否在相同输入和干净环境中重复?
- 失败时是否保存了完整原始输出?
- 题目条件变化后，当前方法的前提是否仍成立?

34.5 命令、脚本与验证片段

以下内容只用于本地题目目录、回环地址、赛事提供的实例或明确授权的实验环境。每次运行都应保留原始输入和完整输出。

```
# SageMath-style sketch for a local challenge.
# Run in Sage, not plain CPython.
# B = Matrix(ZZ, rows)
# R = B.LLL()
# for v in R.rows():
#     candidate = recover(v)
```

```
#     if verify(candidate): print(candidate)

# Always estimate bounds before building the lattice.
bounds={'unknown_bits':48,'equations':16,'modulus_bits':256}
print(bounds)
```

Listing 34.1: 本章常用命令与脚本

34.6　证据保存与结果判定

保存题目原始参数、每个整数的进制与位长、使用的数学关系、脚本版本、候选数量和所有验证等式。对随机性问题保存样本顺序与时间；对加密题保存重新加密或重新签名得到原始数据的检查。

34.7　常见失误与修正方法

- 没有写出未知量边界就开始调参数。
- 矩阵坐标量级差异巨大。
- LLL 返回短向量就直接当作秘密。
- 只在完整大实例调试，无法判断构造错误。
- 忘记符号、中心化或模数倍数。

34.8　小型演练

题目场景

多份 ECDSA 签名的 nonce 高位已知，低 16 位未知。说明格建模前要准备什么。

对每个签名写出 s_i k_i-z_i=r_i d mod q，把 k_i 表示为已知高位加有界低位。移项后得到共享 d 与小误差的同余关系。记录误差界、签名数量、q 位长和缩放因子，先用小 q 与更小未知位构造可验证实例。LLL 候选必须通过 dG=Q 和所有签名方程验证。

34.9 完成检查

交付与复盘检查表

- ☐ 入口、输入、目标输出与限制条件已经写入笔记。
- ☐ 建立了可重复的基线输入与输出。
- ☐ 每个关键结论都有原始证据与验证步骤。
- ☐ 脚本能够识别失败，不会把异常输出当成成功。
- ☐ 从干净目录或新容器运行时能够得到同样结果。

第三十五章　密码学综合案例：从参数审计到脚本验证

本章目标

综合密码题常叠加编码、随机数、RSA 或对称模式。正确流程是分层拆解：先还原字节与数据结构，再审计参数关系，最后选择最小数学攻击。每层都应可逆或可验证。

35.1　原理与题型边界

多阶段题容易因早期一字节错误导致后续数学看似失败。把编码、解压、参数解析、数学求解和明文解析写成独立函数，输出长度、位长和断言。

35.2　识别线索与优先动作

观察线索	可能含义	优先动作
附件包含 JSON 大整数和 Base64 密文	存在数据层与数学层	先规范化解析，不直接复制到计算器
多个 RSA 样本 n 不同但有共同结构	可能共享因子或广播	先做 gcd 矩阵和指数统计
恢复的整数前部像随机填充	可能还需解析块格式或对称密钥	按固定模数字节长度处理
最终得到密钥但解密失败	IV、模式、填充或编码可能读错	用已知文件头和重新加密定位

35.3 标准解题步骤

1. **抄录参数与数据类型**。把整数、字节、十六进制、模数、曲线、随机数和编码层分别记录，避免在字符串与整数之间混淆。
2. **检查基本约束**。为每一层定义输入输出类型和断言。
3. **寻找重复与关系**。先运行廉价参数审计，再选择因式分解或格方法。
4. **建立最小数学模型**。恢复密钥后检查位长、编码和使用位置。
5. **先求可验证的中间量**。从原始附件一条命令完成所有阶段。
6. **实现并加入断言**。脚本对范围、同余、长度、编码和重新加密结果做断言，不只打印候选。
7. **验证并还原字节**。将数学结果按明确位宽和端序转成字节，检查填充、结构和 flag 格式。

35.4 分析时持续追问的问题

- 当前观察是原始事实，还是工具给出的推断?
- 哪个最小输入变化能够区分主要假设?
- 结果能否在相同输入和干净环境中重复?
- 失败时是否保存了完整原始输出?
- 题目条件变化后，当前方法的前提是否仍成立?

35.5 命令、脚本与验证片段

以下内容只用于本地题目目录、回环地址、赛事提供的实例或明确授权的实验环境。每次运行都应保留原始输入和完整输出。

```
# A staged local solver structure.
from pathlib import Path
import json, base64, hashlib

def load(path):
    obj=json.loads(Path(path).read_text())
```

```
    ct=base64.b64decode(obj['ciphertext'],validate=True)
    return obj,ct

def audit(obj):
    for k in ('n','e'):
        v=int(obj[k]); print(k,v.bit_length())

def validate_flag(x):
    if not (x.startswith(b'flag{') and x.endswith(b'}')):
        raise ValueError(x[:64])
obj,ct=load('work/challenge.json'); audit(obj)
print('ct',len(ct),hashlib.sha256(ct).hexdigest())
```

Listing 35.1: 本章常用命令与脚本

35.6　证据保存与结果判定

保存题目原始参数、每个整数的进制与位长、使用的数学关系、脚本版本、候选数量和所有验证等式。对随机性问题保存样本顺序与时间；对加密题保存重新加密或重新签名得到原始数据的检查。

35.7　常见失误与修正方法

- 把所有阶段写在一个函数中，无法定位错误。
- 从网页复制大整数时丢失字符。
- 数学候选未重新代入验证。
- 解密结果使用错误端序或长度。
- 只保留最终明文，没有保存参数审计。

35.8 小型演练

题目场景

JSON 中三个 e=3 RSA 密文包裹同一 AES 密钥，AES-CBC 密文另存 Base64。描述完整求解链。

严格解析 JSON 和 Base64，记录所有位长。验证三个模数互素并用 CRT 合并 RSA 密文，取精确立方根得到固定长度 AES 密钥。检查密钥长度为 16、24 或 32 字节。读取 IV 和 CBC 密文，确认块长并解密，按 PKCS#7 严格去填充。最后验证 flag 格式，并将恢复的密钥重新 RSA 加密、将明文重新 AES 加密以确认整个链条。

35.9 完成检查

交付与复盘检查表

- □ 入口、输入、目标输出与限制条件已经写入笔记。
- □ 建立了可重复的基线输入与输出。
- □ 每个关键结论都有原始证据与验证步骤。
- □ 脚本能够识别失败，不会把异常输出当成成功。
- □ 从干净目录或新容器运行时能够得到同样结果。

第四部分

二进制利用 Pwn 题型

第三十六章　处理器架构、寄存器、端序与调用约定

本章目标

Pwn 的基础是理解目标架构如何传递参数、保存返回地址、对齐栈和执行系统调用。x86-64、i386、ARM、AArch64 与 MIPS 的寄存器、指令长度和端序不同，不能机械套用脚本。

36.1　原理与题型边界

调用约定规定参数位置、返回值、调用者与被调用者保存寄存器、栈对齐和函数序言。小端系统把最低有效字节放在低地址。利用链中的每个地址、参数和栈槽都必须符合目标 ABI。

36.2　识别线索与优先动作

观察线索	可能含义	优先动作
file 显示不同架构或端序	需要对应反汇编器、模拟器和打包方式	先记录 ELF header 和解释器
函数参数在寄存器而非栈中	使用 64 位或 RISC 调用约定	列出前六个参数寄存器
调用 libc 函数时莫名崩溃	可能栈未按 ABI 对齐	在调用前检查栈指针低位
地址字节顺序与十六进制相反	目标为小端	使用 p32/p64 而非字符串翻转

36.3　标准解题步骤

1. **固定二进制与运行环境**。计算 ELF、libc、加载器和补丁文件哈希，记录架构、端序、启动参数与远程协议。
2. **检查保护与链接信息**。使用 file、readelf -h 与 objdump 确认机器类型、位宽、端序和入口。
3. **建立最小崩溃或信息泄露**。在 main 与目标函数处断点，观察参数、返回地址与栈帧。
4. **测量偏移与可控范围**。用循环模式确认输入覆盖的是字节、整数还是指针。
5. **设计最小利用原语**。先构造一次合法函数调用或返回，验证 ABI 与对齐。
6. **在调试器中验证每一步**。检查寄存器、栈、堆、内存映射、调用约定和返回值，不把一次崩溃当作控制成功。
7. **使用脚本本地与远程双模式复现**。设置超时、同步菜单、解析泄露、处理 ASLR，并对失败输出明确诊断。

36.4　分析时持续追问的问题

- 当前观察是原始事实，还是工具给出的推断？
- 哪个最小输入变化能够区分主要假设？
- 结果能否在相同输入和干净环境中重复？
- 失败时是否保存了完整原始输出？
- 题目条件变化后，当前方法的前提是否仍成立？

36.5　常见 ABI 摘要

System V AMD64 前六个整数或指针参数通常位于 RDI、RSI、RDX、RCX、R8、R9，返回值位于 RAX。i386 常由栈传参。AArch64 常用 X0 至 X7 传参，返回值位于 X0。

36.6 命令、脚本与验证片段

以下内容只用于本地题目目录、回环地址、赛事提供的实例或明确授权的实验环境。每次运行都应保留原始输入和完整输出。

```
file work/chall
readelf -h work/chall | tee notes/elf-header.txt
objdump -d -M intel work/chall | sed -n '1,160p' > notes/disasm-head.txt
python3 - <<'PYCODE'
from pwn import *
context.binary=ELF('work/chall',checksec=False)
print(context.arch,context.bits,context.endian)
print(p32(0x12345678).hex(),p64(0x1122334455667788).hex())
PYCODE
```

Listing 36.1: 本章常用命令与脚本

36.7 证据保存与结果判定

保存 ELF、libc 和加载器哈希，checksec、readelf、反汇编、崩溃输入、core、寄存器、内存映射、泄露原始字节和基址计算。最终脚本应同时记录本地与远程参数，并验证关键地址落在预期映射范围。

36.8 常见失误与修正方法

- 看到 64 位就默认所有参数都在栈上。
- 忽略 16 字节栈对齐。
- 用本机架构运行异构二进制并误判行为。
- 手工翻转地址字符串产生长度错误。
- 混淆虚拟地址、文件偏移和运行时地址。

36.9　小型演练

题目场景

x86-64 程序从 gets 返回后进入 system，但在 movaps 指令崩溃。分析原因。

检查崩溃时 RSP 对齐。System V AMD64 ABI 要求调用边界满足特定对齐，ROP 链少了或多了一个返回地址会使 libc 内部使用对齐指令失败。增加一个单独 ret gadget 调整 8 字节，再验证 system 入口的 RSP。不要仅因地址正确就忽略 ABI。

36.10　完成检查

交付与复盘检查表

- □ 入口、输入、目标输出与限制条件已经写入笔记。
- □ 建立了可重复的基线输入与输出。
- □ 每个关键结论都有原始证据与验证步骤。
- □ 脚本能够识别失败，不会把异常输出当成成功。
- □ 从干净目录或新容器运行时能够得到同样结果。

第三十七章　ELF 结构、动态链接与保护机制

本章目标

在利用前应读懂 ELF 的段、节、符号、重定位、GOT、PLT、解释器和动态依赖，并确认 NX、PIE、Canary、RELRO 与 Fortify。保护机制决定可用原语，不等同于“可利用”或“不可利用”。

37.1　原理与题型边界

PIE 使主程序基址随机化，ASLR 还影响栈、堆和共享库。NX 阻止数据页执行，Canary 检测栈破坏，RELRO 影响 GOT 可写性。动态链接通过 PLT/GOT 延迟解析外部函数，泄露已解析 GOT 项可推导 libc 基址。

37.2　识别线索与优先动作

观察线索	可能含义	优先动作
checksec 显示 PIE enabled	静态地址只是偏移	需要主程序地址泄露或相对控制
Full RELRO	GOT 通常不可写	转向泄露、ROP 或其他写目标
NX enabled	栈堆代码不可直接执行	使用代码复用或可执行映射
Canary found	覆盖返回地址前需保留 canary	寻找泄露或不触及 canary 的原语

37.3　标准解题步骤

1. **固定二进制与运行环境**。计算 ELF、libc、加载器和补丁文件哈希，记录架构、端序、启动参数与远程协议。
2. **检查保护与链接信息**。运行 checksec、readelf -l/-S/-d/-r/-s，并保存输出。
3. **建立最小崩溃或信息泄露**。标记代码、只读数据、可写段、GOT、PLT 和初始化数组。
4. **测量偏移与可控范围**。本地启动后查看 /proc/PID/maps，区分文件偏移与运行时地址。
5. **设计最小利用原语**。根据保护选择所需泄露、写入、控制流和执行原语。
6. **在调试器中验证每一步**。检查寄存器、栈、堆、内存映射、调用约定和返回值，不把一次崩溃当作控制成功。
7. **使用脚本本地与远程双模式复现**。设置超时、同步菜单、解析泄露、处理 ASLR，并对失败输出明确诊断。

37.4　分析时持续追问的问题

- 当前观察是原始事实，还是工具给出的推断？
- 哪个最小输入变化能够区分主要假设？
- 结果能否在相同输入和干净环境中重复？
- 失败时是否保存了完整原始输出？
- 题目条件变化后，当前方法的前提是否仍成立？

37.5　命令、脚本与验证片段

以下内容只用于本地题目目录、回环地址、赛事提供的实例或明确授权的实验环境。每次运行都应保留原始输入和完整输出。

```
checksec --file=work/chall | tee notes/checksec.txt
readelf -h -l -S -d -r -s work/chall > notes/readelf-all.txt
objdump -d -M intel work/chall > notes/objdump.txt
```

```
objdump -R work/chall | tee notes/relocations.txt
ldd work/chall | tee notes/ldd.txt
# Run with the supplied loader and libc when provided.
./work/ld-linux-x86-64.so.2 --library-path work ./work/chall
```

Listing 37.1: 本章常用命令与脚本

37.6 证据保存与结果判定

保存 ELF、libc 和加载器哈希，checksec、readelf、反汇编、崩溃输入、core、寄存器、内存映射、泄露原始字节和基址计算。最终脚本应同时记录本地与远程参数，并验证关键地址落在预期映射范围。

37.7 常见失误与修正方法

- 把保护机制清单当作漏洞结论。
- 使用系统 libc 代替题目提供版本。
- 把静态符号地址直接用于 PIE 远程实例。
- 混淆 GOT 项地址与其中存放的函数地址。
- 补丁二进制后未记录新哈希。

37.8 小型演练

题目场景

ELF 开启 NX、PIE、Partial RELRO，无 Canary，提供 libc。规划需要的最小原语。

无 Canary 说明栈溢出可直接覆盖返回地址。NX 要求使用 ROP，而非栈 shellcode。PIE 使主程序 gadget 地址随机，需要程序地址泄露，或利用不依赖完整基址的部分覆盖。Partial RELRO 意味 GOT 可写，但是否需要覆盖取决于已有读写原语。提供 libc 表示典型路线是先泄露 libc 函数地址，再计算基址并调用 system 或其他目标。

37.9　完成检查

交付与复盘检查表

- ☐ 入口、输入、目标输出与限制条件已经写入笔记。
- ☐ 建立了可重复的基线输入与输出。
- ☐ 每个关键结论都有原始证据与验证步骤。
- ☐ 脚本能够识别失败，不会把异常输出当成成功。
- ☐ 从干净目录或新容器运行时能够得到同样结果。

第三十八章　GDB、断点、core 与崩溃定位

本章目标

调试器用于回答精确问题：输入在哪被复制，长度如何计算，哪一字节覆盖了什么，返回前寄存器与栈如何变化。高效调试需要固定输入、断点条件和自动命令，而不是单步浏览整个程序。

38.1　原理与题型边界

ASLR、fork、线程、信号和标准输入缓冲会影响复现。core 文件保留崩溃时寄存器和内存映射。硬件观察点适合跟踪某地址何时被写，但数量有限。远程问题应尽量在本地相同 libc 与加载器下重现。

38.2　识别线索与优先动作

观察线索	可能含义	优先动作
程序崩溃但终端只显示 Segmentation fault	需要 core 或调试器定位	启用 core 并记录 RIP、RSP 与栈
断点命中父进程而逻辑在子进程	程序 fork	设置 follow-fork-mode
输入在 fgets 后少一字节或含换行	I/O 语义影响布局	在调用前后检查缓冲区字节
每次地址不同	ASLR 或 PIE 生效	记录映射并使用相对偏移

38.3　标准解题步骤

1. **固定二进制与运行环境**。计算 ELF、libc、加载器和补丁文件哈希，记录架构、端序、启动参数与远程协议。
2. **检查保护与链接信息**。准备固定 input.bin，确保调试与脚本使用相同字节。
3. **建立最小崩溃或信息泄露**。在危险函数调用前后、函数返回前和崩溃点设置断点。
4. **测量偏移与可控范围**。检查寄存器、栈、内存映射、反汇编和调用栈。
5. **设计最小利用原语**。用 core 或自动命令记录关键状态，避免手工抄写。
6. **在调试器中验证每一步**。检查寄存器、栈、堆、内存映射、调用约定和返回值，不把一次崩溃当作控制成功。
7. **使用脚本本地与远程双模式复现**。设置超时、同步菜单、解析泄露、处理 ASLR，并对失败输出明确诊断。

38.4　分析时持续追问的问题

- 当前观察是原始事实，还是工具给出的推断?
- 哪个最小输入变化能够区分主要假设?
- 结果能否在相同输入和干净环境中重复?
- 失败时是否保存了完整原始输出?
- 题目条件变化后，当前方法的前提是否仍成立?

38.5　命令、脚本与验证片段

以下内容只用于本地题目目录、回环地址、赛事提供的实例或明确授权的实验环境。每次运行都应保留原始输入和完整输出。

```
ulimit -c unlimited
gdb -q work/chall <<'GDB'
set pagination off
set disassembly-flavor intel
```

```
break main
run < work/input.bin
info registers
x/24gx $rsp
info proc mappings
continue
GDB
# Inspect a core produced by the same binary.
gdb -q work/chall core -ex 'set pagination off' \
  -ex 'info registers' -ex 'x/32gx $rsp' -ex 'bt' -ex quit
```

Listing 38.1: 本章常用命令与脚本

38.6　证据保存与结果判定

保存 ELF、libc 和加载器哈希，checksec、readelf、反汇编、崩溃输入、core、寄存器、内存映射、泄露原始字节和基址计算。最终脚本应同时记录本地与远程参数，并验证关键地址落在预期映射范围。

38.7　常见失误与修正方法

- 调试输入与最终 exploit 输入不一致。
- 关闭 ASLR 后忘记远程仍有随机化。
- 只看 backtrace，不检查覆盖字节。
- 断点放在优化后不存在的源码行。
- core 与当前二进制版本不匹配。

38.8　小型演练

题目场景

程序读取 200 字节后崩溃，RIP 显示循环模式的一部分。求精确偏移。

用 pwntools cyclic 生成长度足够的模式，保存为文件并在 GDB 运行。崩溃后根据 RIP 或栈顶实际字节使用 cyclic_find，注意 32/64 位和字节序。用 A*offset

+ B*8 构造确认输入，检查返回时 RIP 变为固定 B 值。只有确认控制位置后才继续构造利用链。

38.9　完成检查

交付与复盘检查表

- □ 入口、输入、目标输出与限制条件已经写入笔记。
- □ 建立了可重复的基线输入与输出。
- □ 每个关键结论都有原始证据与验证步骤。
- □ 脚本能够识别失败，不会把异常输出当成成功。
- □ 从干净目录或新容器运行时能够得到同样结果。

第三十九章　栈溢出、偏移、Canary 与 ret2win

本章目标

栈溢出题的第一目标是建立精确控制：输入长度、缓冲区位置、Canary、保存帧指针和返回地址偏移。ret2win 是最小代码复用，只需把返回地址改为程序内已有目标函数。

39.1　原理与题型边界

局部数组通常位于栈帧中，越界写可覆盖相邻数据。编译器、优化、对齐和可变长度数组会改变布局。Canary 位于控制数据之前，返回前被检查。目标函数可能需要参数或栈对齐。

39.2　识别线索与优先动作

观察线索	可能含义	优先动作
危险读取长度大于栈缓冲区	可能直接溢出	在调用前后检查缓冲区与保存 RIP
崩溃值包含 cyclic 模式	可计算精确偏移	使用对应字节宽度 cyclic_find
返回前调用 __stack_chk_fail	Canary 被破坏	寻找泄露或调整覆盖范围
二进制中有 win、flag 或 print_secret	可能是 ret2win	确认函数入口、参数和调用约定

39.3　标准解题步骤

1. **固定二进制与运行环境**。计算 ELF、libc、加载器和补丁文件哈希，记录架构、端序、启动参数与远程协议。
2. **检查保护与链接信息**。定位危险读取的目标地址和最大长度。
3. **建立最小崩溃或信息泄露**。用循环模式求偏移，并用固定标记确认。
4. **测量偏移与可控范围**。检查 Canary 与保存寄存器是否必须保留。
5. **设计最小利用原语**。把返回地址指向目标函数，补齐参数和对齐。
6. **在调试器中验证每一步**。检查寄存器、栈、堆、内存映射、调用约定和返回值，不把一次崩溃当作控制成功。
7. **使用脚本本地与远程双模式复现**。设置超时、同步菜单、解析泄露、处理 ASLR，并对失败输出明确诊断。

39.4　分析时持续追问的问题

- 当前观察是原始事实，还是工具给出的推断?
- 哪个最小输入变化能够区分主要假设?
- 结果能否在相同输入和干净环境中重复?
- 失败时是否保存了完整原始输出?
- 题目条件变化后，当前方法的前提是否仍成立?

39.5　命令、脚本与验证片段

以下内容只用于本地题目目录、回环地址、赛事提供的实例或明确授权的实验环境。每次运行都应保留原始输入和完整输出。

```
from pwn import *
context.binary=elf=ELF('work/chall',checksec=False)
# Create and later analyze a cyclic pattern.
Path('work/pattern.bin').write_bytes(cyclic(300,n=8))
print('win',hex(elf.symbols.get('win',0)))
OFFSET=120
```

```
payload=flat({OFFSET:[elf.symbols['win']]},word_size=64)
Path('work/payload.bin').write_bytes(payload)
```

Listing 39.1: 本章常用命令与脚本

39.6 证据保存与结果判定

保存 ELF、libc 和加载器哈希，checksec、readelf、反汇编、崩溃输入、core、寄存器、内存映射、泄露原始字节和基址计算。最终脚本应同时记录本地与远程参数，并验证关键地址落在预期映射范围。

39.7 常见失误与修正方法

- 从源码猜偏移而不动态确认。
- cyclic 的 n 参数与查找宽度不一致。
- 直接跳到函数中间导致序言或对齐错误。
- 忽略目标函数参数。
- 有 Canary 时盲目覆盖返回地址。

39.8 小型演练

题目场景

无 PIE、无 Canary 的 64 位程序有 win 函数，gets 缓冲区 64 字节。构造稳定 ret2win。

使用 cyclic 而不是根据 64 字节猜偏移。确认保存 RBP、对齐填充和 RIP 位置。读取 ELF 符号得到 win 地址，必要时在链前加入 ret 对齐。用 payload 文件在 GDB 检查函数入口、RSP 和返回行为，再由脚本发送。若 win 返回到无效地址，可在其后放 exit 或 main。

39.9　完成检查

交付与复盘检查表

- ☐ 入口、输入、目标输出与限制条件已经写入笔记。
- ☐ 建立了可重复的基线输入与输出。
- ☐ 每个关键结论都有原始证据与验证步骤。
- ☐ 脚本能够识别失败，不会把异常输出当成成功。
- ☐ 从干净目录或新容器运行时能够得到同样结果。

第四十章　信息泄露、ret2libc 与动态基址计算

本章目标

ret2libc 通常分两阶段：先调用输出函数泄露 GOT 中的 libc 地址，再返回 main 或读入点；随后根据题目 libc 计算基址，构造 system、execve 或其他目标调用。

40.1　原理与题型边界

运行时函数地址等于 libc 基址加符号偏移。泄露必须保留原始字节，因为地址常含零字节并可能被 puts 截断。基址通常页对齐，计算后应检查多个符号地址和映射范围。

40.2　识别线索与优先动作

观察线索	可能含义	优先动作
提供 libc 与加载器	远程偏移应以提供文件为准	记录哈希并使用同一加载器本地运行
程序有 puts@plt 和可控参数 gadget	可泄露 GOT 项	调用 puts(got) 后返回 main
泄露少于 8 字节	字符串输出遇零终止	按小端补零并说明可见字节
计算基址不页对齐	泄露解析或 libc 版本可能错误	检查原始字节、符号和低 12 位

40.3　标准解题步骤

1. **固定二进制与运行环境**。计算 ELF、libc、加载器和补丁文件哈希，记录架构、端序、启动参数与远程协议。
2. **检查保护与链接信息**。选择已经解析或可通过 PLT 解析的 GOT 项。
3. **建立最小崩溃或信息泄露**。构造第一阶段泄露并返回稳定入口。
4. **测量偏移与可控范围**。解析原始泄露，计算 libc 基址并检查页对齐。
5. **设计最小利用原语**。构造第二阶段目标调用，验证参数字符串地址和栈对齐。
6. **在调试器中验证每一步**。检查寄存器、栈、堆、内存映射、调用约定和返回值，不把一次崩溃当作控制成功。
7. **使用脚本本地与远程双模式复现**。设置超时、同步菜单、解析泄露、处理 ASLR，并对失败输出明确诊断。

40.4　分析时持续追问的问题

- 当前观察是原始事实，还是工具给出的推断?
- 哪个最小输入变化能够区分主要假设?
- 结果能否在相同输入和干净环境中重复?
- 失败时是否保存了完整原始输出?
- 题目条件变化后，当前方法的前提是否仍成立?

40.5　命令、脚本与验证片段

以下内容只用于本地题目目录、回环地址、赛事提供的实例或明确授权的实验环境。每次运行都应保留原始输入和完整输出。

```
from pwn import *
context.binary=elf=ELF('work/chall',checksec=False)
libc=ELF('work/libc.so.6',checksec=False)
p=process([elf.path])
rop=ROP(elf)
rop.call(elf.plt['puts'],[elf.got['puts']])
```

```
rop.call(elf.symbols['main'])
p.sendlineafter(b'> ',flat({120:rop.chain()}))
leak=p.recvline().strip()
puts_addr=u64(leak.ljust(8,b'\0'))
libc.address=puts_addr-libc.symbols['puts']
assert libc.address & 0xfff == 0
log.info('libc=%#x',libc.address)
```

Listing 40.1: 本章常用命令与脚本

40.6　证据保存与结果判定

保存 ELF、libc 和加载器哈希，checksec、readelf、反汇编、崩溃输入、core、寄存器、内存映射、泄露原始字节和基址计算。最终脚本应同时记录本地与远程参数，并验证关键地址落在预期映射范围。

40.7　常见失误与修正方法

- 使用本机 libc 偏移。
- 把文本形式地址和原始地址字节混淆。
- 泄露未同步菜单，读到了提示文本。
- 基址计算后没有页对齐和第二符号验证。
- 第二阶段忽略 system 调用对齐。

40.8　小型演练

题目场景

第一阶段 puts(puts@got) 得到 6 字节泄露。计算 libc 基址并准备第二阶段。

读取到准确的泄露行，按小端在高位补零得到地址。使用提供 libc 的 puts 偏移相减，检查基址低 12 位为零，并可用另一个已知泄露交叉验证。第二阶段寻找 /bin/sh 或题目目标字符串，使用 pop rdi 和 ret 对齐调用 system。脚本应在解析长度异常时停止，而不是继续使用错误地址。

40.9　完成检查

交付与复盘检查表

- ☐ 入口、输入、目标输出与限制条件已经写入笔记。
- ☐ 建立了可重复的基线输入与输出。
- ☐ 每个关键结论都有原始证据与验证步骤。
- ☐ 脚本能够识别失败，不会把异常输出当成成功。
- ☐ 从干净目录或新容器运行时能够得到同样结果。

第四十一章　ROP gadget、调用链与约束满足

本章目标

ROP 通过已有指令片段设置寄存器、读写内存和调用函数。题目难点常在 gadget 副作用、栈消耗、不可用字节、有限输入长度和 PIE。设计链时应把每个 gadget 视为状态转移。

41.1　原理与题型边界

gadget 可能 pop 多个寄存器、修改内存、调整 rsp 或带条件。调用函数需要满足 ABI 和栈对齐。ret2csu 利用初始化代码中的通用寄存器设置与间接调用片段。链条越短越容易验证。

41.2　识别线索与优先动作

观察线索	可能含义	优先动作
缺少直接 pop rdx gadget	需要组合 gadget 或 ret2csu	列出副作用与栈槽
输入长度很短	链条需分阶段或栈迁移	先调用 read 写入更大区域
地址含被过滤字节	需要选择替代 gadget 或分步构造	对每个打包地址检查坏字节
调用后栈偏移异常	gadget 消耗槽位计算错误	逐 gadget 在 GDB 检查 RSP

41.3　标准解题步骤

1. **固定二进制与运行环境**。计算 ELF、libc、加载器和补丁文件哈希，记录架构、端序、启动参数与远程协议。
2. **检查保护与链接信息**。定义调用目标所需寄存器与内存状态。
3. **建立最小崩溃或信息泄露**。列出候选 gadget 的所有副作用和栈消耗。
4. **测量偏移与可控范围**。先完成单次函数调用，再组合读写与第二阶段。
5. **设计最小利用原语**。用 flat 或显式布局标注每个栈槽，并在断点逐项验证。
6. **在调试器中验证每一步**。检查寄存器、栈、堆、内存映射、调用约定和返回值，不把一次崩溃当作控制成功。
7. **使用脚本本地与远程双模式复现**。设置超时、同步菜单、解析泄露、处理 ASLR，并对失败输出明确诊断。

41.4　分析时持续追问的问题

- 当前观察是原始事实，还是工具给出的推断?
- 哪个最小输入变化能够区分主要假设?
- 结果能否在相同输入和干净环境中重复?
- 失败时是否保存了完整原始输出?
- 题目条件变化后，当前方法的前提是否仍成立?

41.5　命令、脚本与验证片段

以下内容只用于本地题目目录、回环地址、赛事提供的实例或明确授权的实验环境。每次运行都应保留原始输入和完整输出。

```
from pwn import *
elf=ELF('work/chall',checksec=False); rop=ROP(elf)
print(rop.find_gadget(['pop rdi','ret']))
print(rop.find_gadget(['ret']))
rop.call(elf.plt['read'],[0,elf.bss(0x800),0x400])
print(rop.dump())
```

```
chain=rop.chain()
for i in range(0,len(chain),8):
    print(i,hex(u64(chain[i:i+8].ljust(8,b'\0'))))
```

Listing 41.1: 本章常用命令与脚本

41.6 证据保存与结果判定

保存 ELF、libc 和加载器哈希，checksec、readelf、反汇编、崩溃输入、core、寄存器、内存映射、泄露原始字节和基址计算。最终脚本应同时记录本地与远程参数，并验证关键地址落在预期映射范围。

41.7 常见失误与修正方法

- 只按 gadget 名称选择，忽略副作用。
- 没有计算每个 pop 对 RSP 的消耗。
- 链条一次构造过长，无法定位失败点。
- 地址坏字节检查只看字符串表示。
- 使用自动 ROP 后不查看生成链。

41.8 小型演练

题目场景

二进制没有 pop rdx，但可调用 read(0,bss,0x400)。如何构造第一阶段。

先检查 ret2csu 或其他多寄存器 gadget，明确它设置 rdi、rsi、rdx 的方式和额外 pop。构造最小 read 调用，把第二阶段写入 bss，然后返回到栈迁移点或可控地址。GDB 在 call 前检查三个参数和函数指针，call 后检查写入长度。自动工具生成链后仍要逐槽审阅。

41.9　完成检查

交付与复盘检查表

- ☐ 入口、输入、目标输出与限制条件已经写入笔记。
- ☐ 建立了可重复的基线输入与输出。
- ☐ 每个关键结论都有原始证据与验证步骤。
- ☐ 脚本能够识别失败，不会把异常输出当成成功。
- ☐ 从干净目录或新容器运行时能够得到同样结果。

第四十二章　栈迁移、SROP 与 seccomp 约束

本章目标

当原始溢出空间不足，可把更大链写入 bss、堆或其他可写区，再通过 leave;ret、pop rsp 或 setcontext 风格原语迁移。SROP 利用信号返回帧一次设置大量寄存器。seccomp 则限制可用系统调用，需要先读取过滤规则。

42.1　原理与题型边界

leave 等价于 mov rsp,rbp; pop rbp。迁移需要可控 rbp 和目标栈布局。SROP 需要触发 rt_sigreturn 系统调用并提供正确架构的 sigframe。seccomp BPF 可能只允许 read、write、openat 等调用，传统 execve 会失败。

42.2　识别线索与优先动作

观察线索	可能含义	优先动作
初始覆盖区只能放两三个地址	需要分阶段写入与迁移	调用 read 写入 bss 后设置新 rsp
存在 syscall;ret 且寄存器控制有限	可能适合 SROP	确认架构、系统调用号和 frame 布局
execve 返回 EPERM 或程序有 seccomp 初始化	系统调用被过滤	提取并解释过滤规则
有 leave;ret 和可控保存 rbp	可进行经典栈迁移	在目标区预置新 rbp 与链

42.3　标准解题步骤

1. **固定二进制与运行环境**。计算 ELF、libc、加载器和补丁文件哈希，记录架构、端序、启动参数与远程协议。
2. **检查保护与链接信息**。确定可写目标区、地址稳定性和第一阶段可用空间。
3. **建立最小崩溃或信息泄露**。构造第二阶段并先在内存中验证布局。
4. **测量偏移与可控范围**。选择迁移 gadget 或 sigreturn 原语，计算目标 RSP。
5. **设计最小利用原语**。根据 seccomp 允许列表设计读取 flag 的最小系统调用序列。
6. **在调试器中验证每一步**。检查寄存器、栈、堆、内存映射、调用约定和返回值，不把一次崩溃当作控制成功。
7. **使用脚本本地与远程双模式复现**。设置超时、同步菜单、解析泄露、处理 ASLR，并对失败输出明确诊断。

42.4　分析时持续追问的问题

- 当前观察是原始事实，还是工具给出的推断？
- 哪个最小输入变化能够区分主要假设？
- 结果能否在相同输入和干净环境中重复？
- 失败时是否保存了完整原始输出？
- 题目条件变化后，当前方法的前提是否仍成立？

42.5　命令、脚本与验证片段

以下内容只用于本地题目目录、回环地址、赛事提供的实例或明确授权的实验环境。每次运行都应保留原始输入和完整输出。

```
from pwn import *
elf=ELF('work/chall',checksec=False)
new_stack=elf.bss(0x800)
# Layout for leave; ret: [next_rbp][first_gadget]...
```

```
stage2=flat(new_stack+0x200,elf.symbols.get('target',0))
print(hex(new_stack),len(stage2))
# Inspect seccomp in a local challenge when seccomp-tools is available.
# seccomp-tools dump ./work/chall
```

Listing 42.1: 本章常用命令与脚本

42.6 证据保存与结果判定

保存 ELF、libc 和加载器哈希，checksec、readelf、反汇编、崩溃输入、core、寄存器、内存映射、泄露原始字节和基址计算。最终脚本应同时记录本地与远程参数，并验证关键地址落在预期映射范围。

42.7 常见失误与修正方法

- 迁移目标内存不可写或地址随机。
- 忘记 leave 会先从新栈 pop rbp。
- SROP frame 使用错误架构或系统调用号。
- 看到 seccomp 就默认只能 ORW，不先读规则。
- 开放文件后忽略返回文件描述符。

42.8 小型演练

题目场景

初始栈只能容纳 read 调用和 leave;ret，bss 固定。设计两阶段迁移。

第一阶段设置 read(0,bss,0x400)，返回后令 rbp 指向 bss 并执行 leave;ret。发送第二阶段时在 bss 起始放置下一 rbp，第二个八字节放第一 gadget，后续为完整链。断点检查 read 已写入、leave 前 rbp 正确、leave 后 rsp=bss+8。若 seccomp 禁止 execve，改用允许的 openat、read、write 读取题目 flag。

42.9　完成检查

交付与复盘检查表

- ☐ 入口、输入、目标输出与限制条件已经写入笔记。
- ☐ 建立了可重复的基线输入与输出。
- ☐ 每个关键结论都有原始证据与验证步骤。
- ☐ 脚本能够识别失败，不会把异常输出当成成功。
- ☐ 从干净目录或新容器运行时能够得到同样结果。

第四十三章　格式化字符串：泄露、写入与参数定位

本章目标

格式化字符串漏洞发生在用户数据被当作格式字符串传给 printf 家族。可利用%p、%x、%s 泄露参数或内存，使用%n 家族写入已输出字符数。第一步是定位参数索引和输入在栈上的位置。

43.1　原理与题型边界

变长参数函数依赖 ABI 从寄存器保存区与栈读取参数。位置参数如%7$p 可直接选取索引。%n 写入 int，%hn 写 16 位，%hhn 写 8 位。写入需要目标地址作为参数，并精确控制累计输出长度。

43.2　识别线索与优先动作

观察线索	可能含义	优先动作
输入包含%p 后输出地址	用户数据被当作格式串	用编号参数稳定定位
输出出现 0x41414141 或输入字节	输入或地址在参数区可见	确定索引与端序
%s 导致崩溃或打印内存	参数被作为指针解引用	只对已知可读地址测试
程序后续使用可写全局或 GOT	可能通过%n 修改	先从无害局部目标验证写原语

43.3　标准解题步骤

1. **固定二进制与运行环境**。计算 ELF、libc、加载器和补丁文件哈希，记录架构、端序、启动参数与远程协议。
2. **检查保护与链接信息**。用短编号序列确定输入和栈参数索引。
3. **建立最小崩溃或信息泄露**。区分直接参数、指针参数和栈上地址。
4. **测量偏移与可控范围**。先建立稳定泄露，再计算 PIE、Canary 或 libc。
5. **设计最小利用原语**。写入时按字节或半字拆分，考虑已输出长度与取模。
6. **在调试器中验证每一步**。检查寄存器、栈、堆、内存映射、调用约定和返回值，不把一次崩溃当作控制成功。
7. **使用脚本本地与远程双模式复现**。设置超时、同步菜单、解析泄露、处理 ASLR，并对失败输出明确诊断。

43.4　分析时持续追问的问题

- 当前观察是原始事实，还是工具给出的推断？
- 哪个最小输入变化能够区分主要假设？
- 结果能否在相同输入和干净环境中重复？
- 失败时是否保存了完整原始输出？
- 题目条件变化后，当前方法的前提是否仍成立？

43.5　命令、脚本与验证片段

以下内容只用于本地题目目录、回环地址、赛事提供的实例或明确授权的实验环境。每次运行都应保留原始输入和完整输出。

```
from pwn import *
context.binary=elf=ELF('work/chall',checksec=False)
p=process(elf.path)
p.sendlineafter(b'> ',b'AAAABBBB|%1$p|%2$p|%3$p|%4$p|%5$p|%6$p|%7$p')
print(p.recvline())
# Generate a local controlled write after the correct offset is known.
```

```
# payload=fmtstr_payload(offset,{elf.symbols['target']:0x1337},write_size='
    short')
```

Listing 43.1: 本章常用命令与脚本

43.6 证据保存与结果判定

保存 ELF、libc 和加载器哈希，checksec、readelf、反汇编、崩溃输入、core、寄存器、内存映射、泄露原始字节和基址计算。最终脚本应同时记录本地与远程参数，并验证关键地址落在预期映射范围。

43.7 常见失误与修正方法

- 一次打印大量%p 导致输出截断而无法定位。
- 使用%s 解引用未知地址。
- %n 累计字符数计算忽略已有前缀。
- 写入顺序导致计数回绕异常。
- 自动生成 payload 后不检查目标地址和索引。

43.8 小型演练

题目场景

输入在第 8 个参数，目标全局变量初值 0，要写 0x1337。

先用%8$p 与输入标记确认偏移。把目标地址放在 payload 合适位置,使用%hn 写两字节，累计输出精确到 0x1337。若程序在 printf 前已输出前缀，不影响当前 printf 内计数，但格式串中的文字会影响。用 GDB 在 printf 后检查目标变量，再运行程序正常逻辑确认影响。

43.9　完成检查

交付与复盘检查表

- ☐ 入口、输入、目标输出与限制条件已经写入笔记。
- ☐ 建立了可重复的基线输入与输出。
- ☐ 每个关键结论都有原始证据与验证步骤。
- ☐ 脚本能够识别失败，不会把异常输出当成成功。
- ☐ 从干净目录或新容器运行时能够得到同样结果。

第四十四章　Shellcode、系统调用与编码约束

本章目标

Shellcode 题要求在可执行内存或通过系统调用原语完成目标。需明确架构、允许字符、最大长度、空字节、输入变换、寄存器初始状态和 seccomp。目标通常可以用最短的读、写、打开或执行序列完成。

44.1　原理与题型边界

位置无关代码不能依赖固定地址。系统调用号与参数寄存器由架构决定。输入可能经过 strcpy、scanf、toupper、十六进制解码或字节过滤，导致普通汇编不可直接使用。自解码器和分阶段读取用于绕过长度与字符约束，但必须在本地验证。

44.2　识别线索与优先动作

观察线索	可能含义	优先动作
程序 mmap 可执行缓冲区并跳转	直接 shellcode 题	检查缓冲地址、长度和初始寄存器
输入禁止零字节或限定字母数字	需要选择等价指令或编码器	汇编后逐字节检查
只允许少量系统调用	需要 ORW 或题目特定调用链	读取 seccomp 规则
第一阶段长度很短但可 read	适合小 loader 读取第二阶段	验证 fd、目标地址和长度

44.3　标准解题步骤

1. **固定二进制与运行环境**。计算 ELF、libc、加载器和补丁文件哈希，记录架构、端序、启动参数与远程协议。
2. **检查保护与链接信息**。确认架构、系统调用 ABI、可执行内存和输入变换。
3. **建立最小崩溃或信息泄露**。先汇编一个无害退出或固定输出 shellcode 验证执行。
4. **测量偏移与可控范围**。检查所有坏字节、长度和位置无关性。
5. **设计最小利用原语**。根据允许系统调用完成题目目标，不添加网络回连或持久化。
6. **在调试器中验证每一步**。检查寄存器、栈、堆、内存映射、调用约定和返回值，不把一次崩溃当作控制成功。
7. **使用脚本本地与远程双模式复现**。设置超时、同步菜单、解析泄露、处理 ASLR，并对失败输出明确诊断。

44.4　分析时持续追问的问题

- 当前观察是原始事实，还是工具给出的推断？
- 哪个最小输入变化能够区分主要假设？
- 结果能否在相同输入和干净环境中重复？
- 失败时是否保存了完整原始输出？
- 题目条件变化后，当前方法的前提是否仍成立？

44.5　命令、脚本与验证片段

以下内容只用于本地题目目录、回环地址、赛事提供的实例或明确授权的实验环境。每次运行都应保留原始输入和完整输出。

```
from pwn import *
context.clear(arch='amd64',os='linux')
# Harmless local demonstration: write a fixed marker, then exit.
```

```
sc=asm(shellcraft.write(1,b'CTF_SC_MARKER\n',14)+shellcraft.exit(0))
print('len',len(sc),'hex',sc.hex())
bad={0x00,0x0a}
print('bad positions',[(i,b) for i,b in enumerate(sc) if b in bad])
Path('work/shellcode.bin').write_bytes(sc)
```

Listing 44.1: 本章常用命令与脚本

44.6 证据保存与结果判定

保存 ELF、libc 和加载器哈希，checksec、readelf、反汇编、崩溃输入、core、寄存器、内存映射、泄露原始字节和基址计算。最终脚本应同时记录本地与远程参数，并验证关键地址落在预期映射范围。

44.7 常见失误与修正方法

- 使用错误架构的系统调用号。
- 汇编后没有检查坏字节。
- shellcode 依赖固定地址。
- 忽略输入函数自动追加换行或零字节。
- 加入反向连接、持久化或超出题目目标的功能。

44.8 小型演练

题目场景

本地程序把最多 40 字节读入 RWX 区，禁止零字节和换行。先证明可执行。

编写只向 stdout 输出固定标记并退出的最小 shellcode。汇编后检查长度与禁用字节，若包含零字节，用寄存器清零、短立即数或栈构造替代。通过程序运行并看到标记证明控制流。之后再按题目需要替换为读取 flag 的受限系统调用序列。

44.9　完成检查

交付与复盘检查表

- □ 入口、输入、目标输出与限制条件已经写入笔记。
- □ 建立了可重复的基线输入与输出。
- □ 每个关键结论都有原始证据与验证步骤。
- □ 脚本能够识别失败，不会把异常输出当成成功。
- □ 从干净目录或新容器运行时能够得到同样结果。

第四十五章　堆基础：chunk、bin、分配与释放轨迹

本章目标

堆题必须先理解题目使用的分配器版本和对象生命周期。以 glibc malloc 为例，chunk 含大小与状态元数据，释放块进入 tcache、fastbin、smallbin、unsorted bin 等结构。版本差异会改变检查与安全链接。

45.1　原理与题型边界

利用不从背诵 bin 开始，而从程序原语开始：可分配多大、可编辑多少、释放后指针是否清空、能否查看释放内容、索引如何验证。通过记录每次操作的 chunk 地址、大小和 bin 变化建立模型。

45.2　识别线索与优先动作

观察线索	可能含义	优先动作
提供 libc 且菜单含 add/edit/show/delete	典型堆对象题	记录每个索引的地址、大小和状态
删除后仍可 show 或 edit	可能 UAF	用固定标记观察释放后内容
相同大小释放再分配复用地址	对应缓存 bin 行为	画出 freelist 变化
大块释放后内容出现 libc 指针	可能进入 unsorted bin	保存原始泄露并计算基址

45.3　标准解题步骤

1. **固定二进制与运行环境**。计算 ELF、libc、加载器和补丁文件哈希，记录架构、端序、启动参数与远程协议。
2. **检查保护与链接信息**。确认 libc 版本、架构与 malloc 实现。
3. **建立最小崩溃或信息泄露**。为菜单操作写脚本，稳定同步并记录索引。
4. **测量偏移与可控范围**。使用固定大小序列观察复用、合并与 bin 迁移。
5. **设计最小利用原语**。先获得泄露或任意写之一，再规划完整链。
6. **在调试器中验证每一步**。检查寄存器、栈、堆、内存映射、调用约定和返回值，不把一次崩溃当作控制成功。
7. **使用脚本本地与远程双模式复现**。设置超时、同步菜单、解析泄露、处理 ASLR，并对失败输出明确诊断。

45.4　分析时持续追问的问题

- 当前观察是原始事实，还是工具给出的推断?
- 哪个最小输入变化能够区分主要假设?
- 结果能否在相同输入和干净环境中重复?
- 失败时是否保存了完整原始输出?
- 题目条件变化后，当前方法的前提是否仍成立?

45.5　命令、脚本与验证片段

以下内容只用于本地题目目录、回环地址、赛事提供的实例或明确授权的实验环境。每次运行都应保留原始输入和完整输出。

```
from pwn import *
context.binary=elf=ELF('work/chall',checksec=False)
p=process(elf.path)
def add(i,size,data=b'A'):
    p.sendlineafter(b'> ',b'1'); p.sendlineafter(b'Index:',str(i).encode())
    p.sendlineafter(b'Size:',str(size).encode()); p.sendafter(b'Data:',data)
```

```
def delete(i):
    p.sendlineafter(b'> ',b'4'); p.sendlineafter(b'Index:',str(i).encode())
# Use deterministic sizes and record every transition in notes.
```

Listing 45.1: 本章常用命令与脚本

45.6 证据保存与结果判定

保存 ELF、libc 和加载器哈希，checksec、readelf、反汇编、崩溃输入、core、寄存器、内存映射、泄露原始字节和基址计算。最终脚本应同时记录本地与远程参数，并验证关键地址落在预期映射范围。

45.7 常见失误与修正方法

- 套用其他 glibc 版本的偏移和检查。
- 菜单脚本不同步导致操作序列与笔记不一致。
- 混淆请求大小、chunk size 和 usable size。
- 只看地址复用就断言可利用。
- 在未理解原语时直接复制复杂 heap exploit。

45.8 小型演练

题目场景

菜单允许 add、show、delete。删除后 show 能看到变化的指针值。建立最小 UAF 证据。

分配两个相同大小块并写入不同标记，记录索引。删除第一个后立即 show，保存返回字节；再分配同大小第三个块，观察是否复用同一地址和数据。若删除后索引仍可读取且内容来自分配器 freelist，说明存在 UAF 读取原语。下一步再判断该泄露能否推导堆地址，不能从一次变化直接跳到任意写。

45.9　完成检查

交付与复盘检查表

- ☐ 入口、输入、目标输出与限制条件已经写入笔记。
- ☐ 建立了可重复的基线输入与输出。
- ☐ 每个关键结论都有原始证据与验证步骤。
- ☐ 脚本能够识别失败，不会把异常输出当成成功。
- ☐ 从干净目录或新容器运行时能够得到同样结果。

第四十六章　UAF、Double Free、Tcache 与堆写原语

本章目标

进阶堆题围绕释放后使用、重复释放、越界写、元数据破坏和 freelist 操作。现代 glibc 对 double free、指针编码与大小一致性有检查，因此必须以题目版本为准。

46.1　原理与题型边界

tcache 单链表保存可复用块。旧版本中修改 next 可让后续分配返回目标地址；启用 safe-linking 后 next 与堆地址混合，需要堆泄露。Double free 可能被 key 或链表检查阻止。最终目标可以是函数指针、程序对象、退出处理结构或其他可写控制数据，不必固定为历史 hook。

46.2　识别线索与优先动作

观察线索	可能含义	优先动作
释放后 edit 仍可写	可修改 freelist 或复用对象	先观察下一次分配地址
重复 delete 报检测错误	存在 double-free 防护	研究是否能通过中间分配或不同 bin 绕过
freelist 指针看似非规范地址	可能启用 safe-linking	结合 chunk 地址验证编码关系
历史 hook 不存在或不可写	glibc 版本较新	寻找程序内函数指针或其他控制目标

46.3　标准解题步骤

1. **固定二进制与运行环境**。计算 ELF、libc、加载器和补丁文件哈希，记录架构、端序、启动参数与远程协议。
2. **检查保护与链接信息**。把可用原语写成读、写、分配、释放和查看的精确定义。
3. **建立最小崩溃或信息泄露**。获取需要的堆或 libc 泄露并验证基址。
4. **测量偏移与可控范围**。在本地最小序列中证明目标地址分配或受控写。
5. **设计最小利用原语**。选择与版本和程序逻辑匹配的控制目标，完成后清理状态。
6. **在调试器中验证每一步**。检查寄存器、栈、堆、内存映射、调用约定和返回值，不把一次崩溃当作控制成功。
7. **使用脚本本地与远程双模式复现**。设置超时、同步菜单、解析泄露、处理 ASLR，并对失败输出明确诊断。

46.4　分析时持续追问的问题

- 当前观察是原始事实，还是工具给出的推断?
- 哪个最小输入变化能够区分主要假设?
- 结果能否在相同输入和干净环境中重复?
- 失败时是否保存了完整原始输出?
- 题目条件变化后，当前方法的前提是否仍成立?

46.5　命令、脚本与验证片段

以下内容只用于本地题目目录、回环地址、赛事提供的实例或明确授权的实验环境。每次运行都应保留原始输入和完整输出。

```
# Safe-linking relationship used by modern glibc tcache:
# encoded_next = next ^ (chunk_address >> 12)
def protect_ptr(chunk_addr,next_addr):
    return next_addr ^ (chunk_addr >> 12)
heap_chunk=0x55555575a2a0
target=0x555555758060
```

```
enc=protect_ptr(heap_chunk,target)
assert protect_ptr(heap_chunk,enc)==target
print(hex(enc))
```

Listing 46.1: 本章常用命令与脚本

46.6 证据保存与结果判定

保存 ELF、libc 和加载器哈希，checksec、readelf、反汇编、崩溃输入、core、寄存器、内存映射、泄露原始字节和基址计算。最终脚本应同时记录本地与远程参数，并验证关键地址落在预期映射范围。

46.7 常见失误与修正方法

- 使用过时的 __free_hook 路线而不检查符号。
- 把 safe-linking 编码当作随机值。
- 没有堆泄露就猜 chunk 高位。
- 写入目标地址但对齐和大小检查不满足。
- 利用链依赖未记录的分配顺序。

46.8 小型演练

题目场景

现代 glibc 题有 UAF edit 和堆泄露，需要让一次分配返回程序全局函数指针。

确认目标地址对齐、可写、请求大小和 tcache bin 匹配。利用堆泄露得到被释放 chunk 地址，按 safe-linking 公式编码目标 next，使用 UAF edit 写入。连续两次相同大小分配，第二次应返回目标地址。先写无害可识别值验证任意写，再将函数指针改为程序内目标函数。每次操作和地址都由脚本日志输出。

46.9　完成检查

交付与复盘检查表

- ☐ 入口、输入、目标输出与限制条件已经写入笔记。
- ☐ 建立了可重复的基线输入与输出。
- ☐ 每个关键结论都有原始证据与验证步骤。
- ☐ 脚本能够识别失败，不会把异常输出当成成功。
- ☐ 从干净目录或新容器运行时能够得到同样结果。

第四十七章　Pwn 综合案例：泄露、基址、链条与远程稳定性

本章目标

综合题把协议同步、信息泄露、PIE/libc 基址、写原语、ROP 或堆布局组合在一起。稳定性来自把每个阶段做成可验证函数，而不是不断增加 sleep 与重试。

47.1　原理与题型边界

完整利用通常包含连接、菜单同步、基线操作、泄露解析、基址检查、原语建立、控制流触发和结果读取。任何阶段返回异常都应立即停止并保存上下文。远程地址变化由泄露解决，不能硬编码一次运行值。

47.2　识别线索与优先动作

观察线索	可能含义	优先动作
本地稳定而远程偶尔失败	协议同步、libc、延迟或实例状态不同	保存原始收发并使用 recvuntil
泄露低字节稳定高字节变化	ASLR 正常	每次运行重新计算基址
堆布局对一次失败后不再工作	异常请求改变了状态	失败时重连并从零开始
flag 输出与 shell 交互不同步	最终读取条件不明确	寻找固定提示或直接执行读文件动作

47.3　标准解题步骤

1. **固定二进制与运行环境**。计算 ELF、libc、加载器和补丁文件哈希，记录架构、端序、启动参数与远程协议。
2. **检查保护与链接信息**。把 exploit 拆为 connect、leak、calculate、build、trigger 和 collect。
3. **建立最小崩溃或信息泄露**。每阶段检查长度、范围、页对齐和返回类型。
4. **测量偏移与可控范围**。本地使用题目加载器与 libc，远程仅替换连接方式。
5. **设计最小利用原语**。设置有限重试，每次重试创建新实例状态并保存失败日志。
6. **在调试器中验证每一步**。检查寄存器、栈、堆、内存映射、调用约定和返回值，不把一次崩溃当作控制成功。
7. **使用脚本本地与远程双模式复现**。设置超时、同步菜单、解析泄露、处理 ASLR，并对失败输出明确诊断。

47.4　分析时持续追问的问题

- 当前观察是原始事实，还是工具给出的推断?
- 哪个最小输入变化能够区分主要假设?
- 结果能否在相同输入和干净环境中重复?
- 失败时是否保存了完整原始输出?
- 题目条件变化后，当前方法的前提是否仍成立?

47.5　命令、脚本与验证片段

以下内容只用于本地题目目录、回环地址、赛事提供的实例或明确授权的实验环境。每次运行都应保留原始输入和完整输出。

```
from pwn import *
context.binary=elf=ELF('work/chall',checksec=False)
libc=ELF('work/libc.so.6',checksec=False)
def start():
```

```
    return remote(args.HOST,args.PORT) if args.REMOTE else process([elf.path])
def parse_leak(raw):
    if not 1<=len(raw)<=8: raise ValueError(f'bad leak length {len(raw)}')
    return u64(raw.ljust(8,b'\0'))
def main():
    io=start()
    # stage_leak(io); stage_control(io); collect(io)
    io.close()
if __name__=='__main__': main()
```

Listing 47.1: 本章常用命令与脚本

47.6 证据保存与结果判定

保存 ELF、libc 和加载器哈希，checksec、readelf、反汇编、崩溃输入、core、寄存器、内存映射、泄露原始字节和基址计算。最终脚本应同时记录本地与远程参数，并验证关键地址落在预期映射范围。

47.7 常见失误与修正方法

- 用 sleep 掩盖菜单同步错误。
- 远程失败后在同一坏状态继续尝试。
- 泄露解析接受任意长度。
- 把本地绝对地址硬编码到远程。
- 得到交互 shell 后没有自动读取和验证 flag。

47.8 小型演练

题目场景

PIE、NX、Canary 全开，格式串可泄露，另有栈溢出。设计阶段化利用。

先用格式串编号参数泄露 Canary、PIE 返回地址和 libc 地址，每个泄露单独验证低位与映射范围。计算三个基址后构造保留 Canary 的栈溢出，使用 PIE gadget 设置参数并调用 libc 目标。脚本在每阶段输出地址和检查结果，任何值不合理就关闭连接重试。最终使用固定命令读取题目 flag，避免人工交互造成不可复现。

47.9　完成检查

交付与复盘检查表

- ☐ 入口、输入、目标输出与限制条件已经写入笔记。
- ☐ 建立了可重复的基线输入与输出。
- ☐ 每个关键结论都有原始证据与验证步骤。
- ☐ 脚本能够识别失败，不会把异常输出当成成功。
- ☐ 从干净目录或新容器运行时能够得到同样结果。

第五部分

逆向工程 Reverse 题型

第四十八章　汇编基础、寄存器、标志位与反编译器阅读

本章目标

逆向首先要能把机器指令与高层语义对应起来。参赛者不必背下全部指令，但要能识别数据移动、算术、比较、条件跳转、调用、返回、栈帧、数组索引和位运算，并知道反编译器会在哪些地方猜错类型。

48.1　原理与题型边界

反编译器根据控制流、调用约定和数据使用推断变量与类型。优化会内联函数、消除局部变量、重排循环并把除法变为乘法与移位。寄存器值只有结合位宽和标志位才有意义，32 位写入在 x86-64 中常会清零高 32 位。

48.2　识别线索与优先动作

观察线索	可能含义	优先动作
大量 cmp/test 后接条件跳转	程序在实现分支、范围检查或状态机	记录比较两端与跳转条件
lea 被反编译成奇怪指针	lea 也可用于纯算术	根据后续使用判断地址还是整数
反编译变量类型不断变化	类型推断不稳定或存在联合使用	回到汇编检查读写宽度
循环被展开或没有明显 for	编译优化改变了结构	依据回边和索引更新重建循环

48.3 标准解题步骤

1. **固定样本与执行条件**。计算样本哈希，记录架构、入口、参数、环境变量、工作目录、标准输入和外部文件。未知样本只在隔离环境运行。
2. **完成低成本静态盘点**。使用 file、readelf 与 objdump 确认架构、端序和调用约定，再阅读入口附近反汇编。
3. **建立程序结构图**。从字符串、导入函数和调用图定位输入、比较与成功分支，避免从入口线性阅读所有代码。
4. **选择最短动态验证路径**。在比较、跳转和目标函数前设置少量断点，观察实际寄存器与内存。
5. **重建关键数据变换**。把关键指令逐条转换为明确位宽的伪代码，特别标注符号扩展、截断、旋转和标志位。
6. **编写独立验证脚本**。把反编译器中的推测改写成明确位宽、端序与溢出规则的脚本，并用程序实际输入输出校验。
7. **从干净副本复现**。保存补丁、断点、跟踪日志和最终输入，重新运行原始样本确认结果。

48.4 分析时持续追问的问题

- 当前观察是原始事实，还是工具给出的推断?
- 哪个最小输入变化能够区分主要假设?
- 结果能否在相同输入和干净环境中重复?
- 失败时是否保存了完整原始输出?
- 题目条件变化后，当前方法的前提是否仍成立?

48.5 命令、脚本与验证片段

以下内容只用于本地题目目录、回环地址、赛事提供的实例或明确授权的实验环境。每次运行都应保留原始输入和完整输出。

```
file work/chall
readelf -h -s -r work/chall > notes/elf-summary.txt
objdump -d -M intel work/chall > notes/disassembly.txt
objdump -s -j .rodata work/chall > notes/rodata.txt

# Inspect a narrow function range rather than the whole binary.
gdb -q work/chall <<'GDB'
set pagination off
set disassembly-flavor intel
break main
run
set disassemble-next-line on
disassemble /r main
info registers
quit
GDB
```

Listing 48.1: 本章常用命令与脚本

48.6　证据保存与结果判定

保存原始样本哈希、文件类型、导入导出、字符串及其偏移、关键函数地址、交叉引用、反编译截图或导出文本、动态输入输出、断点状态和重建脚本。若修改样本，必须同时保留原件、补丁偏移和修改前后哈希。

48.7　常见失误与修正方法

- 把反编译输出当成源码，忽略错误类型和符号。
- 只看指令助记符，不看操作数宽度。
- 混淆有符号与无符号条件跳转。
- 看到 lea 就认定在取地址。
- 从第一条指令开始逐行阅读，迟迟找不到验证逻辑。

48.8 小型演练

题目场景

验证函数先把输入字节加 0x37，再只保留低 8 位，最后用有符号比较判断是否小于零。反编译器显示普通 int 加法。如何重建真实语义？

检查汇编读写宽度。如果字节通过 movzx 读入、加法后写回 al，结果应按 8 位截断。后续若使用 movsx 或 jl，则比较按有符号 8 位解释。脚本中先计算 (x+0x37)&0xff，再在值大于等于 0x80 时减 0x100，最后复现有符号判断。用边界输入 0x48、0x49、0xc8 验证程序与脚本一致。

48.9 完成检查

交付与复盘检查表

- ☐ 入口、输入、目标输出与限制条件已经写入笔记。
- ☐ 建立了可重复的基线输入与输出。
- ☐ 每个关键结论都有原始证据与验证步骤。
- ☐ 脚本能够识别失败，不会把异常输出当成成功。
- ☐ 从干净目录或新容器运行时能够得到同样结果。

第四十九章　静态分析：字符串、导入、交叉引用与控制流

本章目标

静态分析的目标不是阅读全部代码，而是从文件结构、字符串、导入函数、常量和交叉引用中快速找到输入处理、核心变换、成功分支和反调试逻辑。Ghidra、IDA、Binary Ninja 或 radare2 都能完成这些工作，方法比工具界面更重要。

49.1　原理与题型边界

字符串和导入是程序行为的索引。成功提示、错误提示、格式串、文件名、注册表路径、算法常量和库调用都能指向关键函数。控制流图显示分支与循环，但间接跳转、异常处理和尾调用可能使图不完整。

49.2　识别线索与优先动作

观察线索	可能含义	优先动作
存在 success、wrong、flag 等字符串	附近交叉引用可能连接验证分支	查看引用函数及其调用者
导入 memcmp、strcmp、AES、MD5 等函数	可能直接比较或调用标准算法	检查参数来源与比较长度
大量无引用函数	可能是库代码、诱饵或间接调用	先根据调用点和数据引用筛选
只看到间接 call/jmp	函数指针、虚表或跳转表参与控制流	追踪指针写入和表索引

49.3 标准解题步骤

1. **固定样本与执行条件**。计算样本哈希，记录架构、入口、参数、环境变量、工作目录、标准输入和外部文件。未知样本只在隔离环境运行。
2. **完成低成本静态盘点**。提取字符串、导入、导出、节区、符号、重定位与编译器特征。
3. **建立程序结构图**。从成功和失败文本反向追踪，建立输入函数、处理函数、比较函数和输出函数之间的简图。
4. **选择最短动态验证路径**。对关键函数重命名变量、参数与结构字段，并记录依据。
5. **重建关键数据变换**。把静态推断转为可测试输入，例如长度边界、已知前缀或单字节变化。
6. **编写独立验证脚本**。把反编译器中的推测改写成明确位宽、端序与溢出规则的脚本，并用程序实际输入输出校验。
7. **从干净副本复现**。保存补丁、断点、跟踪日志和最终输入，重新运行原始样本确认结果。

49.4 分析时持续追问的问题

- 当前观察是原始事实，还是工具给出的推断?
- 哪个最小输入变化能够区分主要假设?
- 结果能否在相同输入和干净环境中重复?
- 失败时是否保存了完整原始输出?
- 题目条件变化后，当前方法的前提是否仍成立?

49.5 命令、脚本与验证片段

以下内容只用于本地题目目录、回环地址、赛事提供的实例或明确授权的实验环境。每次运行都应保留原始输入和完整输出。

```
strings -a -t x -n 4 work/chall | tee notes/strings.txt
readelf -S -s -r -d work/chall > notes/readelf.txt
objdump -T work/chall | tee notes/dynamic-symbols.txt

# Ghidra headless analysis into a local project.
analyzeHeadless work/ghidra_project ctfproj \
  -import work/chall -overwrite \
  -analysisTimeoutPerFile 300 \
  > captures/ghidra-headless.log 2>&1

# Quick cross-reference-oriented triage with radare2.
r2 -q -c 'aaa; iz; ii; afl; s main; pdf' work/chall \
  > notes/r2-triage.txt
```

Listing 49.1: 本章常用命令与脚本

49.6　证据保存与结果判定

保存原始样本哈希、文件类型、导入导出、字符串及其偏移、关键函数地址、交叉引用、反编译截图或导出文本、动态输入输出、断点状态和重建脚本。若修改样本，必须同时保留原件、补丁偏移和修改前后哈希。

49.7　常见失误与修正方法

- 根据字符串内容直接猜 flag，没有追踪引用。
- 一次自动重命名大量变量，之后无法区分事实与猜测。
- 忽略异常处理和构造函数中的初始化。
- 只看函数调用图，不检查数据交叉引用。
- 把未被静态引用的函数认定为不可达，忽略间接调用。

49.8 小型演练

题目场景

二进制含“Correct”字符串，但唯一静态引用在一个看似从未调用的函数中。如何判断它是否真的不可达?

检查函数地址是否写入函数指针表、初始化数组、虚表或异常处理结构。搜索其地址的字节表示和重定位项，查看间接 call 的目标来源。动态运行时可在该函数入口设置断点，并使用多组输入观察是否命中。若地址通过解密后生成，静态交叉引用可能为空，需要追踪解密和间接跳转。只有静态与动态证据都支持时才能判断不可达。

49.9 完成检查

交付与复盘检查表

- □ 入口、输入、目标输出与限制条件已经写入笔记。
- □ 建立了可重复的基线输入与输出。
- □ 每个关键结论都有原始证据与验证步骤。
- □ 脚本能够识别失败，不会把异常输出当成成功。
- □ 从干净目录或新容器运行时能够得到同样结果。

第五十章　动态分析：断点、系统调用、库调用与输入差分

本章目标

动态分析用于验证静态假设和观察运行时数据。高效做法是围绕关键事件设置断点或跟踪点，例如读取输入、比较缓冲区、解密完成、间接跳转和成功分支，而不是从入口开始单步执行数万条指令。

50.1　原理与题型边界

strace 观察系统调用，ltrace 观察动态库调用，调试器观察寄存器与内存。输入差分通过只改变一个字节来定位数据依赖。断点会改变时序，遇到线程、信号或反调试时需要结合硬件断点、记录重放或补丁。

50.2　识别线索与优先动作

观察线索	可能含义	优先动作
strace 显示读取额外配置文件	关键参数不只来自标准输入	保存文件内容与读取顺序
memcmp 参数在运行时出现明文	可直接观察比较两侧	在调用点断下并导出缓冲区
同一输入每次输出不同	存在随机数、时间或未初始化数据	跟踪随机源与系统调用
断点后程序行为改变	存在时序依赖或反调试	改用硬件断点、日志或临时补丁

50.3 标准解题步骤

1. **固定样本与执行条件**。计算样本哈希，记录架构、入口、参数、环境变量、工作目录、标准输入和外部文件。未知样本只在隔离环境运行。
2. **完成低成本静态盘点**。先用 strace/ltrace 获取输入文件、环境、随机源和重要库调用的概要。
3. **建立程序结构图**。在读取、变换完成、比较和成功分支处设置断点，不在无关初始化中停留。
4. **选择最短动态验证路径**。构造基线输入，再一次改变一个字节，记录受影响的内存区域与分支。
5. **重建关键数据变换**。导出关键缓冲区，写独立脚本复现后再回到原程序验证。
6. **编写独立验证脚本**。把反编译器中的推测改写成明确位宽、端序与溢出规则的脚本，并用程序实际输入输出校验。
7. **从干净副本复现**。保存补丁、断点、跟踪日志和最终输入，重新运行原始样本确认结果。

50.4 分析时持续追问的问题

- 当前观察是原始事实，还是工具给出的推断?
- 哪个最小输入变化能够区分主要假设?
- 结果能否在相同输入和干净环境中重复?
- 失败时是否保存了完整原始输出?
- 题目条件变化后，当前方法的前提是否仍成立?

50.5 命令、脚本与验证片段

以下内容只用于本地题目目录、回环地址、赛事提供的实例或明确授权的实验环境。每次运行都应保留原始输入和完整输出。

```
strace -f -o captures/strace.log ./work/chall < work/input.txt
ltrace -f -o captures/ltrace.log ./work/chall < work/input.txt

gdb -q work/chall <<'GDB'
set pagination off
set follow-fork-mode child
break memcmp
commands
  silent
  printf "memcmp len=%ld\n", $rdx
  x/32bx $rdi
  x/32bx $rsi
  continue
end
run < work/input.txt
quit
GDB
```

Listing 50.1: 本章常用命令与脚本

50.6　证据保存与结果判定

保存原始样本哈希、文件类型、导入导出、字符串及其偏移、关键函数地址、交叉引用、反编译截图或导出文本、动态输入输出、断点状态和重建脚本。若修改样本，必须同时保留原件、补丁偏移和修改前后哈希。

50.7　常见失误与修正方法

- 无目的地逐指令单步，消耗大量时间。
- 在调试器中手工修改状态后忘记最终输入无法复现。
- ltrace 没显示调用就认定不存在，忽略静态链接和内联。
- 输入差分一次改变多个字节。
- 忽略程序读取的环境变量、当前目录和外部文件。

50.8 小型演练

题目场景

程序读取 32 字节输入，经过未知变换后调用 memcmp。请在不完全理解全部代码的情况下确定每个输入字节影响的输出位置。

在 memcmp 前断下并导出左侧缓冲区。建立全零或固定字符基线，然后依次只改变一个输入字节，重复运行并比较导出的输出。记录每个输入位置使哪些输出位置变化，形成依赖矩阵。若一对一映射，可逐字节反推；若多个位置扩散，进一步观察轮结构或分组大小。所有测试固定随机源和外部状态。

50.9 完成检查

交付与复盘检查表

- ☐ 入口、输入、目标输出与限制条件已经写入笔记。
- ☐ 建立了可重复的基线输入与输出。
- ☐ 每个关键结论都有原始证据与验证步骤。
- ☐ 脚本能够识别失败，不会把异常输出当成成功。
- ☐ 从干净目录或新容器运行时能够得到同样结果。

第五十一章　常见数据变换、校验算法与轻量混淆

本章目标

CTF 逆向常把输入经过 XOR、加减、旋转、查表、置换、Base 编码、CRC、哈希或多轮组合后比较。关键是精确恢复运算顺序、位宽、索引和常量，而不是凭“看起来像某算法”直接套库。

51.1　原理与题型边界

位运算在固定宽度整数上进行，溢出是算法的一部分。循环中的索引可能依赖前一轮状态。查表可能同时承担 S 盒、置换或编码。校验和通常可快速排除候选，但并不等同于密码学哈希。

51.2　识别线索与优先动作

观察线索	可能含义	优先动作
常量 0x9e3779b9	可能与 TEA、哈希混合或黄金比例常量相关	结合轮结构与移位判断
连续 256 字节表	可能是 S 盒、CRC 表或字节置换	检查索引和表值分布
大量 rol/ror 与异或	常见于混合函数或自定义编码	明确旋转宽度与轮次
只比较校验值而非完整输出	可能可用约束求解或碰撞	先确定校验覆盖范围

51.3 标准解题步骤

1. **固定样本与执行条件**。计算样本哈希，记录架构、入口、参数、环境变量、工作目录、标准输入和外部文件。未知样本只在隔离环境运行。
2. **完成低成本静态盘点**。标注每个常量、循环边界、索引来源、读写宽度和溢出位置。
3. **建立程序结构图**。用一到两个已知输入在调试器中记录每轮状态。
4. **选择最短动态验证路径**。把函数逐句移植到 Python，使用掩码模拟固定位宽。
5. **重建关键数据变换**。若需要逆运算，按相反顺序撤销操作，并逐层断言。
6. **编写独立验证脚本**。把反编译器中的推测改写成明确位宽、端序与溢出规则的脚本，并用程序实际输入输出校验。
7. **从干净副本复现**。保存补丁、断点、跟踪日志和最终输入，重新运行原始样本确认结果。

51.4 分析时持续追问的问题

- 当前观察是原始事实，还是工具给出的推断?
- 哪个最小输入变化能够区分主要假设?
- 结果能否在相同输入和干净环境中重复?
- 失败时是否保存了完整原始输出?
- 题目条件变化后，当前方法的前提是否仍成立?

51.5 命令、脚本与验证片段

以下内容只用于本地题目目录、回环地址、赛事提供的实例或明确授权的实验环境。每次运行都应保留原始输入和完整输出。

```
from pathlib import Path

MASK32 = 0xffffffff

```

```
def rol32(x, n):
    n &= 31
    x &= MASK32
    return ((x << n) | (x >> (32 - n))) & MASK32

def transform(block):
    x = int.from_bytes(block, 'little')
    x = (x + 0x31415926) & MASK32
    x = rol32(x ^ 0xa5a5a5a5, 7)
    return x.to_bytes(4, 'little')

raw = Path('work/input.bin').read_bytes()
assert len(raw) % 4 == 0
out = b''.join(transform(raw[i:i+4]) for i in range(0, len(raw), 4))
Path('output/rebuilt.bin').write_bytes(out)
print(out.hex())
```

Listing 51.1: 本章常用命令与脚本

51.6　证据保存与结果判定

保存原始样本哈希、文件类型、导入导出、字符串及其偏移、关键函数地址、交叉引用、反编译截图或导出文本、动态输入输出、断点状态和重建脚本。若修改样本，必须同时保留原件、补丁偏移和修改前后哈希。

51.7　常见失误与修正方法

- 忽略 8、16、32 位截断，脚本与程序逐轮偏离。
- 把相关常量当作算法的充分证据。
- 逆运算顺序写错。
- 查表索引的符号扩展导致负数或越界。
- 只验证最终输出，没有比较中间轮状态。

51.8 小型演练

题目场景

验证函数对每个字节先加索引，再旋转 3 位，最后与前一输出字节 XOR。如何构造逆变换?

正向存在链式依赖,但解密时目标输出已知。设 y[i]=rol8((x[i]+i)&0xff,3)^y[i-1]，其中 y[-1] 为固定初值。先计算 t=y[i]^y[i-1]，再对 t 右旋 3 位，最后减去 i 并按 8 位截断得到 x[i]。使用程序生成的一组样本验证每一步，并检查初值与循环起点。

51.9 完成检查

交付与复盘检查表

- □ 入口、输入、目标输出与限制条件已经写入笔记。
- □ 建立了可重复的基线输入与输出。
- □ 每个关键结论都有原始证据与验证步骤。
- □ 脚本能够识别失败，不会把异常输出当成成功。
- □ 从干净目录或新容器运行时能够得到同样结果。

第五十二章　壳、压缩、自修改代码与运行时解包

本章目标

加壳或运行时解包会隐藏原始入口、字符串和导入，使静态分析只能看到解包器。CTF 中常见 UPX、简单段解密、内存复制后改权限以及自定义加载器。目标是找到原始入口和解包后的内存，而不是在壳代码里长期停留。

52.1　原理与题型边界

解包器通常分配或映射内存、解密或解压数据、修复导入与重定位，然后跳转到新代码。自修改代码会写入当前或新代码页。操作系统的 mmap、mprotect、VirtualAlloc、VirtualProtect 等调用提供明显事件。

52.2　识别线索与优先动作

观察线索	可能含义	优先动作
节名异常、入口很小、导入极少	可能是壳或自定义加载器	检查熵、入口调用和内存权限变化
运行时调用 mprotect 赋予执行权限	代码可能刚被解密或复制	在调用后记录新区域
静态字符串很少，运行后出现明文	数据在运行时解密	在解密完成处导出内存
程序跳入堆或匿名映射	原始代码位于新映射	记录映射范围和跳转目标

52.3 标准解题步骤

1. **固定样本与执行条件**。计算样本哈希，记录架构、入口、参数、环境变量、工作目录、标准输入和外部文件。未知样本只在隔离环境运行。
2. **完成低成本静态盘点**。用 file、节区布局、熵和已知壳特征做初步判断。
3. **建立程序结构图**。跟踪 mmap、mprotect、VirtualAlloc 等内存事件，记录地址与权限。
4. **选择最短动态验证路径**。在跳入新区域前暂停，导出内存并标记原始入口。
5. **重建关键数据变换**。对导出内容重新识别，必要时修复文件头、节区或导入，只为便于分析。
6. **编写独立验证脚本**。把反编译器中的推测改写成明确位宽、端序与溢出规则的脚本，并用程序实际输入输出校验。
7. **从干净副本复现**。保存补丁、断点、跟踪日志和最终输入，重新运行原始样本确认结果。

52.4 分析时持续追问的问题

- 当前观察是原始事实，还是工具给出的推断?
- 哪个最小输入变化能够区分主要假设?
- 结果能否在相同输入和干净环境中重复?
- 失败时是否保存了完整原始输出?
- 题目条件变化后，当前方法的前提是否仍成立?

52.5 命令、脚本与验证片段

以下内容只用于本地题目目录、回环地址、赛事提供的实例或明确授权的实验环境。每次运行都应保留原始输入和完整输出。

```
file work/packed
readelf -S -l work/packed | tee notes/packed-layout.txt
strings -a -n 6 work/packed | tee notes/packed-strings.txt
```

```
strace -f -e trace=mmap,mprotect,munmap,read -o captures/memory-events.log ./
    work/packed

gdb -q work/packed <<'GDB'
set pagination off
catch syscall mprotect
run
commands
  silent
  info registers
  info proc mappings
  continue
end
GDB
```

Listing 52.1: 本章常用命令与脚本

52.6　证据保存与结果判定

保存原始样本哈希、文件类型、导入导出、字符串及其偏移、关键函数地址、交叉引用、反编译截图或导出文本、动态输入输出、断点状态和重建脚本。若修改样本，必须同时保留原件、补丁偏移和修改前后哈希。

52.7　常见失误与修正方法

- 看到高熵就断定是加密或恶意代码。
- 直接对原件执行未知解壳工具。
- 导出内存后没有记录基址和权限。
- 把壳修复成可运行文件花费过多时间，忽略只需分析内存。
- 补丁跳过解包后未验证数据和重定位已经完成。

52.8 小型演练

题目场景

程序入口只做内存分配、读取一段高熵数据、调用 mprotect，然后跳入匿名映射。如何取得可分析代码?

在 mprotect 返回后查看参数和进程映射，确认新区域起止地址及执行权限。在最终间接跳转前断下，用调试器 dump binary memory 导出该区域，并记录基址。对导出文件运行 file、strings 和反汇编。若代码使用绝对地址，静态工具中把加载基址设为运行时地址。首先验证导出区域包含稳定函数序言和字符串引用，再继续分析。

52.9 完成检查

交付与复盘检查表

- □ 入口、输入、目标输出与限制条件已经写入笔记。
- □ 建立了可重复的基线输入与输出。
- □ 每个关键结论都有原始证据与验证步骤。
- □ 脚本能够识别失败，不会把异常输出当成成功。
- □ 从干净目录或新容器运行时能够得到同样结果。

第五十三章　反调试、反虚拟机、时间检测与环境检查

本章目标

反调试逻辑通过 ptrace、进程信息、异常、时间差、硬件特征、窗口或设备名称判断分析环境。CTF 的目标通常是识别检查点、理解其返回值如何影响控制流，并用最小补丁或调试设置获得真实逻辑。

53.1　原理与题型边界

反调试检查本身可能是诱饵。绕过方法包括修改返回值、补丁条件跳转、模拟预期环境、隐藏断点或在检查后附加。任何绕过都应记录修改位置，并验证没有跳过必要初始化。

53.2　识别线索与优先动作

观察线索	可能含义	优先动作
调用 ptrace 或查询 TracerPid	程序检查调试器存在	在返回后观察分支并尝试修改返回值
读取时间两次并比较差值	单步执行触发时间阈值	在比较处补丁或固定时间源
检查虚拟机设备、MAC、CPUID	程序识别分析环境	确定检查结果是否真的影响核心逻辑
故意触发异常并依赖处理器	反调试或控制流混淆	跟踪信号与异常处理路径

53.3 标准解题步骤

1. **固定样本与执行条件**。计算样本哈希，记录架构、入口、参数、环境变量、工作目录、标准输入和外部文件。未知样本只在隔离环境运行。
2. **完成低成本静态盘点**。列出所有环境查询、时间源、异常和调试器检测调用。
3. **建立程序结构图**。为每个检查建立输入、返回值和后续分支表。
4. **选择最短动态验证路径**。优先在检查结果使用处修改状态，而不是大范围删除代码。
5. **重建关键数据变换**。绕过后重新确认输入处理、密钥初始化和目标分支仍完整执行。
6. **编写独立验证脚本**。把反编译器中的推测改写成明确位宽、端序与溢出规则的脚本，并用程序实际输入输出校验。
7. **从干净副本复现**。保存补丁、断点、跟踪日志和最终输入，重新运行原始样本确认结果。

53.4 分析时持续追问的问题

- 当前观察是原始事实，还是工具给出的推断?
- 哪个最小输入变化能够区分主要假设?
- 结果能否在相同输入和干净环境中重复?
- 失败时是否保存了完整原始输出?
- 题目条件变化后，当前方法的前提是否仍成立?

53.5 命令、脚本与验证片段

以下内容只用于本地题目目录、回环地址、赛事提供的实例或明确授权的实验环境。每次运行都应保留原始输入和完整输出。

```
# Observe common Linux anti-debug checks.
strace -f -e trace=ptrace,openat,read,clock_gettime,gettimeofday \
  -o captures/anti-debug.log ./work/chall
```

```
gdb -q work/chall <<'GDB'
set pagination off
catch syscall ptrace
run
finish
print $rax
set $rax = 0
continue
GDB

# Compare /proc status only inside the analysis VM.
grep -E 'TracerPid|Seccomp' /proc/$$/status | tee notes/proc-status.txt
```

Listing 53.1: 本章常用命令与脚本

53.6　证据保存与结果判定

保存原始样本哈希、文件类型、导入导出、字符串及其偏移、关键函数地址、交叉引用、反编译截图或导出文本、动态输入输出、断点状态和重建脚本。若修改样本，必须同时保留原件、补丁偏移和修改前后哈希。

53.7　常见失误与修正方法

- 发现 ptrace 就立即全局补丁，破坏其他逻辑。
- 绕过检查后没有验证初始化是否完成。
- 把所有异常都误判为反调试。
- 修改系统时间或宿主环境，影响其他证据。
- 只在调试器内改寄存器，没有形成可复现方案。

53.8 小型演练

题目场景

程序在调试器中总走失败分支，非调试运行则正常。strace 显示 ptrace(PTRACE_TRACEME)。如何最小化验证?

在 ptrace 返回后断下，记录正常与调试状态的返回值和 errno，确认后续条件跳转。只把该返回值改为正常情况下的值并继续，观察是否进入同一初始化与验证逻辑。若成功，可在副本中补丁紧邻的条件跳转，或在启动时使用脚本化调试命令修改返回值。保留原始样本与补丁偏移，并比较绕过前后执行路径。

53.9 完成检查

交付与复盘检查表

- ☐ 入口、输入、目标输出与限制条件已经写入笔记。
- ☐ 建立了可重复的基线输入与输出。
- ☐ 每个关键结论都有原始证据与验证步骤。
- ☐ 脚本能够识别失败，不会把异常输出当成成功。
- ☐ 从干净目录或新容器运行时能够得到同样结果。

第五十四章　自定义虚拟机、字节码与状态机逆向

本章目标

自定义 VM 题把验证逻辑编码为字节码，由解释器循环取指、解码和执行。解题重点是恢复虚拟寄存器、栈、内存、指令格式和控制流，再决定写反汇编器、模拟器还是直接求约束。

54.1　原理与题型边界

典型解释器包含程序计数器、opcode 分派、操作数读取和状态更新。分派可使用 switch、跳转表、函数指针或异常。字节码可能是变长指令，也可能在执行前解密。恢复语义时要区分宿主机器状态与虚拟机状态。

54.2　识别线索与优先动作

观察线索	可能含义	优先动作
大循环读取数组并按字节分派	典型解释器取指循环	定位 PC 更新和 opcode 分派
大量结构相似的小处理块	每块可能对应一种虚拟指令	记录输入操作数与状态变化
间接跳转目标来自表	可能是 direct threaded dispatch	导出表并映射处理函数
输入只在虚拟内存中使用	核心校验位于字节码	写模拟器跟踪虚拟状态

54.3 标准解题步骤

1. **固定样本与执行条件**。计算样本哈希，记录架构、入口、参数、环境变量、工作目录、标准输入和外部文件。未知样本只在隔离环境运行。
2. **完成低成本静态盘点**。找到解释器循环、虚拟 PC、寄存器或栈结构和字节码缓冲区。
3. **建立程序结构图**。为每个 opcode 记录长度、操作数编码、状态变化和跳转行为。
4. **选择最短动态验证路径**。编写反汇编器，把字节码转为可读指令并标注目标地址。
5. **重建关键数据变换**。实现最小模拟器，逐步对比原程序中的虚拟状态，之后再求输入。
6. **编写独立验证脚本**。把反编译器中的推测改写成明确位宽、端序与溢出规则的脚本，并用程序实际输入输出校验。
7. **从干净副本复现**。保存补丁、断点、跟踪日志和最终输入，重新运行原始样本确认结果。

54.4 分析时持续追问的问题

- 当前观察是原始事实，还是工具给出的推断?
- 哪个最小输入变化能够区分主要假设?
- 结果能否在相同输入和干净环境中重复?
- 失败时是否保存了完整原始输出?
- 题目条件变化后，当前方法的前提是否仍成立?

54.5 命令、脚本与验证片段

以下内容只用于本地题目目录、回环地址、赛事提供的实例或明确授权的实验环境。每次运行都应保留原始输入和完整输出。

```python
from pathlib import Path

code = Path('work/bytecode.bin').read_bytes()
OPLEN = {0x10: 3, 0x20: 3, 0x30: 2, 0x40: 3, 0xff: 1}

pc = 0
while pc < len(code):
    op = code[pc]
    if op == 0x10:
        print(f'{pc:04x}: MOVI r{code[pc+1]}, {code[pc+2]:#x}')
    elif op == 0x20:
        print(f'{pc:04x}: XOR  r{code[pc+1]}, r{code[pc+2]}')
    elif op == 0x30:
        print(f'{pc:04x}: CMPZ r{code[pc+1]}')
    elif op == 0x40:
        target = int.from_bytes(code[pc+1:pc+3], 'little')
        print(f'{pc:04x}: JNZ  {target:#x}')
    elif op == 0xff:
        print(f'{pc:04x}: HALT')
    else:
        raise ValueError(f'unknown opcode {op:#x} at {pc:#x}')
    pc += OPLEN[op]
```

Listing 54.1: 本章常用命令与脚本

54.6　证据保存与结果判定

保存原始样本哈希、文件类型、导入导出、字符串及其偏移、关键函数地址、交叉引用、反编译截图或导出文本、动态输入输出、断点状态和重建脚本。若修改样本，必须同时保留原件、补丁偏移和修改前后哈希。

54.7　常见失误与修正方法

- 只根据处理函数大小猜 opcode 语义。
- 忘记分支指令会改变虚拟 PC。
- 把宿主寄存器当成虚拟寄存器。
- 写模拟器但没有逐步对照原程序状态。
- 忽略字节码执行前的解密或重定位。

54.8 小型演练

题目场景

解释器有 12 个 opcode，最终只比较虚拟寄存器 r3 是否为零。怎样决定使用模拟、符号执行还是直接逆运算？

先写反汇编器恢复控制流，并用具体输入实现解释器的确定性模拟。如果程序基本线性且操作可逆，直接从目标状态逆推最快。如果存在少量分支和位向量运算，可把输入字节和寄存器建模为约束交给 Z3。如果循环次数依赖输入或状态空间很大，先通过动态跟踪和静态数据流削减分支，再选择局部符号执行。无论哪种方法，都用原解释器验证最终输入。

54.9 完成检查

交付与复盘检查表

- □ 入口、输入、目标输出与限制条件已经写入笔记。
- □ 建立了可重复的基线输入与输出。
- □ 每个关键结论都有原始证据与验证步骤。
- □ 脚本能够识别失败，不会把异常输出当成成功。
- □ 从干净目录或新容器运行时能够得到同样结果。

第五十五章　.NET、Java、Python 字节码与托管程序

本章目标

托管程序通常保留更多类型、类名、方法签名和元数据，静态恢复比本地代码容易，但仍可能有混淆、反射、动态加载、资源加密和本地库调用。应先利用运行时格式信息，再决定是否进入底层字节码。

55.1　原理与题型边界

Java class 使用 JVM 字节码，.NET 程序使用 CIL 与元数据，Python pyc 保存代码对象和字节码。反编译输出接近源码，但编译器合成结构、异步状态机、闭包和混淆器会改变可读性。反射和动态加载使静态调用关系不完整。

55.2　识别线索与优先动作

观察线索	可能含义	优先动作
文件为 PE 但导入 mscoree.dll	可能是.NET 程序	检查 CLR 元数据并使用托管反编译器
JAR 中有大量 class 和资源	逻辑与密钥可能分散在类和资源中	列出清单、入口类和资源文件
调用 Class.forName 或反射 API	目标方法在运行时解析	记录字符串与加载类名
包含 native 方法或 JNI/PInvoke	核心逻辑进入本地库	同时分析对应 so 或 DLL

55.3 标准解题步骤

1. **固定样本与执行条件**。计算样本哈希，记录架构、入口、参数、环境变量、工作目录、标准输入和外部文件。未知样本只在隔离环境运行。
2. **完成低成本静态盘点**。确认运行时、入口点、目标框架版本、清单、资源与依赖。
3. **建立程序结构图**。反编译后从输入、比较、成功文本与资源加载位置建立调用链。
4. **选择最短动态验证路径**。对反射、动态加载和本地调用单独记录实际目标。
5. **重建关键数据变换**。把关键算法移植到脚本，保留 Java/.NET 的整数溢出、字符串和编码语义。
6. **编写独立验证脚本**。把反编译器中的推测改写成明确位宽、端序与溢出规则的脚本，并用程序实际输入输出校验。
7. **从干净副本复现**。保存补丁、断点、跟踪日志和最终输入，重新运行原始样本确认结果。

55.4 分析时持续追问的问题

- 当前观察是原始事实，还是工具给出的推断?
- 哪个最小输入变化能够区分主要假设?
- 结果能否在相同输入和干净环境中重复?
- 失败时是否保存了完整原始输出?
- 题目条件变化后，当前方法的前提是否仍成立?

55.5 命令、脚本与验证片段

以下内容只用于本地题目目录、回环地址、赛事提供的实例或明确授权的实验环境。每次运行都应保留原始输入和完整输出。

```
# Java archive triage.
unzip -l work/challenge.jar | tee notes/jar-list.txt
unzip -p work/challenge.jar META-INF/MANIFEST.MF | tee notes/manifest.txt
javap -classpath work/challenge.jar -c -p com.ctf.Main > notes/javap-main.txt

# Python bytecode metadata and disassembly.
python3 - <<'PYCODE'
import dis, marshal, pathlib
p = pathlib.Path('work/module.pyc').read_bytes()
# The header length depends on Python version; use a matching interpreter.
code = marshal.loads(p[16:])
print(code.co_names)
dis.dis(code)
PYCODE

# Basic .NET metadata when the local tool is available.
file work/challenge.exe
strings -a -el work/challenge.exe | tee notes/dotnet-wide-strings.txt
```

Listing 55.1: 本章常用命令与脚本

55.6　证据保存与结果判定

保存原始样本哈希、文件类型、导入导出、字符串及其偏移、关键函数地址、交叉引用、反编译截图或导出文本、动态输入输出、断点状态和重建脚本。若修改样本，必须同时保留原件、补丁偏移和修改前后哈希。

55.7　常见失误与修正方法

- 直接用错误版本 Python 读取 pyc。
- 反编译器生成的高层结构与真实字节码不一致时仍照抄。
- 忽略资源文件、配置和嵌入程序集。
- 看到反射就放弃静态分析，没有追踪类名字符串。
- 只分析托管层，遗漏 native 库中的核心校验。

55.8 小型演练

题目场景

JAR 的 Main 只通过反射调用名称被 Base64 编码的方法，方法又读取加密资源。如何梳理调用链?

先从清单确定入口类，查看 Main 的字节码和反编译结果，恢复 Base64 解码后的类名与方法名。搜索对应类，确认反射参数类型。跟踪资源名到 getResourceAsStream，提取资源并记录解密函数。将名称解码、资源解密和目标方法调用分别写成脚本，逐段与 JVM 运行结果比较。若目标方法通过 JNI 进入本地库，再把导出符号与 Java native 声明对应起来。

55.9 完成检查

交付与复盘检查表

- ☐ 入口、输入、目标输出与限制条件已经写入笔记。
- ☐ 建立了可重复的基线输入与输出。
- ☐ 每个关键结论都有原始证据与验证步骤。
- ☐ 脚本能够识别失败，不会把异常输出当成成功。
- ☐ 从干净目录或新容器运行时能够得到同样结果。

第五十六章　逆向综合案例：从样本盘点到独立求解器

本章目标

综合题往往同时包含压缩或解包、反调试、输入变换、校验和状态机。解题时应把它拆成独立阶段，每阶段都有输入、输出和验证点，避免在一个巨大的反编译函数中同时处理所有问题。

56.1　原理与题型边界

可靠路线通常是：固定样本，找到输入与成功分支，绕过但不删除环境检查，导出运行时数据，重建核心变换，最后由原程序验证。补丁只用于观察和缩短路径，最终说明中应区分补丁获得的信息与真正求解逻辑。

56.2　识别线索与优先动作

观察线索	可能含义	优先动作
入口只见解包器，运行后出现验证字符串	先处理运行时解包	在跳入新区域前导出内存
校验前有时间和调试检查	环境检查可能阻断真实逻辑	记录返回值并做最小绕过
核心函数包含多轮位运算和查表	需要逐轮重建	导出轮状态与表数据
最终只比较 16 字节摘要	可通过逆运算、约束或候选校验求输入	确定摘要是否可逆及输入约束

56.3 标准解题步骤

1. **固定样本与执行条件**。计算样本哈希，记录架构、入口、参数、环境变量、工作目录、标准输入和外部文件。未知样本只在隔离环境运行。
2. **完成低成本静态盘点**。建立原始样本和所有外部资源的哈希清单。
3. **建立程序结构图**。用系统调用跟踪与字符串交叉引用找到运行时入口和输入路径。
4. **选择最短动态验证路径**。导出解包代码并最小绕过反调试，确认成功分支位置。
5. **重建关键数据变换**。逐轮记录变换状态，写求解器并由未修改原程序验证。
6. **编写独立验证脚本**。把反编译器中的推测改写成明确位宽、端序与溢出规则的脚本，并用程序实际输入输出校验。
7. **从干净副本复现**。保存补丁、断点、跟踪日志和最终输入，重新运行原始样本确认结果。

56.4 分析时持续追问的问题

- 当前观察是原始事实，还是工具给出的推断?
- 哪个最小输入变化能够区分主要假设?
- 结果能否在相同输入和干净环境中重复?
- 失败时是否保存了完整原始输出?
- 题目条件变化后，当前方法的前提是否仍成立?

56.5 命令、脚本与验证片段

以下内容只用于本地题目目录、回环地址、赛事提供的实例或明确授权的实验环境。每次运行都应保留原始输入和完整输出。

```
# Reproducible reverse-engineering case directory.
mkdir -p case/{original,work,notes,captures,scripts,output}
sha256sum case/original/* | tee case/notes/SHA256SUMS
file case/original/* | tee case/notes/file.txt
```

```
5
6  # Run the independent solver, then validate with the untouched sample.
7  python3 case/scripts/solve.py case/original/challenge.bin \
8    | tee case/output/candidate.txt
9
10 env -i PATH=/usr/bin:/bin LANG=C \
11   timeout 5s case/original/challenge.bin \
12   < case/output/candidate.txt \
13   > case/captures/validation.stdout \
14   2> case/captures/validation.stderr
15 printf 'exit=%s\n' "$?" | tee case/captures/validation.meta
```

Listing 56.1: 本章常用命令与脚本

56.6　证据保存与结果判定

保存原始样本哈希、文件类型、导入导出、字符串及其偏移、关键函数地址、交叉引用、反编译截图或导出文本、动态输入输出、断点状态和重建脚本。若修改样本，必须同时保留原件、补丁偏移和修改前后哈希。

56.7　常见失误与修正方法

- 把补丁后直接进入成功分支当成解题。
- 没有区分解包数据、算法常量和输入相关状态。
- 只保存反编译工程，没有导出关键结论。
- 求解器接受旧缓存，干净运行失败。
- 最终输入只在调试器修改内存后有效。

56.8　小型演练

题目场景

样本运行时解包，检查调试器，读取 20 字节输入，经过四轮混合后比较摘要。设计完整交付。

交付应包含原样本与哈希、解包区域的基址和导出文件、反调试检查位置与最小绕过记录、四轮算法的位宽和常量说明、每轮已知样本状态、独立 solve.py、候选输入以及未修改样本的验证日志。最终说明明确哪一步只是为了观察，哪一步真正从目标摘要推导输入。新环境执行一条命令应先求候选，再把候选喂给原样本并看到成功文本。

56.9 完成检查

交付与复盘检查表

- ☐ 入口、输入、目标输出与限制条件已经写入笔记。
- ☐ 建立了可重复的基线输入与输出。
- ☐ 每个关键结论都有原始证据与验证步骤。
- ☐ 脚本能够识别失败，不会把异常输出当成成功。
- ☐ 从干净目录或新容器运行时能够得到同样结果。

第六部分

数字取证、隐写与 Misc 题型

第五十七章　文件签名、格式结构、数据拼接与文件雕刻

本章目标

取证题常把多个格式拼接、修改扩展名、损坏文件头、删除目录项或把负载嵌入未使用区域。文件雕刻的目标是根据结构恢复对象，并证明起止边界，而不是仅在十六进制中搜索一个魔数。

57.1　原理与题型边界

多数格式有头部、长度字段、索引、块结构、结束标记或校验。魔数可能自然出现在压缩数据中，因此单个签名只能作为候选。可靠恢复要结合长度、内部引用、校验和与解析器结果。

57.2　识别线索与优先动作

观察线索	可能含义	优先动作
扩展名与 file 结果不一致	文件被改名或多层封装	检查头部、尾部和容器目录
解析器提示尾随数据	主文件后可能拼接另一对象	定位真实结束位置并验证尾部
文件头损坏但内部结构可见	可能只需修复少量字段	对照同格式样本和规范
磁盘镜像中出现孤立魔数	可能是删除文件或数据片段	结合连续块和格式长度雕刻

57.3　标准解题步骤

1. **保存原始证据与上下文**。计算哈希，记录题面、来源、获取时间、时区、文件大小和任何已知采集方式。所有转换都在副本上进行。
2. **进行低侵入盘点**。用 file、xxd、strings、binwalk 签名和格式专用工具盘点，但不让自动提取覆盖证据。
3. **建立结构或时间基线**。标记候选头、长度字段、块序列、结束标记和文件总长度。
4. **提出可检验假设**。为每个候选边界写出依据，排除随机匹配。
5. **使用专用工具提取**。使用 dd 或 Python 按精确偏移提取，再由独立解析器验证。
6. **交叉验证结果**。至少使用结构约束、第二种工具、原始字节或上下文事件验证关键结论。
7. **导出可复现过程**。保存命令、过滤器、偏移、时区、脚本、提取文件哈希和最终证据链。

57.4　分析时持续追问的问题

- 当前观察是原始事实，还是工具给出的推断?
- 哪个最小输入变化能够区分主要假设?
- 结果能否在相同输入和干净环境中重复?
- 失败时是否保存了完整原始输出?
- 题目条件变化后，当前方法的前提是否仍成立?

57.5　命令、脚本与验证片段

以下内容只用于本地题目目录、回环地址、赛事提供的实例或明确授权的实验环境。每次运行都应保留原始输入和完整输出。

```
sha256sum original/evidence.bin | tee notes/SHA256SUMS
file original/evidence.bin | tee notes/file.txt
xxd -g 1 -l 256 original/evidence.bin > notes/head.hex
tail -c 512 original/evidence.bin | xxd -g 1 > notes/tail.hex
binwalk original/evidence.bin | tee notes/binwalk.txt

# Extract a proven byte range from a copy.
dd if=original/evidence.bin of=work/carved.bin \
  bs=1 skip=65536 count=24576 status=none
sha256sum work/carved.bin | tee notes/carved.sha256
file work/carved.bin
```

Listing 57.1: 本章常用命令与脚本

57.6　证据保存与结果判定

保存原始证据哈希、采集上下文、工具版本、查询或过滤器、所有提取文件的来源偏移与哈希、时间戳所用时区以及关键截图或原始字节。任何修复、解压、转换、挂载和解码必须在副本中进行，并保留前后对应关系。

57.7　常见失误与修正方法

- 只搜索魔数就直接切分，忽略长度和校验。
- 在原件上使用修复工具。
- 自动雕刻生成很多文件后逐个猜，没有记录偏移。
- 把解析器能打开当作内容完整。
- 忽略文件尾部、填充区和容器注释。

57.8　小型演练

题目场景

一张 PNG 能正常显示，但尾部多出 18 KB。如何判断尾部是否为有效 ZIP，而不是随机字节？

解析 PNG 块直到 IEND，计算 IEND 块结束偏移并与总长度比较。对尾部搜索 ZIP 本地文件头、中央目录和结束记录，检查目录中偏移是否落在尾部范围。按 PNG 真实结束位置提取尾部，再用 unzip -t 和列表模式验证。保存 PNG 与 ZIP 的哈希、切分偏移和解析器输出。仅看到 PK 字节不足以证明 ZIP 有效。

57.9　完成检查

交付与复盘检查表

- □ 入口、输入、目标输出与限制条件已经写入笔记。
- □ 建立了可重复的基线输入与输出。
- □ 每个关键结论都有原始证据与验证步骤。
- □ 脚本能够识别失败，不会把异常输出当成成功。
- □ 从干净目录或新容器运行时能够得到同样结果。

第五十八章　磁盘镜像、分区表与文件系统恢复

本章目标

磁盘取证题可能要求从原始镜像、E01、虚拟磁盘或分区镜像中恢复删除文件、浏览目录、读取日志和分析未分配空间。应先理解镜像层级，再选择只读挂载或 Sleuth Kit 等工具。

58.1　原理与题型边界

磁盘可含分区表、多个分区、文件系统元数据、分配与未分配块、日志和交换空间。删除通常只移除目录项或标记块可用，内容可能仍在。时间戳语义随文件系统而异。

58.2　识别线索与优先动作

观察线索	可能含义	优先动作
mmls 显示多个分区	文件系统从分区偏移开始	记录扇区大小和起始扇区
目录项显示 deleted	元数据仍指向原数据块	用 inode 或元数据地址恢复
挂载失败但 fsstat 可识别	文件系统可能损坏或镜像不是整个磁盘	使用只读取证工具而非修复
同一文件有多种时间戳	创建、修改、元数据变更和访问语义不同	记录时区与时间戳类型

58.3　标准解题步骤

1. **保存原始证据与上下文**。计算哈希，记录题面、来源、获取时间、时区、文件大小和任何已知采集方式。所有转换都在副本上进行。
2. **进行低侵入盘点**。识别镜像格式、总大小、扇区大小、分区表和每个分区偏移。
3. **建立结构或时间基线**。对目标分区运行 fsstat、fls 或等价工具，建立目录与时间基线。
4. **提出可检验假设**。根据题面寻找删除项、异常扩展名、最近修改、用户目录和未分配空间。
5. **使用专用工具提取**。按 inode、元数据地址或块范围恢复，避免以读写方式挂载。
6. **交叉验证结果**。至少使用结构约束、第二种工具、原始字节或上下文事件验证关键结论。
7. **导出可复现过程**。保存命令、过滤器、偏移、时区、脚本、提取文件哈希和最终证据链。

58.4　分析时持续追问的问题

- 当前观察是原始事实，还是工具给出的推断？
- 哪个最小输入变化能够区分主要假设？
- 结果能否在相同输入和干净环境中重复？
- 失败时是否保存了完整原始输出？
- 题目条件变化后，当前方法的前提是否仍成立？

58.5　命令、脚本与验证片段

以下内容只用于本地题目目录、回环地址、赛事提供的实例或明确授权的实验环境。每次运行都应保留原始输入和完整输出。

```
sha256sum original/disk.raw | tee notes/disk.sha256
file original/disk.raw | tee notes/disk-file.txt
mmls original/disk.raw | tee notes/partitions.txt

# Example: partition starts at sector 2048, sector size 512.
OFFSET=$((2048*512))
fsstat -o 2048 original/disk.raw > notes/fsstat.txt
fls -r -p -o 2048 original/disk.raw > notes/fls.txt

# Recover an item by metadata address shown by fls.
icat -o 2048 original/disk.raw 128-1 > work/recovered.bin
sha256sum work/recovered.bin
file work/recovered.bin

# Optional read-only loop mount inside an analysis VM.
sudo mount -o ro,loop,offset=$OFFSET original/disk.raw /mnt/ctf_disk
```

Listing 58.1: 本章常用命令与脚本

58.6 证据保存与结果判定

保存原始证据哈希、采集上下文、工具版本、查询或过滤器、所有提取文件的来源偏移与哈希、时间戳所用时区以及关键截图或原始字节。任何修复、解压、转换、挂载和解码必须在副本中进行，并保留前后对应关系。

58.7 常见失误与修正方法

- 把分区起始扇区直接当字节偏移。
- 以读写方式挂载并触发日志回放。
- 使用文件系统修复工具改变证据。
- 只看当前目录，忽略删除项和未分配空间。
- 混淆主机时区、文件系统时区和应用日志时区。

58.8　小型演练

题目场景

原始磁盘镜像包含一个 NTFS 分区。题目提示用户删除了 secret.txt。请设计恢复与验证流程。

用 mmls 记录 NTFS 分区起始扇区，使用 fls -r -d 列出删除项并搜索文件名。记录 secret.txt 的元数据地址，再用 icat 按该地址恢复。对恢复文件计算哈希，使用 file 与内容结构验证。若目录项已丢失，再在未分配空间使用 blkls 导出并按文件格式雕刻。全过程不挂载为可写，不运行 chkdsk。

58.9　完成检查

交付与复盘检查表

- □ 入口、输入、目标输出与限制条件已经写入笔记。
- □ 建立了可重复的基线输入与输出。
- □ 每个关键结论都有原始证据与验证步骤。
- □ 脚本能够识别失败，不会把异常输出当成成功。
- □ 从干净目录或新容器运行时能够得到同样结果。

第五十九章　网络流量 PCAP：会话重组、协议分析与对象提取

本章目标

PCAP 题要求从大量数据包中恢复会话、文件、凭据片段、DNS 编码、异常流量或攻击时间线。有效方法是先做协议和端点统计，再聚焦少量会话，而不是逐包浏览。

59.1　原理与题型边界

抓包包含链路层、网络层、传输层和应用层。TCP 需要重组并处理重传、乱序和分段。TLS 加密后通常只能观察握手、证书、SNI、流量特征，除非题目提供密钥或会话密钥。

59.2　识别线索与优先动作

观察线索	可能含义	优先动作
少数主机产生大量连接	可能是扫描、自动化或数据传输	统计端点、端口和会话数
HTTP 有对象或表单	可直接重组请求与响应	导出对象并保留流编号
DNS 查询名很长且高熵	可能编码数据或隧道	提取标签、顺序和响应
TCP 分析显示重传与乱序	直接按包拼接会出错	使用流重组而非手工连接 payload

59.3　标准解题步骤

1. **保存原始证据与上下文**。计算哈希，记录题面、来源、获取时间、时区、文件大小和任何已知采集方式。所有转换都在副本上进行。
2. **进行低侵入盘点**。读取 capture metadata，统计协议、端点、端口、会话、时间范围和异常包。
3. **建立结构或时间基线**。根据题面建立过滤器，选出候选流并记录 stream 编号。
4. **提出可检验假设**。使用 Follow Stream、tshark 字段或协议解析器重组应用数据。
5. **使用专用工具提取**。导出文件或字段后重新识别、计算哈希并关联到原流与时间。
6. **交叉验证结果**。至少使用结构约束、第二种工具、原始字节或上下文事件验证关键结论。
7. **导出可复现过程**。保存命令、过滤器、偏移、时区、脚本、提取文件哈希和最终证据链。

59.4　分析时持续追问的问题

- 当前观察是原始事实，还是工具给出的推断?
- 哪个最小输入变化能够区分主要假设?
- 结果能否在相同输入和干净环境中重复?
- 失败时是否保存了完整原始输出?
- 题目条件变化后，当前方法的前提是否仍成立?

59.5　命令、脚本与验证片段

以下内容只用于本地题目目录、回环地址、赛事提供的实例或明确授权的实验环境。每次运行都应保留原始输入和完整输出。

```
capinfos original/traffic.pcap | tee notes/capinfos.txt
tshark -r original/traffic.pcap -q -z io,phs | tee notes/protocol-hierarchy.txt
tshark -r original/traffic.pcap -q -z endpoints,ip | tee notes/endpoints.txt
tshark -r original/traffic.pcap -q -z conv,tcp | tee notes/tcp-conversations.
    txt

# List HTTP requests with stream identifiers.
tshark -r original/traffic.pcap \
  -Y 'http.request' \
  -T fields -e frame.time_epoch -e tcp.stream -e ip.src \
  -e http.request.method -e http.host -e http.request.uri \
  > notes/http-requests.tsv

# Extract a selected TCP stream as raw hex.
tshark -r original/traffic.pcap -q -z follow,tcp,raw,7 \
  > work/tcp-stream-7.txt

# DNS query names for later ordered decoding.
tshark -r original/traffic.pcap -Y 'dns.flags.response == 0' \
  -T fields -e frame.number -e frame.time_epoch -e ip.src -e dns.qry.name \
  > notes/dns-queries.tsv
```

Listing 59.1: 本章常用命令与脚本

59.6　证据保存与结果判定

保存原始证据哈希、采集上下文、工具版本、查询或过滤器、所有提取文件的来源偏移与哈希、时间戳所用时区以及关键截图或原始字节。任何修复、解压、转换、挂载和解码必须在副本中进行，并保留前后对应关系。

59.7　常见失误与修正方法

- 把数据包顺序直接当应用字节顺序，忽略重传和分段。
- 导出对象后未记录来源流与帧号。
- 用显示过滤器后以为数据从文件中删除。
- 仅看默认解析，忽略端口上运行的非标准协议。
- 没有统一时区，无法与主机日志关联。

59.8　小型演练

题目场景

抓包中 DNS 查询名形如 0001.Zmxh...challenge.local，序号偶尔乱序且有重传。如何恢复数据?

用 tshark 提取帧号、时间、源地址和完整查询名。拆出序号与编码标签，按序号去重，重传只保留相同内容的一份。检查序号是否连续，拼接标签后按题面提示做 Base64 或 Base32 解码。对缺失序号回到抓包确认是否被过滤。保存原查询表、去重规则、拼接字符串和解码结果哈希。

59.9　完成检查

交付与复盘检查表

- ☐ 入口、输入、目标输出与限制条件已经写入笔记。
- ☐ 建立了可重复的基线输入与输出。
- ☐ 每个关键结论都有原始证据与验证步骤。
- ☐ 脚本能够识别失败，不会把异常输出当成成功。
- ☐ 从干净目录或新容器运行时能够得到同样结果。

第六十章　内存取证：进程、网络、命令行、注入区域与文件对象

本章目标

内存镜像保留运行时进程、模块、网络连接、命令行、环境、缓存、文件对象和部分已解密数据。Volatility 3 等框架通过操作系统符号解释结构。第一步是确认镜像与操作系统信息，再围绕题面构建进程和时间线。

60.1　原理与题型边界

内存是采集时刻的快照，数据可能部分换出或已释放。不同插件从不同结构枚举对象，结果不一定一致。隐藏或终止进程可能在扫描插件中出现但不在活动链表中。

60.2　识别线索与优先动作

观察线索	可能含义	优先动作
pslist 与 psscan 结果不同	可能有已退出、隐藏或残留进程对象	比较 PID、时间和父子关系
netscan 显示异常外连或监听	进程可能与网络事件相关	关联 PID、创建时间和端口
cmdline/console 含命令历史	可还原用户操作或题目步骤	保存原文并验证相关文件对象
可执行私有内存区域异常	可能是解包、JIT 或注入代码	检查权限、内容和所属进程

60.3　标准解题步骤

1. **保存原始证据与上下文**。计算哈希，记录题面、来源、获取时间、时区、文件大小和任何已知采集方式。所有转换都在副本上进行。
2. **进行低侵入盘点**。确认镜像哈希、大小、采集时间、操作系统与内核符号是否匹配。
3. **建立结构或时间基线**。运行进程、父子关系、命令行、网络、模块与时间插件建立基线。
4. **提出可检验假设**。根据异常 PID 深入查看内存映射、句柄、文件对象、环境和字符串。
5. **使用专用工具提取**。导出候选进程、区域或文件，重新计算哈希并用静态工具验证。
6. **交叉验证结果**。至少使用结构约束、第二种工具、原始字节或上下文事件验证关键结论。
7. **导出可复现过程**。保存命令、过滤器、偏移、时区、脚本、提取文件哈希和最终证据链。

60.4　分析时持续追问的问题

- 当前观察是原始事实，还是工具给出的推断?
- 哪个最小输入变化能够区分主要假设?
- 结果能否在相同输入和干净环境中重复?
- 失败时是否保存了完整原始输出?
- 题目条件变化后，当前方法的前提是否仍成立?

60.5　命令、脚本与验证片段

以下内容只用于本地题目目录、回环地址、赛事提供的实例或明确授权的实验环境。每次运行都应保留原始输入和完整输出。

```
# Volatility 3 command pattern; use symbols matching the challenge image.
python3 vol.py -f original/memory.raw windows.info > notes/windows-info.txt
python3 vol.py -f original/memory.raw windows.pslist > notes/pslist.txt
python3 vol.py -f original/memory.raw windows.psscan > notes/psscan.txt
python3 vol.py -f original/memory.raw windows.pstree > notes/pstree.txt
python3 vol.py -f original/memory.raw windows.cmdline > notes/cmdline.txt
python3 vol.py -f original/memory.raw windows.netscan > notes/netscan.txt

# Inspect one candidate process.
python3 vol.py -f original/memory.raw windows.dlllist --pid 4242 > notes/
    pid4242-dlls.txt
python3 vol.py -f original/memory.raw windows.malfind --pid 4242 --dump \
  --dump-dir work/malfind > notes/pid4242-malfind.txt
sha256sum work/malfind/* > notes/malfind-hashes.txt
```

Listing 60.1: 本章常用命令与脚本

60.6　证据保存与结果判定

保存原始证据哈希、采集上下文、工具版本、查询或过滤器、所有提取文件的来源偏移与哈希、时间戳所用时区以及关键截图或原始字节。任何修复、解压、转换、挂载和解码必须在副本中进行，并保留前后对应关系。

60.7　常见失误与修正方法

- 未确认系统版本和符号就解释插件空结果。
- 只运行一个进程插件。
- 看到可执行私有内存就直接认定恶意，忽略 JIT。
- 导出文件后未记录 PID、虚拟地址和插件参数。
- 把内存中的字符串出现当作该进程实际使用。

60.8　小型演练

题目场景

内存镜像中 pslist 没有 3136，但 psscan 有该 PID，netscan 显示它曾连接题目服务器。如何判断其角色?

比较 psscan 的创建与退出时间、父 PID 和镜像名，在 pstree 与 cmdline 中寻找关联。使用 handles、dlllist、vadinfo 或相应插件检查残留对象，搜索网络端点和文件句柄。若进程已退出，部分结构缺失是正常现象。把网络连接时间与日志或 PCAP 对齐，并导出可用内存区域。结论应说明哪些字段来自完整活动结构，哪些来自扫描残留。

60.9　完成检查

交付与复盘检查表

- □ 入口、输入、目标输出与限制条件已经写入笔记。
- □ 建立了可重复的基线输入与输出。
- □ 每个关键结论都有原始证据与验证步骤。
- □ 脚本能够识别失败，不会把异常输出当成成功。
- □ 从干净目录或新容器运行时能够得到同样结果。

第六十一章　日志分析、时区统一与事件时间线

本章目标

日志题的主要难点是来源多、格式不一、时区不同、字段含义不一致以及事件存在延迟。应先保留原顺序和原时区，再生成统一时间线，不能直接排序后覆盖证据。

61.1　原理与题型边界

应用日志、Web 访问日志、系统日志、审计日志、云事件和文件时间各自记录不同观察点。一次动作可能在多个来源中留下时间不同的事件。时间线用于关联，不代表每个时间戳都同样可信。

61.2　识别线索与优先动作

观察线索	可能含义	优先动作
时间戳有 Z、偏移或无时区	来源时区语义不同	为每个来源建立解析规则
同一请求在多日志中相差数秒	排队、代理或时钟偏差存在	用请求 ID 和稳定事件校准
日志行顺序与时间不一致	多线程、缓冲或汇总写入	保留原行号并另建排序视图
字段被 URL、JSON 或 shell 转义	需要先按格式解析	不要用简单 split 破坏内容

61.3　标准解题步骤

1. **保存原始证据与上下文**。计算哈希，记录题面、来源、获取时间、时区、文件大小和任何已知采集方式。所有转换都在副本上进行。
2. **进行低侵入盘点**。为每个日志记录文件哈希、来源系统、采集方式、原时区和字段定义。
3. **建立结构或时间基线**。编写解析器输出统一字段：UTC 时间、来源、事件类型、主体、对象、原行号和原文摘要。
4. **提出可检验假设**。利用请求 ID、PID、用户名、IP、文件哈希和会话标识关联事件。
5. **使用专用工具提取**。生成按时间排序的派生视图，同时保留原文件与原行号。
6. **交叉验证结果**。至少使用结构约束、第二种工具、原始字节或上下文事件验证关键结论。
7. **导出可复现过程**。保存命令、过滤器、偏移、时区、脚本、提取文件哈希和最终证据链。

61.4　分析时持续追问的问题

- 当前观察是原始事实，还是工具给出的推断?
- 哪个最小输入变化能够区分主要假设?
- 结果能否在相同输入和干净环境中重复?
- 失败时是否保存了完整原始输出?
- 题目条件变化后，当前方法的前提是否仍成立?

61.5　命令、脚本与验证片段

以下内容只用于本地题目目录、回环地址、赛事提供的实例或明确授权的实验环境。每次运行都应保留原始输入和完整输出。

```
# Preserve original line numbers before parsing.
nl -ba original/access.log > work/access-numbered.log

python3 - <<'PYCODE'
from datetime import datetime, timezone
from pathlib import Path
import json

out=[]
for n,line in enumerate(Path('original/events.jsonl').read_text().splitlines()
    ,1):
    obj=json.loads(line)
    dt=datetime.fromisoformat(obj['time'].replace('Z','+00:00')).astimezone(
    timezone.utc)
    out.append({
        'time_utc': dt.isoformat(),
        'source':'events.jsonl',
        'line':n,
        'event':obj.get('event'),
        'actor':obj.get('user'),
        'object':obj.get('path'),
    })
out.sort(key=lambda x:(x['time_utc'],x['source'],x['line']))
Path('work/timeline.json').write_text(json.dumps(out,indent=2,ensure_ascii=
    False))
PYCODE
```

Listing 61.1: 本章常用命令与脚本

61.6　证据保存与结果判定

保存原始证据哈希、采集上下文、工具版本、查询或过滤器、所有提取文件的来源偏移与哈希、时间戳所用时区以及关键截图或原始字节。任何修复、解压、转换、挂载和解码必须在副本中进行，并保留前后对应关系。

61.7　常见失误与修正方法

- 把没有时区的时间默认为本机时区。
- 排序后丢失原行号和原始顺序。
- 用字符串切分 JSON、CSV 或带引号字段。

- 把缺少日志当作事件没有发生。
- 只凭相近时间关联事件，忽略请求 ID 和主体。

61.8　小型演练

题目场景

Web 日志为 UTC，主机日志标注 +08:00，文件时间由磁盘镜像工具输出但时区未知。如何建立可靠时间线?

分别记录三个来源的时区语义。Web 与主机日志转换为 UTC，同时保留原时间字符串。对文件时间，不应擅自假定；先查采集工具参数、文件系统类型和题面环境。可用一个同时出现在 Web、主机和文件系统中的已知事件估计偏差，但在时间线中标注这是校准值。所有记录保留来源、原行号和置信度。

61.9　完成检查

交付与复盘检查表

- □ 入口、输入、目标输出与限制条件已经写入笔记。
- □ 建立了可重复的基线输入与输出。
- □ 每个关键结论都有原始证据与验证步骤。
- □ 脚本能够识别失败，不会把异常输出当成成功。
- □ 从干净目录或新容器运行时能够得到同样结果。

第六十二章　图像隐写、像素通道、调色板与元数据

本章目标

图像题可能把信息放在元数据、附加块、最低有效位、颜色通道、调色板顺序、透明度、像素排列或视觉差分中。开始时先确认真实格式与像素模型，再根据统计异常选择工具。

62.1　原理与题型边界

PNG 无损保存像素和块结构，JPEG 使用有损变换，GIF 与索引色 PNG 通过调色板引用颜色。LSB 提取需要明确通道顺序、位序、扫描方向和终止条件。任意尝试参数容易产生大量看似可读的假阳性。

62.2　识别线索与优先动作

观察线索	可能含义	优先动作
图片尺寸或颜色模式异常	可能利用通道、行填充或调色板	记录宽高、位深、通道和块
元数据有注释或自定义字段	可能直接存放提示或负载	提取原始元数据并记录字段
单个通道噪声分布不同	可能有 LSB 或位平面信息	分离通道并查看位平面
两张图片肉眼相同	可能通过像素差或结构差编码	逐像素比较并定位差异

62.3　标准解题步骤

1. **保存原始证据与上下文**。计算哈希，记录题面、来源、获取时间、时区、文件大小和任何已知采集方式。所有转换都在副本上进行。
2. **进行低侵入盘点**。运行 file、identify、exiftool 和格式结构工具，记录尺寸、模式、位深、块与元数据。
3. **建立结构或时间基线**。对 RGB、RGBA、调色板和灰度分别理解像素存储，不先假设 RGB 顺序。
4. **提出可检验假设**。检查通道直方图、最低位分布、位平面和相邻像素差。
5. **使用专用工具提取**。根据题面约束选择扫描顺序与位序，写脚本提取并验证格式。
6. **交叉验证结果**。至少使用结构约束、第二种工具、原始字节或上下文事件验证关键结论。
7. **导出可复现过程**。保存命令、过滤器、偏移、时区、脚本、提取文件哈希和最终证据链。

62.4　分析时持续追问的问题

- 当前观察是原始事实，还是工具给出的推断?
- 哪个最小输入变化能够区分主要假设?
- 结果能否在相同输入和干净环境中重复?
- 失败时是否保存了完整原始输出?
- 题目条件变化后，当前方法的前提是否仍成立?

62.5　命令、脚本与验证片段

以下内容只用于本地题目目录、回环地址、赛事提供的实例或明确授权的实验环境。每次运行都应保留原始输入和完整输出。

```
file original/image.png
identify -verbose original/image.png > notes/identify.txt
exiftool -a -u -g1 original/image.png > notes/exiftool.txt
pngcheck -v original/image.png > notes/pngcheck.txt

python3 - <<'PYCODE'
from PIL import Image
from pathlib import Path
im=Image.open('original/image.png').convert('RGBA')
bits=[]
for r,g,b,a in im.getdata():
    bits.extend([r&1,g&1,b&1])
out=bytearray()
for i in range(0,len(bits)-7,8):
    v=0
    for bit in bits[i:i+8]:
        v=(v<<1)|bit
    out.append(v)
Path('work/rgb-lsb-msb.bin').write_bytes(out)
print(len(out),out[:32])
PYCODE
```

Listing 62.1: 本章常用命令与脚本

62.6　证据保存与结果判定

保存原始证据哈希、采集上下文、工具版本、查询或过滤器、所有提取文件的来源偏移与哈希、时间戳所用时区以及关键截图或原始字节。任何修复、解压、转换、挂载和解码必须在副本中进行，并保留前后对应关系。

62.7　常见失误与修正方法

- 直接运行所有隐写参数组合并从随机输出中找可读片段。
- 把 JPEG 当作稳定逐像素 LSB 容器。
- 忽略 alpha 通道、调色板和位深。
- 重新保存图片后再分析，破坏原始数据。
- 提取后没有验证长度、魔数或校验。

62.8　小型演练

题目场景

一张 RGBA PNG 中 RGB 通道 LSB 看似随机，alpha 通道只有 254 和 255。如何分析?

先统计 alpha 值分布和位置，确认 254/255 是否近似均衡且与透明视觉无关。按标准扫描顺序把 alpha&1 提取为位流，同时尝试明确的位序，不做无限组合。检查输出头部是否有已知魔数、长度字段或题面 flag 前缀。若位流按行有结构，可把 alpha 位平面直接渲染为黑白图。记录通道、扫描顺序、位序和输出哈希。

62.9　完成检查

交付与复盘检查表

- □ 入口、输入、目标输出与限制条件已经写入笔记。
- □ 建立了可重复的基线输入与输出。
- □ 每个关键结论都有原始证据与验证步骤。
- □ 脚本能够识别失败，不会把异常输出当成成功。
- □ 从干净目录或新容器运行时能够得到同样结果。

第六十三章　音频、视频、频谱与帧级分析

本章目标

音视频题可能在波形、频谱、左右声道差、静音区、采样最低位、DTMF、摩尔斯码、帧序列、字幕、容器附件或编码元数据中隐藏信息。先区分容器、编码流和解码后的样本。

63.1　原理与题型边界

音频由采样率、位深、声道和样本序列构成；视频容器可含多路视频、音频、字幕、附件和元数据。重新编码会改变样本与帧，取证分析应先直接复制流或无损导出。

63.2　识别线索与优先动作

观察线索	可能含义	优先动作
频谱中有文字或规则条纹	信息可能编码在频率随时间变化	生成线性与对数频谱
左右声道相似但不完全相同	差分或单声道可能隐藏信号	分别导出 L、R、L-R 与 L+R
视频含多个流或附件	容器内有额外字幕、封面或数据	使用 ffprobe 列出全部流
特定帧短暂出现二维码或文本	需要逐帧或按场景提取	保存帧号与时间戳

63.3　标准解题步骤

1. **保存原始证据与上下文**。计算哈希，记录题面、来源、获取时间、时区、文件大小和任何已知采集方式。所有转换都在副本上进行。
2. **进行低侵入盘点**。用 ffprobe、soxi 或 mediainfo 记录容器、编码、采样率、位深、声道、时长和所有流。
3. **建立结构或时间基线**。不重新编码地提取音频、字幕、附件或原始帧，保留时间基。
4. **提出可检验假设**。对音频观察波形、频谱、声道差、静音与样本位；对视频建立缩略图与帧差。
5. **使用专用工具提取**。对候选信号使用匹配的解码器，并把结果关联回样本区间或帧号。
6. **交叉验证结果**。至少使用结构约束、第二种工具、原始字节或上下文事件验证关键结论。
7. **导出可复现过程**。保存命令、过滤器、偏移、时区、脚本、提取文件哈希和最终证据链。

63.4　分析时持续追问的问题

- 当前观察是原始事实，还是工具给出的推断?
- 哪个最小输入变化能够区分主要假设?
- 结果能否在相同输入和干净环境中重复?
- 失败时是否保存了完整原始输出?
- 题目条件变化后，当前方法的前提是否仍成立?

63.5　命令、脚本与验证片段

以下内容只用于本地题目目录、回环地址、赛事提供的实例或明确授权的实验环境。每次运行都应保留原始输入和完整输出。

```
ffprobe -hide_banner -show_format -show_streams \
  original/media.mkv > notes/ffprobe.txt 2>&1

# Copy streams without transcoding when possible.
ffmpeg -i original/media.mkv -map 0:a:0 -c copy work/audio.flac
ffmpeg -i original/media.mkv -map 0:s:0 work/subtitles.srt

# Split stereo channels and derive a difference signal.
ffmpeg -i work/audio.flac -filter_complex \
  '[0:a]channelsplit=channel_layout=stereo[L][R]' \
  -map '[L]' work/left.wav -map '[R]' work/right.wav
ffmpeg -i work/audio.flac -af 'pan=mono|c0=c0-c1' work/difference.wav

# Extract one frame per second with timestamps in filenames.
ffmpeg -i original/media.mkv -vf fps=1 work/frame-%06d.png
```

Listing 63.1: 本章常用命令与脚本

63.6　证据保存与结果判定

保存原始证据哈希、采集上下文、工具版本、查询或过滤器、所有提取文件的来源偏移与哈希、时间戳所用时区以及关键截图或原始字节。任何修复、解压、转换、挂载和解码必须在副本中进行，并保留前后对应关系。

63.7　常见失误与修正方法

- 分析前重新编码，破坏最低位和精确帧。
- 只听音频，不看频谱和声道。
- 只提取第一路流，遗漏字幕或附件。
- 帧截图未记录帧号或时间。
- 把压缩编码伪影误判为隐藏信号。

63.8　小型演练

题目场景

立体声 WAV 听起来正常，但左右声道差分后出现脉冲。怎样判断是摩尔斯码、DTMF 还是随机噪声？

先无损导出左右声道并计算差分，观察脉冲持续时间和间隔。如果只有两个时长层级和规则间隔，测试摩尔斯码；若每个片段同时存在两条稳定频率，测试 DTMF；若脉冲幅度对应二进制，检查采样最低位。记录时间窗口与频谱，使用一小段手工验证，再批量解码。结果必须能回到原始时间位置复查。

63.9　完成检查

交付与复盘检查表

- ☐ 入口、输入、目标输出与限制条件已经写入笔记。
- ☐ 建立了可重复的基线输入与输出。
- ☐ 每个关键结论都有原始证据与验证步骤。
- ☐ 脚本能够识别失败，不会把异常输出当成成功。
- ☐ 从干净目录或新容器运行时能够得到同样结果。

第六十四章　文档、压缩包、PDF、Office 与元数据取证

本章目标

文档题常利用压缩容器结构、嵌入对象、修订记录、隐藏文本、宏、附件、字体、图层、增量更新或元数据。Office Open XML 本质上是 ZIP，PDF 可包含多次增量保存，压缩包还可能使用弱口令或路径技巧。

64.1　原理与题型边界

文档显示内容只是对象树的一种渲染。隐藏对象、关系文件、注释、修订和附件可能不出现在页面上。未知文档解析器具有较大攻击面，应在隔离环境使用静态提取工具，避免直接启用宏。

64.2　识别线索与优先动作

观察线索	可能含义	优先动作
DOCX/XLSX/PPTX 可被 unzip 列表	内容位于 XML、关系和媒体文件	复制后解压并搜索文本与关系
PDF 尾部有多个%%EOF	可能有增量更新或追加对象	比较对象版本和交叉引用
文档含宏或嵌入对象	可能包含题目逻辑或隐藏数据	静态提取，不在日常系统启用
压缩包加密但题目给出已知文件	可能适用已知明文或口令线索	先确认算法与候选空间

64.3　标准解题步骤

1. **保存原始证据与上下文**。计算哈希，记录题面、来源、获取时间、时区、文件大小和任何已知采集方式。所有转换都在副本上进行。
2. **进行低侵入盘点**。识别真实格式、容器目录、元数据、嵌入对象、附件、脚本和宏。
3. **建立结构或时间基线**。对 OOXML 解压副本，检查 document、sharedStrings、comments、relationships、media 和 customXml。
4. **提出可检验假设**。对 PDF 记录对象、页面、附件、JavaScript、字体和增量更新。
5. **使用专用工具提取**。提取对象后重新识别与哈希，并在隔离环境处理可执行内容。
6. **交叉验证结果**。至少使用结构约束、第二种工具、原始字节或上下文事件验证关键结论。
7. **导出可复现过程**。保存命令、过滤器、偏移、时区、脚本、提取文件哈希和最终证据链。

64.4　分析时持续追问的问题

- 当前观察是原始事实，还是工具给出的推断？
- 哪个最小输入变化能够区分主要假设？
- 结果能否在相同输入和干净环境中重复？
- 失败时是否保存了完整原始输出？
- 题目条件变化后，当前方法的前提是否仍成立？

64.5　命令、脚本与验证片段

以下内容只用于本地题目目录、回环地址、赛事提供的实例或明确授权的实验环境。每次运行都应保留原始输入和完整输出。

```
file original/document.docx
exiftool -a -u -g1 original/document.docx > notes/docx-metadata.txt
unzip -l original/document.docx > notes/docx-list.txt
mkdir -p work/docx && unzip -q original/document.docx -d work/docx
rg -n -i 'flag|secret|comment|hidden' work/docx > notes/docx-search.txt || true

file original/report.pdf
pdfinfo original/report.pdf > notes/pdfinfo.txt
pdfdetach -list original/report.pdf > notes/pdf-attachments.txt
qpdf --check original/report.pdf > notes/qpdf-check.txt 2>&1
qpdf --qdf --object-streams=disable original/report.pdf work/report-qdf.pdf

# List archive safely before extraction.
unzip -l original/archive.zip | tee notes/archive-list.txt
7z l -slt original/archive.zip > notes/archive-technical.txt
```

Listing 64.1: 本章常用命令与脚本

64.6 证据保存与结果判定

保存原始证据哈希、采集上下文、工具版本、查询或过滤器、所有提取文件的来源偏移与哈希、时间戳所用时区以及关键截图或原始字节。任何修复、解压、转换、挂载和解码必须在副本中进行，并保留前后对应关系。

64.7 常见失误与修正方法

- 直接双击未知文档并启用宏。
- 只看渲染页面，忽略附件和对象。
- 解压不可信归档时未防范绝对路径和父目录路径。
- 在原 PDF 上使用修复或优化工具。
- 猜口令前没有估算候选空间与加密算法。

64.8　小型演练

题目场景

PDF 显示一页普通文本，但文件尾有两个%%EOF，qpdf 显示增量更新。如何寻找被覆盖内容?

保留原 PDF，使用 qpdf 的 QDF 模式或对象查看工具列出交叉引用和对象版本。定位最后一次更新新增或重定义的页面内容流，再查看前一版本指向的旧内容流。必要时从旧 xref 或对象偏移提取流并解压。记录对象号、代数、文件偏移和解码步骤。渲染旧内容时使用副本，不能把修复后的 PDF 替代原证据。

64.9　完成检查

交付与复盘检查表

- □ 入口、输入、目标输出与限制条件已经写入笔记。
- □ 建立了可重复的基线输入与输出。
- □ 每个关键结论都有原始证据与验证步骤。
- □ 脚本能够识别失败，不会把异常输出当成成功。
- □ 从干净目录或新容器运行时能够得到同样结果。

第六十五章　硬件、串口、逻辑分析与简单总线数据

本章目标

硬件类 CTF 可能提供逻辑分析仪采样、UART 串口日志、SPI/I2C 总线、固件镜像、示波器 CSV、键盘扫描或 RFID 数据。重点是先恢复物理或链路参数，再解释高层协议。

65.1　原理与题型边界

UART 需要波特率、数据位、校验位、停止位和空闲电平；SPI 需要时钟极性与相位、片选和位序；I2C 使用起始、地址、读写位、应答与停止。参数错误常产生近似可读但不稳定的数据。

65.2　识别线索与优先动作

观察线索	可能含义	优先动作
逻辑波形有固定宽度脉冲	可由最短位宽估计符号率	测量多个边沿间隔并取共同基准
解码串口大部分可读但偶尔错字	波特率或采样点略有偏差	比较候选波特率与帧错误
两条数据线加片选或时钟	可能为 SPI	确定 CLK、MOSI/MISO、CS 和模式
地址后有 ACK/NACK	可能为 I2C	按事务分组并识别寄存器访问

65.3　标准解题步骤

1. **保存原始证据与上下文**。计算哈希，记录题面、来源、获取时间、时区、文件大小和任何已知采集方式。所有转换都在副本上进行。
2. **进行低侵入盘点**。记录采样率、通道名称、电平、触发条件、文件格式和题面接线信息。
3. **建立结构或时间基线**。从波形统计估计位宽、时钟、空闲电平和事务边界。
4. **提出可检验假设**。使用解码器测试少量合理参数，比较帧错误与协议一致性。
5. **使用专用工具提取**。导出字节流，保留时间戳与方向，再分析上层命令或文件格式。
6. **交叉验证结果**。至少使用结构约束、第二种工具、原始字节或上下文事件验证关键结论。
7. **导出可复现过程**。保存命令、过滤器、偏移、时区、脚本、提取文件哈希和最终证据链。

65.4　分析时持续追问的问题

- 当前观察是原始事实，还是工具给出的推断?
- 哪个最小输入变化能够区分主要假设?
- 结果能否在相同输入和干净环境中重复?
- 失败时是否保存了完整原始输出?
- 题目条件变化后，当前方法的前提是否仍成立?

65.5　命令、脚本与验证片段

以下内容只用于本地题目目录、回环地址、赛事提供的实例或明确授权的实验环境。每次运行都应保留原始输入和完整输出。

```
# Inspect a sigrok capture and decode a local UART sample.
sigrok-cli -i original/capture.sr -P uart \
  -C D0=RX:baudrate=115200:format=ascii \
```

```
  -A uart=rx-data > work/uart.txt

# Export protocol annotations as CSV for reproducible analysis.
sigrok-cli -i original/capture.sr \
  -P i2c:scl=SCL:sda=SDA \
  -A i2c=address-read,address-write,data-read,data-write \
  > work/i2c-annotations.txt

# Examine a firmware image before executing anything.
file original/firmware.bin
xxd -g 1 -l 256 original/firmware.bin > notes/firmware-head.hex
strings -a -t x -n 6 original/firmware.bin > notes/firmware-strings.txt
binwalk original/firmware.bin > notes/firmware-signatures.txt
```

Listing 65.1: 本章常用命令与脚本

65.6　证据保存与结果判定

保存原始证据哈希、采集上下文、工具版本、查询或过滤器、所有提取文件的来源偏移与哈希、时间戳所用时区以及关键截图或原始字节。任何修复、解压、转换、挂载和解码必须在副本中进行，并保留前后对应关系。

65.7　常见失误与修正方法

- 根据一个可读词就接受错误串口参数。
- 没有记录采样率和通道映射。
- 把 SPI 的 MISO 与 MOSI 混在一个字节流。
- 忽略片选，跨事务拼接数据。
- 直接在真实硬件写入未知固件。

65.8　小型演练

题目场景

UART 波形空闲高电平，最短稳定位宽约 8.68 微秒，8 位数据无校验。应选择什么参数并如何验证？

1/8.68 微秒约为 115200 波特，因此先尝试 115200、8N1、LSB first。解码后检查起始位低、停止位高、帧错误数量和文本或协议结构。再尝试相邻常见波特率作为对照，若 115200 显著减少帧错误且连续数据合理，则记录参数。对关键字节回到波形手工核对一个完整帧。

65.9　完成检查

交付与复盘检查表

- □ 入口、输入、目标输出与限制条件已经写入笔记。
- □ 建立了可重复的基线输入与输出。
- □ 每个关键结论都有原始证据与验证步骤。
- □ 脚本能够识别失败，不会把异常输出当成成功。
- □ 从干净目录或新容器运行时能够得到同样结果。

第六十六章　OSINT 题：公开资料检索、时间验证与隐私边界

本章目标

OSINT 题通常要求从公开网页、图片、地图、证书、代码仓库、存档或元数据识别地点、时间、人物别名或事件。分析必须限定在题目给出的虚构身份、公开线索和赛事规则内，不应扩大到现实个人隐私或联系无关人员。

66.1　原理与题型边界

公开信息会变化、删除或被搜索引擎重新排序。可靠结论需要多个独立来源、明确时间和原始页面证据。图片地理定位应结合建筑、道路、地形、太阳、交通标志和元数据，而不是依赖单一相似图。

66.2　识别线索与优先动作

观察线索	可能含义	优先动作
图片有独特建筑、路牌或地形	可进行地理定位	分离可见线索并逐项检索验证
题目给出用户名或仓库片段	可能跨公开平台复用	只在题目范围检索并记录时间
页面已删除或内容变化	需要网页存档或缓存	比较不同日期快照
答案依赖日期或天气	需要确认事件发生时间而非当前状态	使用当时的公开记录交叉验证

66.3　标准解题步骤

1. **保存原始证据与上下文**。计算哈希，记录题面、来源、获取时间、时区、文件大小和任何已知采集方式。所有转换都在副本上进行。
2. **进行低侵入盘点**。提取题面中的原始文字、图片裁剪、时间、语言、用户名、域名和文件元数据。
3. **建立结构或时间基线**。把线索分成可独立验证的子问题，不先锁定单一答案。
4. **提出可检验假设**。使用至少两个互相独立的公开来源验证地点、时间或身份关系。
5. **使用专用工具提取**。保存页面标题、访问日期、截图、存档时间和推理链，避免只留搜索结果摘要。
6. **交叉验证结果**。至少使用结构约束、第二种工具、原始字节或上下文事件验证关键结论。
7. **导出可复现过程**。保存命令、过滤器、偏移、时区、脚本、提取文件哈希和最终证据链。

66.4　分析时持续追问的问题

- 当前观察是原始事实，还是工具给出的推断?
- 哪个最小输入变化能够区分主要假设?
- 结果能否在相同输入和干净环境中重复?
- 失败时是否保存了完整原始输出?
- 题目条件变化后，当前方法的前提是否仍成立?

66.5　命令、脚本与验证片段

以下内容只用于本地题目目录、回环地址、赛事提供的实例或明确授权的实验环境。每次运行都应保留原始输入和完整输出。

```
# Local metadata and image preparation before any public search.
exiftool -a -u -g1 original/clue.jpg > notes/clue-metadata.txt
identify -verbose original/clue.jpg > notes/clue-image.txt

# Extract readable text from supplied HTML or repository files locally.
rg -n -i 'username|location|date|email|domain|commit' work/clues \
  > notes/local-text-hits.txt || true

# Inspect a supplied Git repository without contacting remotes.
git -C work/repo log --all --date=iso-strict \
  --pretty=format:'%H%x09%ad%x09%an%x09%s' > notes/git-log.tsv
git -C work/repo remote -v > notes/git-remotes.txt
```

Listing 66.1: 本章常用命令与脚本

66.6 证据保存与结果判定

保存原始证据哈希、采集上下文、工具版本、查询或过滤器、所有提取文件的来源偏移与哈希、时间戳所用时区以及关键截图或原始字节。任何修复、解压、转换、挂载和解码必须在副本中进行，并保留前后对应关系。

66.7 常见失误与修正方法

- 只凭一个搜索结果或相似图片下结论。
- 把当前网页状态当作题目发生时状态。
- 向现实个人发送消息或尝试登录账户。
- 忽略来源之间的复制关系，把同一信息误当多个独立证据。
- 只保存链接，没有保存访问日期和页面内容。

66.8　小型演练

题目场景

题目图片显示山谷、公路编号一角和左侧通行车辆，要求回答拍摄地点。怎样组织证据?

先记录可见线索：道路行驶方向、路牌字体和颜色、山体形态、植被、道路护栏、太阳方向和可能语言。分别检索道路编号体系与交通规则，形成少量候选国家或地区。再用地图和公开街景或照片比较道路弯曲、山脊轮廓和建筑。最终答案至少由道路编号规则与独特地形两个独立线索支持，并保存访问日期与截图。若无法排除多个地点，应继续寻找细节而不是猜。

66.9　完成检查

交付与复盘检查表

- ☐ 入口、输入、目标输出与限制条件已经写入笔记。
- ☐ 建立了可重复的基线输入与输出。
- ☐ 每个关键结论都有原始证据与验证步骤。
- ☐ 脚本能够识别失败，不会把异常输出当成成功。
- ☐ 从干净目录或新容器运行时能够得到同样结果。

第六十七章　Misc：二维码、条码、键盘轨迹、算法题与冷门语言

本章目标

Misc 题覆盖二维码、条码、棋盘、迷宫、键盘记录、正则、压缩流、网络编码、约束求解、冷门编程语言和组合算法。共同方法是先识别数据模型和约束，再选择专用解码器或编写短脚本。

67.1　原理与题型边界

题面外观常带有强提示：固定网格可能是二维码或矩阵，重复键码可能对应 HID，奇怪源码可能是 Brainfuck、Whitespace 或栈语言。专用格式解码失败时，应检查旋转、镜像、错误纠正、缺失边框和符号映射。

67.2　识别线索与优先动作

观察线索	可能含义	优先动作
黑白方格有定位图案	可能是 QR、Data Matrix 或自定义位图	确认尺寸、静区、定位块和旋转
数据为 USB HID 键码	可还原键盘按键与修饰键	按报告描述符和布局解析
源码只含少数字符	可能是 esolang 或编码	统计字符集和控制结构
要求最短路径、填数或满足条件	适合图算法、动态规划或约束求解	先估算状态空间并定义变量

67.3　标准解题步骤

1. **保存原始证据与上下文**。计算哈希，记录题面、来源、获取时间、时区、文件大小和任何已知采集方式。所有转换都在副本上进行。
2. **进行低侵入盘点**。盘点字符集、尺寸、重复模式、字段长度、已知前缀和题面提示。
3. **建立结构或时间基线**。识别标准格式时先验证结构标记，避免仅凭视觉相似。
4. **提出可检验假设**。把输入转为明确的数据结构，例如网格、图、键码事件或约束变量。
5. **使用专用工具提取**。用小样本和断言验证解析，再运行完整求解并由题目规则校验。
6. **交叉验证结果**。至少使用结构约束、第二种工具、原始字节或上下文事件验证关键结论。
7. **导出可复现过程**。保存命令、过滤器、偏移、时区、脚本、提取文件哈希和最终证据链。

67.4　分析时持续追问的问题

- 当前观察是原始事实，还是工具给出的推断?
- 哪个最小输入变化能够区分主要假设?
- 结果能否在相同输入和干净环境中重复?
- 失败时是否保存了完整原始输出?
- 题目条件变化后，当前方法的前提是否仍成立?

67.5　命令、脚本与验证片段

以下内容只用于本地题目目录、回环地址、赛事提供的实例或明确授权的实验环境。每次运行都应保留原始输入和完整输出。

```
# Try standard barcode decoders on a supplied local image.
zbarimg --raw original/code.png | tee notes/zbar-result.txt

# Example USB HID boot-keyboard decoder skeleton.
python3 - <<'PYCODE'
from pathlib import Path
KEYS={4:'a',5:'b',6:'c',7:'d',8:'e',9:'f',10:'g',11:'h',12:'i',13:'j'}
out=[]
for line in Path('work/hid-reports.txt').read_text().splitlines():
    report=bytes.fromhex(line.replace(':',' '))
    modifier=report[0]
    for code in report[2:8]:
        if code in KEYS:
            ch=KEYS[code]
            out.append(ch.upper() if modifier & 0x22 else ch)
print(''.join(out))
PYCODE

# A deterministic graph shortest path baseline.
python3 - <<'PYCODE'
from collections import deque
start=(0,0); goal=(9,9)
blocked=set()
q=deque([(start,0)]); seen={start}
while q:
    (r,c),d=q.popleft()
    if (r,c)==goal:
        print(d); break
    for dr,dc in ((1,0),(-1,0),(0,1),(0,-1)):
        n=(r+dr,c+dc)
        if 0<=n[0]<10 and 0<=n[1]<10 and n not in blocked and n not in seen:
            seen.add(n); q.append((n,d+1))
PYCODE
```

Listing 67.1: 本章常用命令与脚本

67.6　证据保存与结果判定

保存原始证据哈希、采集上下文、工具版本、查询或过滤器、所有提取文件的来源偏移与哈希、时间戳所用时区以及关键截图或原始字节。任何修复、解压、转换、挂载和解码必须在副本中进行，并保留前后对应关系。

67.7　常见失误与修正方法

- 识别出可能格式后立即使用一个工具，失败就换方向。
- 二维码重建时忽略静区、方向和模块尺寸。
- HID 解码忽略按键释放、修饰键和键盘布局。
- 算法题直接暴力枚举，没有估算状态空间。
- 从随机解码结果中挑选看似可读文本。

67.8　小型演练

题目场景

一张损坏二维码缺少右下角部分，但版本、三个定位块和大部分模块仍在。如何恢复？

先根据定位块确定旋转、模块尺寸和版本，重建标准静区。读取格式信息以确定纠错级别和掩码，若格式信息损坏可枚举有限候选并用 BCH 校验筛选。缺失模块标为擦除，交给支持 Reed-Solomon 擦除恢复的解码器，或根据码字映射填入未知。每次修复保存网格和候选，最终由二维码结构校验和内容格式共同验证。

67.9　完成检查

交付与复盘检查表

- □ 入口、输入、目标输出与限制条件已经写入笔记。
- □ 建立了可重复的基线输入与输出。
- □ 每个关键结论都有原始证据与验证步骤。
- □ 脚本能够识别失败，不会把异常输出当成成功。
- □ 从干净目录或新容器运行时能够得到同样结果。

第六十八章　取证综合案例：多源证据关联与可复现报告

本章目标

综合取证题可能同时提供磁盘镜像、内存、PCAP、日志和文档。真正难点是把来源不同、时区不同、可信度不同的证据关联起来，构建一条能解释题目事件的时间线，并明确每个结论的来源。

68.1　原理与题型边界

多源关联依赖稳定标识，例如文件哈希、进程 PID、请求 ID、用户名、IP、端口、会话令牌、路径和时间。某个来源的缺失不等于事件不存在，某个字符串出现也不等于已执行。报告应区分直接证据、强推断与待验证事项。

68.2　识别线索与优先动作

观察线索	可能含义	优先动作
PCAP 中下载文件与磁盘文件同哈希	网络与主机事件可直接关联	记录流、时间、路径和哈希
内存进程命令行指向删除文件	可连接执行与文件系统痕迹	恢复文件并比较时间
日志时间与抓包相差固定值	时钟偏差或时区不同	用多个共同事件估计偏差
文档元数据用户名与主机账户一致	可能关联创建者但不是充分证明	寻找路径、时间和进程补充证据

68.3　标准解题步骤

1. **保存原始证据与上下文**。计算哈希，记录题面、来源、获取时间、时区、文件大小和任何已知采集方式。所有转换都在副本上进行。
2. **进行低侵入盘点**。为每个证据源建立哈希、时区、采集方式和可用字段清单。
3. **建立结构或时间基线**。分别完成磁盘、内存、网络和日志的独立基线，不先强行拼接。
4. **提出可检验假设**。选择共同事件校准时间和标识，再建立跨源关联表。
5. **使用专用工具提取**。对每个题目结论列出至少一条直接证据和一条交叉验证。
6. **交叉验证结果**。至少使用结构约束、第二种工具、原始字节或上下文事件验证关键结论。
7. **导出可复现过程**。保存命令、过滤器、偏移、时区、脚本、提取文件哈希和最终证据链。

68.4　分析时持续追问的问题

- 当前观察是原始事实，还是工具给出的推断?
- 哪个最小输入变化能够区分主要假设?
- 结果能否在相同输入和干净环境中重复?
- 失败时是否保存了完整原始输出?
- 题目条件变化后，当前方法的前提是否仍成立?

68.5　命令、脚本与验证片段

以下内容只用于本地题目目录、回环地址、赛事提供的实例或明确授权的实验环境。每次运行都应保留原始输入和完整输出。

```
# A simple cross-source evidence table built from normalized TSV files.
python3 - <<'PYCODE'
from pathlib import Path
import csv, json
```

```
rows=[]
for name in ('network.tsv','process.tsv','files.tsv','logs.tsv'):
    p=Path('work')/name
    if not p.exists():
        continue
    with p.open(newline='') as f:
        for row in csv.DictReader(f,delimiter='\t'):
            row['source']=name
            rows.append(row)
rows.sort(key=lambda r:(r.get('time_utc',''),r['source']))
Path('output/cross-source-timeline.json').write_text(
    json.dumps(rows,indent=2,ensure_ascii=False),encoding='utf-8')
print(len(rows))
PYCODE

sha256sum original/* > notes/source-hashes.txt
sha256sum work/* output/* > notes/derived-hashes.txt 2>/dev/null || true
```

Listing 68.1: 本章常用命令与脚本

68.6　证据保存与结果判定

保存原始证据哈希、采集上下文、工具版本、查询或过滤器、所有提取文件的来源偏移与哈希、时间戳所用时区以及关键截图或原始字节。任何修复、解压、转换、挂载和解码必须在副本中进行，并保留前后对应关系。

68.7　常见失误与修正方法

- 先确定故事再选择支持它的证据。
- 跨源时间未校准就按秒级顺序解释。
- 把相同文本误认为相同对象，没有比较哈希或标识。
- 派生文件没有记录来源偏移、插件或过滤器。
- 报告只给结论，不给复现命令和证据路径。

68.8　小型演练

题目场景

提供 disk.raw、memory.raw、traffic.pcap 和 access.log。要求回答某文件何时下载、由哪个进程打开、随后被移动到哪里。

先独立分析 PCAP，提取下载响应并记录流、UTC 时间和文件哈希。磁盘中按哈希或文件名定位文件与移动后的路径，记录文件系统时间语义。内存中查找浏览器或下载工具进程、命令行、句柄和文件对象，关联 PID。访问日志提供请求 ID、账户或代理时间。用共同请求和哈希校准时间，最终表格列出网络下载、进程持有、磁盘写入和移动四个事件，每行附来源文件、命令、对象标识和置信度。

68.9　完成检查

交付与复盘检查表

- □ 入口、输入、目标输出与限制条件已经写入笔记。
- □ 建立了可重复的基线输入与输出。
- □ 每个关键结论都有原始证据与验证步骤。
- □ 脚本能够识别失败，不会把异常输出当成成功。
- □ 从干净目录或新容器运行时能够得到同样结果。

第七部分

移动端、智能合约、容器与云环境

第六十九章　Android APK: 清单、组件、资源、字节码与本地库

本章目标

Android CTF 题常把线索放在 AndroidManifest、资源、SharedPreferences、SQLite、Intent、深链、导出组件、WebView、Java/Kotlin 字节码或 JNI 本地库中。先静态盘点应用结构，再在模拟器中验证组件和数据流。

69.1　原理与题型边界

APK 是 ZIP 容器，包含编译资源、DEX、清单、签名和可选本地库。组件能否被外部调用由 exported、intent-filter 和权限决定。应用签名、调试标志、网络安全配置和备份设置会影响题目入口。

69.2　识别线索与优先动作

观察线索	可能含义	优先动作
Manifest 中有 exported activity/service/receiver	组件可能被题目客户端直接触发	记录 action、data、权限和参数
存在 assets、raw 或 res/xml	密钥、配置或网络策略可能在资源中	解包并追踪资源引用
Java 层很薄但有 System.loadLibrary	核心逻辑位于 native so	定位 JNI 导出和 RegisterNatives

观察线索	可能含义	优先动作
WebView 启用 JavaScript interface	网页内容与本地接口形成信任边界	追踪加载来源和暴露方法

69.3 标准解题步骤

1. **固定附件与运行平台**。计算应用包、镜像、合约、配置和依赖哈希，记录平台版本、架构、网络、账户与题目实例范围。
2. **盘点结构与清单**。使用 apktool、aapt、jadx、unzip 和签名工具列出清单、资源、DEX、证书与本地库。
3. **建立数据与权限模型**。建立组件、Intent、URI、权限、文件、数据库、网络和 native 调用的数据流图。
4. **定位信任边界**。从输入点追踪到比较、存储、解密和成功提示，区分客户端检查与服务器检查。
5. **执行最小本地验证**。在专用模拟器中使用 adb 启动组件、提供本地 Intent、查看 logcat 与应用私有状态。
6. **编写可重复脚本或测试**。将手工点击、交易、镜像提取或策略判断写成固定输入的测试，并保存完整输出。
7. **使用原始环境复验**。清理缓存与状态，从新模拟器、本地区块链、干净容器或策略副本重跑。

69.4 分析时持续追问的问题

- 当前观察是原始事实，还是工具给出的推断?
- 哪个最小输入变化能够区分主要假设?
- 结果能否在相同输入和干净环境中重复?
- 失败时是否保存了完整原始输出?
- 题目条件变化后，当前方法的前提是否仍成立?

69.5　命令、脚本与验证片段

以下内容只用于本地题目目录、回环地址、赛事提供的实例或明确授权的实验环境。每次运行都应保留原始输入和完整输出。

```
sha256sum original/challenge.apk | tee notes/apk.sha256
file original/challenge.apk
unzip -l original/challenge.apk > notes/apk-list.txt
aapt dump badging original/challenge.apk > notes/aapt-badging.txt
aapt dump xmltree original/challenge.apk AndroidManifest.xml > notes/manifest-
    tree.txt
apksigner verify --print-certs original/challenge.apk > notes/signature.txt

apktool d -f original/challenge.apk -o work/apktool > captures/apktool.log 2>&1
jadx -d work/jadx original/challenge.apk > captures/jadx.log 2>&1
rg -n -i 'flag|secret|token|loadLibrary|addJavascriptInterface|intent' \
  work/jadx work/apktool > notes/android-hits.txt || true

# Only against a dedicated local emulator with the challenge installed.
adb shell pm list packages | grep challenge
adb shell dumpsys package com.example.challenge > notes/package-dumpsys.txt
adb logcat -c
adb shell am start -n com.example.challenge/.MainActivity
adb logcat -d > captures/logcat.txt
```

Listing 69.1: 本章常用命令与脚本

69.6　证据保存与结果判定

保存原始附件和依赖哈希、平台与工具版本、清单和权限、反编译或解包结果、网络与日志、账户或地址状态、交易回执或容器层摘要、配置差异以及最终自动化测试。任何动态修改都要记录对象、偏移、方法、前后状态和恢复方式。

69.7　常见失误与修正方法

- 只搜索字符串，没有追踪资源 ID 和数据流。
- 把反编译 Java 当作准确源码，忽略 Kotlin 合成代码和优化。
- 在个人手机安装未知 APK。
- 只看主 Activity，遗漏导出 Service、Receiver、Provider 和深链。

- Java 层找不到逻辑后未分析 native 库。

69.8 小型演练

题目场景

APK 的主界面总提示失败，但 Manifest 有一个导出的 activity，接收 ctf://verify?code=... 深链。如何验证入口?

在清单中确认 activity、scheme、host、path 和是否要求权限。追踪 getIntent 与 Uri 参数到验证函数，记录 code 的格式和变换。在专用模拟器中使用 adb shell am start -a android.intent.action.VIEW -d 指定本地深链并明确组件，观察 logcat 与界面。若成功只由客户端判断，还要检查题目是否要求从资源、数据库或 native 函数恢复正确 code，而不是简单绕过界面。

69.9 完成检查

交付与复盘检查表

- □ 入口、输入、目标输出与限制条件已经写入笔记。
- □ 建立了可重复的基线输入与输出。
- □ 每个关键结论都有原始证据与验证步骤。
- □ 脚本能够识别失败，不会把异常输出当成成功。
- □ 从干净目录或新容器运行时能够得到同样结果。

第七十章　iOS IPA、Mach-O 与应用数据分析

本章目标

iOS 题通常提供 IPA、应用容器备份、模拟器数据或 Mach-O。静态分析关注 Info.plist、URL Scheme、Entitlements、资源、Objective-C 类与方法、Swift 符号、本地数据库和加密常量。动态验证应使用专用模拟器或题目设备。

70.1　原理与题型边界

IPA 是包含 Payload/App.app 的 ZIP。Mach-O 可包含多个架构切片，Objective-C 运行时保留类名、选择器和协议，Swift 符号可被 demangle。Entitlements 反映 Keychain 组、应用组、推送与调试等权限。

70.2　识别线索与优先动作

观察线索	可能含义	优先动作
Info.plist 有 URL types 或 query schemes	应用支持深链或查询其他应用	追踪 openURL 与参数解析
Entitlements 含 application-groups	数据可能位于共享容器	检查题目提供的容器目录
大量 Objective-C selector 可读	可从方法名快速定位验证逻辑	使用 nm、strings 和反编译器交叉引用
存在 Frameworks 或 dylib	核心逻辑可能在嵌入框架	逐个计算哈希并分析依赖

70.3 标准解题步骤

1. **固定附件与运行平台**。计算应用包、镜像、合约、配置和依赖哈希，记录平台版本、架构、网络、账户与题目实例范围。
2. **盘点结构与清单**。解压 IPA，列出 Info.plist、Entitlements、可执行文件、Frameworks、资源和签名信息。
3. **建立数据与权限模型**。建立 URL、通知、文件、数据库、Keychain 标识、网络与方法调用的数据流。
4. **定位信任边界**。从成功文本、输入控件、比较函数、选择器和 Swift 符号定位核心逻辑。
5. **执行最小本地验证**。在专用模拟器中验证深链、文件读取和状态变化，记录控制台日志与容器路径。
6. **编写可重复脚本或测试**。将手工点击、交易、镜像提取或策略判断写成固定输入的测试，并保存完整输出。
7. **使用原始环境复验**。清理缓存与状态，从新模拟器、本地区块链、干净容器或策略副本重跑。

70.4 分析时持续追问的问题

- 当前观察是原始事实，还是工具给出的推断?
- 哪个最小输入变化能够区分主要假设?
- 结果能否在相同输入和干净环境中重复?
- 失败时是否保存了完整原始输出?
- 题目条件变化后，当前方法的前提是否仍成立?

70.5 命令、脚本与验证片段

以下内容只用于本地题目目录、回环地址、赛事提供的实例或明确授权的实验环境。每次运行都应保留原始输入和完整输出。

```
sha256sum original/challenge.ipa | tee notes/ipa.sha256
mkdir -p work/ipa && unzip -q original/challenge.ipa -d work/ipa
find work/ipa/Payload -maxdepth 3 -type f -print > notes/ipa-files.txt

APP=$(find work/ipa/Payload -maxdepth 1 -name '*.app' -type d | head -n 1)
plutil -p "$APP/Info.plist" > notes/info-plist.txt
file "$APP"/* | tee notes/app-file-types.txt

# Replace Challenge with the CFBundleExecutable value from Info.plist.
BIN="$APP/Challenge"
file "$BIN" > notes/macho-file.txt
otool -hv "$BIN" > notes/macho-header.txt
otool -L "$BIN" > notes/macho-libraries.txt
nm -m "$BIN" > notes/macho-symbols.txt
strings -a -t x "$BIN" > notes/macho-strings.txt
```

Listing 70.1: 本章常用命令与脚本

70.6　证据保存与结果判定

保存原始附件和依赖哈希、平台与工具版本、清单和权限、反编译或解包结果、网络与日志、账户或地址状态、交易回执或容器层摘要、配置差异以及最终自动化测试。任何动态修改都要记录对象、偏移、方法、前后状态和恢复方式。

70.7　常见失误与修正方法

- 只看 Info.plist，不分析 Mach-O 和嵌入框架。
- 把设备架构切片与模拟器架构混淆。
- 通过修改返回值直接显示成功，却没有恢复题目要求的数据。
- 分析应用数据时忽略保护类、共享容器和数据库 WAL。
- 在个人设备安装未知配置或应用。

70.8 小型演练

题目场景

IPA 中有自定义 URL Scheme ctfapp，成功提示位于 Swift 方法中，输入还会读取 bundled rules.json。如何组织分析?

从 Info.plist 确认 URL Scheme 和入口代理方法，使用 strings、符号和反编译器定位处理 URL 的 Swift 或 Objective-C 方法。提取 rules.json 并记录格式，追踪其字段如何参与输入变换。将变换写成独立脚本，使用 IPA 原资源。最后在专用模拟器通过深链传入候选并观察成功提示。若使用了数据库或应用组文件，先清理应用状态再复验。

70.9 完成检查

交付与复盘检查表

- ☐ 入口、输入、目标输出与限制条件已经写入笔记。
- ☐ 建立了可重复的基线输入与输出。
- ☐ 每个关键结论都有原始证据与验证步骤。
- ☐ 脚本能够识别失败，不会把异常输出当成成功。
- ☐ 从干净目录或新容器运行时能够得到同样结果。

第七十一章　智能合约：存储布局、调用上下文与常见漏洞原语

本章目标

智能合约 CTF 通常在本地区块链或专用测试实例中要求改变某个状态。常见题型包括访问控制、重入、错误的 tx.origin、delegatecall、存储碰撞、签名重放、随机数、强制转账和逻辑状态机。解题要同时理解 Solidity 源码、EVM 执行和链上状态。

71.1　原理与题型边界

合约状态位于 32 字节槽，动态数组和映射通过哈希定位。msg.sender、tx.origin、msg.value、delegatecall 上下文和外部调用顺序会改变权限与存储。交易失败会回滚状态，但事件与调试追踪能揭示执行路径。

71.2　识别线索与优先动作

观察线索	可能含义	优先动作
权限检查使用 tx.origin	中间合约可能改变调用链语义	在本地链构造两层调用验证
外部 call 在余额更新之前	可能存在重入窗口	画出调用与状态更新顺序
delegatecall 目标可控	代码在调用者存储上下文执行	比较两合约存储布局
签名未绑定链、合约或 nonce	可能重放或跨域复用	重建签名消息和唯一性条件

71.3 标准解题步骤

1. **固定附件与运行平台**。计算应用包、镜像、合约、配置和依赖哈希，记录平台版本、架构、网络、账户与题目实例范围。
2. **盘点结构与清单**。记录编译器版本、优化设置、合约地址、部署交易、ABI、字节码和初始状态。
3. **建立数据与权限模型**。画出角色、函数、修饰器、外部调用、存储槽、资产和目标条件。
4. **定位信任边界**。使用 view 调用、存储读取和交易 trace 验证源码理解。
5. **执行最小本地验证**。在本地链编写最小攻击合约或测试，用断言检查每个状态变化。
6. **编写可重复脚本或测试**。将手工点击、交易、镜像提取或策略判断写成固定输入的测试，并保存完整输出。
7. **使用原始环境复验**。清理缓存与状态，从新模拟器、本地区块链、干净容器或策略副本重跑。

71.4 分析时持续追问的问题

- 当前观察是原始事实，还是工具给出的推断?
- 哪个最小输入变化能够区分主要假设?
- 结果能否在相同输入和干净环境中重复?
- 失败时是否保存了完整原始输出?
- 题目条件变化后，当前方法的前提是否仍成立?

71.5 存储槽的基本关系

静态值通常从槽 0 开始按布局规则存放。映射项可概括为

$$\mathrm{slot}(m[k]) = \mathrm{keccak256}(\mathrm{pad}(k) \| \mathrm{pad}(p)),$$

其中 p 是映射变量自身的槽号。动态数组数据区通常由数组槽号的哈希决定。实际分析还要考虑结构体、打包、继承与代理布局。

71.6　命令、脚本与验证片段

以下内容只用于本地题目目录、回环地址、赛事提供的实例或明确授权的实验环境。每次运行都应保留原始输入和完整输出。

```
# Example Foundry workflow on a local challenge project.
forge build | tee captures/forge-build.txt
forge test -vvv | tee captures/forge-test.txt
anvil --chain-id 31337 > captures/anvil.log 2>&1 &
ANVIL_PID=$!
trap 'kill $ANVIL_PID' EXIT

# Inspect local bytecode and storage.
cast code 0x0000000000000000000000000000000000001000 \
  --rpc-url http://127.0.0.1:8545 > notes/contract-code.txt
cast storage 0x0000000000000000000000000000000000001000 0 \
  --rpc-url http://127.0.0.1:8545 > notes/storage-slot-0.txt
cast call 0x0000000000000000000000000000000000001000 \
  'solved()(bool)' --rpc-url http://127.0.0.1:8545

# Trace only transactions submitted to the local chain.
cast run 0xLOCAL_TRANSACTION_HASH \
  --rpc-url http://127.0.0.1:8545 > captures/tx-trace.txt
```

Listing 71.1: 本章常用命令与脚本

71.7　证据保存与结果判定

保存原始附件和依赖哈希、平台与工具版本、清单和权限、反编译或解包结果、网络与日志、账户或地址状态、交易回执或容器层摘要、配置差异以及最终自动化测试。任何动态修改都要记录对象、偏移、方法、前后状态和恢复方式。

71.8　常见失误与修正方法

- 只读 Solidity 源码，不核对编译版本和实际字节码。
- 混淆 msg.sender 与 tx.origin。

- 假设槽按变量声明简单递增，忽略打包、继承和映射。
- 只看交易成功状态，不检查最终目标变量。
- 在公共主网或不属于题目的合约上试验。

71.9 小型演练

题目场景

本地题目 Vault 使用 tx.origin == owner 限制 withdraw，并要求 Vault 余额变为零。设计验证。

部署一个中间合约，其函数调用 Vault.withdraw。由 owner 账户调用中间合约时，Vault 内 tx.origin 仍为 owner，而 msg.sender 是中间合约。如果提款接收者取自 msg.sender，中间合约将收到资金；若接收者固定为 owner，则只需满足目标余额条件。用本地测试断言调用前后 owner、调用者、Vault 余额和 solved 状态，并保存 trace。结论说明漏洞来自授权主体选择错误。

71.10 完成检查

交付与复盘检查表

- □ 入口、输入、目标输出与限制条件已经写入笔记。
- □ 建立了可重复的基线输入与输出。
- □ 每个关键结论都有原始证据与验证步骤。
- □ 脚本能够识别失败，不会把异常输出当成成功。
- □ 从干净目录或新容器运行时能够得到同样结果。

第七十二章　Docker 镜像、层、Entrypoint、挂载与容器题

本章目标

容器题可能给出 Dockerfile、镜像 tar、compose 配置或运行中的本地服务。线索常在历史层、删除但仍存在的文件、环境变量、构建参数、Entrypoint、挂载、用户、capabilities 和网络中。镜像最终文件系统不是全部历史。

72.1　原理与题型边界

Docker 镜像由内容寻址层组成。上层删除文件通常通过 whiteout 隐藏，下层数据仍在镜像归档中。运行时又叠加可写层、环境、挂载、网络与安全配置。容器内 root 不等于宿主 root，但危险挂载和特权配置会扩大边界。

72.2　识别线索与优先动作

观察线索	可能含义	优先动作
最终容器中找不到文件但 history 有 COPY/RUN	文件可能仍在旧层	导出镜像并逐层检查
compose 挂载宿主路径或 socket	题目边界依赖运行配置	记录挂载、只读标志和用户
Entrypoint 与 CMD 组合复杂	实际参数可能与手工运行不同	检查 inspect 中最终配置
容器内环境含 token 或路径	构建或运行配置提供线索	区分镜像配置和运行时注入

72.3 标准解题步骤

1. **固定附件与运行平台**。计算应用包、镜像、合约、配置和依赖哈希，记录平台版本、架构、网络、账户与题目实例范围。
2. **盘点结构与清单**。保存 Dockerfile、compose、镜像 ID、RepoDigest、配置 JSON、历史和层摘要。
3. **建立数据与权限模型**。分别建立镜像文件系统、历史层和运行时挂载环境的模型。
4. **定位信任边界**。定位 Entrypoint、CMD、用户、工作目录、环境、端口、网络和 capability。
5. **执行最小本地验证**。在隔离主机以最小权限启动，验证服务入口和层中文件，不使用不必要的 privileged。
6. **编写可重复脚本或测试**。将手工点击、交易、镜像提取或策略判断写成固定输入的测试，并保存完整输出。
7. **使用原始环境复验**。清理缓存与状态，从新模拟器、本地区块链、干净容器或策略副本重跑。

72.4 分析时持续追问的问题

- 当前观察是原始事实，还是工具给出的推断?
- 哪个最小输入变化能够区分主要假设?
- 结果能否在相同输入和干净环境中重复?
- 失败时是否保存了完整原始输出?
- 题目条件变化后，当前方法的前提是否仍成立?

72.5 命令、脚本与验证片段

以下内容只用于本地题目目录、回环地址、赛事提供的实例或明确授权的实验环境。每次运行都应保留原始输入和完整输出。

```
docker image inspect ctf/challenge:local > notes/image-inspect.json
docker history --no-trunc ctf/challenge:local > notes/image-history.txt

docker image save ctf/challenge:local -o work/image.tar
mkdir -p work/image && tar -xf work/image.tar -C work/image
find work/image -maxdepth 3 -type f -printf '%s\t%p\n' \
  | sort -n > notes/image-files.txt

# Create a flattened filesystem without starting the image.
CID=$(docker create --network none ctf/challenge:local)
docker export "$CID" -o work/rootfs.tar
docker rm "$CID"
mkdir -p work/rootfs && tar -xf work/rootfs.tar -C work/rootfs
rg -n -i 'flag|secret|token|password' work/rootfs > notes/rootfs-hits.txt ||
    true

# Inspect compose configuration after variable substitution.
docker compose -f original/compose.yml config > notes/compose-resolved.yml
```

Listing 72.1: 本章常用命令与脚本

72.6　证据保存与结果判定

保存原始附件和依赖哈希、平台与工具版本、清单和权限、反编译或解包结果、网络与日志、账户或地址状态、交易回执或容器层摘要、配置差异以及最终自动化测试。任何动态修改都要记录对象、偏移、方法、前后状态和恢复方式。

72.7　常见失误与修正方法

- 只检查运行中的最终文件系统，忽略历史层。
- 直接运行不可信镜像并给予 privileged。
- 挂载 Docker socket 或宿主根目录进行分析。
- 把镜像环境变量与题目实例运行时变量混为一谈。
- 发现旧层秘密后没有验证它是否为当前题目有效值。

72.8 小型演练

题目场景

Dockerfile 先 COPY secret.txt，下一层 RUN rm secret.txt，最终容器中不存在该文件。如何恢复并证明来源?

用 docker image save 导出镜像，读取 manifest.json 获得层顺序。逐层检查 tar，找到包含 secret.txt 的层，再确认后续层的 whiteout 或删除行为。按层 tar 提取原文件，计算哈希，记录镜像 ID、层 digest、归档路径和 Dockerfile 对应指令。不要仅从 history 的命令文本猜内容。最后验证文件格式与题目约束，区分历史遗留值和运行时有效值。

72.9 完成检查

交付与复盘检查表

- □ 入口、输入、目标输出与限制条件已经写入笔记。
- □ 建立了可重复的基线输入与输出。
- □ 每个关键结论都有原始证据与验证步骤。
- □ 脚本能够识别失败，不会把异常输出当成成功。
- □ 从干净目录或新容器运行时能够得到同样结果。

第七十三章　云策略、IAM、K8s 清单与基础设施配置题

本章目标

云与 Kubernetes CTF 常提供 IAM policy、Terraform、CloudFormation、Kubernetes YAML、审计日志或本地模拟环境，要求识别权限路径、错误信任关系、公开资源、Secret 暴露或工作负载配置问题。优先进行离线策略分析，不访问真实云账户。

73.1　原理与题型边界

权限结论由主体、动作、资源、条件、显式拒绝、信任策略和会话上下文共同决定。Kubernetes 还涉及 ServiceAccount、RoleBinding、ClusterRole、Namespace、Secret、Volume、Pod Security 与网络。通配符不一定直接等于完全控制，条件与资源范围必须一起解释。

73.2　识别线索与优先动作

观察线索	可能含义	优先动作
策略允许 sts:AssumeRole 或等价动作	可能存在角色跳转链	画出主体到角色的信任图
Action 或 Resource 有通配符	权限范围可能过宽	结合条件和显式拒绝计算有效权限
Pod 使用 hostPath、privileged 或宿主命名空间	工作负载边界显著扩大	记录具体挂载、能力和服务账户

观察线索	可能含义	优先动作
Secret 以环境或卷注入	数据可能出现在清单、日志或容器	追踪 Secret 来源和可读主体

73.3 标准解题步骤

1. **固定附件与运行平台**。计算应用包、镜像、合约、配置和依赖哈希，记录平台版本、架构、网络、账户与题目实例范围。
2. **盘点结构与清单**。解析所有策略、信任关系、变量替换、模块输出、角色绑定、服务账户和 Secret 引用。
3. **建立数据与权限模型**。建立主体、角色、动作、资源、条件与显式拒绝的图，标出目标资源。
4. **定位信任边界**。对 Kubernetes 建立 namespace、ServiceAccount、RBAC、Pod、Volume 和网络关系。
5. **执行最小本地验证**。使用本地策略模拟、静态检查或最小测试配置验证，不对真实云账户发请求。
6. **编写可重复脚本或测试**。将手工点击、交易、镜像提取或策略判断写成固定输入的测试，并保存完整输出。
7. **使用原始环境复验**。清理缓存与状态，从新模拟器、本地区块链、干净容器或策略副本重跑。

73.4 分析时持续追问的问题

- 当前观察是原始事实，还是工具给出的推断?
- 哪个最小输入变化能够区分主要假设?
- 结果能否在相同输入和干净环境中重复?
- 失败时是否保存了完整原始输出?
- 题目条件变化后，当前方法的前提是否仍成立?

73.5　命令、脚本与验证片段

以下内容只用于本地题目目录、回环地址、赛事提供的实例或明确授权的实验环境。每次运行都应保留原始输入和完整输出。

```
# Normalize and search supplied infrastructure files locally.
find original/iac -type f -print | sort > notes/iac-files.txt
rg -n -i 'assumerole|principal|action|resource|condition|hostPath|privileged|
    serviceAccount|secret' \
  original/iac > notes/iac-hits.txt || true

# Render Terraform configuration without applying it.
terraform -chdir=original/iac init -backend=false > captures/tf-init.txt 2>&1
terraform -chdir=original/iac validate > captures/tf-validate.txt 2>&1
terraform -chdir=original/iac show -json plan.tfplan > work/plan.json 2>/dev/
    null || true

# Analyze local Kubernetes manifests only.
kubectl apply --dry-run=client -f original/k8s/ -o yaml > work/k8s-normalized.
    yaml
kubectl auth can-i --list \
  --as=system:serviceaccount:challenge:solver \
  --namespace challenge \
  --kubeconfig work/local-kind.kubeconfig > notes/rbac-can-i.txt

# Static policy checks can be added without deployment.
yq '.items[] | {kind,metadata:.metadata.name,spec}' work/k8s-normalized.yaml \
  > notes/k8s-summary.txt
```

Listing 73.1: 本章常用命令与脚本

73.6　证据保存与结果判定

保存原始附件和依赖哈希、平台与工具版本、清单和权限、反编译或解包结果、网络与日志、账户或地址状态、交易回执或容器层摘要、配置差异以及最终自动化测试。任何动态修改都要记录对象、偏移、方法、前后状态和恢复方式。

73.7　常见失误与修正方法

- 看到通配符就忽略条件、资源和显式拒绝。
- 只看身份策略，不看角色信任策略。

- 直接把题目凭据配置到日常云账户。
- Kubernetes 分析忽略 namespace 和 ServiceAccount。
- 对清单执行 apply，而题目只需要离线分析。

73.8 小型演练

题目场景

提供三份 IAM 策略和两个角色信任策略，要求找出从 userA 到读取 bucketFlag 的权限路径。如何分析?

先标准化每条 Statement，列出 Effect、Action、Resource、Condition 和 Principal。画出 userA 可直接执行的动作，尤其是 AssumeRole、PassRole 或策略修改类动作，再核对目标角色信任策略是否接受 userA 或其可到达角色。对每条跳转检查条件和会话限制。最终路径写成 userA 到 roleX 到 roleY 到 bucketFlag，并为每一步引用具体 Statement。若存在显式 Deny，应说明它是否覆盖路径。仅凭 Allow 通配符不能跳过信任策略。

73.9 完成检查

交付与复盘检查表

- □ 入口、输入、目标输出与限制条件已经写入笔记。
- □ 建立了可重复的基线输入与输出。
- □ 每个关键结论都有原始证据与验证步骤。
- □ 脚本能够识别失败，不会把异常输出当成成功。
- □ 从干净目录或新容器运行时能够得到同样结果。

第七十四章　现代平台综合案例：移动端、API、容器与链上状态联动

本章目标

综合题可能让移动应用调用本地 API，API 运行在容器中，再把结果写入本地区块链。分层分析时要为每一层建立独立输入输出和信任边界，避免把客户端显示、服务端授权和链上最终状态混为一件事。

74.1　原理与题型边界

端到端链路可拆为应用输入、Intent 或 UI、HTTP 请求、容器路由、数据库或秘密、合约调用与最终状态。每层都可能校验、转换或缓存。真正的成功条件通常由题面指定，例如 API 返回 flag、合约 solved 为 true 或链上余额变化。

74.2　识别线索与优先动作

观察线索	可能含义	优先动作
APK 中有 API base URL 与合约地址	客户端连接两种后端状态	分别验证 HTTP 与链上来源
容器有旧层配置和运行时环境	客户端看到的值可能来自不同阶段	追踪实际路由与配置优先级
API 返回交易哈希但界面显示失败	链上状态可能已成功	独立查询交易回执和目标变量
重装应用后行为变化	客户端缓存或本地数据库参与	清理状态并记录初始化流程

74.3 标准解题步骤

1. **固定附件与运行平台**。计算应用包、镜像、合约、配置和依赖哈希，记录平台版本、架构、网络、账户与题目实例范围。
2. **盘点结构与清单**。固定 APK、容器镜像、compose、合约源码或字节码、链快照和初始账户。
3. **建立数据与权限模型**。画出从移动输入到 API，再到合约调用与目标状态的完整数据流。
4. **定位信任边界**。为每层建立独立基线和日志，确认谁负责认证、转换、存储与成功判定。
5. **执行最小本地验证**。编写端到端脚本或测试，同时保留应用、HTTP、容器和交易证据。
6. **编写可重复脚本或测试**。将手工点击、交易、镜像提取或策略判断写成固定输入的测试，并保存完整输出。
7. **使用原始环境复验**。清理缓存与状态，从新模拟器、本地区块链、干净容器或策略副本重跑。

74.4 分析时持续追问的问题

- 当前观察是原始事实，还是工具给出的推断?
- 哪个最小输入变化能够区分主要假设?
- 结果能否在相同输入和干净环境中重复?
- 失败时是否保存了完整原始输出?
- 题目条件变化后，当前方法的前提是否仍成立?

74.5 命令、脚本与验证片段

以下内容只用于本地题目目录、回环地址、赛事提供的实例或明确授权的实验环境。每次运行都应保留原始输入和完整输出。

```
# Collect a reproducible multi-layer snapshot.
sha256sum original/challenge.apk original/image.tar original/contracts/* \
  > notes/all-source-hashes.txt

docker compose -f original/compose.yml config > notes/compose-resolved.yml
docker compose -f original/compose.yml up -d
trap 'docker compose -f original/compose.yml down -v' EXIT

curl --silent --show-error --include \
  http://127.0.0.1:8000/api/status \
  > captures/api-status.txt

cast call 0x0000000000000000000000000000000000001000 \
  'solved()(bool)' --rpc-url http://127.0.0.1:8545 \
  > captures/contract-solved.txt

adb logcat -c
adb shell am force-stop com.example.challenge
adb shell monkey -p com.example.challenge 1 >/dev/null
adb logcat -d > captures/android-end-to-end.log
```

Listing 74.1: 本章常用命令与脚本

74.6　证据保存与结果判定

保存原始附件和依赖哈希、平台与工具版本、清单和权限、反编译或解包结果、网络与日志、账户或地址状态、交易回执或容器层摘要、配置差异以及最终自动化测试。任何动态修改都要记录对象、偏移、方法、前后状态和恢复方式。

74.7　常见失误与修正方法

- 只修改 APK 客户端显示，未满足服务端或链上目标。
- 容器重启后依赖旧卷和缓存。
- 交易返回哈希就认定成功，没有检查 receipt 和状态。
- 同一 token 在应用、API 和合约中的编码处理未统一。
- 交付缺少一层日志，无法解释端到端失败。

74.8 小型演练

题目场景

APK 发送一个经过本地变换的 code 到容器 API, API 用旧镜像层中的 seed 生成签名并调用本地合约。题目要求合约 solved 为 true。如何组织完整解法?

先从 APK 恢复 code 变换并用模拟器请求日志验证。再从镜像历史层提取 seed，确认 API 实际加载路径和签名消息格式。用本地 HTTP 基线重现 API 请求，保存响应和容器日志。最后读取合约 ABI、nonce 和签名验证逻辑，提交交易并检查 receipt、事件与 solved 状态。自动化脚本应从原始 APK、镜像和合约配置生成请求，启动干净 compose 和本地链，最后断言 solved 为 true。

74.9 完成检查

交付与复盘检查表

- ☐ 入口、输入、目标输出与限制条件已经写入笔记。
- ☐ 建立了可重复的基线输入与输出。
- ☐ 每个关键结论都有原始证据与验证步骤。
- ☐ 脚本能够识别失败，不会把异常输出当成成功。
- ☐ 从干净目录或新容器运行时能够得到同样结果。

第八部分

竞赛工程、协作与训练方法

第七十五章　开赛盘点、题目分流、动态分值与时间预算

本章目标

团队开赛时需要快速建立题目地图，识别低成本入口、专长匹配、依赖关系和动态分值变化。好的分流不会把所有人锁在高分题上，也不会让低分题因无人确认而长期闲置。

75.1　原理与题型边界

题目价值由预期得分、成功概率、所需时间、并行性和解锁其他题的可能性共同决定。动态分值只反映解题人数或平台规则，不必然反映当前团队的边际成本。每个方向都需要检查点与退出条件。

75.2　识别线索与优先动作

观察线索	可能含义	优先动作
多个低分题尚未打开附件	可能存在快速签到、编码或配置题	安排一人完成全场低成本盘点
高分题已有明确原语	继续投入可能比新题更划算	记录距离目标还缺哪些原语
三人同时在同一方向重复枚举	分工和状态共享不足	拆成环境、静态、动态或脚本任务
题目分值下降很快	其他队伍已普遍解决	检查是否遗漏常见入口或公开提示

75.3　标准解题步骤

1. **明确当前目标与时间窗口**。写清题目、分值、当前入口、已有证据、下一检查点和停止条件。
2. **建立共享状态**。建立共享题目表，记录类别、分值、附件、远程状态、负责人、入口和下一动作。
3. **分配可并行任务**。按专长分配，同时保留一名机动人员处理环境、快速题和复现。
4. **按检查点交换证据**。每 20 至 40 分钟进行短检查，交换新事实和阻塞点，不重复完整讲解。
5. **把成功路径固化**。拿到可重复结果后立即整理提交和脚本，再决定是否继续优化。
6. **进行独立复现**。由未参与原解题过程的队员从原件开始执行，记录缺失步骤与环境假设。
7. **赛后复盘并转化为训练**。把卡点归类为知识、工具、环境、实验设计或协作问题，形成下一轮练习任务。

75.4　分析时持续追问的问题

- 当前观察是原始事实，还是工具给出的推断？
- 哪个最小输入变化能够区分主要假设？
- 结果能否在相同输入和干净环境中重复？
- 失败时是否保存了完整原始输出？
- 题目条件变化后，当前方法的前提是否仍成立？

75.5　命令、脚本与验证片段

以下内容只用于本地题目目录、回环地址、赛事提供的实例或明确授权的实验环境。每次运行都应保留原始输入和完整输出。

```
# A simple TSV board that remains readable without special software.
cat > notes/scoreboard.tsv <<'TSV'
challenge   category    points  owner   state   entry   next_check  next_action
welcome misc    50  alice   solved  base64  09:20   write minimal script
vault   pwn 300 bob working stack overflow  09:35   leak libc
archive forensics   200 carol   triaged deleted zip 09:30   verify partition
    offset
TSV

# Show active and blocked work.
awk -F'\t' 'NR==1 || $5=="working" || $5=="blocked"' notes/scoreboard.tsv \
  | column -t -s $'\t'

# Timestamp short checkpoints without replacing detailed notes.
printf '%s\t%s\t%s\n' "$(date --iso-8601=seconds)" vault \
  'libc leak stable; need stack alignment' >> notes/checkpoints.tsv
```

Listing 75.1: 本章常用命令与脚本

75.6　证据保存与结果判定

保存题目状态表、负责人、开始与检查点时间、原始证据、关键假设、已排除方向、成功脚本、提交记录、环境版本和复现日志。赛后复盘要能够区分技术原因、环境原因和沟通原因，而不是只记录“不会”或“没时间”。

75.7　常见失误与修正方法

- 按分值从高到低机械排序。
- 所有人同时看同一题却没有拆分任务。
- 没有停止条件，在一个方向上持续投入。
- 题目状态只写“在做”，没有入口和下一动作。
- 拿到 flag 后无人整理，之后无法复现或提交。

75.8　小型演练

题目场景

四人队伍开赛一小时，八道题中两道已解，一道高分 Pwn 有稳定崩溃但无泄露，三道低分题尚未打开。如何调整？

保留一人继续 Pwn，但明确下一检查点必须得到泄露或证明某替代原语。安排两人分别盘点低分 Web、Crypto 和 Misc，要求十分钟内写出附件类型、最可能入口和是否需要专用环境。第四人负责复现已解题、整理提交并机动支援。共享表中把 Pwn 的已知偏移、保护和缺失原语写清，避免换人后重新开始。下一次检查根据低分题入口和 Pwn 进展重新分配。

75.9　完成检查

交付与复盘检查表

- □ 入口、输入、目标输出与限制条件已经写入笔记。
- □ 建立了可重复的基线输入与输出。
- □ 每个关键结论都有原始证据与验证步骤。
- □ 脚本能够识别失败，不会把异常输出当成成功。
- □ 从干净目录或新容器运行时能够得到同样结果。

第七十六章　团队协作、共享笔记、证据交接与 Git 使用

本章目标

协作的目标是让队友能接着做，而不是只知道“有人正在做”。每道题都应有统一目录、负责人、事实、假设、实验、下一步和可执行脚本。Git 适合版本化文本与小脚本，大型附件和敏感实例信息需要单独管理。

76.1　原理与题型边界

高质量交接包含上下文、原始证据、已排除方向和失败判定。口头交流快但易丢失，笔记稳定但不能变成大段流水账。提交频率应以可验证里程碑为单位，而不是每条临时输出。

76.2　识别线索与优先动作

观察线索	可能含义	优先动作
队友接手后先问附件在哪里	目录和 README 不统一	为每题建立固定结构和入口说明
同一脚本出现多个私有版本	没有主分支或提交约定	用短分支与小提交合并
失败方向被重复尝试	笔记只记录成功结论	保留输入、输出和排除依据
大型 pcap 或镜像进入 Git	仓库膨胀且操作变慢	只提交哈希、下载说明和派生小文件

76.3　标准解题步骤

1. **明确当前目标与时间窗口**。写清题目、分值、当前入口、已有证据、下一检查点和停止条件。
2. **建立共享状态**。共享目录中每题包含 README、original 哈希、scripts、notes、captures 和 output。
3. **分配可并行任务**。按可并行问题拆分分支，例如环境复现、协议分析、算法重建和 exploit。
4. **按检查点交换证据**。检查点只交换新事实、阻塞点、下一动作和需要的协助，原始日志通过文件传递。
5. **把成功路径固化**。成功后整理一条干净命令、依赖与输出，提交脚本和说明。
6. **进行独立复现**。由未参与原解题过程的队员从原件开始执行，记录缺失步骤与环境假设。
7. **赛后复盘并转化为训练**。把卡点归类为知识、工具、环境、实验设计或协作问题，形成下一轮练习任务。

76.4　分析时持续追问的问题

- 当前观察是原始事实，还是工具给出的推断?
- 哪个最小输入变化能够区分主要假设?
- 结果能否在相同输入和干净环境中重复?
- 失败时是否保存了完整原始输出?
- 题目条件变化后，当前方法的前提是否仍成立?

76.5　命令、脚本与验证片段

以下内容只用于本地题目目录、回环地址、赛事提供的实例或明确授权的实验环境。每次运行都应保留原始输入和完整输出。

```
# Initialize a challenge workspace with a conservative ignore file.
mkdir -p team/vault/{original,work,scripts,notes,captures,output}
cat > team/vault/.gitignore <<'GITIGNORE'
original/*
work/*
captures/*.pcap
captures/*.raw
captures/core*
!original/SHA256SUMS
GITIGNORE

cat > team/vault/README.md <<'README'
# vault

- Owner:
- Scope:
- Known facts:
- Current hypotheses:
- Ruled out:
- Reproduce:
- Next action:
README

git -C team add vault/README.md vault/.gitignore
git -C team commit -m 'vault: add triage workspace'

# Exchange a patch instead of copying an entire divergent directory.
git -C team format-patch -1 --stdout > output/latest-change.patch
```

Listing 76.1: 本章常用命令与脚本

76.6　证据保存与结果判定

保存题目状态表、负责人、开始与检查点时间、原始证据、关键假设、已排除方向、成功脚本、提交记录、环境版本和复现日志。赛后复盘要能够区分技术原因、环境原因和沟通原因，而不是只记录“不会”或“没时间”。

76.7　常见失误与修正方法

- 把聊天记录当作唯一笔记。
- 提交包含远程凭据、会话 Cookie 或个人密钥。

- 大提交混合环境、算法和无关格式化。
- 用 Git 强制覆盖队友未提交工作。
- README 只有最终 flag，没有失败条件和复现命令。

76.8　小型演练

题目场景

队友 A 已经定位 Web 题的隐藏 API，队友 B 要接手写自动化，但 A 只有代理历史和口头描述。怎样完成有效交接?

A 导出最小原始请求和响应，删除个人 Cookie 后放入 captures，README 写明认证前提、必需头部、参数语义、稳定响应特征和已排除测试。再提供一条 curl 基线和代理项目或 HAR 的位置。B 先在相同实例重放基线，确认状态码、长度和关键字段一致，再编写脚本。脚本提交时把主机、端口和 token 改为参数，不提交真实会话。

76.9　完成检查

交付与复盘检查表

- □ 入口、输入、目标输出与限制条件已经写入笔记。
- □ 建立了可重复的基线输入与输出。
- □ 每个关键结论都有原始证据与验证步骤。
- □ 脚本能够识别失败，不会把异常输出当成成功。
- □ 从干净目录或新容器运行时能够得到同样结果。

第七十七章　Writeup、复现实验、证据图与责任边界

本章目标

优秀 Writeup 让读者从题面和原件开始，理解入口、关键推理、验证步骤和最终脚本。它不需要记录每一次盲目尝试，但必须保留决定方向的证据，并说明示例只适用于题目环境。

77.1　原理与题型边界

Writeup 的结构可分为环境与附件、题型识别、关键事实、漏洞或算法原理、利用或求解步骤、脚本、验证和复盘。截图应辅助证明，不应代替可复制文本。涉及赛事尚未结束、第三方系统或真实漏洞时需要遵守披露规则。

77.2　识别线索与优先动作

观察线索	可能含义	优先动作
正文从最终 payload 开始	缺少入口与前提，读者无法迁移理解	先写观察证据和约束
只有截图没有命令或数据	结果不可搜索和复现	同时提供文本、偏移和脚本
脚本依赖绝对路径或旧缓存	作者电脑外无法运行	在干净环境执行并记录依赖
公开时仍包含实例 token 或个人信息	存在泄露风险	发布前进行范围与秘密审查

77.3　标准解题步骤

1. **明确当前目标与时间窗口**。写清题目、分值、当前入口、已有证据、下一检查点和停止条件。
2. **建立共享状态**。从题面、附件哈希、环境和目标条件开始，写明授权范围。
3. **分配可并行任务**。按事实到假设到验证组织关键路径，删除无关尝试但保留重要排除。
4. **按检查点交换证据**。提供最小脚本、依赖、运行命令、预期中间输出和失败判断。
5. **把成功路径固化**。由另一人从干净环境复现，再发布或归档。
6. **进行独立复现**。由未参与原解题过程的队员从原件开始执行，记录缺失步骤与环境假设。
7. **赛后复盘并转化为训练**。把卡点归类为知识、工具、环境、实验设计或协作问题，形成下一轮练习任务。

77.4　分析时持续追问的问题

- 当前观察是原始事实，还是工具给出的推断?
- 哪个最小输入变化能够区分主要假设?
- 结果能否在相同输入和干净环境中重复?
- 失败时是否保存了完整原始输出?
- 题目条件变化后，当前方法的前提是否仍成立?

77.5　命令、脚本与验证片段

以下内容只用于本地题目目录、回环地址、赛事提供的实例或明确授权的实验环境。每次运行都应保留原始输入和完整输出。

```
# Reproduce from a clean local virtual environment.
python3 -m venv work/repro-venv
. work/repro-venv/bin/activate
```

```
python3 -m pip install --require-hashes -r scripts/requirements.txt \
  > captures/pip-install.txt 2>&1

python3 scripts/solve.py \
  --input original/challenge.bin \
  --output output/result.txt \
  > captures/repro.stdout \
  2> captures/repro.stderr
printf 'exit=%s\n' "$?" > captures/repro.meta
sha256sum original/challenge.bin scripts/solve.py output/result.txt \
  > output/repro-hashes.txt

# Scan the publication tree for obvious local secrets before release.
rg -n -i 'cookie:|authorization:|api[_-]?key|private[_-]?key|session=' \
  writeup scripts notes || true
```

Listing 77.1: 本章常用命令与脚本

77.6　证据保存与结果判定

保存题目状态表、负责人、开始与检查点时间、原始证据、关键假设、已排除方向、成功脚本、提交记录、环境版本和复现日志。赛后复盘要能够区分技术原因、环境原因和沟通原因，而不是只记录“不会”或“没时间”。

77.7　常见失误与修正方法

- 把 Writeup 写成工具命令清单，没有推理和验证。
- 保留大量无关尝试，关键路径被淹没。
- 只给最终 flag 或 payload。
- 使用赛事外真实目标截图。
- 在比赛结束前公开题目关键细节。

77.8　小型演练

题目场景

你要整理一道 SQL 注入题，最终脚本有效，但中间手工测试很多。哪些内容应写入正式 Writeup?

保留题面和环境、原始请求基线、导致怀疑 SQL 的稳定差异、确认语法与列数的最小实验、数据库类型依据、受限枚举策略、最终脚本和干净复现。删去没有提供新信息的随机 payload，但可在复盘中概括早期误判。所有请求指向本地或赛事示例地址，移除实例 token。说明哪些错误响应或时间样本证明了结论。

77.9　完成检查

交付与复盘检查表

- ☐ 入口、输入、目标输出与限制条件已经写入笔记。
- ☐ 建立了可重复的基线输入与输出。
- ☐ 每个关键结论都有原始证据与验证步骤。
- ☐ 脚本能够识别失败，不会把异常输出当成成功。
- ☐ 从干净目录或新容器运行时能够得到同样结果。

第七十八章　环境故障、工具差异、编码问题与系统化排错

本章目标

比赛中的大量时间损失来自环境而非题目：Python 包版本、libc、架构、Java 版本、代理证书、DNS、换行、文件权限、时区、容器缓存和终端编码。排错应先缩小层级，再比较已知正常与失败环境。

78.1　原理与题型边界

系统化排错遵循输入、程序、依赖、操作系统、网络和外部状态的层次。错误消息与退出码是证据。重装全部工具通常会破坏可比较性，并可能掩盖真正差异。

78.2　识别线索与优先动作

观察线索	可能含义	优先动作
队友脚本成功而本机失败	环境或输入存在差异	比较哈希、版本、命令和环境变量
容器重建后行为不变	缓存层或卷保留旧状态	检查 build cache、volume 和 compose 配置
文本解析多一字符或少一字符	CRLF、末尾换行、BOM 或编码不同	十六进制比较原始字节
连接域名到错误服务	DNS、Host、SNI 或代理配置差异	分别检查解析、TCP、TLS 和 HTTP

78.3　标准解题步骤

1. **明确当前目标与时间窗口**。写清题目、分值、当前入口、已有证据、下一检查点和停止条件。
2. **建立共享状态**。把失败压缩为最小命令、最小输入和完整错误输出。
3. **分配可并行任务**。比较成功与失败环境的文件哈希、版本、架构、locale、时区、环境变量和网络路径。
4. **按检查点交换证据**。一次只改变一项，优先验证输入、路径、权限、依赖和协议边界。
5. **把成功路径固化**。修复后写入依赖清单、容器配置或启动脚本，防止再次出现。
6. **进行独立复现**。由未参与原解题过程的队员从原件开始执行，记录缺失步骤与环境假设。
7. **赛后复盘并转化为训练**。把卡点归类为知识、工具、环境、实验设计或协作问题，形成下一轮练习任务。

78.4　分析时持续追问的问题

- 当前观察是原始事实，还是工具给出的推断?
- 哪个最小输入变化能够区分主要假设?
- 结果能否在相同输入和干净环境中重复?
- 失败时是否保存了完整原始输出?
- 题目条件变化后，当前方法的前提是否仍成立?

78.5　命令、脚本与验证片段

以下内容只用于本地题目目录、回环地址、赛事提供的实例或明确授权的实验环境。每次运行都应保留原始输入和完整输出。

```
# Capture a compact environment report.
{
  uname -a
```

```
  id
  pwd
  locale
  date --iso-8601=seconds
  python3 --version
  python3 -m pip freeze
  openssl version -a
  file work/chall
  sha256sum work/chall work/input.bin
  env | sort
} > notes/environment-report.txt 2>&1

# Compare raw text inputs including line endings and BOM.
xxd -g 1 work/input-good.txt > notes/input-good.hex
xxd -g 1 work/input-bad.txt > notes/input-bad.hex
diff -u notes/input-good.hex notes/input-bad.hex | tee notes/input-byte-diff.
    txt

# Rebuild a local container without cache and remove named state deliberately.
docker compose -f original/compose.yml down -v
docker compose -f original/compose.yml build --no-cache \
  > captures/rebuild.txt 2>&1
docker compose -f original/compose.yml up -d
```

Listing 78.1: 本章常用命令与脚本

78.6　证据保存与结果判定

保存题目状态表、负责人、开始与检查点时间、原始证据、关键假设、已排除方向、成功脚本、提交记录、环境版本和复现日志。赛后复盘要能够区分技术原因、环境原因和沟通原因，而不是只记录“不会”或“没时间”。

78.7　常见失误与修正方法

- 看到 ImportError 就升级所有包。
- 失败环境与成功环境输入文件哈希不同却未发现。
- 关闭安全机制或使用 root 作为通用修复。
- 网络问题直接归咎服务器，没有分层检查。
- 修复只存在终端历史，没有进入项目配置。

78.8　小型演练

题目场景

同一 Pwn exploit 在队友机器稳定，本机在进入 system 时崩溃。怎样排查?

先比较 ELF、libc、加载器和 exploit 哈希，再比较架构、pwntools 版本、启动命令和 ASLR 设置。确认本机是否真的使用题目 libc，检查泄露基址计算与映射。若地址一致但在 libc 内 movaps 崩溃，比较 RSP 对齐和链长度。一次只修改加载器或对齐 gadget，保存 GDB 状态。最终把正确加载命令和依赖写入脚本，而不是依赖本机全局库。

78.9　完成检查

交付与复盘检查表

- ☐ 入口、输入、目标输出与限制条件已经写入笔记。
- ☐ 建立了可重复的基线输入与输出。
- ☐ 每个关键结论都有原始证据与验证步骤。
- ☐ 脚本能够识别失败，不会把异常输出当成成功。
- ☐ 从干净目录或新容器运行时能够得到同样结果。

第七十九章　训练计划、题库选择、错题档案与能力增长

本章目标

持续提升来自有目的的重复，而不是只追求做题数量。训练应在知识学习、短题练习、限时模拟、复现优秀 Writeup 和赛后复盘之间形成循环，并维护可以检索的错题档案。

79.1　原理与题型边界

技能可分为识别、原理、工具、实验设计、脚本化和表达。某题做不出可能因为不知道概念，也可能因为环境不稳、没有验证假设或无法自动化。训练任务应针对具体缺口。

79.2　识别线索与优先动作

观察线索	可能含义	优先动作
看 Writeup 能懂但独立不会做	识别和起步能力不足	隔一段时间无提示重做同类题
知道漏洞但脚本经常失败	协议、边界和工程能力不足	训练最小复现与错误处理
比赛中卡住，赛后很快看懂	时间预算和题型信号不足	做限时分类与入口练习
错题笔记只有答案	无法形成可迁移知识	记录线索、关键实验和失败原因

79.3　标准解题步骤

1. **明确当前目标与时间窗口**。写清题目、分值、当前入口、已有证据、下一检查点和停止条件。
2. **建立共享状态**。建立个人能力矩阵，按题型记录“能识别、能手工、能脚本、能解释”的等级。
3. **分配可并行任务**。每周选择一个窄主题学习原理，再完成三到五道递进练习。
4. **按检查点交换证据**。每两周做一次限时混合题，训练分流、停止条件和交付。
5. **把成功路径固化**。错题在 24 小时、7 天和 30 天后无提示复现，更新模板与检查表。
6. **进行独立复现**。由未参与原解题过程的队员从原件开始执行，记录缺失步骤与环境假设。
7. **赛后复盘并转化为训练**。把卡点归类为知识、工具、环境、实验设计或协作问题，形成下一轮练习任务。

79.4　分析时持续追问的问题

- 当前观察是原始事实，还是工具给出的推断?
- 哪个最小输入变化能够区分主要假设?
- 结果能否在相同输入和干净环境中重复?
- 失败时是否保存了完整原始输出?
- 题目条件变化后，当前方法的前提是否仍成立?

79.5　十二周训练节奏建议

第 1 至 2 周巩固 Linux、字节、Python 与网络基线；第 3 至 5 周选择一个主方向深入；第 6 周进行混合限时模拟；第 7 至 9 周补第二方向并复现经典题；第 10 周练团队交接；第 11 周进行完整赛制模拟；第 12 周整理模板、错题和下一周期能力矩阵。每周至少保留一个能够从原件一条命令复现的完整题目。

79.6 命令、脚本与验证片段

以下内容只用于本地题目目录、回环地址、赛事提供的实例或明确授权的实验环境。每次运行都应保留原始输入和完整输出。

```
# A plain-text practice ledger.
cat > notes/practice.tsv <<'TSV'
date    challenge   category    time_min    result  main_gap    retry_date    artifact
2026-07-10  local-sqli-01   web 45  solved  column-count reasoning  2026-07-17    scripts/solve.py
2026-07-10  ret2libc-02 pwn 90  partial libc base validation    2026-07-12    notes/ret2libc.md
TSV

# Summarize time and gaps with a short Python script.
python3 - <<'PYCODE'
from pathlib import Path
import csv, collections
rows=list(csv.DictReader(Path('notes/practice.tsv').open(),delimiter='\t'))
by_cat=collections.defaultdict(lambda:[0,0])
gaps=collections.Counter()
for r in rows:
    by_cat[r['category']][0]+=1
    by_cat[r['category']][1]+=int(r['time_min'])
    gaps[r['main_gap']]+=1
print('category summary')
for k,(n,t) in sorted(by_cat.items()): print(k,n,t)
print('gaps')
for k,v in gaps.most_common(): print(v,k)
PYCODE
```

Listing 79.1: 本章常用命令与脚本

79.7 证据保存与结果判定

保存题目状态表、负责人、开始与检查点时间、原始证据、关键假设、已排除方向、成功脚本、提交记录、环境版本和复现日志。赛后复盘要能够区分技术原因、环境原因和沟通原因，而不是只记录“不会”或“没时间”。

79.8 常见失误与修正方法

- 题库随机跳跃，没有围绕具体能力建立递进。

- 做完立即看答案，没有延迟复现。
- 只统计 solved 数量，不记录耗时和卡点。
- 模板越积越多但从不在比赛中使用。
- 只练擅长类别，团队短板长期存在。

79.9　小型演练

题目场景

你能看懂 Web Writeup，但比赛中常不知道第一步做什么。制定四周修正计划。

第一周只练请求重构和差分，每天两道小题，要求保存基线、输入位置表和一条 curl。第二周练认证、访问控制与注入识别，每题在二十分钟内写出三个可检验假设。第三周无提示重做前两周题，并把成功路径脚本化。第四周做两次九十分钟混合模拟，记录何时应该继续或换题。每道错题在七天后复现，档案必须写题型线索、关键证据、最小实验和失败原因。

79.10　完成检查

交付与复盘检查表

- □ 入口、输入、目标输出与限制条件已经写入笔记。
- □ 建立了可重复的基线输入与输出。
- □ 每个关键结论都有原始证据与验证步骤。
- □ 脚本能够识别失败，不会把异常输出当成成功。
- □ 从干净目录或新容器运行时能够得到同样结果。

附录 A　实验目录、环境记录与题目工作表

本附录提供一套可以直接用于每道题的目录和记录模板。模板的目的不是增加文书工作，而是确保原始输入、失败实验、成功脚本和环境差异都能被快速定位。比赛中可以删减字段，但不应删除原件哈希、基线命令、关键证据和最终复现。

```
mkdir -p challenge/{original,work,scripts,captures,notes,output}
sha256sum challenge/original/* > challenge/notes/SHA256SUMS

cat > challenge/README.md <<'README'
# Challenge name

## Scope and target
- Category:
- Points:
- Flag format:
- Authorized host/port:
- Instance lifetime:

## Environment
- OS / architecture:
- Runtime and versions:
- Supplied libc / loader / image:

## Known facts
1.
2.

## Current hypotheses
| ID | Hypothesis | Supporting evidence | Falsifying test | State |
|----|------------|---------------------|-----------------|-------|

## Experiments
| Time | Input/change | Expected | Actual | Artifact | Conclusion |
|------|--------------|----------|--------|----------|------------|

## Reproduce
```sh
# one clean command
```

```

```
Next action
README

script -q -f challenge/captures/session-$(date +%Y%m%d-%H%M%S).log
```

Listing A.1: 题目目录与 README 模板

A.1　环境快照字段

字段	示例内容	用途
样本哈希	SHA-256、文件大小	确认队员分析的是同一原件
运行平台	Linux x86-64、Android API 34、本地 Anvil	解释架构、ABI 与平台差异
依赖版本	Python、libc、JDK、Node、Solidity 编译器	复现算法和运行时行为
命令行	完整参数、工作目录、环境变量	消除隐含状态
网络范围	回环地址或赛事主机、端口、协议	控制授权边界
时间与时区	ISO 8601、UTC 偏移	关联日志与实例生命周期
失败判定	退出码、状态码、异常文本、断言	避免异常输出被当作成功
派生文件	来源偏移、转换命令、前后哈希	维护证据链

A.2　最小实验记录表

编号	事实或假设	唯一变化	原始结果	判断
E01	基线输入可重复	无	记录状态码、长度、时间或寄存器	基线成立或不稳定
E02	字段 A 影响分支	只改变字段 A	保存完整响应或内存差异	支持、否定或信息不足

编号	事实或假设	唯一变化	原始结果	判断
E03	环境差异导致失败	只替换依赖或加载器	比较哈希、映射与错误位置	定位环境层级
E04	候选满足目标条件	从原件运行最终脚本	保存中间值和最终验证	完成复现

附录 B　Linux、网络与文本处理命令速查

下表只列出比赛中高频且容易遗忘的用途。命令参数应根据题目文件和授权范围调整，运行前先使用只读或列表模式，输出通过 tee 保存。对于未知文件名，优先使用空字符分隔的 find 与 xargs 组合。

命令	主要用途	注意事项
sha256sum file	计算样本哈希	把结果与文件大小一起记录
file file	按内容识别格式	扩展名只作提示
stat file	查看权限、大小和时间	注明文件系统与时区
xxd -g 1 -l 256 file	查看头部字节	偏移默认十六进制显示
od -An -tx1 -v file	连续输出字节	适合脚本管道
strings -a -t x -n 5 file	提取 ASCII 字符串与偏移	宽字符另用 -e l 或专用工具
readelf -h -l -S -s -r file	ELF 头、段、节、符号、重定位	保存原始输出
objdump -d -M intel file	反汇编 ELF	先缩小函数范围
find dir -type f -print0	安全枚举文件	与 xargs -0 配合
rg -n -i pattern dir	递归搜索文本	二进制数据需专门处理
awk -F TAB ...	按字段处理文本	保留原行号与原文件
sort | uniq -c | sort -nr	统计唯一值频次	排序前保存原始顺序
jq -S . file.json	规范化 JSON	不要用字符串切分 JSON
tar -tf archive.tar	只列出归档	检查路径后再解压
unzip -l archive.zip	只列 ZIP 条目	警惕父目录与绝对路径
7z l -slt archive	显示技术字段	可查看方法、CRC、大小
curl -i –max-redirs 0 URL	保存 HTTP 状态、头部与跳转	显式控制 Cookie 与代理

命令	主要用途	注意事项
openssl s_client -connect host:port -servername name	检查 TLS 与 SNI	只连接题目服务
nc -nv host port	建立简单 TCP 基线	TCP 是字节流
socat -v - TCP:host:port	显示双向字节	适合记录本地交互
tcpdump -i lo -nn -s0 -w out.pcap filter	抓取本地流量	先确认接口与过滤器
tshark -r in.pcap -Y filter	离线显示过滤	与 capture filter 语法不同
dig name TXT +noall +answer	查询指定 DNS 记录	记录解析器和时间
timeout 10s command	限制单次运行时间	保存退出码 124 等状态
/usr/bin/time -v command	记录资源与运行时间	与 shell 内建 time 区分

```
# Safe batch hashing for filenames containing spaces or newlines.
find work -type f -print0 | while IFS= read -r -d '' f; do
  printf '%s\t' "$f"
  sha256sum "$f"
done > notes/hashes.tsv

# Preserve each stage of a text pipeline.
rg -n 'ERROR|FAIL' original/app.log | tee work/error-lines.txt
cut -d' ' -f1-3 work/error-lines.txt | tee work/error-times.txt
sort work/error-times.txt | uniq -c | sort -nr > output/error-summary.txt

# Compare byte-level differences, including line endings and BOM.
xxd -g 1 work/a.txt > work/a.hex
xxd -g 1 work/b.txt > work/b.hex
diff -u work/a.hex work/b.hex | tee notes/byte-diff.txt
```

Listing B.1: 安全批处理与中间结果保存

附录 C　常见文件签名、容器结构与编码速查

格式	典型签名	常见偏移	验证要点
PNG	89 50 4E 47 0D 0A 1A 0A	0	块结构，IEND 后可能有尾随数据
JPEG	FF D8 FF	0	通常以 FF D9 结束，但需结合段解析
GIF87a/GIF89a	47 49 46 38 37/39 61	0	索引色与多帧
BMP	42 4D	0	文件头含像素数据偏移
TIFF little endian	49 49 2A 00	0	IFD 通过偏移连接
TIFF big endian	4D 4D 00 2A	0	注意端序
PDF	25 50 44 46	0 或文件前部	可有增量更新与多个 EOF
ZIP/APK/OOXML	50 4B 03 04	0 或嵌入位置	中央目录决定完整结构
7-Zip	37 7A BC AF 27 1C	0	头部含校验
RAR 4	52 61 72 21 1A 07 00	0	RAR5 签名略长
RAR 5	52 61 72 21 1A 07 01 00	0	块结构
gzip	1F 8B	0	后有压缩方法与标志
bzip2	42 5A 68	0	后接块大小字符
xz	FD 37 7A 58 5A 00	0	容器有索引与尾部
ELF	7F 45 4C 46	0	后续字节给出位宽与端序

格式	典型签名	常见偏移	验证要点
PE	4D 5A	0	PE 签名位于 e_lfanew 指定位置
Mach-O 32/64	FE ED FA CE / FE ED FA CF 等	0	还存在反端序与 Fat magic
Java class	CA FE BA BE	0	包含版本与常量池
DEX	64 65 78 0A	0	后接版本和零字节
WebAssembly	00 61 73 6D	0	后接版本
SQLite 3	SQLite format 3 + 00	0	固定 16 字节签名
PCAP	D4 C3 B2 A1 或 A1 B2 C3 D4	0	签名同时反映端序
PCAPNG	0A 0D 0D 0A	0	由多种块组成
WAV	RIFF....WAVE	0 与 8	RIFF 长度为小端
AVI	RIFF....AVI	0 与 8	RIFF 容器
WebP	RIFF....WEBP	0 与 8	RIFF 容器
FLAC	66 4C 61 43	0	元数据块后为帧
Ogg	4F 67 67 53	0	页序列与 CRC
MP3	49 44 33 或同步字	0	ID3 可选，帧同步需验证
ISO 9660	43 44 30 30 31	通常 0x8001	卷描述符位置固定

C.1　编码识别线索

编码或表示	线索	验证方法
十六进制文本	字符集限于 0-9 A-F，长度通常为偶数	严格解码，重新编码应恢复输入
标准 Base64	字母数字加 + /，可有 = 填充	使用 validate 模式并检查输出结构
URL-safe Base64	字母数字加 - _	区分标准字符集与填充
Base32	A-Z 与 2-7，常有 =	长度与字符集约束

编码或表示	线索	验证方法
URL 编码	百分号后两个十六进制字符	区分路径与表单中 + 的含义
UTF-16LE	可见 ASCII 字节后常有 00	按码元解码并检查 BOM
小端整数	低有效字节位于低地址	用字段范围和邻近结构验证
二进制补码	最高位参与符号解释	明确位宽后转换
zlib/deflate	常见 78 01、78 9C、78 DA 头	使用容器与校验验证

```
from base64 import b64decode, urlsafe_b64decode, b32decode
from pathlib import Path
from urllib.parse import unquote_to_bytes
import hashlib, zlib

raw=Path('original/value.txt').read_bytes().strip()
print('input',len(raw),hashlib.sha256(raw).hexdigest())

candidates=[]
if len(raw)%2==0:
    try: candidates.append(('hex',bytes.fromhex(raw.decode('ascii'))))
    except Exception: pass
try: candidates.append(('base64',b64decode(raw,validate=True)))
except Exception: pass
try: candidates.append(('base64url',urlsafe_b64decode(raw+b'='*((-len(raw))%4))
    ))
except Exception: pass
try: candidates.append(('url',unquote_to_bytes(raw.decode('ascii'))))
except Exception: pass

for name,data in candidates:
    print(name,len(data),data[:32],hashlib.sha256(data).hexdigest())
```

Listing C.1: 分层解码候选与结果记录

附录 D　HTTP 状态、头部、请求模板与 Web 检查表

HTTP 测试需要保存原始请求，而不是只保留代理中的参数表。相同参数在路径、查询、表单、JSON、Cookie 和头部中可能经过不同解析器。重定向、缓存、压缩和动态字段会影响差分，先建立最小基线再改变单个变量。

状态码	常见语义	分析注意
200	请求成功	仍需检查正文是否为错误页面
201	资源已创建	关注 Location 与返回对象
202	已接受异步处理	后续状态可能另查
204	成功但无正文	不要按 JSON 解析
301	永久重定向	客户端可能缓存
302	临时重定向	历史客户端可能改变方法
303	使用 GET 获取结果	常见表单提交后跳转
307	临时重定向并保留方法	消息体通常保留
308	永久重定向并保留方法	方法与消息体保留
400	请求语法或解析错误	可用于区分解析层
401	需要认证	检查 WWW-Authenticate
403	已识别但拒绝	与未认证、访问控制区分
404	资源不存在或被隐藏	可能是路由或权限伪装
405	方法不允许	检查 Allow
409	状态冲突	竞态、版本或重复操作
413	请求体过大	上传和代理限制
415	媒体类型不支持	Content-Type 解析分支
422	语义校验失败	结构已解析但字段无效
429	速率限制	遵守 Retry-After 与赛事限制
500	应用内部错误	保留响应但不要破坏服务
502	网关收到无效上游响应	区分代理和应用
503	服务暂不可用	实例启动、过载或维护

状态码	常见语义	分析注意
504	网关超时	上游延迟或不可达

头部	含义与检查点
Host	虚拟主机选择，HTTP/1.1 必需
Content-Type	决定消息体解析器
Content-Length	声明消息体字节长度
Transfer-Encoding	分块或其他传输编码
Cookie	客户端会话状态
Set-Cookie	服务端设置 Cookie 与属性
Authorization	认证凭据或 token
Origin	跨源请求来源
Referer	导航来源，可能用于弱校验
Location	重定向目标
Cache-Control	缓存策略
Vary	响应缓存所依赖的请求头
X-Forwarded-For	代理传递的来源地址，不能默认可信
X-Forwarded-Host	代理传递的主机
Content-Security-Policy	限制脚本、资源与导航来源
Access-Control-Allow-Origin	CORS 允许的读取来源
Access-Control-Allow-Credentials	CORS 是否允许凭据
WWW-Authenticate	认证方案与挑战
ETag/If-Match	缓存与并发版本控制
Range/Content-Range	分段读取与响应范围

```
# Minimal GET baseline without automatic redirects.
curl --http1.1 --silent --show-error --include --max-redirs 0 \
  'http://127.0.0.1:8000/items?id=1' \
  -H 'Host: challenge.local' \
  -o captures/get-baseline.txt

# JSON request with separate response headers and body.
```

```
curl --silent --show-error \
  -D captures/create.headers \
  -o captures/create.body \
  -w 'code=%{http_code} bytes=%{size_download} time=%{time_total}\n' \
  -X POST 'http://127.0.0.1:8000/api/items' \
  -H 'Content-Type: application/json' \
  -H 'Cookie: session=LOCAL_TEST_TOKEN' \
  --data-binary @work/request.json

# Multipart upload of a harmless local test file.
curl --silent --show-error --include \
  -F 'description=baseline' \
  -F 'file=@work/sample.txt;type=text/plain;filename=sample.txt' \
  'http://127.0.0.1:8000/upload' \
  > captures/upload-response.txt

# Show each redirect as a separate response when investigating a chain.
curl --silent --show-error --include --max-redirs 0 \
  'http://127.0.0.1:8000/start'
```

Listing D.1: 可重放 HTTP 基线模板

交付与复盘检查表

- ☐ 方法、路径、查询、头部、Cookie 和消息体都有原始副本。
- ☐ 已经关闭自动跳转并逐跳检查 Location。
- ☐ 已建立未认证、普通用户和目标角色的权限矩阵。
- ☐ 参数缺失、空值、重复、类型和边界长度已分别测试。
- ☐ 状态码、长度、关键字段和时间差使用多次样本验证。
- ☐ 客户端 JavaScript、源映射、接口路径和隐藏字段已经检查。
- ☐ 文件名、Content-Type、文件内容和保存路径被分别验证。
- ☐ 所有测试只针对题目提供的本地或赛事实例。

附录 E　Python 解题模板库

E.1　健壮的命令行求解器

```python
from __future__ import annotations
import argparse, hashlib, logging
from pathlib import Path

LOG=logging.getLogger('solver')

def load(path:Path)->bytes:
    data=path.read_bytes()
    if not data:
        raise ValueError(f'empty input: {path}')
    LOG.info('input=%s size=%d sha256=%s',path,len(data),hashlib.sha256(data).
    hexdigest())
    return data

def solve(data:bytes)->bytes:
    # Replace with the challenge transformation.
    return data[::-1]

def validate(result:bytes)->None:
    if not (result.startswith(b'flag{') and result.endswith(b'}')):
        raise ValueError(f'unexpected result: {result[:64]!r}')

def main()->int:
    p=argparse.ArgumentParser()
    p.add_argument('input',type=Path)
    p.add_argument('-o','--output',type=Path)
    p.add_argument('-v','--verbose',action='store_true')
    a=p.parse_args()
    logging.basicConfig(level=logging.DEBUG if a.verbose else logging.INFO)
    result=solve(load(a.input)); validate(result)
    if a.output: a.output.write_bytes(result+b'\n')
    else: print(result.decode('ascii'))
    return 0

if __name__=='__main__':
    raise SystemExit(main())
```

Listing E.1: 带输入哈希、验证和退出码的求解器

E.2　TCP 精确读取与带长度消息

```python
import socket, struct

def recv_exact(sock:socket.socket,n:int)->bytes:
    if n<0: raise ValueError('negative length')
    chunks=[]; left=n
    while left:
        block=sock.recv(left)
        if not block:
            raise EOFError(f'connection closed with {left} bytes missing')
        chunks.append(block); left-=len(block)
    return b''.join(chunks)

with socket.create_connection(('127.0.0.1',31337),timeout=3) as s:
    s.settimeout(3)
    size=struct.unpack('<I',recv_exact(s,4))[0]
    if size>1_000_000:
        raise ValueError(f'unreasonable frame length: {size}')
    frame=recv_exact(s,size)
    print(frame)
```

Listing E.2: TCP 长度前缀消息模板

E.3　二进制结构解析

```python
from dataclasses import dataclass
from pathlib import Path
import struct

@dataclass
class Record:
    kind:int
    flags:int
    length:int
    payload:bytes

def parse_records(data:bytes)->list[Record]:
    pos=0; out=[]
    while pos<len(data):
        if len(data)-pos<8:
            raise ValueError(f'truncated header at {pos:#x}')
        kind,flags,length=struct.unpack_from('<HHI',data,pos)
        pos+=8
        if length>len(data)-pos:
```

```
            raise ValueError(f'length {length} exceeds remaining bytes at {pos:#x}')
        payload=data[pos:pos+length]; pos+=length
        out.append(Record(kind,flags,length,payload))
    return out

data=Path('original/records.bin').read_bytes()
for i,r in enumerate(parse_records(data)):
    print(i,r.kind,r.flags,r.length,r.payload[:16].hex())
```

Listing E.3: 带边界检查的二进制解析器

E.4　HTTP 会话与差分

```
import json, time
from pathlib import Path
import requests

BASE='http://127.0.0.1:8000'
s=requests.Session()
s.headers.update({'User-Agent':'ctf-local-reproducer/1.0'})

def request(name,method,path,**kwargs):
    start=time.perf_counter()
    r=s.request(method,BASE+path,timeout=5,allow_redirects=False,**kwargs)
    elapsed=time.perf_counter()-start
    Path(f'captures/{name}.body').write_bytes(r.content)
    Path(f'captures/{name}.json').write_text(json.dumps({
        'status':r.status_code,'headers':dict(r.headers),'bytes':len(r.content),
        'elapsed':elapsed,'url':r.url},indent=2),encoding='utf-8')
    return r

baseline=request('baseline','GET','/profile',params={'id':'1'})
changed=request('changed','GET','/profile',params={'id':'2'})
print(baseline.status_code,len(baseline.content))
print(changed.status_code,len(changed.content))
```

Listing E.4: 保存响应证据的 requests 模板

附录 F　密码学公式、判断表与脚本模板

F.1　基础关系

欧几里得算法计算 $\gcd(a,b)$。若 $\gcd(a,n)=1$，则 a 在模 n 下存在逆元 a^{-1}，满足

$$a\,a^{-1}\equiv 1 \pmod{n}.$$

RSA 中 $n=pq$，常见欧拉函数为 $\varphi(n)=(p-1)(q-1)$，私钥指数满足

$$d\equiv e^{-1} \pmod{\varphi(n)}.$$

加解密关系为 $c\equiv m^e \pmod n$ 与 $m\equiv c^d \pmod n$。这些式子只有在编码、填充和参数条件明确时才能直接用于题目。

中国剩余定理在模数两两互素时把多个同余式合并为唯一的模乘积解。哈希长度扩展只适用于特定 Merkle-Damgård 结构和错误的 secret-prefix MAC，不适用于 HMAC。CTR 与流密码重用 nonce 或密钥流时，两个密文异或会消去密钥流。

题目线索	首选数学动作	验证条件
多个 RSA 模数共享因子	对所有模数做批量 gcd	确认公钥集合来源和重复素数
小 e 且 m^e<n	整数 e 次根	必须验证幂没有模回绕
同一消息、同一 e、多个互素模数	Håstad 广播条件	先 CRT 合并，再取整数根
已知 p+q 或 phi(n)	解二次方程或直接求 d	验证 p*q=n
DSA/ECDSA nonce 重用	由两个签名恢复 nonce 与私钥	消息哈希和签名格式必须一致
CBC 错误反馈区分填充	可能形成 padding oracle	统计稳定响应并限制请求

题目线索	首选数学动作	验证条件
CTR/GCM nonce 重用	密钥流或认证关系受损	比较 nonce、密钥与消息结构
LCG 连续输出	建立模线性关系	明确模数、截断和输出位
MT19937 完整 32 位输出	可逆 temper 后恢复状态	需要足够连续输出
近似线性关系与小误差	考虑格规约	先正确缩放并验证候选

```python
from math import gcd, isqrt

def egcd(a,b):
    if b==0: return a,1,0
    g,x1,y1=egcd(b,a%b)
    return g,y1,x1-(a//b)*y1

def invmod(a,n):
    g,x,_=egcd(a,n)
    if g!=1: raise ValueError('inverse does not exist')
    return x%n

def iroot(n,k):
    lo,hi=0,1
    while hi**k<n: hi*=2
    while lo+1<hi:
        mid=(lo+hi)//2
        if mid**k<n: lo=mid
        else: hi=mid
    return hi if hi**k==n else lo

# Shared-prime check for local challenge public moduli.
moduli=[]
for i,n in enumerate(moduli):
    for j in range(i):
        p=gcd(n,moduli[j])
        if 1<p<n:
            q=n//p
            print(i,j,p,q)
```

Listing F.1: 模逆、整数根与共享因子模板

附录 G　Pwn、GDB 与 pwntools 速查

架构/ABI	常见整数参数	返回值	关键注意
x86-64 System V	RDI, RSI, RDX, RCX, R8, R9	RAX	调用边界注意 16 字节对齐
i386 cdecl	通常由栈从右到左传参	EAX	调用者清理参数
AArch64	X0 至 X7	X0	LR 为 X30，SP 16 字节对齐
ARM 32-bit AAPCS	R0 至 R3，其余栈	R0	注意 ARM/Thumb 状态
MIPS o32	A0 至 A3，其余栈	V0	常见延迟槽与端序差异

保护	影响	典型应对思路
NX	数据页不可执行	优先代码复用、现有函数或可执行映射
PIE	主程序基址随机	需要程序地址泄露、相对控制或部分覆盖
ASLR	栈、堆、库等随机	泄露地址并验证基址对齐
Canary	返回前检测栈破坏	泄露并原样保留，或使用不触及它的原语
Partial RELRO	部分重定位后仍有可写 GOT	是否覆盖取决于实际读写能力
Full RELRO	GOT 通常只读	转向其他写目标和代码复用
Fortify	部分 libc 调用带边界检查	检查编译优化和实际调用

GDB 命令	用途
info registers	显示寄存器
x/24gx $rsp	按 8 字节查看栈
x/32bx ADDRESS	查看原始字节
disassemble /r FUNC	显示函数与机器码
info proc mappings	查看进程映射
vmmap	插件中的映射视图
break *ADDRESS	按地址断点
watch *ADDRESS	写观察点
rwatch *ADDRESS	读观察点
catch syscall NAME	系统调用捕获点
set follow-fork-mode child	跟随子进程
set $rax=VALUE	临时修改寄存器
dump binary memory OUT START END	导出内存
generate-core-file OUT	生成 core
set disable-randomization off	恢复 ASLR 行为

```python
from pwn import *
import argparse

parser=argparse.ArgumentParser()
parser.add_argument('--remote',action='store_true')
parser.add_argument('--host',default='127.0.0.1')
parser.add_argument('--port',type=int,default=31337)
a=parser.parse_args()

elf=ELF('work/chall',checksec=False)
context.binary=elf
context.log_level='debug'

if a.remote:
    io=remote(a.host,a.port,timeout=4)
else:
    io=process([elf.path])

# Establish protocol synchronization before sending the payload.
io.recvuntil(b'> ')
offset=cyclic_find(0x6161616c)  # replace with measured crash value
payload=flat({offset:elf.symbols.get('win',0)})
```

```
log.info('payload length=%d',len(payload))
io.sendline(payload)
result=io.recvall(timeout=3)
if b'flag{' not in result:
    raise SystemExit(f'expected flag marker, got {result[:120]!r}')
print(result.decode('latin-1',errors='replace'))
```

Listing G.1: 本地与远程双模式 pwntools 模板

附录 H　逆向工程指令、跳转与工具速查

跳转	条件	常见含义
je / jz	ZF=1	相等或结果为零
jne / jnz	ZF=0	不相等或非零
ja	CF=0 且 ZF=0	无符号大于
jae	CF=0	无符号大于等于
jb	CF=1	无符号小于
jbe	CF=1 或 ZF=1	无符号小于等于
jg	ZF=0 且 SF=OF	有符号大于
jge	SF=OF	有符号大于等于
jl	SF 不等于 OF	有符号小于
jle	ZF=1 或 SF 不等于 OF	有符号小于等于
jo / jno	OF=1 / 0	有符号溢出存在或不存在
js / jns	SF=1 / 0	结果符号位

指令	分析要点
mov/movzx/movsx	移动、零扩展、符号扩展
lea	地址计算或纯算术
test	按位与但不保存结果，常用于零检查
cmp	减法式比较但不保存结果
shl/shr/sar	左移、逻辑右移、算术右移
rol/ror	固定宽度旋转
imul/mul	有符号或无符号乘法
idiv/div	有符号或无符号除法
cmovcc	条件移动，减少显式分支
setcc	按条件写一个字节

指令	分析要点
call/ret	调用与返回，结合 ABI 分析
push/pop	栈操作，注意位宽和对齐

```
# Static triage.
file work/chall
readelf -h -l -S -s -r -d work/chall > notes/readelf.txt
strings -a -t x -n 5 work/chall > notes/strings.txt
objdump -d -M intel work/chall > notes/disassembly.txt

# Dynamic triage around system and library boundaries.
strace -f -o captures/strace.log ./work/chall < work/input.txt
ltrace -f -o captures/ltrace.log ./work/chall < work/input.txt

# Exact-width helpers for translating decompiled arithmetic.
python3 - <<'PYCODE'
MASK8=0xff; MASK32=0xffffffff
def rol8(x,n):
    n&=7; x&=MASK8
    return ((x<<n)|(x>>(8-n)))&MASK8
def s8(x):
    x&=MASK8
    return x-0x100 if x&0x80 else x
print(rol8(0x81,3),s8(0xff))
PYCODE
```

Listing H.1: 静态、动态与固定位宽验证模板

附录 I　取证命令、Wireshark 过滤器与 Volatility 插件速查

显示过滤器	用途
http.request	所有 HTTP 请求
http.response	所有 HTTP 响应
tcp.stream == 7	指定 TCP 流
dns.flags.response == 0	DNS 查询
dns.qry.type == 16	TXT 查询
tls.handshake.extensions_server_name	含 SNI 的 TLS 握手
tcp.analysis.retransmission	TCP 重传
tcp.analysis.out_of_order	乱序包
frame contains ”flag{”	原始帧包含字节串
ip.addr == 10.0.0.5	任一方向涉及 IP
tcp.port == 31337	任一方向涉及端口
http.request.method == ”POST”	POST 请求
http.content_type contains ”json”	JSON 相关响应
usb.capdata	USB 捕获数据字段

命令	用途
mmls image	列出分区与起始扇区
fsstat -o SECTOR image	文件系统信息
fls -r -p -o SECTOR image	递归目录与删除项
icat -o SECTOR image META	按元数据地址恢复文件
blkls -o SECTOR image	导出未分配块
ils -o SECTOR image	列出 inode 或元数据记录

命令	用途
tsk_recover -o SECTOR image out	批量恢复分配或删除文件
exiftool -a -u -g1 file	元数据完整视图
binwalk file	签名扫描，不把结果当最终边界
qpdf –check file.pdf	检查 PDF 结构

Volatility 3 插件	主要用途
windows.info	Windows 内核与层信息
windows.pslist	活动进程链表
windows.psscan	扫描进程对象
windows.pstree	父子关系
windows.cmdline	命令行
windows.netscan	网络端点
windows.dlllist	进程模块
windows.handles	句柄
windows.filescan	扫描文件对象
windows.dumpfiles	导出文件对象
windows.vadinfo	虚拟地址描述符
windows.malfind	可疑可执行私有区域
linux.pslist	Linux 进程
linux.lsof	Linux 打开文件
linux.sockstat	Linux 套接字

```
# PCAP baseline and selected exports.
capinfos original/traffic.pcap > notes/capinfos.txt
tshark -r original/traffic.pcap -q -z io,phs > notes/protocols.txt
tshark -r original/traffic.pcap -q -z endpoints,ip > notes/endpoints.txt
tshark -r original/traffic.pcap -q -z conv,tcp > notes/conversations.txt
tshark -r original/traffic.pcap -Y 'http.request' \
  -T fields -e frame.time_epoch -e tcp.stream -e ip.src \
  -e http.request.method -e http.host -e http.request.uri \
  > notes/http.tsv

# Memory baseline with Volatility 3.
python3 vol.py -f original/memory.raw windows.info > notes/memory-info.txt
python3 vol.py -f original/memory.raw windows.pslist > notes/pslist.txt
python3 vol.py -f original/memory.raw windows.psscan > notes/psscan.txt
```

```
python3 vol.py -f original/memory.raw windows.netscan > notes/netscan.txt
```

Listing I.1: 网络与内存取证基线

附录 J　移动端、合约、容器与云配置速查

命令	用途
aapt dump badging app.apk	Android 包名、版本、入口概览
aapt dump xmltree app.apk AndroidManifest.xml	编译清单树
apktool d app.apk	资源和 smali 解包
jadx -d out app.apk	Java/Kotlin 反编译
apksigner verify –print-certs app.apk	签名证书
adb shell dumpsys package PKG	已安装应用组件与权限
adb logcat	模拟器日志
plutil -p Info.plist	解析 Apple plist
otool -L binary	Mach-O 动态依赖
nm -m binary	Mach-O 符号
forge test -vvv	本地 Solidity 测试与 trace
cast call ADDRESS SIG	本地链只读调用
cast storage ADDRESS SLOT	读取本地链存储槽
docker image inspect IMAGE	镜像配置 JSON
docker history –no-trunc IMAGE	镜像历史层命令
docker image save IMAGE -o image.tar	导出镜像与层
docker compose config	解析 compose 变量后的配置
terraform validate	离线验证 Terraform
kubectl apply –dry-run=client	本地解析 Kubernetes 清单
kubectl auth can-i –list –as …	在本地集群评估 RBAC

交付与复盘检查表

- □ 移动应用的清单、组件、资源、证书、数据库、网络与 native 库均已盘点。
- □ 深链和导出组件只在专用模拟器测试。
- □ 合约的编译版本、ABI、字节码、部署状态和目标条件已经固定。
- □ 所有交易只发往题目本地链或专用测试实例。
- □ Docker 镜像最终文件系统与历史层分别检查。
- □ 未知镜像没有使用 privileged、Docker socket 或宿主根挂载。
- □ IAM 分析同时考虑身份策略、信任策略、条件、资源和显式拒绝。
- □ Kubernetes 分析明确 namespace、ServiceAccount、RoleBinding、Volume 和安全上下文。

附录 K　按题型执行的最终检查清单

K.1　通用

交付与复盘检查表

- □ 原件哈希与题面已保存。
- □ flag 格式和提交要求明确。
- □ 事实、假设与实验分开记录。
- □ 基线输入与输出可重复。
- □ 每次实验只改变一个主要变量。
- □ 失败输出与退出码有保存。
- □ 最终脚本从干净环境运行成功。
- □ 未访问题目范围之外的目标。

K.2　Web

交付与复盘检查表

- □ 原始请求包含方法、路径、头部、Cookie 与消息体。
- □ 跳转链逐跳检查。
- □ 匿名、普通用户和目标角色建立权限矩阵。
- □ 查询、表单、JSON、Cookie 与头部输入分别检查。
- □ 客户端代码、接口路径和源映射已查看。

- ☐ 文件名、内容、MIME 与保存路径分别验证。
- ☐ 时间差异使用多次样本。
- ☐ 脚本限制速率并识别异常响应。

K.3　Crypto

交付与复盘检查表

- ☐ 明确题目目标是解密、伪造、恢复密钥还是预测状态。
- ☐ 所有整数的位宽、端序与编码明确。
- ☐ 先检查 gcd、重复参数、nonce 和边界条件。
- ☐ 每一步数学推导有代回验证。
- ☐ 候选明文通过填充、格式或重新加密校验。
- ☐ 随机数样本顺序与截断方式明确。
- ☐ 格模型有正确缩放和候选代回。
- ☐ 没有把编码误称为加密。

K.4　Pwn

交付与复盘检查表

- ☐ ELF、libc 与加载器哈希一致。
- ☐ 架构、ABI、端序和保护机制明确。
- ☐ 偏移由循环模式或内存证据测量。
- ☐ 泄露字节长度、端序和基址对齐被验证。
- ☐ ROP 每个参数和栈对齐已检查。
- ☐ 堆 exploit 的 glibc 版本与状态可重复。

- ☐ 本地与远程共用同一逻辑，只改变连接参数。
- ☐ 成功输出由脚本断言。

K.5 Reverse

交付与复盘检查表

- ☐ 输入、参数、环境与外部文件固定。
- ☐ 字符串和导入通过交叉引用验证。
- ☐ 反编译类型与汇编宽度交叉检查。
- ☐ 动态断点围绕关键事件而非无目的单步。
- ☐ 数据变换明确溢出、端序、轮次和索引。
- ☐ 补丁只用于观察，并记录偏移。
- ☐ 求解器与原程序中间状态一致。
- ☐ 未修改样本验证最终输入。

K.6 Forensics

交付与复盘检查表

- ☐ 原证据只读且有哈希。
- ☐ 所有时间戳的来源与时区明确。
- ☐ 提取文件记录来源偏移、流、PID 或元数据地址。
- ☐ 自动工具结果由结构或第二工具验证。
- ☐ 排序视图保留原行号和原顺序。
- ☐ PCAP 使用会话重组而非直接拼包。
- ☐ 内存插件结果结合多个枚举来源。

- ☐ 报告区分直接证据与推断。

K.7　Mobile/Contract/Cloud

交付与复盘检查表

- ☐ 应用或配置附件哈希与平台版本明确。
- ☐ 客户端检查与服务端或链上目标区分。
- ☐ 组件、权限、深链和本地存储有数据流图。
- ☐ 合约调用上下文、存储槽与最终状态被验证。
- ☐ 容器层、运行时挂载和环境分别检查。
- ☐ 云权限包含信任关系与条件。
- ☐ 测试只在模拟器、本地链、干净容器或本地集群。
- ☐ 端到端结果包含每层日志。

附录 L　术语表

术语	说明
ABI	应用二进制接口，规定调用约定、数据布局、系统调用等二进制层规则。
ASLR	地址空间布局随机化，使栈、堆、共享库等运行地址变化。
Base64	把字节编码为有限字符集的表示，不提供保密性。
BSS	未显式初始化的全局或静态数据区域。
Canary	栈保护值，返回前检查是否被覆盖。
CBC	分组密码链接模式，每块与前一密文块相关。
CFB	将分组密码用作类流模式的一种工作模式。
CIL	Common Intermediate Language，.NET 托管字节码。
CORS	浏览器跨源读取控制机制，由响应头决定。
CRC	循环冗余校验，适合检测传输错误，不是密码学哈希。
CSRF	利用浏览器自动携带凭据触发跨站请求的攻击类别。
CSP	内容安全策略，限制浏览器可加载或执行的资源来源。
CTR	计数器模式，把分组密码输出作为密钥流。
DEX	Android Dalvik/ART 使用的可执行字节码容器。
DOM XSS	客户端脚本把不可信数据写入危险 DOM sink 导致的 XSS。
ELF	Linux 与类 Unix 常见的可执行与目标文件格式。
Entropy	信息熵或字节分布度量，高熵可由压缩、加密或随机数据产生。
EVM	以太坊虚拟机，执行智能合约字节码。
GOT	全局偏移表，动态链接程序中保存符号解析结果。
HMAC	基于密码学哈希和密钥的消息认证结构。
HTTP Parameter Pollution	重复参数在多个组件中被不同方式解释。

术语	说明
IDOR	通过直接改变对象标识访问未授权对象的访问控制缺陷。
IFD	TIFF 文件中的图像文件目录结构。
IAT	PE 文件的导入地址表。
JWT	由头部、载荷和签名或 MAC 组成的令牌格式。
KASLR	内核地址空间布局随机化。
LSB	最低有效位，图像与音频隐写常用载体。
MAC	消息认证码，用于完整性和来源认证。
Mach-O	Apple 平台常见的可执行文件格式。
Nonce	只使用一次或在给定密钥下必须唯一的参数。
NX	不可执行页保护，阻止在数据页直接执行代码。
OAuth	委托授权框架，涉及客户端、资源所有者、授权服务器和资源服务器。
OOB	越界访问，读取或写入对象边界之外。
Padding Oracle	通过可区分反馈判断填充是否有效的侧信道。
PCAP	保存网络数据包的捕获文件格式。
PCAPNG	支持多接口和扩展块的新一代抓包格式。
PE	Windows 可执行文件格式。
PIE	位置无关可执行文件，使主程序也可随机化加载。
PLT	过程链接表，动态调用外部符号的跳板。
PRNG	伪随机数生成器，由内部状态确定性产生输出。
RCE	远程代码执行，在 CTF 中只应针对题目实例验证。
RELRO	重定位只读保护，影响 GOT 等区域的可写性。
ROP	返回导向编程，通过已有代码片段构造计算。
S-box	代换盒，密码算法中进行非线性替换。
SameSite	Cookie 属性，限制跨站请求中是否携带 Cookie。
Seccomp	Linux 系统调用过滤机制。
SNI	TLS 握手中声明目标服务器名称的扩展。
SSRF	服务端根据用户影响的地址发起请求。
SSTI	服务端模板注入，不可信输入进入模板语法解释。
Stack Pivot	把栈指针转移到可控内存区域。
Symbolic Execution	把输入表示为符号并沿程序路径积累约束。

术语	说明
Tcache	现代 glibc 中的线程本地小块缓存。
TOCTOU	检查时与使用时状态发生变化的竞态类别。
TLS	传输层安全协议，为应用数据提供加密与认证。
UAF	Use After Free，释放后继续使用对象。
Unicode Normalization	把等价 Unicode 序列转换为规范形式。
VAD	Windows 进程虚拟地址描述符。
VM Bytecode	由自定义或托管虚拟机解释执行的指令序列。
WAL	预写式日志，SQLite 等系统用来记录尚未合并的事务。
XSS	不可信数据在浏览器脚本上下文执行。
XXE	XML 外部实体处理导致的文件读取或外部请求问题。
Z3	常用于位向量、整数和布尔约束求解的 SMT 求解器。
字节序	多字节整数在内存或文件中的字节排列方式。
补码	固定宽度有符号整数常用的二进制表示。
魔数	文件或协议开头等位置用于识别格式的固定字节。
文件雕刻	依据签名和结构从原始数据中恢复文件对象。
交叉引用	代码或数据对某地址、符号或对象的引用关系。
基线	在最少变化条件下可重复的输入与输出。
信息增益	一次实验减少不确定性的程度。
原语	能够稳定实现的基本能力，如任意读、有限写或控制流。
重放	使用保存的原始输入在相同条件下再次执行。
证据链	从原始输入到派生结果的来源、操作和哈希关系。
状态机	由状态、输入、转移和输出组成的行为模型。
动态链接	程序运行或加载时解析外部库符号。
静态分析	不执行目标程序而检查结构、代码和数据。
动态分析	在受控执行中观察运行时行为。
时区校准	把不同来源时间转换并验证到共同时间基准。
置信度	结论受证据支持的程度，应与事实和推断区分。

附录 M　可打印复盘表与训练记录

下列空表可打印或复制到每道题的 README。填写时使用具体事实和文件路径，不使用“差不多”“应该是”一类无法验证的描述。复盘重点是下一次遇到相似信号时能否更快建立基线和验证假设。

M.1　题目入口表

题目与类别	
目标与 flag 格式	
附件、远程与范围	
第一批可验证事实	
主要假设	
下一检查点与停止条件	

M.2　实验记录表

编号	时间	输入或唯一变化	原始结果与文件	结论和下一步

M.3　最终复现表

原件哈希	
依赖与版本	

一条运行命令	
关键中间输出	
成功判定	
失败诊断	
范围与安全说明	

M.4　赛后复盘表

最早出现的有效线索	
没有及时利用的证据	
主要卡点类别	知识 / 工具 / 环境 / 实验设计 / 协作 / 时间管理
可固化的脚本或模板	

七天后复现日期	
三十天后复现结果	
下一组训练题	

参考资料

参考文献

[1] OWASP Foundation. *Web Security Testing Guide*. https://owasp.org/www-project-web-security-testing-guide/

[2] PortSwigger. *Web Security Academy*. https://portswigger.net/web-security

[3] Python Software Foundation. *Python Documentation*. https://docs.python.org/3/

[4] Gallopsled. *pwntools Documentation*. https://docs.pwntools.com/

[5] Free Software Foundation. *Debugging with GDB*. https://sourceware.org/gdb/current/onlinedocs/gdb.html/

[6] National Security Agency. *Ghidra Documentation*. https://ghidra-sre.org/

[7] Wireshark Foundation. *Wireshark User's Guide*. https://www.wireshark.org/docs/

[8] Volatility Foundation. *Volatility 3 Documentation*. https://volatility3.readthedocs.io/

[9] Docker Inc. *Docker Documentation*. https://docs.docker.com/

[10] Solidity Project. *Solidity Documentation*. https://docs.soliditylang.org/

[11] The Cryptopals Crypto Challenges. https://cryptopals.com/

[12] Intel Corporation. *Intel 64 and IA-32 Architectures Software Developer Manuals*.

[13] Arm Limited. *Arm Architecture Reference Manuals*.